

Bases de datos espaciales

Alexys H. Rodríguez-Avellaneda Ph.D.

2026-09-21

Tabla de contenidos

Bienvenida	1
Objetivos y alcance formativo	1
Material de la clase	2
Cómo usar este libro	2
Requisitos previos del entorno	2
Licencia y citación del material	3
Licencia de uso	3
Cómo citar este material	3
Contacto y colaboración	4
Prefacio	5
Enfoque metodológico y computacional	5
Pilares operativos de la práctica computacional	6
Reproducibilidad y entorno de trabajo	6
Organización de la obra	6
1 Syllabus: bases de datos espaciales	7
1.1 Información general	7
1.2 Justificación académica	7
1.3 Propósito formativo y competencias	8
1.3.1 Propósito general	8
1.3.2 Competencias del curso	8
1.4 Objetivos de aprendizaje	8
1.5 Estructura curricular articulada en cuatro ejes	9
1.6 Programación semanal y contenidos detallados (16 semanas)	9
1.6.1 Eje 1: fundamentos de diseño, modelado y arquitecturas de datos (semanas 1 a 3)	9
1.6.2 Eje 2: motores relacionales espaciales con PostgreSQL y PostGIS (semanas 4 a 7)	10
1.6.3 Evaluación de medio término y balance pedagógico (semana 8)	11
1.6.4 Eje 3: geodatabases corporativas, diseño y formatos avanzados (semanas 9 a 10)	11
1.6.5 Eje 4: paradigmas emergentes y seminario de proyectos finales (semanas 11 a 16)	12
1.7 Sistema de evaluación y ponderación porcentual	13
1.8 Estructura de entregables del proyecto final en el repositorio Quarto	13
1.9 Políticas del curso, entregas y régimen disciplinario	14
1.9.1 Gestión de entregas y control de versiones vía GitHub	14
1.9.2 Régimen de entregas extemporáneas y penalizaciones	14
1.9.3 Asistencia, puntualidad y justificación de inasistencias	14
1.9.4 Plan de compensación y recuperación de sesiones por días festivos	15
1.9.5 Integridad académica, ética en investigación y código de conducta	15
1.10 Referencias bibliográficas	15

I	Instalación y entornos	17
	Contenido modular de la sección	19
2	Guía de instalación	21
2.1	Introducción	21
2.2	Instalación de Visual Studio Code	21
2.3	Instalación del orquestador de contenedores	21
2.3.1	Limpieza del entorno de virtualización	22
2.3.2	Descarga y preparación de recursos	22
2.4	Despliegue operativo y administración de contenedores	22
2.5	Instalación de herramientas cliente complementarias	23
3	Instalación y aprovisionamiento de PostgreSQL 18 y PostGIS 3.6	25
3.1	Arquitectura de directorios y parámetros del sistema anfitrión	25
3.2	Descripción y funcionalidad de herramientas del ecosistema	26
3.3	Despliegue del motor relacional PostgreSQL	27
3.4	Integración de la extensión espacial con StackBuilder	27
3.5	Configuración de variables de entorno y terminal integrada	28
3.6	Inicialización manual de la base de datos espacial	28
3.6.1	Conexión administrativa al clúster	28
3.6.2	Creación de la base de datos	29
3.6.3	Habilitación de extensiones espaciales	29
3.6.4	Auditoría de versiones y enlaces de librerías	29
3.7	Verificación analítica del módulo espacial	29
3.8	Resumen de la estructura desplegada	30
3.9	Configuración de PostgreSQL local para acceso desde Docker	30
3.9.1	Habilitación de la escucha de red en postgresql.conf	30
3.9.2	Control de acceso y autenticación en pg_hba.conf	30
3.9.3	Reinicio del servicio en el motor de base de datos	31
4	Despliegue Docker y verificación de la base de datos	33
4.1	Introducción	33
4.2	Orquestación y verificación del motor de contenedores	33
4.2.1	Verificación del servicio y catálogo de imágenes	33
4.2.2	Despliegue de servicios mediante Docker Compose	34
4.3	Gestión de variables de entorno y contextos de ejecución	34
4.3.1	Coexistencia de clústeres y topología de puertos según el contexto de ejecución	36
4.3.2	Mecanismo de autenticación libpq y gestión de credenciales	36
4.3.3	Parámetros de línea de comandos en herramientas PostgreSQL	37
4.3.4	Dualidad operativa: compilación documental vs terminal interactiva	37
4.4	Verificación de versiones y extensiones espaciales	39
4.5	Auditoría de catálogos y bases de datos	40
4.6	Inspección estructural de la base de datos de estudio	41
4.6.1	Esquemas y tablas espaciales	41
4.6.2	Validación de capas vectoriales en el clúster	43
4.6.3	Verificación de índices espaciales	43
4.7	Replicación de la base de datos en una instalación local nativa	44
4.7.1	Creación del catálogo y habilitación espacial	44
4.7.2	Restauración mediante inyección del script SQL	45
4.7.3	Verificación de integridad en la instancia local	45
5	Respaldo y recuperación de la base de datos sig_db_unal	47
5.1	Persistencia de variables y configuración de entorno	47
5.2	Estrategias de extracción de respaldos lógicos	48
5.2.1	Respaldo desde el clúster dockerizado	48

5.2.2	Respaldo desde la instancia en el sistema anfitrión	49
5.3	Procedimientos de recuperación y restauración (sistema anfitrión)	50
5.3.1	Escenario 1: restauración total desde backup completo	50
5.3.2	Escenario 2: restauración por componentes (estructura y datos)	54
5.4	Auditoría de integridad posterior a la restauración	60
5.4.1	Auditoría automatizada desatendida mediante CLI	61
5.4.2	Auditoría interactiva en cliente psql	64

II Fundamentos y diseño de bases de datos espaciales 67

6	Arquitectura de bases de datos espaciales y estándares OGC	69
6.0.1	Terminal integrada (psql)	69
6.0.2	Python (SQLAlchemy, Pandas y GeoPandas)	69
6.0.3	pgAdmin 4	71
6.1	Objetivos del capítulo	71
6.2	De sistemas de archivos a bases de datos relacionales	71
6.2.1	Componentes y funciones de un SGBD	73
6.2.2	Topología cliente-servidor y tránsito en la pila de red	77
6.2.3	Importancia y características de los SGBD	77
6.2.4	Perfiles de usuario y responsabilidades del administrador	78
6.2.5	Niveles de abstracción e independencia de datos	79
6.3	Evolución de modelos de datos y la dimensión espacial	81
6.3.1	Coexistencia paradigmática y persistencia políglota	85
6.3.2	La dimensión espacial como catalizador del modelo objeto-relacional	86
6.3.3	Consideraciones arquitectónicas: extensibilidad y motores de cómputo analítico	88
6.4	Arquitecturas para aplicaciones de sistemas de información geográfica	89
6.4.1	Arquitectura ad-hoc	90
6.4.2	Arquitectura no integrada (dual o híbrida)	90
6.4.3	Arquitectura integrada (SGBD espacial)	90
6.4.4	Distribución de responsabilidades entre el SIG y el SGBD objeto-relacional	91
6.5	PostgreSQL y la extensión PostGIS	92
6.5.1	Verificación de PostGIS en el sistema anfitrión (Windows)	92
6.5.2	Terminal (psql)	92
6.5.3	Python (SQLAlchemy)	93
6.5.4	Python (Pandas)	93
6.6	Estándar OGC Simple Features for SQL (ISO 19125)	94
6.6.1	Jerarquía de tipos geométricos OGC	94
6.6.2	Representación espacial de primitivas y colecciones	94
6.6.3	Serialización y formatos estándar: WKT y WKB	95
6.6.4	Representación espacial de primitivas y colecciones en PostGIS	98
6.6.5	Terminal (psql)	98
6.6.6	Python (SQLAlchemy)	99
6.6.7	Python (Shapely)	100
6.6.8	Python (GeoPandas)	101
6.6.9	R	103
6.6.10	Julia	107
6.7	Catálogos de metadatos espaciales	109
6.7.1	El catálogo spatial_ref_sys	109
6.7.2	Terminal (psql)	111
6.7.3	Python (SQLAlchemy)	111
6.7.4	Python (Pandas)	111
6.7.5	Terminal (psql)	112
6.7.6	Python (SQLAlchemy)	113

6.7.7	Python (Pandas)	114
6.7.8	Terminal (psql)	115
6.7.9	Python (SQLAlchemy)	115
6.7.10	Terminal (psql)	116
6.7.11	Python (SQLAlchemy)	117
6.7.12	Python (Pandas)	118
6.7.13	La vista de catálogo geometry_columns	119
6.7.14	Terminal (psql)	119
6.7.15	Python (SQLAlchemy)	120
6.7.16	Python (Pandas)	120
6.7.17	Terminal (psql)	122
6.7.18	Python (SQLAlchemy)	122
6.7.19	Python (Pandas)	123
6.7.20	Asignación de identificador espacial en las vistas del esquema	124
6.7.21	Terminal (psql)	124
6.7.22	Python (SQLAlchemy)	124
6.7.23	Terminal (psql)	125
6.7.24	Python (SQLAlchemy)	126
6.7.25	Python (Pandas)	126
6.8	Verificación práctica en la consola integrada	127
6.8.1	Terminal (psql)	127
6.8.2	Python (SQLAlchemy)	128
6.8.3	Python (Pandas)	129
6.9	Articulación con los proyectos de investigación	130
7	Modelado conceptual, diseño lógico e implementación física espacial	133
7.1	Fases en el ciclo de vida de una base de datos espacial	133
7.1.1	De la abstracción territorial al esquema operacional	133
7.1.2	Niveles de modelado: conceptual, lógico y físico	134
7.1.3	Hoja de ruta taxonómica y conceptual	135
7.2	Modelado conceptual y abstracción territorial (modelo E-R)	136
7.2.1	Modelado de entidades	137
7.2.2	Modelado de relaciones y reglas de lectura	137
7.2.3	Cardinalidad mediante rangos numéricos y equivalencia con Barker	141
7.2.4	Modelado de atributos y dominios	142
7.2.5	Resumen de estándares y convenciones de diseño para entidades y atributos	147
7.2.6	Paradigmas y alternativas de representación conceptual	149
7.3	Diseño conceptual geoespacial mediante diagramas de clases UML	151
7.3.1	Fundamentos de orientación a objetos y correspondencia con el modelo Entidad-Relación	151
7.3.2	Componentes y convenciones del perfil geoespacial UML	152
7.3.3	Modelado territorial integrado en UML	153
7.4	Transformación conceptual a lógica relacional	155
7.4.1	El modelo relacional y reglas de derivación estructural	156
7.4.2	Notación estructural y código cromático de claves	157
7.4.3	Reglas de integridad y gestión de identificadores	157
7.4.4	Operaciones de combinación relacional y espacial	159
7.4.5	Normalización relacional y formas normales avanzadas	160
7.4.6	Evolución paradigmática: almacenamiento relacional puro frente a objeto-relacional . .	165
7.5	Implementación física y migración espacial desde el repositorio legado	166
7.5.1	Arquitectura de la base de datos legado frente al nuevo repositorio local	166
7.5.2	Creación del catálogo físico y extensiones espaciales	168
7.5.3	Actualización del sistema de referencia espacial MAGNA-SIRGAS Origen Nacional (EPSG:9377)	168
7.5.4	Estructura DDL con restricciones declarativas e índices	169

7.5.5	Migración de datos y reproyección de coordenadas	170
7.5.6	Geometría derivada: disolución topológica de microrregiones	171
7.5.7	Operaciones de combinación relacional y espacial en el nuevo esquema	172
7.5.8	Asociación muchos a muchos en el modelo conceptual	174
7.6	Ejercicios de diseño conceptual, lógico e implementación física	177
III	Proyectos aplicados	181
8	Guía metodológica para el proyecto final	183
8.1	Estructura de la sesión de dos horas	183
8.2	Estructura de carpetas y paquete de entregables	183
8.3	Estándares técnicos y directivas de código en Quarto	184
8.4	Pipeline de compilación y publicación continua	185
9	Temas de proyecto final en bases de datos espaciales	187
9.1	Tema 01: Procesamiento analítico columnar y formatos nativos de nube (DuckDB espacial y GeoParquet)	188
9.2	Tema 02: Escalabilidad y procesamiento masivo distribuido (Apache Sedona y PySpark) . . .	188
9.3	Tema 03: Sistemas de mallas espaciales discretas globales (DGGS: Uber H3 y Google S2) . .	189
9.4	Tema 04: Bases de datos espaciales orientadas a grafos y análisis de redes (Neo4j Spatial vs. pgRouting)	189
9.5	Tema 05: Indexación geoespacial en motores NoSQL para telemetría y flujos en tiempo real (Elasticsearch / OpenSearch vs. MongoDB)	189
9.6	Tema 06: Programación procedural espacial en servidor y extensiones nativas (PL/pgSQL avanzado, PL/Python y Rust/PGRX)	190
9.7	Tema 07: Ecosistemas analíticos desacoplados y flujos geoespaciales con GeoLibre	190
10	Ficha de propuesta de tema de investigación	193
10.1	Información general	193
10.2	Descripción del proyecto	193
10.3	Objetivos de aprendizaje	194
10.4	Preguntas de investigación orientadoras	194
10.5	Desarrollo práctico y metodología del taller	194
10.6	Entregables esperados	195
11	Rúbrica de evaluación para el reporte en QMD y el código	197
	Apéndices	201
A	Quarto: orquestación y configuración	201
A.1	Opciones del encabezado YAML	201
A.2	Resumen de comandos Quarto	202
A.3	Gestión eficiente de renderizado en Quarto	202
A.3.1	Hoja de referencia: comandos de renderizado	202
A.3.2	Estrategias adicionales para el flujo de edición	203
A.4	Control de ejecución en Quarto: opciones de bloques	204
A.4.1	Detalle funcional y aplicación técnica de las directivas	204
A.4.2	Sintaxis de directivas según el motor de ejecución	205
A.4.3	Recomendaciones de uso según el contexto	205
A.5	Instalación de extensiones de Quarto	206
A.5.1	Proceso de instalación	206
A.5.2	Gestión de archivos del proyecto	206
A.5.3	Aplicación en el documento	207

A.6	Compilación de la guía del curso	207
A.6.1	Compilación del proyecto completo	207
A.6.2	Compilación de archivos individuales	207
A.6.3	Notas de estudio y personalización	208
B	Docker: gestión de contenedores e imágenes	209
B.1	Instalación del software	209
B.1.1	Sistema anfitrión (Windows)	209
B.1.2	Procedimiento general (agnóstico)	210
B.2	Preparación del orquestador	210
B.2.1	Reasignación del almacenamiento de imágenes virtuales	210
B.2.2	Optimización de la memoria de intercambio (swap)	211
B.3	Descarga y preparación de archivos	213
B.4	Carga de contenedores orquestados	213
B.5	Contenedores instalados	213
B.6	Puesta en marcha y detención de contenedores	214
B.6.1	Inicio y reanudación de servicios	214
B.6.2	Detención y desmontaje de servicios	214
B.7	Localización de archivos y persistencia	215
B.8	Vinculación de contenedores en la terminal integrada	215
B.9	Configuración de puertos y topología de conectividad	215
B.9.1	Coexistencia de clústeres y resolución de red según el contexto de ejecución	216
B.10	Credenciales de la base de datos	217
B.11	Lógica de arquitectura (Dockerfile y docker-compose)	217
B.12	Guía de acceso: topología de red	217
B.12.1	Conexión desde el sistema anfitrión (Exterior)	217
B.12.2	Conexión intraclúster (Interior)	217
B.13	Limpieza del entorno de contenedores	218
B.13.1	Diagnóstico de consumo de almacenamiento	218
B.13.2	Desmantelamiento de servicios y volúmenes vinculados	218
B.13.3	Purga integral de artefactos y caché de compilación	218
B.13.4	Auditoría y confirmación de estado limpio	219
B.14	Verificación de registros de instalación	219
B.15	Comandos de orquestación	219
B.16	Resumen de la infraestructura instalada	219
B.17	Validación del entorno Python	220
B.17.1	Ingreso al entorno interactivo	220
B.17.2	Guía visual del entorno	220
B.18	Validación del entorno R	220
B.18.1	Visualización interactiva con httpgd	220
B.19	Validación del entorno Julia	221
B.19.1	Ejecución de rutinas mediante evaluación cruzada	221
B.20	Verificación de conectividad multilenguaje	221
B.20.1	Python	221
B.20.2	R	221
B.20.3	Julia	222
C	Git y GitHub: Sincronización Local-Remota	227
C.1	Instalación de Git	227
C.2	Creación de cuenta en GitHub	227
C.3	Control de versiones - flujo de trabajo en PowerShell	227
C.4	Diagnóstico de archivos “invisibles” o errores de rastreo	228
C.5	Subir el contenido (carpeta local hacia GitHub)	228
C.6	Autenticación segura SSH - Windows	230

C.6.1	Requisito previo: creación de cuenta en GitHub	231
C.6.2	Requisito previo: instalación de Git	231
C.6.3	Preparación del entorno local	231
C.6.4	Gestión de llaves SSH en Windows	231
C.6.5	Registro de la identidad en GitHub	232
C.6.6	Activación del agente SSH del sistema	232
C.6.7	Verificación de la autenticación	232
C.7	Autenticación segura SSH - Linux (Docker)	233
C.7.1	Preparación del Entorno Global	233
C.7.2	Gestión de llaves SSH (Algoritmo Ed25519)	233
C.7.3	Registro de Identidad en GitHub	234
C.7.4	Activación del Agente SSH	234
C.7.5	Verificación Final	234
C.8	Descargar contenido (GitHub hacia carpeta local)	235
C.8.1	Opción A: sincronizar mediante SSH (recomendado)	235
C.8.2	Opción B: sincronizar mediante HTTPS	236
C.9	Notas de flujo de trabajo	237
D	Visual Studio Code (VSCode): Extensiones e Interfaz	239
D.1	Instalación	239
D.2	Acceso y configuración en VSCode	239
D.3	Inicialización del visor gráfico	239
D.4	Atajos	239
D.4.1	La paleta de comandos VSCode	239
D.4.2	Atajos de teclado rápidos VSCode	240
D.4.3	Atajos de teclado JupyterLab (notebooks)	240
D.5	Instalación Tinytex	241
D.6	Instalación de extensiones	241
D.6.1	Persistencia y ciclo de vida del contenedor	241
D.6.2	Clasificación y catálogo de extensiones	242
D.6.3	Notas técnicas sobre herramientas específicas	244
D.6.4	Sinergia de las herramientas de PostgreSQL en Visual Studio Code	245
E	Temas técnicos sobre Windows	247
E.1	Herramientas de administración	247
E.2	Comandos (PowerShell)	248
F	Temas Técnicos sobre Linux	249
F.1	Aprovisionamiento y actualización del cliente PostgreSQL en el contenedor	249
F.2	Instalación y verificación de Graphviz en el contenedor	249
F.2.1	Diagnóstico previo del entorno	250
F.2.2	Aprovisionamiento del paquete Graphviz	250
F.2.3	Verificación posterior de los binarios	250
F.3	Comandos comunes de terminal	250
F.4	Verificación de librerías geoespaciales	251
F.4.1	¿Cómo leer la salida de dpkg?	251
G	PostgreSQL	253
G.1	Introducción al cliente interactivo y por lotes (psql)	253
G.1.1	Anatomía de comandos: metacomandos de barra invertida frente a sentencias SQL	253
G.1.2	Parámetros de invocación y modificadores de línea de comandos	254
G.1.3	Guía de inicio rápido para la administración con psql	257
H	Lista de acrónimos y glosario de conceptos técnicos	261
H.1	Lista de acrónimos	261

H.2	Glosario de conceptos técnicos	264
I	Errata	277
	Referencias Bibliográficas	279

Listado de Figuras

4.1	Ciclo de vida y auto-inicialización declarativa del clúster espacial con volumen persistente . . .	35
4.2	Matriz de resolución de red y mapeo de puertos según el contexto de ejecución	36
4.3	Jerarquía de resolución en cascada de parámetros de conexión en la biblioteca libpq	38
4.4	Organización física de esquemas, capas vectoriales e índices espaciales en la base de datos de estudio	42
6.1	Centralización e integración de datos espaciales y alfanuméricos en un SGBD espacial.	72
6.2	Componentes de un sistema de bases de datos, según Silberschatz, Korth y Sudarshan (2002).	73
6.3	Arquitectura de componentes y ejecución de transacciones, según Elmasri y Navathe (2007).	76
6.4	Arquitectura ANSI/SPARC de tres niveles y correspondencias de independencia de datos.	80
6.5	Cronología de modelos de bases de datos con alineación temporal formal.	82
6.6	Convergencia de requerimientos de datos hacia el modelo objeto-relacional.	87
6.7	Evolución de las arquitecturas de integración entre SIG y bases de datos.	90
6.8	Distribución de capas y responsabilidades entre el software SIG y el SGBD objeto-relacional.	91
6.9	Diagrama de clases UML para la jerarquía de geometrías según ISO 19125 / OGC Simple Feature Access.	96
6.10	Instanciación espacial y morfología de tipos geométricos según el estándar OGC.	97
6.11	Instanciación espacial y morfología de tipos geométricos según el estándar OGC en Python con Shapely.	102
6.12	Instanciación espacial y morfología de tipos geométricos según el estándar OGC en Python con GeoPandas.	104
6.13	Instanciación espacial y morfología de tipos geométricos según el estándar OGC en R.	106
6.14	Instanciación espacial y morfología de tipos geométricos según el estándar OGC en Julia.	110
7.1	Fases en el desarrollo de una base de datos espacial operacional.	134
7.2	Correspondencia entre los componentes del modelo conceptual y el diseño lógico relacional.	135
7.3	Componentes estructurales y reglas de anotación formal en un diagrama Entidad-Relación (1:M).	136
7.4	Estructura conceptual de una entidad con convenciones de nomenclatura y atributos.	137
7.5	Flujo bidireccional de análisis en la asociación conceptual entre entidades espaciales.	137
7.6	Simbología gráfica normalizada para relaciones en el modelo conceptual (Barker).	138
7.7	Asociación 1:M obligatoria en ambos extremos con línea 100% continua.	139
7.8	Asociación 1:M con opcionalidad mediante tramo continuo y tramo discontinuo.	140
7.9	Concurrencia entre la notación gráfica de Barker y los rangos numéricos Look-Across.	141
7.10	Correspondencia conceptual entre la especificación formal de la entidad municipios y la materialización de sus instancias territoriales.	142
7.11	Integración conceptual de clases espaciales primarias y entidades alfanuméricas agregadas en notación Barker.	146
7.12	Representación de una asociación territorial binaria (1:N) bajo la notación clásica de Peter Chen.	150
7.13	Esquema conceptual territorial en notación de clases UML (ISO 19109) estructurado por paquetes temáticos y dominios discretos.	154
7.14	Transición del modelo conceptual (Barker) al diseño lógico relacional con código cromático de claves y signatura formal del esquema.	158

7.15 Desarticulación histórica de primitivas vectoriales bajo el modelo relacional puro sin tipos espaciales nativos.	167
7.16 Modelo conceptual UML con paquetes temáticos y asociación muchos a muchos directa en el subsistema productivo.	175
7.17 Resolución del esquema relacional M:N mediante tabla intermedia asociativa, código cromático y signatura formal.	176
H.1 Representación matricial y dimensional del filtrado espacial clásico frente a la extracción de embeddings mediante encoders.	269

Listado de Tablas

1.1	Distribución de componentes evaluativos y ponderación porcentual	13
3.1	Parámetros nominales de configuración aplicados en el despliegue del clúster local	25
3.2	Utilidades y librerías del ecosistema PostgreSQL y PostGIS	26
4.1	Especificaciones de conexión para clústeres local y dockerizado	36
5.1	Formatos estándar de extracción lógica mediante pg_dump	48
6.1	Componentes principales del SGBD según Silberschatz y sus responsabilidades funcionales. .	75
6.2	Niveles de abstracción arquitectónica y su instanciación técnica.	81
6.3	Clasificación de sistemas gestores de bases de datos según su cronología formal.	83
6.4	Objetivos operativos, compromisos arquitectónicos y vigencia de los modelos de datos en entornos geoespaciales.	85
6.5	Comparación arquitectónica entre la topología cliente-servidor y el modelo de despliegue embebido. .	89
6.6	Primitivas y colecciones geométricas del estándar OGC.	94
6.7	Matriz de articulación conceptual entre los fundamentos del capítulo y los proyectos finales de investigación.	130
7.1	Convenciones gráficas y semánticas para relaciones en el modelo conceptual	138
7.2	Mapeo formal entre la simbología de Barker y la notación de rangos numéricos	141
7.4	Convenciones de nomenclatura conceptual y prefijos mnemotécnicos en entidades territoriales. .	148
7.5	Correspondencia formal entre el modelo Entidad-Relación de Barker y el diagrama de clases UML. .	152
7.6	Convenciones semánticas y estructurales del perfil conceptual UML	154
7.7	Contraste estructural entre combinaciones relacionales y espaciales en motores objeto-relacionales. .	159
7.8	Relación operaciones	163
9.1	Matriz de asignación de temas para grupos de posgrado	187
10.1	Información general de la propuesta de investigación	193
11.1	Matriz de evaluación del proyecto de investigación y código fuente	197
A.1	Comandos esenciales para la gestión de flujos en Quarto	203
A.2	Matriz de opciones de visibilidad, ejecución y captura de errores para bloques de código . . .	204
A.3	Sintaxis de directivas según el motor de ejecución	205
A.4	Estrategias recomendadas de etiquetado por etapa analítica	205
B.1	Mapeo de puertos para los servicios de la infraestructura dockerizada	216
B.2	Matriz de enrutamiento y puertos según el vector de ejecución	216
B.3	Matriz de comandos de orquestación de Docker	219
B.4	Inventario de arquitectura y componentes del laboratorio	220
C.1	Comandos esenciales para la gestión de Git en terminal	227

D.3	Extensiones de Visual Studio Code para el entorno de bases de datos espaciales	242
D.4	Matriz funcional de herramientas para interacción con PostgreSQL	245
G.1	Metacomandos esenciales de inspección y administración en psql	254
G.2	Parámetros y modificadores de línea de comandos para utilidades cliente de PostgreSQL y PostGIS	255
H.1	Catálogo de acrónimos y siglas técnicas normalizadas.	261
H.2	Comparación técnica y matemática entre PCA y modelos de embeddings.	266
H.3	Comparación técnica entre paradigmas arquitectónicos OLTP y OLAP.	272

Bienvenida

El presente documento constituye la guía técnica, metodológica y computacional del curso de posgrado **Bases de datos espaciales** de la Maestría en Geomática de la Universidad Nacional de Colombia.

Este recurso ha sido estructurado como un marco riguroso para el modelado conceptual, la administración, el diseño formal de esquemas, la optimización de consultas espaciales (*Spatial SQL*) y la experimentación con sistemas de gestión de bases de datos espaciales (SGBDE) y arquitecturas analíticas contemporáneas.

Objetivos y alcance formativo

El propósito central consiste en desarrollar competencias avanzadas para la ingeniería de datos geoespaciales, la estructuración de geodatabases corporativas y el análisis masivo de datos vectoriales y ráster bajo estándares de reproducibilidad y Ciencia Abierta:

- **Fundamentos y diseño conceptual:** Formulación de modelos entidad-relación espaciales normalizados (1FN, 2FN, 3FN), diseño visual asistido con pgModeler y cumplimiento de las especificaciones OGC *Simple Features for SQL* e ISO 19107.
- **Motores relacionales espaciales con PostgreSQL y PostGIS:** Administración de instancias, tipado cartesiano (*geometry*) y geodésico (*geography*), indexación multidimensional (GiST, SP-GiST, BRIN), análisis topológico con la matriz DE-9IM, topología persistente y estrategias de respaldo estructurado con tablas de contenido (TOC).
- **Geodatabases corporativas y flujos ETL:** Arquitecturas basadas en archivos y multiusuario, ingeniería guiada por modelos (ShapeChange), vistas de diseño integradas (*data design views*), reglas de atributos, almacenamiento ráster y construcción de pipelines reproducibles de extracción, transformación y carga.
- **Ecosistema de conexión políglota:** Interacción directa mediante la consola de comandos (*psql*), herramientas de administración gráfica (*pgAdmin 4*) y controladores analíticos en Python (*psycopg*, *GeoPandas*), R (*DBI*, *sf*) y Julia (*LibPQ.jl*, *GeoDataFrames.jl*).
- **Paradigmas analíticos emergentes:** Evaluación experimental de motores analíticos columnares (DuckDB espacial, GeoParquet), procesamiento distribuido (Apache Sedona, PySpark), mallas espaciales discretas globales (DGGS H3/S2), bases de datos de grafos (Neo4j Spatial, pgRouting), motores NoSQL y extensiones compiladas nativas en Rust.

Material de la clase

El material de la clase se distribuye como un proyecto Quarto alojado en el repositorio:

- Protocolo SSH: `git@github.com:alexysshr/bde2026.git`
- Interfaz web HTTPS: <https://github.com/alexysshr/bde2026>

Para acceder y sincronizar este material se requiere:

- Inscripción formal en la asignatura *Bases de datos espaciales* (UNAL 2026).
- Disponer de una cuenta registrada en GitHub.
- Contar con la herramienta de control de versiones `git` instalada en el sistema anfitrión.

La documentación procedimental sobre Git y GitHub se detalla en Apéndice C, donde se presentan las guías para clonar proyectos y gestionar credenciales criptográficas de acceso.

Cómo usar este libro

A lo largo de los capítulos, las bases conceptuales, las guías metodológicas y los talleres prácticos se encuentran organizados de forma secuencial. En los módulos que integran flujos analíticos políglotas, las implementaciones computacionales se estructuran en paneles con pestañas (*tabsets*) para contrastar la sintaxis y el rendimiento entre entornos:

i Navegación y reproducibilidad

Los bloques de código contienen directivas YAML de Quarto (`#| label: ...`, `#| eval: ...`) con prefijos taxonómicos estandarizados (`setup-`, `docker-`, `ddl-`, `dml-`, `query-`, `audit-`, `export-`) para facilitar la trazabilidad y ejecución en la terminal integrada de Visual Studio Code o Positron.

i Enfoque instrumental y prerequisites de programación

Si bien este material integra demostraciones dentro de un ecosistema analítico políglota, el eje procedimental y evaluativo del curso se fundamenta en la interacción directa mediante la consola de comandos con el cliente `psql`, complementada periódicamente a través del entorno gráfico `pgAdmin 4`.

Las implementaciones presentadas en Python, R y Julia se incorporan exclusivamente como marcos de referencia comparativa para consulta autónoma. En consecuencia, no se exigen conocimientos previos de programación ni dominio de sintaxis en dichos lenguajes para el desarrollo y culminación de los talleres prácticos.

Requisitos previos del entorno

La compilación de los talleres y guías disponibles en formato `.qmd` exige la configuración previa de las siguientes utilidades, detalladas en el módulo de instalación y en los anexos técnicos:

- Git (sistema anfitrión)
- Docker Desktop (sistema anfitrión)

- Contenedores Docker preconfigurados para el curso (orquestración de servicios y motores de bases de datos)
- Entorno de desarrollo sobre la terminal integrada de Visual Studio Code (sistema anfitrión)
- Motor de bases de datos PostgreSQL con extensión espacial PostGIS (sistema anfitrión)
- Clientes de administración relacional y espacial (sistema anfitrión y contenedor)
- Software de Sistemas de Información Geográfica de escritorio: QGIS y ArcGIS Pro (sistema anfitrión)

Licencia y citación del material

Licencia de uso

Este material pedagógico y científico se distribuye bajo una licencia **Creative Commons Atribución-NoComercial 4.0 Internacional (CC BY-NC 4.0)**.

Alcance y condiciones de uso

Bajo los principios de la Ciencia Abierta (*Open Science*) y las directrices FAIR (*Findable, Accessible, Interoperable, Reusable*), se autoriza:

1. **Compartir:** Copiar y redistribuir el material en cualquier medio o formato.
2. **Adaptar:** Remezclar, transformar y construir sobre el contenido para propósitos estrictamente académicos y de investigación.

Condiciones obligatorias:

- **Atribución:** Debe otorgarse el crédito académico correspondiente, proveer un enlace a la licencia e indicar explícitamente si se efectuaron modificaciones.
- **Uso no comercial:** No se autoriza la explotación comercial ni el uso de este material con fines de lucro directo o indirecto.

Cómo citar este material

Para la utilización de los esquemas DDL, consultas analíticas, guías de laboratorio o datos en proyectos de investigación o publicaciones técnicas, se deben incluir las siguientes referencias:

Referencia en formato APA (7.^a edición)

Rodríguez-Avellaneda, A. H. (2026). *Bases de datos espaciales* [Material de curso]. Maestría en Geomática, Universidad Nacional de Colombia. <https://github.com/alexysshr/bde2026>

Referencia en formato BibTeX (para LaTeX y gestores bibliográficos)

```
@misc{rodriguez2026basesdatos,
  author    = {Rodríguez-Avellaneda, Alexys H.},
  title     = {Bases de datos espaciales},
  year      = {2026},
  howpublished = {Material de curso, Maestría en Geomática},
  publisher = {Universidad Nacional de Colombia},
  url       = {[https://github.com/alexysshr/bde2026] (https://github.com/alexysshr/bde2026)},
  note      = {Licencia CC BY-NC 4.0}
}
```

Contacto y colaboración

Este documento constituye una obra técnica en construcción y actualización permanente. Para reportar inconsistencias en los scripts DDL/DML, erratas conceptuales o proponer extensiones a los módulos de laboratorio, se contemplan los siguientes mecanismos:

1. **Apertura de un Issue:** Notificar fallas de compilación, inconsistencias en consultas SQL o correcciones de redacción.
2. **Envío de un Pull Request:** Someter correcciones directas en los archivos `.qmd` o incorporar nuevos talleres y flujos reproducibles.



Reconocimiento a herramientas de IA

La estructuración de plantillas de configuración, la resolución de dependencias en contenedores Docker y la optimización de flujos de integración continua han sido asistidas mediante herramientas de Inteligencia Artificial, bajo la validación, diseño conceptual y supervisión técnica del autor principal.

Prefacio

El presente documento constituye el material de referencia técnica, metodológica y computacional del curso de posgrado **Bases de datos espaciales**. El propósito central de esta obra es proporcionar un marco riguroso para el modelado conceptual, la administración de infraestructura, la optimización de consultas (*Spatial SQL*) y la experimentación con sistemas de gestión de bases de datos espaciales (SGBDE) y arquitecturas analíticas contemporáneas.

En el contexto contemporáneo de la geomática y la ciencia de datos espaciales, el crecimiento sostenido en volumen y dimensionalidad de la información geográfica exige superar el almacenamiento en formatos planos desarticulados y la dependencia exclusiva de interfaces de escritorio. El dominio de los motores relacionales espaciales, el diseño normalizado de esquemas y el procesamiento analítico a gran escala constituyen competencias críticas para garantizar la integridad topológica, la concurrencia y la reproducibilidad técnica en flujos de investigación y producción técnica avanzada.

Enfoque metodológico y computacional

El desarrollo temático de la asignatura y la estructura del material se fundamentan en los principios de la Ciencia Abierta (*Open Science*), las directrices FAIR (*Findable, Accessible, Interoperable, Reusable*) y la reproducibilidad computacional integral:

- **Modelado conceptual y estándares internacionales:** Aplicación de las especificaciones del *Open Geospatial Consortium* (OGC, *Simple Features for SQL*), la norma ISO 19107, normalización relacional formal (1FN, 2FN, 3FN) y diseño visual de esquemas asistido con pgModeler.
- **Motores relacionales y Spatial SQL:** Administración avanzada de clústeres PostgreSQL y PostGIS, tipado geométrico cartesiano y geodésico, análisis topológico mediante la matriz DE-9IM, indexación multidimensional (GiST, SP-GiST, BRIN), diagnóstico de planes de ejecución y topología vectorial persistente.
- **Geodatabases corporativas y flujos ETL:** Arquitecturas basadas en archivos y motores multiusuario, ingeniería guiada por modelos (ShapeChange), vistas de diseño integradas (*data design views*), reglas de atributos, almacenamiento de coberturas ráster y construcción de flujos de extracción, transformación y carga reproducibles.
- **Ecosistema de conexión políglota:** Interacción directa mediante la consola nativa (`psql`) y entornos de administración gráfica (pgAdmin 4), complementada con librerías analíticas en Python (`psycopg`, `GeoPandas`), R (`DBI`, `sf`) y Julia (`LibPQ.jl`, `GeoDataFrames.jl`).
- **Seminario de paradigmas emergentes:** Evaluación experimental de tecnologías analíticas

contemporáneas, tales como procesamiento masivo distribuido (Apache Sedona, PySpark), motores columnares y formatos nativos de nube (DuckDB, GeoParquet), mallas espaciales discretas globales (DGGs H3/S2), grafos espaciales (Neo4j, pgRouting), motores NoSQL y extensiones compiladas nativas en Rust.

Pilares operativos de la práctica computacional

El aprendizaje aplicado se consolida mediante tres fases secuenciales de trabajo técnico:

1. **Despliegue y administración del entorno:** Configuración e inicialización de motores de bases de datos espaciales desde la línea de comandos y orquestación en contenedores, garantizando entornos reproducibles e independientes del sistema anfitrión.
2. **Implementación de esquemas y explotación analítica:** Ejecución de sentencias DDL para la definición de estructuras, manipulación DML para ingestión masiva y resolución de problemas geográficos complejos mediante Spatial SQL y análisis topológico.
3. **Integración con clientes y flujos analíticos:** Conexión de repositorios espaciales con software SIG de escritorio (QGIS, ArcGIS Pro) y librerías espaciales en ecosistemas de computación científica.

Reproducibilidad y entorno de trabajo

Los talleres aplicados, ejercicios guiados y proyectos experimentales se encuentran estructurados para ejecutarse desde la terminal integrada de Visual Studio Code o Positron. La totalidad de los servicios de bases de datos relacionales, extensiones espaciales y entornos analíticos políglotas se suministra orquestada mediante contenedores Docker.

Esta aproximación asegura un ambiente de desarrollo homogéneo, desacoplado de las particularidades del sistema operativo del sistema anfitrión (*host*), lo cual mitiga discrepancias en versiones de dependencias dinámicas (GEOS, PROJ, GDAL). Para los módulos que integran geodatabases corporativas y herramientas propietarias complementarias como ArcGIS Pro, los procedimientos se desarrollan sobre el sistema anfitrión atendiendo a los requerimientos de instalación local correspondientes.

Organización de la obra

El texto se encuentra organizado de manera modular para articular el rigor conceptual con la instrumentación práctica:

- **Syllabus y lineamientos del curso:** Expone las competencias formativas, la programación temática estructurada a lo largo de las 16 semanas académicas, las políticas operativas y los criterios de evaluación.
- **Capítulos conceptuales y talleres aplicados:** Desarrolla la fundamentación teórica, las directivas procedimentales y los bloques de código auditados para las prácticas de laboratorio.
- **Proyectos de investigación experimental:** Profundiza en el análisis comparativo, benchmarking y evaluación de desempeño computacional de los paradigmas geoespaciales emergentes, incluyendo su marco metodológico, código reproducible y rúbrica técnica de evaluación.

Capítulo 1

Syllabus: bases de datos espaciales

1.1 Información general

- **Nivel de formación:** Posgrado (Maestría y Doctorado)
- **Docente:** Alexys Rodríguez Avellaneda, Ph.D.
- **Número de sesiones:** 16 semanas (sesiones presenciales / asistidas de 4 horas)
- **Marco científico y metodológico:** Ciencia Abierta (*Open Science*), principios FAIR (*Findable, Accessible, Interoperable, Reusable*) y reproducibilidad computacional integral
- **Entorno computacional y artefactos reproducibles:** Terminal integrada de Visual Studio Code / Positron, orquestación en contenedores Docker, control de versiones en GitHub y publicación automatizada con Quarto
- **Ecosistema de conexión políglota para bases de datos:** Interfaz de línea de comandos (`psql`), administración gráfica (`pgAdmin 4`), y controladores analíticos en Python (`psycopg`, `GeoPandas`, `SQLAlchemy`), R (`DBI`, `sf`) y Julia (`LibPQ.jl`, `GeoDataFrames.jl`)

1.2 Justificación académica

El incremento en la frecuencia y volumen de captura de información geográfica exige que los profesionales de posgrado dominen la arquitectura interna, el modelado conceptual y la optimización de sistemas de gestión de bases de datos espaciales (SGBDE).

El programa académico se estructura alrededor de cuatro ejes fundamentales:

1. **Fundamentos de diseño, modelado y arquitecturas en general:** Principios de modelado conceptual espacial, normalización relacional, diseño gráfico de esquemas y estándares de interoperabilidad del *Open Geospatial Consortium* (OGC) e ISO.
2. **PostgreSQL y PostGIS:** Administración del motor relacional, ingeniería directa de esquemas con `pgModeler`, interacción políglota (`psql`, `pgAdmin`, Python, R, Julia), utilidades CLI (`shp2psql`, `psql2shp`), respaldos estructurados con TOC (`pg_dump`, `pg_restore`), indexación espacial avanzada, optimización en *Spatial SQL* y topología persistente.
3. **Geodatabases:** Arquitecturas corporativas y basadas en archivos (*Enterprise* y *File Geodatabases*),

ingeniería guiada por modelos (ShapeChange), paradigmas modernos de modelado visual integrado, reglas de atributos, topología y documentación automatizada de esquemas.

4. **Temas emergentes (proyectos finales):** Paradigmas de computación distribuida, motores OLAP columnares, mallas espaciales discretas globales (DGGS), bases de datos orientadas a grafos y motores NoSQL, liderados y ejecutados experimentalmente por los estudiantes.

1.3 Propósito formativo y competencias

1.3.1 Propósito general

Desarrollar la capacidad técnica y conceptual para diseñar, implementar, administrar y explotar analíticamente sistemas de bases de datos espaciales relacionales y arquitecturas analíticas contemporáneas en flujos reproducibles.

1.3.2 Competencias del curso

Al culminar el programa académico, se consolidan las siguientes competencias:

- **Fundamentación relacional y lenguaje SQL:** Comprender la arquitectura interna de los sistemas gestores de bases de datos relacionales, el álgebra relacional subyacente y la formulación de consultas analíticas estándar mediante SQL.
- **Modelado e implementación espacial:** Diseñar esquemas conceptuales y lógicos que capturen relaciones espaciales complejas, materializando los modelos mediante extensiones espaciales estandarizadas (PostGIS, OGC Simple Features) y herramientas de ingeniería de esquemas.
- **Ingeniería de datos e integración:** Articular repositorios espaciales centralizados con clientes de visualización, sistemas de información geográfica de escritorio y entornos analíticos de programación.

1.4 Objetivos de aprendizaje

Al finalizar el periodo académico, el estudiante estará en capacidad de:

- Diseñar esquemas conceptuales y lógicos de bases de datos espaciales aplicando reglas de normalización, restricciones de integridad referencial y estándares internacionales (OGC Simple Features e ISO 19107).
- Utilizar herramientas de diseño gráfico asistido (pgModeler) para formular modelos entidad-relación espaciales y generar scripts DDL reproducibles sobre PostgreSQL/PostGIS.
- Conectar, consultar y manipular bases de datos espaciales mediante clientes nativos (psql, pgAdmin) y flujos programáticos políglotas en Python, R y Julia.
- Modelar, estructurar y documentar geodatabases corporativas combinando arquitecturas guiadas por modelos (ShapeChange) y el paradigma moderno de diseño visual integrado, esquemas XML y reportes automatizados de diccionario de datos.
- Administrar y optimizar instancias relacionales espaciales mediante utilidades CLI, indexación avanzada (GiST, SP-GiST, BRIN), análisis de planes de ejecución y consultas analíticas complejas en *Spatial SQL*.
- Formular, liderar y evaluar experimentalmente proyectos de investigación aplicada sobre tecnologías geoespaciales emergentes, dirigiendo talleres prácticos en vivo (*live coding*) y documentando los resultados en reportes reproducibles bajo Quarto.

1.5 Estructura curricular articulada en cuatro ejes

```

Eje 1: Fundamentos y diseño (Sem 1-3)
  Sem 1: Arquitectura y estándares OGC
  Sem 2: Modelado conceptual y normalización
  Sem 3: Álgebra relacional, DDL y modelado visual con pgModeler (Taller 1)
Eje 2: PostgreSQL + PostGIS (Sem 4-7)
  Sem 4: Implementación física, CLI, pgAdmin y conexión políglota (psql, Python, R, Julia)
  Sem 5: Spatial SQL, DE-9IM y vistas analíticas (Taller 2)
  Sem 6: Indexación GiST, optimización y respaldos avanzados TOC
  Sem 7: Topología vectorial persistente (Taller 3)
Evaluación de medio término (Sem 8)
  Examen teórico individual + Asesoría de proyectos
Eje 3: Geodatabases, diseño y gestión de datos (Sem 9-10)
  Sem 9: Enterprise Geodatabases, ShapeChange y diseño moderno
  Sem 10: Datos ráster, lógica procedural y flujos ETL (Taller 4)
Eje 4: Temas emergentes y proyectos finales (Sem 11-16)
  Sem 11: DuckDB espacial y GeoParquet (Tema 01)
  Sem 12: Apache Sedona y PySpark (Tema 02)
  Sem 13: Mallas discretas H3 y S2 (Tema 03)
  Sem 14: Neo4j Spatial vs. pgRouting (Tema 04)
  Sem 15: Elasticsearch vs. MongoDB espacial (Tema 05)
  Sem 16: Procedimientos nativos en Rust (Tema 06) + Cierre

```

1.6 Programación semanal y contenidos detallados (16 semanas)

1.6.1 Eje 1: fundamentos de diseño, modelado y arquitecturas de datos (semanas 1 a 3)

Semana 1: Arquitectura de sistemas de bases de datos espaciales y estándares OGC

- Comparación entre el modelo de almacenamiento en archivos planos y los sistemas de gestión de bases de datos relacionales espaciales.
- Estándar OGC *Simple Features for SQL* (SFS) y norma ISO 19107 (*Spatial Schema*).
- Jerarquía de tipos de datos geométricos vectoriales: `Point`, `LineString`, `Polygon`, colecciones homogéneas y `GeometryCollection`.
- Configuración del entorno de desarrollo sobre la terminal integrada de Visual Studio Code y orquestación de servicios en contenedores locales.

Semana 2: Modelado conceptual, lógico y normalización de datos espaciales

- Diagramas Entidad-Relación extendidos para información espacial (notaciones conceptuales geográficas).
- Aplicación de formas normales (1FN, 2FN y 3FN) y restricciones de integridad referencial (`ON UPDATE`, `ON DELETE CASCADE` / `SET NULL`).
- Sistemas de referencia de coordenadas espaciales (catálogo EPSG, tabla `spatial_ref_sys`, proyecciones cartográficas y transformaciones de datum).
- Estándares de codificación e intercambio: WKT (*Well-Known Text*), WKB (*Well-Known Binary*) y GeoJSON.

Semana 3: Álgebra relacional espacial, definición de esquemas DDL y modelado visual con pgModeler

- Operaciones del álgebra relacional proyectadas al contexto espacial: selección, proyección, unión, diferencia y producto cartesiano.
- Diseño visual de esquemas espaciales: fundamentos del modelado gráfico relacional.

- Modelado asistido con **pgModeler**: creación gráfica de tablas, tipos espaciales PostGIS, llaves foráneas, restricciones CHECK e índices.
- Generación automatizada de scripts DDL (*Forward Engineering*) y sincronización de esquemas.
- **Evaluación práctica: Taller 1: Modelado conceptual, normalización y diseño visual de esquemas DDL con pgModeler.**

1.6.2 Eje 2: motores relacionales espaciales con PostgreSQL y PostGIS (semanas 4 a 7)

Semana 4: Implementación física, administración y ecosistema cliente políglota

- Arquitectura cliente-servidor de PostgreSQL y activación modular de PostGIS.
- Despliegue del esquema físico a partir de modelos generados en pgModeler e ingeniería inversa (*Reverse Engineering*).
- Catálogos del sistema y tablas de metadatos: `geometry_columns` y `spatial_ref_sys`.
- Tipo de dato `geometry` (cartesiano/plano) frente al tipo `geography` (esferoidal/geodésico).
- Ingestión y extracción por consola de comandos: `shp2pgsql`, `pgsql2shp` y `ogr2ogr`.
- Interfaces de administración y conexión políglota:
- *Consola nativa*: Cliente interactivo `psql` (variables de sesión `search_path`, `client_encoding`, ejecución de scripts con `\i`).
- *Entorno gráfico*: Administración visual de esquemas y servidores con `pgAdmin 4`.
- *Entornos de desarrollo analítico*: Conexión y lectura de datos espaciales desde Python (`psycopg`, `GeoPandas`), R (`DBI`, `sf`) y Julia (`LibPQ.jl`, `GeoDataFrames.jl`).

Semana 5: Consultas analíticas en Spatial SQL, modelo topológico DE-9IM y vistas espaciales

- Matriz de intersección de 9 dimensiones extendida (DE-9IM) y formulación de predicados espaciales.
- Predicados topológicos estándar: `ST_Intersects`, `ST_Contains`, `ST_Within`, `ST_Disjoint`, `ST_Touches`, `ST_Crosses`, `ST_Overlaps`.
- Funciones de análisis, construcción y agregación espacial: `ST_Buffer`, `ST_Intersection`, `ST_Difference`, `ST_Union`, `ST_Centroid`.
- Creación y registro de vistas espaciales agregadas (`CREATE VIEW`) con identificadores únicos para interoperabilidad con software SIG.
- Extracción de métricas espaciales hacia DataFrames en Python, R y Julia.
- **Evaluación práctica: Taller 2: Consultas espaciales avanzadas, uniones espaciales (Spatial Joins), vistas y agregación con PostGIS.**

Semana 6: Indexación espacial, optimización de consultas y administración avanzada de respaldos

- Principios teóricos del árbol R (*R-Tree*) e implementación de índices GiST (*Generalized Search Tree*) y SP-GiST en PostGIS.
- Índices de rango de bloques (BRIN) aplicados a series de telemetría masiva ordenada.
- Operadores espaciales optimizados por índice: delimitador de caja envolvente (`&&`) y búsqueda de vecinos más cercanos (*k-NN* vía `<->`).
- Perfilamiento y análisis de planes de ejecución mediante `EXPLAIN` (`ANALYZE`, `BUFFERS`) para diagnóstico de cuellos de botella.

- Estrategias de respaldo y migración: generación de dumps personalizados (`pg_dump -F c`), extracción de esquemas y restauración ordenada mediante tablas de contenido (`pg_restore -l`, `pg_restore -L`).

Semana 7: Topología vectorial persistente y validación de reglas

- Fundamento del modelo de nodos, aristas (*edges*) y caras (*faces*) con la extensión `postgis_topology`.
- Garantía de cobertura continua, eliminación de solapes (*overlaps*) y control de polígonos espurios (*slivers*).
- Funciones de validación y reparación automática de geometrías (`ST_IsValid`, `ST_IsValidReason`, `ST_MakeValid`).
- **Evaluación práctica: Taller 3: Optimización con índices espaciales, respaldos con TOC, planes de ejecución y topología.**

1.6.3 Evaluación de medio término y balance pedagógico (semana 8)

Semana 8: Examen teórico individual y asesoría de proyectos de investigación

- **Evaluación teórica:** Examen individual escrito sobre diseño conceptual, álgebra relacional, normalización, cálculo espacial, optimización en *Spatial SQL* y administración de esquemas.
- Sesión de acompañamiento y validación del diseño experimental para los seis equipos del proyecto final.

1.6.4 Eje 3: geodatabases corporativas, diseño y formatos avanzados (semanas 9 a 10)

Semana 9: Arquitecturas de geodatabases, diseño guiado por modelos y paradigma moderno

- Conceptos generales de geodatabases: almacenamiento relacional de objetos espaciales, clases de entidad (*feature classes*), conjuntos de datos (*feature datasets*) y catálogos.
- Comparación entre geodatabases basadas en archivos (*File Geodatabase*, *GeoPackage*) y geodatabases corporativas multiusuario (*Enterprise Geodatabase* desplegada sobre PostgreSQL).
- Enfoque guiado por modelos (*Model-Driven Architecture* - MDA) y estándares abiertos: transformación automatizada de modelos conceptuales UML a esquemas de geodatabase mediante **ShapeChange**.
- Paradigma moderno de diseño y gestión de geodatabases:
- *Modelado visual integrado (data design views)*: Configuración directa e interactiva de campos, subtipos, dominios (rango y valores codificados), valores contingentes y reglas de atributos (*attribute rules* para cálculo, restricción y validación).
- *Extracción y clonación del esquema*: Exportación de arquitecturas completas mediante documentos de espacio de trabajo XML (*Export XML Workspace Document* de tipo `SCHEMA_ONLY`) para versionamiento en repositorios Git.
- *Documentación automática del esquema*: Generación del diccionario de datos estructurado en formato Markdown/HTML mediante la herramienta nativa *Generate Geodatabase Schema Report*.
- Reglas de topología vectorial en geodatabases: tolerancias de agrupamiento, validación y gestión de excepciones.

Semana 10: Almacenamiento ráster, lógica procedural en servidor y flujos ETL

- Almacenamiento y teselado de datos ráster con la extensión `postgis_raster` y catálogos de mosaico en geodatabases.

- Álgebra de mapas en base de datos (`ST_MapAlgebra`) y extracción de estadísticas zonales (`ST_SummaryStats`).
- Funciones almacenadas en PL/pgSQL y disparadores espaciales (*triggers*) para automatización de integridad en servidor.
- Construcción de pipelines de extracción, transformación y carga (ETL) utilizando librerías geoespaciales (GDAL/OGR en Python).
- **Evaluación práctica: Taller 4: Diseño de geodatabases, reglas de integridad, topología, documentación y flujos ETL.**

1.6.5 Eje 4: paradigmas emergentes y seminario de proyectos finales (semanas 11 a 16)

Para una cohorte de 10 a 15 estudiantes, se conforman 6 equipos de trabajo (2 a 3 estudiantes por grupo). Cada semana se destina íntegramente a una temática de investigación.

El equipo asignado asume la dirección del aula durante la sesión de 2 horas (120 minutos), estructurada estrictamente en tres bloques secuenciales:

- **Bloque 1 (40 minutos):** Presentación del marco conceptual, arquitectura tecnológica y diseño experimental.
- **Bloque 2 (65 minutos):** Taller práctico guiado en tiempo real (*live coding*) con la cohorte conectada al entorno.
- **Bloque 3 (15 minutos):** Debate técnico, análisis de limitaciones y retroalimentación docente.

Semana 11: Proyecto final 01 — Procesamiento analítico columnar y formatos nativos de nube

- *Tecnología:* DuckDB espacial y GeoParquet.
- *Enfoque del taller:* Consultas analíticas vectoriales directas sobre almacenamiento en la nube, evaluación de *predicate pushdown* y límites de huella de memoria.
- *Directores de sesión:* Equipo 01.

Semana 12: Proyecto final 02 — Escalabilidad y procesamiento masivo distribuido

- *Tecnología:* Apache Sedona y PySpark.
- *Enfoque del taller:* *Spatial Join* masivo sobre clúster distribuido, particionamiento espacial (*Quad-Tree / KDB-Tree*) y análisis del grafo acíclico dirigido (*DAG*).
- *Directores de sesión:* Equipo 02.

Semana 13: Proyecto final 03 — Sistemas de mallas espaciales discretas globales (DGGS)

- *Tecnología:* Mallas jerárquicas Uber H3 y Google S2.
- *Enfoque del taller:* Agregación espacial de series temporales masivas, reclasificación multi-resolución y evaluación de distorsión métrica frente a índices continuos.
- *Directores de sesión:* Equipo 03.

Semana 14: Proyecto final 04 — Bases de datos espaciales orientadas a grafos y análisis de redes

- *Tecnología:* Neo4j Spatial frente a pgRouting.
- *Enfoque del taller:* Construcción de topologías de transporte, ruteo dinámico con Dijkstra / A* y cálculo de isócronas bajo concurrencia.

- *Directores de sesión:* Equipo 04.

Semana 15: Proyecto final 05 — Indexación geoespacial en motores NoSQL para telemetría y flujos en tiempo real

- *Tecnología:* Elasticsearch / OpenSearch frente a MongoDB espacial.
- *Enfoque del taller:* Ingestión de eventos de alta frecuencia, consultas concurrentes de *geo-fencing* e indexación con árboles BKD y mallas 2dsphere.
- *Directores de sesión:* Equipo 05.

Semana 16: Proyecto final 06 — Programación procedural en servidor y extensiones nativas

- *Tecnología:* PL/pgSQL avanzado, PL/Python y extensiones en lenguaje compilado (Rust vía PGRX).
- *Enfoque del taller:* Evaluación de la sobrecarga de serialización WKB entre cliente y servidor, y *benchmarking* de algoritmos geométricos en extensiones nativas.
- *Directores de sesión:* Equipo 06.
- *Cierre académico:* Integración automatizada del libro del curso compilado con Quarto y balance general de la asignatura.

1.7 Sistema de evaluación y ponderación porcentual

Tabla 1.1: Distribución de componentes evaluativos y ponderación porcentual

Componente	Instrumento y alcance	Semana de entrega / aplicación	Porcentaje
Talleres prácticos guiados	4 talleres aplicados (7.5% cada uno)	Semanas 3, 5, 7 y 10	30%
Examen teórico de medio término	Prueba escrita individual sobre Ejes 1 y 2	Semana 8	20%
Reporte científico del proyecto final	Capítulo reproducible en Quarto (.qmd)	Semana asignada (S11 a S16)	20%
Implementación técnica y reproducibilidad	Repositorio GitHub con entorno Docker / Colab y datos	Semana asignada (S11 a S16)	15%
Dirección pedagógica y taller guiado	Liderazgo de la sesión de 2 horas (40m / 65m / 15m)	Semana asignada (S11 a S16)	15%
Total			100%

1.8 Estructura de entregables del proyecto final en el repositorio Quarto

Cada equipo de investigación debe crear su carpeta correspondiente en el directorio `proyectos_finales/` respetando la estructura modular estandarizada:

```

proyectos_finales/
  tema_XX_[nombre_tema]/
    XX_proyecto_[nombre_tema].qmd
    XX_rubrica_[nombre_tema].qmd
    XX_slides_[nombre_tema].qmd
    XX_taller_[nombre_tema].qmd
    references.bib

```

(Donde *XX* corresponde al identificador de dos dígitos del tema: 01 a 06).

El paquete de entregables se compone de:

- `XX_proyecto_[nombre_tema].qmd`: Reporte científico completo que se compila como capítulo en el libro del curso.
- `XX_rubrica_[nombre_tema].qmd`: Matriz de evaluación cuantitativa y cualitativa formulada para la práctica.
- `XX_slides_[nombre_tema].qmd`: Presentación interactiva en Quarto RevealJS para el Bloque 1.
- `XX_taller_[nombre_tema].qmd`: Guía de laboratorio ejecutable paso a paso para el Bloque 2.
- `references.bib`: Archivo BibTeX con la literatura técnica y científica citada.

1.9 Políticas del curso, entregas y régimen disciplinario

1.9.1 Gestión de entregas y control de versiones vía GitHub

Todas las asignaciones prácticas, talleres y reportes del curso se gestionan de manera obligatoria a través de la organización de GitHub del curso.

- **Enlaces de asignación:** Cada taller y proyecto cuenta con un enlace individual o grupal suministrado por el docente a través del cual se genera automáticamente el repositorio privado de entrega.
- **Trazabilidad de commits:** Los entregables deben reflejar un historial de versiones progresivo y verificable. No se aceptan entregas estructuradas mediante un único *commit* final masivo.
- **Estado de compilación:** El código de las guías y reportes en Quarto (`.qmd`) debe compilar sin errores en las pruebas automatizadas del repositorio.

1.9.2 Régimen de entregas extemporáneas y penalizaciones

Las fechas y horas límites fijadas en cada asignación son de estricto cumplimiento.

- **Penalización por retraso:** Cada semana completa o fracción de semana de retraso en la entrega de cualquier taller o reporte implica una deducción automática de **5 unidades (puntos)** sobre la calificación final obtenida en dicha actividad.
- **Límite de recepción extemporánea:** Ningún trabajo se evalúa si la entrega supera dos semanas posteriores a la fecha original de corte.

1.9.3 Asistencia, puntualidad y justificación de inasistencias

Dada la naturaleza teórico-práctica y el nivel de posgrado de la asignatura, la presencia activa es fundamental para el desarrollo de las competencias.

- **Control de asistencia:** Se realiza registro formal de asistencia al inicio de cada una de las sesiones de clase.
- **Asistencia obligatoria:** Se exige un porcentaje mínimo de asistencia del 80% para la aprobación del curso.
- **Justificación de inasistencias:** Toda ausencia debe ser soportada mediante un documento formal válido (incapacidad médica o justificación laboral/académica oficial) remitido al docente dentro de los cinco días hábiles posteriores a la inasistencia.

1.9.4 Plan de compensación y recuperación de sesiones por días festivos

Dado que el curso se programa los días lunes, el calendario académico coincide con la pérdida de al menos cuatro (4) jornadas de clase debido a días festivos.

- **Mecanismo de concertación:** Durante la primera sesión presencial, el docente y la cohorte definirán y concertarán el cronograma de recuperación horaria.
- **Modalidad de compensación:** Las sesiones equivalentes se desarrollarán mediante encuentros sincrónicos remotos y talleres prácticos virtuales estructurados en el repositorio del curso, asegurando el cumplimiento íntegro de las 64 horas de formación presencial/asistida.

1.9.5 Integridad académica, ética en investigación y código de conducta

La investigación y práctica en bases de datos espaciales exige honestidad científica y transparencia metodológica.

- **Originalidad y código abierto:** El código fuente desarrollado debe ser original o citar explícitamente las librerías, repositorios y fuentes de inspiración utilizadas, respetando los términos de sus licencias.
- **Uso ético de asistentes de IA:** Las herramientas de asistencia de código basadas en inteligencia artificial son bienvenidas como aceleradores de productividad, siempre que el estudiante comprenda, justifique y pueda explicar cada línea de código implementada. Queda prohibida la generación no supervisada y acrítica de reportes teóricos.
- **Plagio y fraude académico:** El plagio de textos, la copia directa entre estudiantes o la falsificación de métricas de rendimiento en los experimentos analíticos se sancionará de acuerdo con los reglamentos disciplinarios de posgrado vigentes.

1.10 Referencias bibliográficas

- Güting, R. H., & Schneider, M. (2005). *Moving objects databases*. Morgan Kaufmann.
- Li, C. C., & Shan, J. (Eds.). (2015). *Advances in spatial and temporal databases*. Springer International Publishing.
- Marquez, A. (2015). *PostGIS essentials*. Packt Publishing Ltd.
- Obe, R. O., & Hsu, L. S. (2021). *PostGIS in action* (3rd ed.). Manning Publications.
- Open Geospatial Consortium. (2011). *OpenGIS implementation standard for geographic information - Simple feature access - Part 1: Common architecture* (OGC 06-103r4).
- Rigaux, P., Scholl, M., & Voisard, A. (2001). *Spatial databases: With application to GIS*. Morgan Kaufmann.
- Shekhar, S., & Chawla, S. (2003). *Spatial databases: A tour*. Prentice Hall.

- Yeung, A. K., & Hall, G. B. (2007). *Spatial database systems: Design, implementation and project management*. Springer Science & Business Media.

Parte I

Instalación y entornos

El despliegue de una infraestructura homogénea y reproducible es condición indispensable para la administración de motores relacionales espaciales y el procesamiento analítico. En esta parte se detallan los procedimientos para configurar el entorno de trabajo sobre el sistema anfitrión, la orquestación de servicios en contenedores Docker, la gestión de instancias PostgreSQL/PostGIS y las políticas de respaldo estructurado.

Contenido modular de la sección

La presente parte se compone de los siguientes módulos operativos:

- **Configuración del entorno de desarrollo:** Integración de Visual Studio Code y Docker Desktop.
- **Instalación del motor relacional:** Despliegue de PostgreSQL y activación de la extensión PostGIS.
- **Infraestructura y orquestación:** Despliegue de contenedores y verificación de conectividad por terminal.
- **Administración de respaldos:** Estrategias de extracción y restauración mediante `pg_dump` y `pg_restore`.

Capítulo 2

Guía de instalación

2.1 Introducción

En esta guía se detallan los procedimientos requeridos para:

- Instalar y configurar Visual Studio Code.
- Instalar y parametrizar Docker Desktop.
- Desplegar y orquestar los contenedores del curso.

Para garantizar un entorno preconfigurado y homogéneo, se utiliza la orquestación de contenedores con Docker ([Inc., 2025](#)). Las imágenes a instalar aprovisionan, de forma aislada y coordinada, un motor analítico habilitado con los lenguajes Python, R y Julia, así como el motor relacional PostgreSQL y la extensión espacial PostGIS con una base de datos pedagógica estructurada.

Esta configuración establece un marco formal de **reproducibilidad científica** sustentado en una arquitectura híbrida: **Visual Studio Code** como entorno de desarrollo en el equipo local ([Microsoft Corporation, 2026](#)) y **Docker** como laboratorio de ejecución aislado ([Inc., 2025](#)). Dicha aproximación garantiza la compatibilidad de versiones de software y elimina discrepancias asociadas a la plataforma operativa del sistema anfitrión.

El procedimiento detalla la instalación y carga de componentes mediante un flujo desacoplado de la red (*offline*), diseñado para operar de forma confiable en infraestructuras con restricciones de conectividad.

2.2 Instalación de Visual Studio Code

En el Apéndice D se documenta el proceso de instalación del entorno y la parametrización de las principales extensiones requeridas para el flujo analítico.

2.3 Instalación del orquestador de contenedores

El repositorio central del curso exige la presencia operativa de Docker Desktop y de dos contenedores preconfigurados. Estos servicios integran el motor PostgreSQL 18 con PostGIS 3.6, la base de datos espacial inicial para los talleres analíticos y el ecosistema de procesamiento políglota (Python, R y Julia).

En el Apéndice B se expone la guía de instalación de Docker Desktop, complementada por el procedimiento de optimización inicial detallado en la Sección B.2 para asegurar un rendimiento óptimo del subsistema de almacenamiento.

! Optimización previa de recursos

Antes de importar las imágenes del curso, es mandatorio ejecutar la rutina de ajuste descrita en la Sección B.2 para evitar la degradación de memoria y espacio en disco del sistema anfitrión.

2.3.1 Limpieza del entorno de virtualización

Si existen despliegues previos de Docker Desktop y se requiere iniciar el aprovisionamiento sobre un estado limpio, consulte el protocolo de purga técnica en la Sección B.13.

! Advertencia sobre remoción de volúmenes

El procedimiento documentado en la Sección B.13 elimina la totalidad de contenedores, imágenes y volúmenes existentes en el motor local. Se debe respaldar cualquier recurso previo ajeno al curso antes de proceder.

2.3.2 Descarga y preparación de recursos

El código fuente, las guías de laboratorio y los archivos de configuración residen en un repositorio privado en GitHub. La sincronización y entrega formal de actividades requiere la autenticación individual en la plataforma. Consulte el protocolo de clonación y obtención de artefactos en la Sección B.3.

i Requisito de cuenta en GitHub

Es indispensable disponer de una cuenta activa en GitHub para habilitar los permisos de acceso al repositorio privado, clonar las plantillas de trabajo y gestionar la trazabilidad de los talleres evaluativos.

2.4 Despliegue operativo y administración de contenedores

La puesta en marcha de los servicios orquestados se desarrolla mediante los siguientes procedimientos modulares:

- **Carga de imágenes binarias:** Para importar los paquetes precompilados sin depender de descargas remotas masivas, consulte la Sección B.4.
- **Inicialización de servicios:** Las instrucciones de arranque en segundo plano mediante `docker compose` se detallan en la Sección B.6.1.
- **Acoplamiento en la terminal integrada:** El procedimiento de vinculación de la sesión de trabajo interactiva desde Visual Studio Code hacia el contenedor analítico se documenta en la Sección B.8.
- **Detención y apagado seguro:** Las directivas para suspender o desmontar los servicios sin comprometer la persistencia de datos se exponen en la Sección B.6.2.
- **Catálogo de servicios activos:** El inventario de imágenes, puertos y contenedores desplegados se encuentra sistematizado en la Sección B.5.

2.5 Instalación de herramientas cliente complementarias

Una vez inicializado el contenedor analítico y abierta la sesión en la terminal integrada de Visual Studio Code, se deben ejecutar las directivas de aprovisionamiento descritas en la Sección [F.1](#) y la Sección [F.2](#). Esto garantiza la disponibilidad del cliente nativo `psql` correspondiente a la versión mayor del clúster PostgreSQL (v18), así como las utilidades de renderizado topológico y grafos (Graphviz).

Capítulo 3

Instalación y aprovisionamiento de PostgreSQL 18 y PostGIS 3.6

El despliegue de un sistema de gestión de bases de datos relacionales espaciales (SGBDE) de alto rendimiento requiere la articulación precisa entre el motor relacional PostgreSQL y la suite de extensiones PostGIS. Esta integración dota al motor de tipos de datos geométricos y geográficos alineados con los estándares del Open Geospatial Consortium (OGC), funciones analíticas vectoriales, soporte ráster y motores topológicos.

A continuación, se detalla el procedimiento de instalación, configuración de parámetros del sistema anfitrión, aprovisionamiento de variables de entorno y validación funcional del clúster espacial.

3.1 Arquitectura de directorios y parámetros del sistema anfitrión

La instalación debe realizarse a partir del paquete binario oficial de 64 bits (`postgresql-18.x-x64`), el cual se puede descargar de <https://www.postgresql.org/download/>. Durante la ejecución del asistente, se deben definir las rutas base del software, el directorio segregado de almacenamiento de datos (`data`) y las credenciales maestras de administración.

La siguiente tabla resume los parámetros nominales de configuración aplicados en el despliegue del clúster:

Tabla 3.1: Parámetros nominales de configuración aplicados en el despliegue del clúster local

Componente	Parámetro / Ruta sugerida	Descripción técnica
Installation directory	<code><ruta_instalacion>/PostgreSQL/18</code>	Directorio raíz para binarios, librerías compartidas y encabezados.
Data directory	<code><ruta_instalacion>/PostgreSQL/18/data</code>	Espacio físico exclusivo donde residen las tablas, índices y catálogos.
Database port	5432	Puerto TCP estándar de escucha del servicio relacional.
Superuser	<code>postgres</code>	Rol principal de administración de base de datos con privilegios absolutos.

Componente	Parámetro / Ruta sugerida	Descripción técnica
Contraseña superusuario	geomatica2025	Clave de acceso asignada al rol administrativo postgres .
Service account	Cuenta de servicio de red estándar	Identidad del sistema anfitrión que administra el servicio del motor.
Database service	postgresql-x64-18	Nombre del servicio registrado para gestión de arranque en segundo plano.
Command-line tools	<ruta_instalacion>/PostgreSQL/18/bin	Ubicación de ejecutables y utilidades de terminal (psql , pg_dump , etc.).
pgAdmin 4	<ruta_instalacion>/PostgreSQL/18/pgAdmin 4	Interfaz gráfica desacoplada para gestión, monitoreo y consultas.
StackBuilder	<ruta_instalacion>/PostgreSQL/18	Gestor modular para descarga de extensiones y drivers complementarios.
Installation log	<ruta_temporal>/install-postgresql.log	Registro detallado de eventos de compilación, copia y registro de llaves.

3.2 Descripción y funcionalidad de herramientas del ecosistema

El paquete distribuido integra herramientas administrativas de interfaz gráfica, clientes de línea de comandos, cargadores de datos espaciales y motores de cálculo geométrico esenciales para el análisis geoespacial avanzado.

La siguiente tabla describe la utilidad técnica de cada herramienta instalada:

Tabla 3.2: Utilidades y librerías del ecosistema PostgreSQL y PostGIS

Herramienta / Componente	Tipo	Funcionalidad principal en el flujo espacial
psql	CLI (Terminal)	Cliente interactivo de línea de comandos para ejecución de DDL, DML y administración del motor.
pgAdmin 4	GUI (Gráfica)	Entorno visual para administración de esquemas, optimización de consultas, inspección y visualización de registros.
StackBuilder	Gestor de paquetes	Aplicación para la selección, descarga y vinculación de librerías complementarias y controladores al clúster.
pg_dump / pg_restore	CLI (Terminal)	Utilidades de extracción y restauración de respaldos lógicos (texto plano, formato tar o binario personalizado).
shp2pgsql	CLI (Cargador)	Convertor de formatos vectoriales Shapefile a sentencias de inserción SQL estructuradas para PostGIS.

Herramienta / Componente	Tipo	Funcionalidad principal en el flujo espacial
raster2pgsql	CLI (Cargador)	Herramienta para particionar, piramidar y cargar coberturas ráster multiespectrales en tablas relacionales.
GEOS	Motor algorítmico	Librería computacional encargada de operaciones topológicas 2D (ST_Intersects , ST_Buffer , ST_Intersection).
PROJ	Motor geodésico	Librería para transformaciones de coordenadas, proyección cartográfica y conversión de datum/SRID.
GDAL	Motor de traducción	Capa de abstracción para lectura y escritura de formatos geospaciales ráster y vectoriales integrados al motor.

3.3 Despliegue del motor relacional PostgreSQL

El procedimiento de instalación se realiza mediante el instalador interactivo siguiendo una secuencia ordenada:

1. Ejecutar el instalador binario con privilegios administrativos en el sistema anfitrión.
2. Indicar el directorio de instalación deseado (por ejemplo, `D:\Programs\PostgreSQL\18` o `<ruta_instalacion>/PostgreSQL/18`).
3. Definir el directorio de datos (**data**), garantizando que se localice en un volumen con espacio suficiente para los catálogos y capas geográficas del curso.
4. Establecer la contraseña obligatoria del superusuario **postgres** como **geomatica2025**.
5. Conservar el puerto de conexión estándar 5432 y la configuración regional predeterminada del clúster (*Locale* por defecto).
6. Confirmar el resumen de parámetros y proceder con la copia de componentes binarios.
7. Al finalizar la transferencia, mantener marcada la casilla para inicializar **StackBuilder** antes de cerrar el asistente.

3.4 Integración de la extensión espacial con StackBuilder

PostGIS debe incorporarse como una extensión binaria compatible con la versión mayor del motor:

1. En la ventana inicial de StackBuilder, seleccionar la instancia activa del clúster (puerto 5432).
2. Desplegar la categoría **Spatial Extensions**.
3. Marcar la opción correspondiente al paquete espacial:

PostGIS 3.6 Bundle for PostgreSQL 18 (64 bit) v3.6.1

4. Definir un directorio de descarga temporal en el almacenamiento local.
5. Durante la instalación de los componentes espaciales, **no activar** la casilla orientada a crear

automáticamente una base de datos espacial genérica (*Create spatial database*). La creación y configuración de la base de datos se ejecutará manualmente para controlar la parametrización de esquemas y extensiones.

6. Aceptar la integración de las librerías dependientes (GEOS, PROJ, GDAL y controladores ráster asociados) hasta completar el asistente.

3.5 Configuración de variables de entorno y terminal integrada

Para ejecutar los binarios del gestor desde la terminal integrada de Visual Studio Code sin declarar la ruta absoluta, es indispensable registrar la carpeta `bin` en la variable de entorno `PATH` del sistema anfitrión.

1. Abrir la configuración avanzada del sistema en el panel de control del sistema anfitrión y acceder a **Variables de entorno**.
2. En la sección **Variables del sistema**, seleccionar la variable `Path` y presionar **Editar**.
3. Agregar una nueva entrada con la ruta absoluta hacia los ejecutables:

```
<ruta_instalacion>/PostgreSQL/18/bin
```

4. Guardar los cambios aplicados.
5. Iniciar una nueva sesión de la terminal integrada de Visual Studio Code y verificar el reconocimiento del binario principal `psql`:

```
#| label: setup-verify-psql
#| eval: false
psql --version
```

Más información sobre `psql` en el Sección [G.1](#):

- Metacomandos de barra invertida frente a sentencias SQL (Sección [G.1.1](#) o en Tabla [G.1](#)).
- Parámetros de invocación y modificadores de línea de comandos (Sección [G.1.2](#) o en Tabla [G.2](#)).
- Guía de inicio rápido para la administración con `psql` (Sección [G.1.3](#)).

3.6 Inicialización manual de la base de datos espacial

El aprovisionamiento manual permite aislar los catálogos y registrar únicamente las extensiones requeridas por la infraestructura de datos espaciales.

3.6.1 Conexión administrativa al clúster

Abrir la terminal integrada de Visual Studio Code y conectarse a la instancia local utilizando el usuario administrativo:

```
#| label: setup-connect-psql
#| eval: false
psql -U postgres
```

El sistema solicitará la contraseña configurada (`geomatica2025`).

3.6.2 Creación de la base de datos

Crear el contenedor lógico destinado a las prácticas analíticas del curso:

```
#| label: ddl-create-spatialdb
#| eval: false
CREATE DATABASE spatialdb;
```

Conectarse a la base de datos recién creada:

```
#| label: setup-switch-database
#| eval: false
\c spatialdb
```

3.6.3 Habilitación de extensiones espaciales

Ejecutar las directivas DDL para activar el núcleo vectorial, el soporte ráster y el módulo topológico:

```
#| label: ddl-enable-postgis-extensions
#| eval: false
CREATE EXTENSION postgis;
CREATE EXTENSION postgis_raster;
CREATE EXTENSION postgis_topology;
```

3.6.4 Auditoría de versiones y enlaces de librerías

Comprobar que PostGIS se encuentre vinculado correctamente con GEOS, PROJ y los componentes de estadísticas espaciales:

```
#| label: audit-postgis-version
#| eval: false
SELECT postgis_version();
```

Resultado esperado en consola:

```
3.6 USE_GEOS=1 USE_PROJ=1 USE_STATS=1
```

3.7 Verificación analítica del módulo espacial

Para confirmar la operatividad de los constructores geométricos basados en el estándar OGC SFS (*Simple Feature Access*), se ejecuta una consulta de prueba instanciando una geometría de tipo punto en dos dimensiones:

```
#| label: query-test-point-geometry
#| eval: false
SELECT ST_AsText(ST_Point(10, 20));
```

Respuesta esperada:

```
POINT(10 20)
```

3.8 Resumen de la estructura desplegada

Al culminar el procedimiento, el árbol de directorios en el sistema anfitrión presentará la siguiente jerarquía funcional:

```
<ruta_instalacion>/PostgreSQL/18/
  bin/          <- Ejecutables del motor, clientes CLI y cargadores espaciales
  data/         <- Almacenamiento físico de tablas, índices y catálogos
  include/      <- Encabezados de desarrollo en C/C++ para extensiones
  lib/          <- Librerías dinámicas del motor y módulos compilados de PostGIS
  pgAdmin 4/    <- Aplicación de administración y monitoreo visual
  share/        <- Scripts SQL de extensión, plantillas y archivos de soporte
  StackBuilder/ <- Utilidad de mantenimiento y actualización de componentes
```

El clúster PostgreSQL 18 con PostGIS 3.6 queda completamente operativo, validado y disponible para la ingesta y manipulación de datos geográficos a través de la terminal integrada de Visual Studio Code.

3.9 Configuración de PostgreSQL local para acceso desde Docker

Dado que el material y las prácticas del curso se compilan dentro de contenedores Docker (ver Capítulo 2), la instancia de PostgreSQL alojada en el sistema anfitrión debe permitir conexiones entrantes originadas desde las interfaces virtuales de dichos contenedores. Por omisión, el motor restringe la escucha exclusivamente a la interfaz local (*localhost* o *127.0.0.1*), lo que bloquea las solicitudes provenientes del entorno contenedorizado.

A continuación, se detalla el procedimiento para habilitar la escucha de red, autorizar la autenticación y aplicar las nuevas directivas operativas.

3.9.1 Habilitación de la escucha de red en postgresql.conf

Localizar el archivo principal `postgresql.conf` dentro del directorio de datos del clúster local. Se debe asegurar la presencia y activación de la siguiente directiva:

```
listen_addresses = '*'
```

Esta directiva instruye al motor a enlazar los puertos de escucha a todas las interfaces de red disponibles en el equipo local, incluyendo los adaptadores puente (*bridge*) creados por Docker.

3.9.2 Control de acceso y autenticación en pg_hba.conf

El parámetro anterior únicamente habilita la escucha en los puertos; la política de admisión debe autorizarse de forma explícita en el archivo de control de acceso `pg_hba.conf`. Se debe incorporar la siguiente directiva al final del archivo:

host	all	all	0.0.0.0/0	scram-sha-256
------	-----	-----	-----------	---------------

Esta entrada autoriza conexiones TCP/IP (**host**) hacia cualquier base de datos (**all**) para todos los usuarios registrados (**all**), desde cualquier segmento IPv4 (0.0.0.0/0), exigiendo el protocolo criptográfico **scram-sha-256** para la validación de credenciales.

3.9.3 Reinicio del servicio en el motor de base de datos

Tras modificar los archivos `postgresql.conf` y `pg_hba.conf`, es obligatorio reiniciar el proceso del motor para que cargue la nueva tabla de enrutamiento y autenticación en memoria.

3.9.3.1 Consola de comandos (agnóstico)

Desde la terminal integrada de Visual Studio Code con permisos de administración, ejecutar el comando agnóstico de control del clúster indicando la ruta del directorio de datos:

```
#| label: setup-restart-pg-engine
#| eval: false
pg_ctl -D "<ruta_al_directorio_de_datos>" restart
```

3.9.3.2 Sistema anfitrión (Windows)

1. Presione la combinación de teclas **Windows + R** para abrir el diálogo de ejecución.
2. Escriba `services.msc` y presione **Enter**.
3. En la lista de servicios del sistema, localice la entrada `postgresql-x64-18` (PostgreSQL Server 18).
4. Haga clic derecho sobre el servicio y seleccione la opción **Restart** (Reiniciar).

Capítulo 4

Despliegue Docker y verificación de la base de datos

4.1 Introducción

Antes de continuar con este capítulo, se debe contar con las siguientes herramientas instaladas y operativas en el equipo local:

- VSCode, Docker Desktop y contenedores para el curso: ver Capítulo 2
- PostgreSQL y PostGIS en el sistema anfitrión: ver Capítulo 3
- Configuración de PostgreSQL y PostGIS en el sistema anfitrión: ver Sección 3.9
- Configuración del cliente `psql` en el Docker: ver Sección F.1
- Instalación y verificación de Graphviz en el contenedor: ver Sección F.2

En esta sección se configura el entorno de contenedores para acceder a la base de datos espacial pedagógica (`sig_db_unal`), la cual contiene PostgreSQL 18 y PostGIS 3.6 preconfigurados con la cartografía de estudio. Adicionalmente, se presenta el procedimiento para replicar, cargar y auditar este catálogo en la instalación nativa del equipo local.

4.2 Orquestación y verificación del motor de contenedores

Previo a la configuración de parámetros de red y ejecución de consultas, es indispensable validar que el servicio de contenedores se encuentra en ejecución y que las imágenes base residen en el catálogo local.

4.2.1 Verificación del servicio y catálogo de imágenes

Para comprobar el estado operativo del motor desde la consola de comandos del equipo local:

```
# Comprobación de versión y estado operativo del servicio
docker --version
docker info
```

Tras ejecutar el aprovisionamiento descrito en la Capítulo 2, verifique la presencia de las imágenes requeridas:

```
docker images
```

El catálogo debe listar las siguientes imágenes:

- `postgis_unal:v1.12`
- `image_sig_unal:v1.11_produccion_final`

4.2.2 Despliegue de servicios mediante Docker Compose

Una vez validadas las imágenes, se inicia la infraestructura en segundo plano desde el directorio raíz del proyecto en el sistema anfitrión:

```
# Despliegue de los contenedores y creación de volúmenes persistentes
docker compose up -d

# Verificación del estado de ejecución de los servicios
docker compose ps
```

Durante el primer despliegue, el servicio `db-postgis` (`contenedor_postgis_unal`) detecta que el volumen persistente `postgis_data_unal` se encuentra vacío y ejecuta automáticamente la rutina de auto-inicialización declarativa:

- Inicializa el clúster de base de datos PostgreSQL 18.
- Habilita las extensiones espaciales requeridas (`postgis`, `postgis_topology`, `postgis_raster`).
- Ejecuta el script SQL embebido en la imagen, aprovisionando de forma automática los esquemas temáticos `base750` y `divpol` con su cartografía vectorial dentro de la base de datos `sig_db_unal`.

En la Figura 4.1 se esquematiza el mecanismo de auto-inicialización condicional y la persistencia de datos administrada por el motor de contenedores.

Este proceso de carga se realiza una única vez; en ejecuciones posteriores, el motor se enlaza directamente a los datos persistidos en el volumen, reduciendo el tiempo de arranque a pocos segundos. Para el desarrollo y compilación del material, se recomienda ejecutar el flujo de trabajo directamente desde la terminal integrada de Visual Studio Code con el contenedor configurado (ver Sección B.8).

4.3 Gestión de variables de entorno y contextos de ejecución

El flujo de trabajo técnico del curso contempla la coexistencia de dos motores de base de datos independientes, junto con dos modos diferenciados de ejecución: la compilación documental desatendida mediante Quarto/Knitr y la interacción manual interactiva desde la terminal integrada.

Las instancias del motor corresponden a:

1. **Instancia en el equipo local:** Instalada directamente sobre el sistema anfitrión (*host*). Las credenciales de acceso (usuario, contraseña, puerto) corresponden a las establecidas durante su aprovisionamiento local.

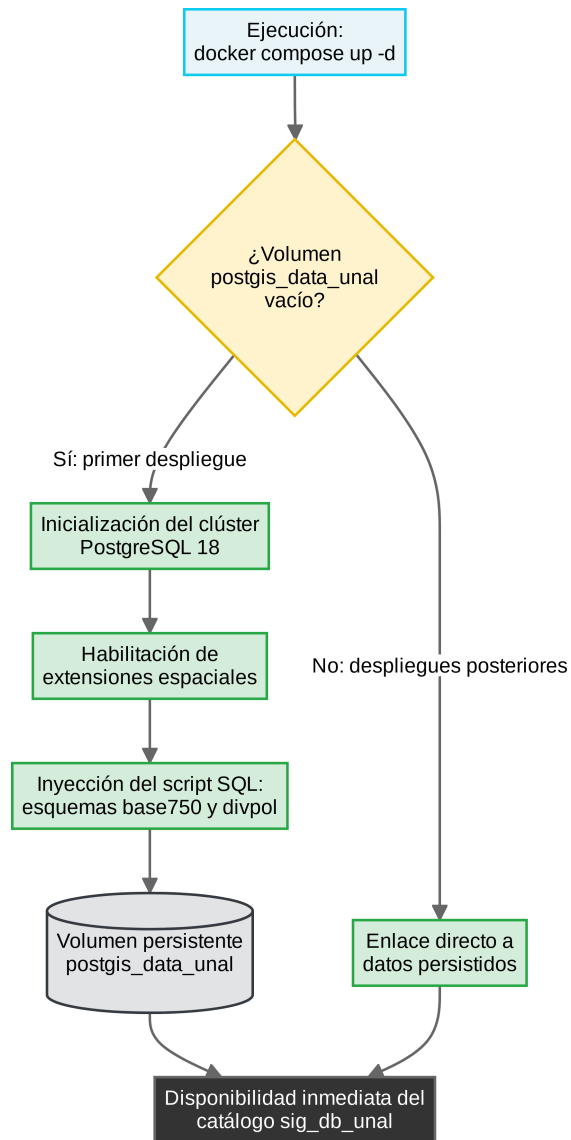


Figura 4.1: Ciclo de vida y auto-inicialización declarativa del clúster espacial con volumen persistente

2. **Instancia en el contenedor (db-postgis):** Aprovechada de forma estandarizada dentro de la imagen `postgis_unal:v1.12`. Las credenciales son homogéneas para fines pedagógicos.

4.3.1 Coexistencia de clústeres y topología de puertos según el contexto de ejecución

Para profundizar en la matriz de puertos, el enrutamiento y la resolución de red según el entorno de ejecución, ver Sección B.9 (o consultar la Tabla B.2).

La Figura 4.2 detalla el aislamiento de red entre el sistema anfitrión y los contenedores, delimitando los puntos de acceso y redireccionamiento de puertos para cada servicio.

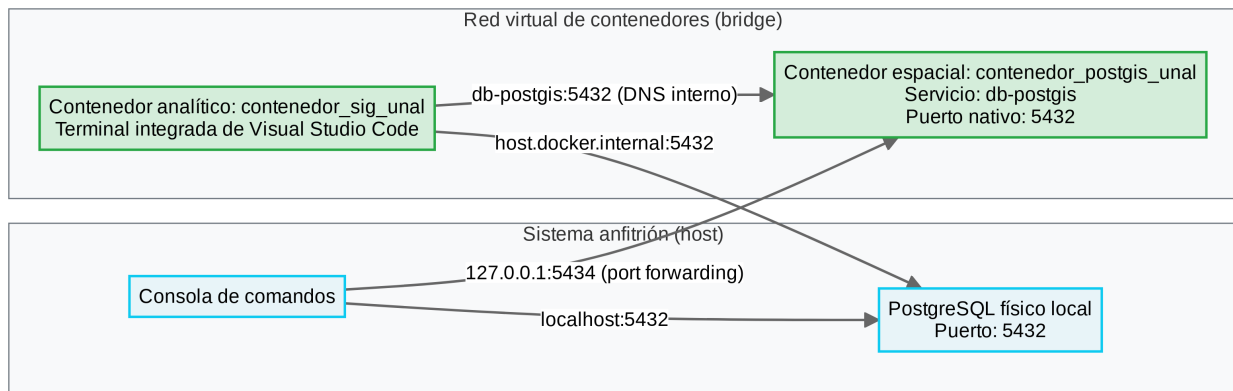


Figura 4.2: Matriz de resolución de red y mapeo de puertos según el contexto de ejecución

4.3.2 Mecanismo de autenticación libpq y gestión de credenciales

El cliente interactivo `psql` y las herramientas espaciales utilizan la biblioteca `libpq`, la cual inspecciona la variable de entorno `PGPASSWORD` para autenticar conexiones de manera automática.

Para gestionar la alternancia entre ambos clústeres sin requerir el ingreso interactivo de credenciales en cada consulta, se definen variables de almacenamiento (`LOCAL_PGPASSWORD` y `DOCKER_PGPASSWORD`). Antes de ejecutar una consulta hacia una instancia determinada, se transfiere el valor correspondiente a la variable activa `PGPASSWORD`.

Tabla 4.1: Especificaciones de conexión para clústeres local y dockerizado

Parámetro técnico	Instancia local física	Instancia en contenedor (db-postgis)	Función técnica
Host / Interfaz	<code>host.docker.internal</code>	<code>db-postgis</code>	Enrutamiento desde el entorno analítico
Puerto de red	5432 (estándar)	5432 (interno)	Puerto TCP nativo del servicio
Usuario / Rol	<code>postgres</code>	<code>profe_unal</code>	Rol con privilegios administrativos DDL/DML

Parámetro técnico	Instancia local física	Instancia en contenedor (db-postgis)	Función técnica
Contraseña	Asignada localmente	geomatica2025	Credencial de autenticación del clúster
Base de datos	postgres / sig_db_unal	sig_db_unal	Catálogo relacional con extensión PostGIS

4.3.3 Parámetros de línea de comandos en herramientas PostgreSQL

Las utilidades cliente de PostgreSQL (`psql`, `pg_dump`, `pg_restore`) y los cargadores espaciales de PostGIS (`shp2pgsql`, `raster2pgsql`) comparten un conjunto estandarizado de modificadores de línea de comandos gestionados por la biblioteca `libpq`.

Para consultar el desglose detallado de modificadores, su equivalencia con variables de entorno y las utilidades compatibles, ver Sección [G.1.2](#) (o consultar la Tabla [G.2](#)).

4.3.4 Dualidad operativa: compilación documental vs terminal interactiva

La ejecución de sentencias en este libro opera bajo dos contextos técnicos que resuelven las variables de conexión de manera diferente:

1. **Contexto de compilación reproducible (Knitr/Quarto):** Cada bloque de código ejecutable `{bash}` se procesa en una subshell aislada y efímera. Las variables exportadas mediante `export VAR=valor` dentro de un bloque mueren al finalizar dicho bloque y no son heredadas por los bloques siguientes. Para asegurar que todos los bloques del capítulo compartan los mismos parámetros de red sin fallos, el documento inyecta las variables en la tabla del proceso padre mediante `Sys.setenv()` en un bloque inicial de R (`setup-variables-sistema-r`). Cada subshell hija generada por Knitr hereda automáticamente estas variables del sistema.
2. **Contexto de ejecución interactiva (terminal de Visual Studio Code):** Cuando el estudiante ejecuta comandos directamente en la consola interactiva (ya sea dentro del contenedor de desarrollo o en el sistema anfitrión), no existe un proceso R intermediario. En este escenario, es obligatorio exportar las variables de entorno (`export PG*`) en la propia sesión de la shell para que `psql` y las herramientas clientes consuman automáticamente las credenciales sin requerir parámetros redundantes en cada instrucción.

La biblioteca `libpq` evalúa los parámetros bajo el esquema de precedencia jerárquico presentado en la Figura [4.3](#).

Para sesiones interactivas en la terminal, se deben cargar los parámetros copiando el siguiente bloque de inicialización:

```
# =====
# 1. Parámetros de conexión: Instancia física en el equipo local (desde contenedor)
# =====
export LOCAL_PGHOST="host.docker.internal"
export LOCAL_PGPORT="5432"
export LOCAL_PGUSER="postgres"

# Nota para el estudiante: Reemplace 'geomatica2025' por la contraseña de su instalación local.
export LOCAL_PGPASSWORD='geomatica2025'
```

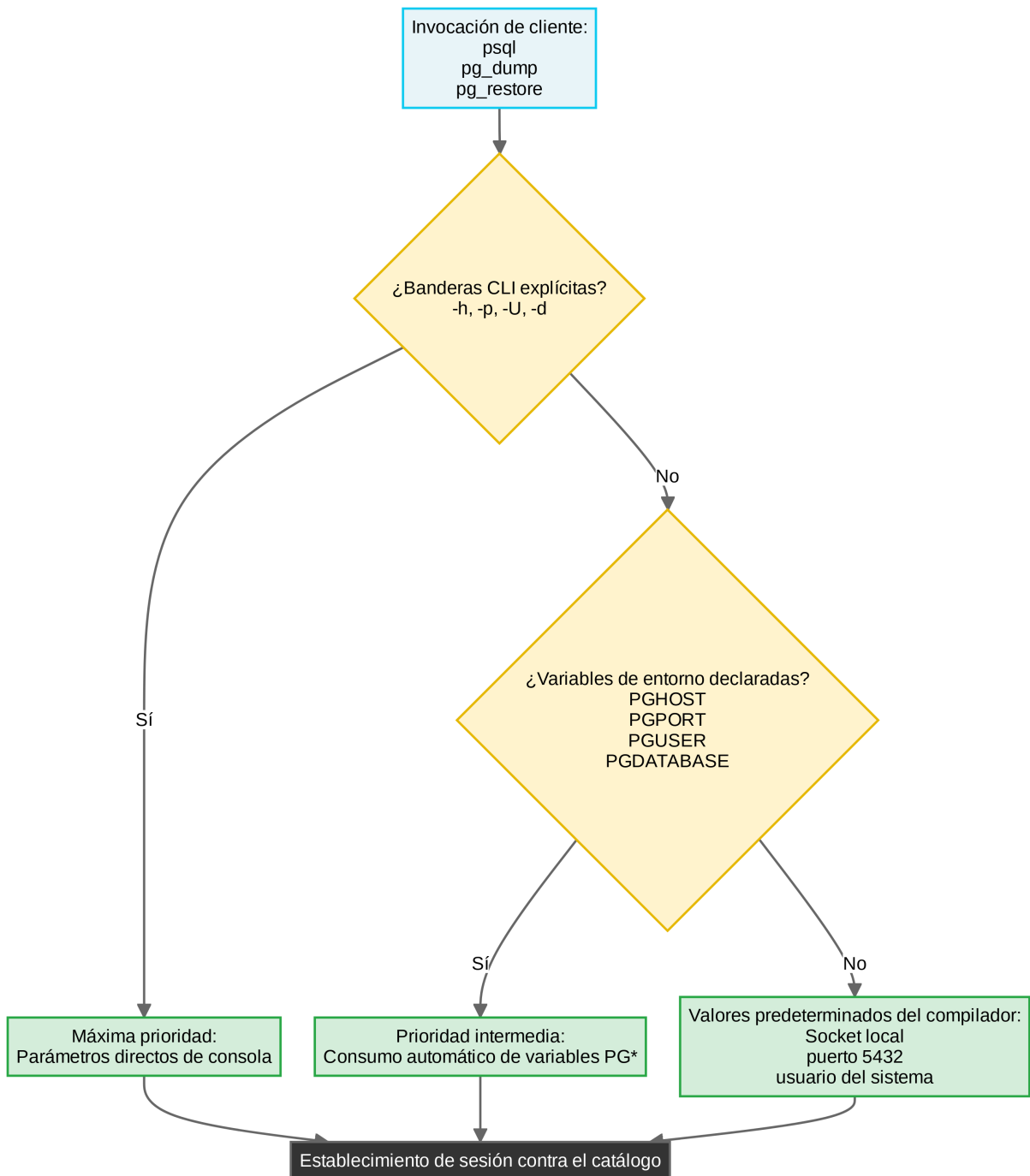


Figura 4.3: Jerarquía de resolución en cascada de parámetros de conexión en la biblioteca libpq

```

export LOCAL_PGDATABASE="postgres"

# =====
# 2. Parámetros de conexión: Clúster espacial en el contenedor (db-postgis)
# =====
export DOCKER_PGHOST="db-postgis"
export DOCKER_PGPORT="5432"
export DOCKER_PGUSER="profe_unal"
export DOCKER_PGPASSWORD='geomatica2025'
export DOCKER_PGDATABASE="sig_db_unal"

# =====
# 3. Variables estándar activas para libpq (consumo automático por defecto)
# =====
export PGHOST="${DOCKER_PGHOST}"
export PGPORT="${DOCKER_PGPORT}"
export PGUSER="${DOCKER_PGUSER}"
export PGPASSWORD="${DOCKER_PGPASSWORD}"
export PGDATABASE="${DOCKER_PGDATABASE}"

# =====
# 4. Verificación de las variables asignadas en la sesión
# =====
echo "--- Instancia local (host) ---"
echo "Host: ${LOCAL_PGHOST}:${LOCAL_PGPORT} | Usuario: ${LOCAL_PGUSER} \vert{} BD:${LOCAL_PGDATABASE}"

echo "--- Instancia dockerizada ---"
echo "Host: ${DOCKER_PGHOST}:${DOCKER_PGPORT} | Usuario: ${DOCKER_PGUSER} \vert{} BD:${DOCKER_PGDATABASE}"

echo "--- Variables estándar activas (libpq) ---"
echo "Destino por defecto: ${PGUSER}@${PGHOST}:${PGPORT}/${PGDATABASE}"

```

--- Instancia local (host) ---

Host: host.docker.internal:5432 | Usuario: postgres \vert{} BD:postgres

--- Instancia dockerizada ---

Host: db-postgis:5432 | Usuario: profe_unal \vert{} BD:sig_db_unal

--- Variables estándar activas (libpq) ---

Destino por defecto: profe_unal@db-postgis:5432/sig_db_unal

i Nomenclatura canónica de conexión (usuario@host:puerto/base_de_datos)

La estructura compuesta `${PGUSER}@${PGHOST}:${PGPORT}/${PGDATABASE}` (por ejemplo, `profe_unal@db-postgis:5432/sig_db_unal`) corresponde a la notación canónica para la identificación unívoca de un extremo de conexión (*target endpoint*), fundamentada en el estándar de URI (*Uniform Resource Identifier*) de PostgreSQL (RFC 3986) y cadenas DSN (*Data Source Name*). Este descriptor unifica los tres planos funcionales requeridos:

- **Identidad del rol autenticado (profe_unal@):** Determina las credenciales y los privilegios de ejecución DDL/DML.
- **Dirección y puerto del socket de red (db-postgis:5432):** Especifica la ruta de enrutamiento hacia la instancia.
- **Catálogo relacional objetivo (/sig_db_unal):** Define la base de datos relacional y espacial vinculada a la sesión.

4.4 Verificación de versiones y extensiones espaciales

Para asegurar la disponibilidad operativa del clúster y validar la correcta habilitación de la extensión espacial, se asigna la credencial del contenedor a la variable activa `PGPASSWORD` y se ejecutan las consultas de diagnóstico preliminar.

i Modificadores de línea de comandos para control de formato en psql

Las salidas tabulares emitidas por `psql` para funciones de introspección del sistema suelen superar el ancho del área imprimible en documentos PDF o generar barras de desplazamiento horizontal innecesarias en HTML. Para garantizar legibilidad y evitar desbordamientos, se emplean los modificadores estandarizados de la utilidad:

- `-x (--expanded)`: Activa la visualización expandida vertical (pares atributo-valor por registro), idónea para inspeccionar tuplas únicas con campos de texto extensos.
- `-P format=wrapped`: Habilita la envoltura y división automática de líneas dentro de las celdas tabulares.
- `-P columns=90`: Delimita el ancho máximo del renderizado a 90 caracteres, forzando el corte dentro de los márgenes tipográficos de la página.

Para consultar el inventario general de parámetros, modificadores y equivalencias con variables de entorno de `libpq`, ver la Tabla G.2 en la Sección G.1.2.

```
export PGPASSWORD="${DOCKER_PGPASSWORD}"

# 1. Versión del motor relacional PostgreSQL (modo expandido y ajuste a 90 columnas)
psql -h ${DOCKER_PGHOST} -p ${DOCKER_PGPORT} -U ${DOCKER_PGUSER} -d ${DOCKER_PGDATABASE} \
  -x -P format=wrapped -P columns=90 \
  -c "SELECT version();"

# 2. Comprobación básica de operatividad de la extensión PostGIS
psql -h ${DOCKER_PGHOST} -p ${DOCKER_PGPORT} -U ${DOCKER_PGUSER} -d ${DOCKER_PGDATABASE} \
  -c "SELECT PostGIS_Version();"
```

```
-[ RECORD 1 ]-----
version | PostgreSQL 18.1 (Debian 18.1-1.pgdg13+2) on x86_64-pc-linux-gnu, compiled by gc.
        | .c (Debian 14.2.0-19) 14.2.0, 64-bit
```

```
postgis_version
```

```
-----
3.6 USE_GEOS=1 USE_PROJ=1 USE_STATS=1
(1 row)
```

4.5 Auditoría de catálogos y bases de datos

Para examinar los catálogos instanciados en el clúster dockerizado, su volumetría física en disco, propietario y codificación:

```
export PGPASSWORD="${DOCKER_PGPASSWORD}"

query_catalogos="
SELECT
    datname AS base_de_datos,
    pg_catalog.pg_get_userbyid(datdba) AS propietario,
    pg_size_pretty(pg_catalog.pg_database_size(datname)) AS tamaño,
    pg_catalog.pg_encoding_to_char(encoding) AS codificacion,
    datcollate AS ordenamiento
FROM pg_catalog.pg_database
WHERE datistemplate = false
ORDER BY pg_catalog.pg_database_size(datname) DESC;
"
```

```
psql -h ${DOCKER_PGHOST} -p ${DOCKER_PGPORT} -U ${DOCKER_PGUSER} -d ${DOCKER_PGDATABASE} -c
↳ "${query_catalogos}"
```

base_de_datos	propietario	tamano	codificacion	ordenamiento
sig_db_unal	profe_unal	109 MB	UTF8	C.UTF-8
postgres	postgres	7902 kB	UTF8	C.UTF-8

(2 rows)

4.6 Inspección estructural de la base de datos de estudio

La base de datos pedagógica `sig_db_unal` contiene los esquemas modulares preparados para los talleres prácticos del curso.

La organización física del catálogo relacional se detalla en la Figura 4.4, ilustrando la articulación entre las extensiones espaciales habilitadas y los esquemas temáticos cartográficos.

4.6.1 Esquemas y tablas espaciales

Se comprueba la existencia de los esquemas temáticos (`base750` y `divpol`):

```
export PGPASSWORD="${DOCKER_PGPASSWORD}"
# Listado de esquemas activos
psql -h ${DOCKER_PGHOST} -p ${DOCKER_PGPORT} -U ${DOCKER_PGUSER} -d ${DOCKER_PGDATABASE} -c "\dn"
# Inventario de tablas por esquema temático
query_tablas="
SELECT
    schemaname AS esquema,
    tablename AS tabla
FROM pg_tables
WHERE schemaname IN ('base750', 'divpol')
ORDER BY schemaname, tablename;
"
psql -h ${DOCKER_PGHOST} -p ${DOCKER_PGPORT} -U ${DOCKER_PGUSER} -d ${DOCKER_PGDATABASE} -c
↳ "${query_tablas}"
```

List of schemas

Name	Owner
base750	profe_unal
cron	profe_unal
divpol	profe_unal
public	pg_database_owner
topology	profe_unal

(5 rows)

esquema	tabla
base750	hidro_lineas

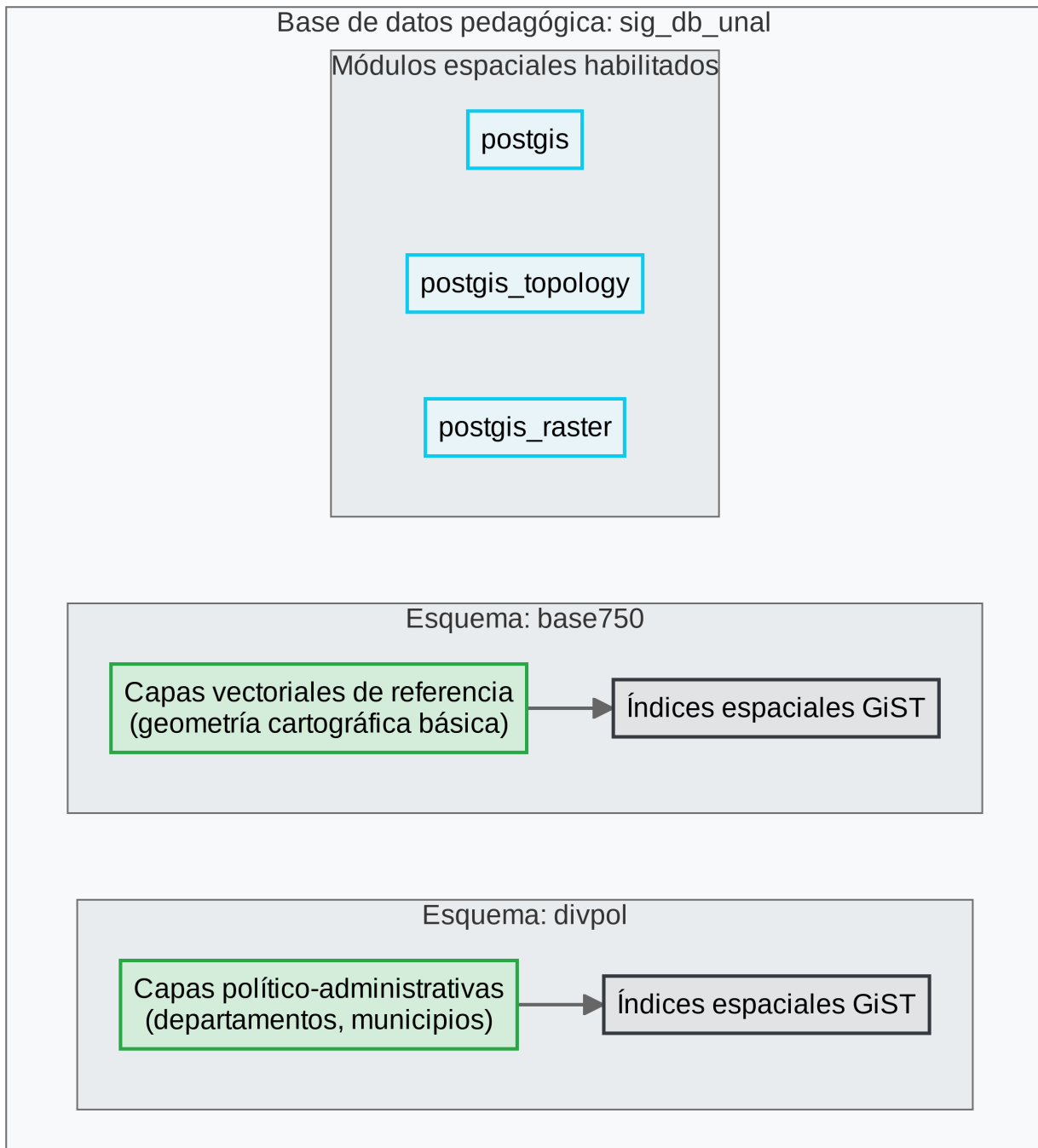


Figura 4.4: Organización física de esquemas, capas vectoriales e índices espaciales en la base de datos de estudio

```

base750 | vias_lineas
base750 | vias_puntos
base750 | vias_texto
divpol  | departamentos
divpol  | municipios
divpol  | nacion
(7 rows)

```

4.6.2 Validación de capas vectoriales en el clúster

Para verificar que las capas vectoriales cargadas en los esquemas temáticos disponen de tipado geométrico y asignación de identificador de referencia espacial (SRID), se consulta la vista `geometry_columns`:

```

export PGPASSWORD="${DOCKER_PGPASSWORD}"

query_geom="
SELECT
  f_table_schema AS esquema,
  f_table_name AS capa,
  f_geometry_column AS columna_geom,
  coord_dimension AS dimension,
  srid,
  type AS tipo_geometria
FROM geometry_columns
WHERE f_table_schema IN ('base750', 'divpol')
ORDER BY f_table_schema, f_table_name;
"

psql -h ${DOCKER_PGHOST} -p ${DOCKER_PGPORT} -U ${DOCKER_PGUSER} -d ${DOCKER_PGDATABASE} -c
  ↪ "${query_geom}"

```

esquema	capa	columna_geom	dimension	srid	tipo_geometria
base750	hidro_lineas	the_geom	2	21892	GEOMETRY
base750	vias_lineas	the_geom	2	21892	GEOMETRY
base750	vias_puntos	the_geom	2	21892	ST_Point
base750	vias_texto	the_geom	2	21892	ST_Point
divpol	capa_departamentos	the_geom	2	0	GEOMETRY
divpol	capa_municipios	the_geom	2	0	GEOMETRY
divpol	departamentos	the_geom	2	21892	GEOMETRY
divpol	municipios	the_geom	2	21892	GEOMETRY

(8 rows)

4.6.3 Verificación de índices espaciales

Para garantizar eficiencia computacional en las operaciones de cruce y filtrado topológico, las capas deben disponer de índices basados en árboles de búsqueda generalizada (GiST):

```

export PGPASSWORD="${DOCKER_PGPASSWORD}"

query_indices="
SELECT
  schemaname AS esquema,
  tablename AS tabla,
  indexname AS nombre_indice,
  indexdef AS definicion

```

```

FROM pg_indexes
WHERE schemaname IN ('base750', 'divpol') AND indexname LIKE '%gist%'
ORDER BY schemaname, tablename;
"

psql -h ${DOCKER_PGHOST} -p ${DOCKER_PGPORT} -U ${DOCKER_PGUSER} -d ${DOCKER_PGDATABASE} -P format=wrapped
→ -P columns=90 -c "${query_indices}"

```

esquema	tabla	nombre_indice	definicion
base750	hidro_lineas	hidro_lineas_the_geom_gis.	CREATE INDEX hidro_lineas_the_geom_.
		.t	.gist ON base750.hidro_lineas USING .
			.gist (the_geom)
base750	vias_lineas	vias_lineas_the_geom_gist	CREATE INDEX vias_lineas_the_geom_g.
			.ist ON base750.vias_lineas USING gi.
			.st (the_geom)
base750	vias_puntos	vias_puntos_the_geom_gist	CREATE INDEX vias_puntos_the_geom_g.
			.ist ON base750.vias_puntos USING gi.
			.st (the_geom)
base750	vias_texto	vias_texto_the_geom_gist	CREATE INDEX vias_texto_the_geom_gi.
			.st ON base750.vias_texto USING gist.
			. (the_geom)
divpol	departamento.	departamentos_the_geom_gi.	CREATE INDEX departamentos_the_geom.
	.s	.st	.gist ON divpol.departamentos USING.
			. gist (the_geom)
divpol	municipios	municipios_the_geom_gist	CREATE INDEX municipios_the_geom_gi.
			.st ON divpol.municipios USING gist .
			.(the_geom)

(6 rows)

4.7 Replicación de la base de datos en una instalación local nativa

En escenarios donde se requiera trabajar directamente sobre la instalación física local de PostgreSQL y PostGIS (puerto 5432) sin recurrir al contenedor, es posible reconstruir la base de datos `sig_db_unal` a partir del archivo de volcado SQL `init_unal_spatialdb.sql`.

4.7.1 Creación del catálogo y habilitación espacial

Para aprovisionar el rol docente y la base de datos en el motor local:

```

# 1. Asignar la contraseña del motor local para libpq
# Nota para el estudiante: Reemplace 'geomatica2025' por la contraseña configurada en su equipo local.
export PGPASSWORD='geomatica2025'

# 2. Creación del rol docente en la instancia local
query_rol="
DO \${$
BEGIN
  IF NOT EXISTS (SELECT FROM pg_roles WHERE rolname = 'profe_unal') THEN
    CREATE ROLE profe_unal WITH SUPERUSER CREATEDB CREATEROLE LOGIN PASSWORD 'geomatica2025';
  END IF;
END

```

```
\$\\$;
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d postgres -c "${query_rol}"

# 3. Creación de la base de datos pedagógica asignando como propietario a profe_unal
psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d postgres -c "CREATE DATABASE sig_db_unal
↳ WITH OWNER profe_unal ENCODING 'UTF8';"

# 4. Habilitación de la extensión espacial PostGIS en el catálogo creado
psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d sig_db_unal -c "CREATE EXTENSION IF NOT
↳ EXISTS postgis;"
```

4.7.2 Restauración mediante inyección del script SQL

Con el catálogo creado y la extensión PostGIS habilitada en el motor local, se procesa el archivo `init_unal_spatialdb.sql`:

```
# Inyección de estructura DDL, cartografía vectorial e índices espaciales
psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d sig_db_unal -f init_unal_spatialdb.sql
```

4.7.3 Verificación de integridad en la instancia local

Para comprobar que la base de datos en el equipo local contenga la misma volumetría y estructura que el clúster dockerizado:

```
# Nota para el estudiante: Reemplace 'geomatica2025' por la contraseña de su instalación local.
export PGPASSWORD='geomatica2025'

# 1. Conteo de registros por tabla en la instancia física local
query_conteo="
SELECT
    schemaname AS esquema,
    relname AS tabla,
    n_live_tup AS registros_locales
FROM pg_stat_user_tables
WHERE schemaname IN ('base750', 'divpol')
ORDER BY schemaname, relname;
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d sig_db_unal -c "${query_conteo}"

# 2. Comprobación del catálogo formal geometry_columns en el motor local
query_geom_local="
SELECT
    f_table_schema AS esquema,
    f_table_name AS capa,
    f_geometry_column AS columna_geom,
    srid,
    type AS tipo_geometria
FROM geometry_columns
WHERE f_table_schema IN ('base750', 'divpol')
ORDER BY f_table_schema, f_table_name;
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d sig_db_unal -c "${query_geom_local}"
```

esquema	tabla	registros_locales
base750	hidro_lineas	0
base750	vias_lineas	0
base750	vias_puntos	0
base750	vias_texto	0

```

divpol | departamentos |          0
divpol | municipios    |          0
divpol | nacion        |          0
(7 rows)

```

```

esquema |      capa      | columna_geom | srid | tipo_geometria
-----+-----+-----+-----+-----
base750 | hidro_lineas   | the_geom     | 21892 | GEOMETRY
base750 | vias_lineas     | the_geom     | 21892 | GEOMETRY
base750 | vias_puntos     | the_geom     | 21892 | ST_Point
base750 | vias_texto      | the_geom     | 21892 | ST_Point
divpol  | capa_departamentos | the_geom     | 21892 | GEOMETRY
divpol  | capa_municipios  | the_geom     | 21892 | GEOMETRY
divpol  | departamentos   | the_geom     | 21892 | GEOMETRY
divpol  | municipios       | the_geom     | 21892 | GEOMETRY
(8 rows)

```

Con este procedimiento, la base de datos queda completamente operativa y sincronizada tanto en el contenedor como en la instalación física local.

Capítulo 5

Respaldo y recuperación de la base de datos sig_db_unal

La gestión y preservación de infraestructuras de datos espaciales exige la implementación de protocolos sistemáticos de respaldo (*backup*) y recuperación ante desastres (*disaster recovery*). En entornos analíticos basados en PostgreSQL 18 y PostGIS 3.6, la persistencia de geometrías vectoriales conformes al estándar OGC SFS (*Simple Feature Access*) y de catálogos topológicos demanda el uso de utilidades nativas de extracción lógica optimizadas.

A continuación, se formalizan los métodos estándar para la generación de copias de seguridad de la base de datos pedagógica sig_db_unal y su posterior reconstrucción, considerando tanto el clúster dockerizado de desarrollo como la instancia relacional nativa alojada en el sistema anfitrión (host).

5.1 Persistencia de variables y configuración de entorno

El protocolo de conexión de las herramientas cliente de PostgreSQL (`pg_dump`, `pg_restore`, `psql`) opera sobre la biblioteca `libpq`, la cual resuelve parámetros de conexión en cascada evaluando las variables de entorno del sistema.

Para operar desde la terminal integrada de Visual Studio Code dentro del contenedor analítico y direccionar peticiones hacia cualquiera de las dos instancias sin solicitar contraseñas de forma interactiva, se declaran los parámetros de red, credenciales administrativas y la resolución dinámica del directorio de almacenamiento:

```
# =====
# 1. Directorio de persistencia para copias de respaldo
# =====
# Resolución condicional: valida si la sesión se ubica en la raíz o en chapters/
if [ -d "backups_pg18" ]; then
  export BACKUP_DIR="backups_pg18"
else
  export BACKUP_DIR="../backups_pg18"
fi

# =====
# 2. Parámetros de conexión: Clúster espacial en contenedor (db-postgis)
# =====
export DOCKER_PGHOST="db-postgis"
export DOCKER_PGPORT="5432"
```

```

export DOCKER_PGUSER="profe_unal"
export DOCKER_PGPASSWORD='geomatica2025'
export DOCKER_PGDATABASE="sig_db_unal"

# =====
# 3. Parámetros de conexión: Instancia física en el sistema anfitrión (host)
# =====
export LOCAL_PGHOST="host.docker.internal"
export LOCAL_PGPORT="5432"
export LOCAL_PGUSER="postgres"
export LOCAL_PGPASSWORD='geomatica2025'
export LOCAL_PGDATABASE="sig_db_unal"

# =====
# 4. Variables estándar libpq por defecto (apuntando al contenedor activo)
# =====
export PGHOST="${DOCKER_PGHOST}"
export PGPORT="${DOCKER_PGPORT}"
export PGUSER="${DOCKER_PGUSER}"
export PGPASSWORD="${DOCKER_PGPASSWORD}"
export PGDATABASE="${DOCKER_PGDATABASE}"

```

5.2 Estrategias de extracción de respaldos lógicos

La utilidad `pg_dump` extrae definiciones de esquemas, extensiones espaciales y registros tabulares en dos formatos principales:

Tabla 5.1: Formatos estándar de extracción lógica mediante `pg_dump`

Formato	Modificador	Extensión	Características técnicas
Texto plano (SQL)	<code>-F p</code>	<code>.sql</code>	Archivo de texto estructurado con sentencias DDL/DML legibles por el analizador sintáctico del motor.
Personalizado (<i>Custom</i>)	<code>-F c</code>	<code>.backup</code> / <code>.dump</code>	Formato binario comprimido de alta eficiencia que permite restauración paralela y selectiva.

5.2.1 Respaldo desde el clúster dockerizado

Para generar copias de respaldo de la base de datos `sig_db_unal` instanciada en el servicio `db-postgis`, se aplican las credenciales del rol docente (`profe_unal`).

5.2.1.1 Respaldo exclusivo de estructura (schema only)

Archivo ligero que contiene únicamente las definiciones de tablas, vistas, funciones, índices espaciales GiST, disparadores y extensiones, omitiendo los datos. Es la opción ideal para crear entornos de desarrollo, pruebas o versionar esquemas limpios.

```

export PGPASSWORD="${DOCKER_PGPASSWORD}"

pg_dump -h ${DOCKER_PGHOST} -p ${DOCKER_PGPORT} -U ${DOCKER_PGUSER} -d ${DOCKER_PGDATABASE} -s -F p -f
↳ "${BACKUP_DIR}/sig_db_unal_docker_estructura.sql" -P format=wrapped -P columns=90

```

Parámetros implementados:

- **-s:** Extrae exclusivamente el esquema (definición DDL de objetos sin datos).
- **-F p:** Formato texto plano (SQL legible).
- **-f:** Ruta y nombre del archivo de salida.

5.2.1.2 Respaldo exclusivo de datos (data only)

Archivo comprimido que contiene exclusivamente los registros de la base de datos y las geometrías vectoriales, sin sentencias DDL de creación de objetos.

```
export PGPASSWORD="${DOCKER_PGPASSWORD}"
pg_dump -h ${DOCKER_PGHOST} -p ${DOCKER_PGPORT} -U ${DOCKER_PGUSER} -d ${DOCKER_PGDATABASE} -a -F c -f
↳ "${BACKUP_DIR}/sig_db_unal_docker_datos.dump"
```

Parámetros implementados:

- **-a:** Extrae exclusivamente los datos (DML).
- **-F c:** Formato *Custom* (binario comprimido que permite restauración selectiva).

5.2.1.3 Respaldo maestro integral (master) - Recomendado

Copia maestra que incluye tanto la estructura como los datos. Constituye el archivo principal para planes de contingencia, portabilidad completa y recuperación ante desastres (*disaster recovery*).

```
export PGPASSWORD="${DOCKER_PGPASSWORD}"
pg_dump -h ${DOCKER_PGHOST} -p ${DOCKER_PGPORT} -U ${DOCKER_PGUSER} -d ${DOCKER_PGDATABASE} -N cron -F c
↳ -f "${BACKUP_DIR}/sig_db_unal_docker_completo.backup"
```

Parámetros implementados:

- **-F c:** Formato *Custom* (binario estructurado y comprimido, estándar para bases de datos geoespaciales modernas).
- **-N cron:** Excluye el esquema *cron* durante la extracción. El contenedor tiene instalada y habilitada la extensión *pg_cron* (planificador de tareas en segundo plano), mientras que la instancia del motor relacional en el sistema anfitrión (host) no cuenta con dicha librería compilada ni registrada en sus parámetros de servidor.

5.2.2 Respaldo desde la instancia en el sistema anfitrión

Cuando la base de datos *sig_db_unal* ha sido replicada o modificada directamente en la instancia física del sistema anfitrión, la extracción desde la terminal integrada de Visual Studio Code se ejecuta redirigiendo el host hacia *host.docker.internal* y utilizando el rol administrativo *postgres*:

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"
pg_dump -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} -F c -f
↳ "${BACKUP_DIR}/sig_db_unal_local_completo.backup"
```

5.3 Procedimientos de recuperación y restauración (sistema anfitrión)

La reconstrucción del catálogo espacial a partir de los respaldos generados se efectúa mediante dos escenarios operacionales estándar.

5.3.1 Escenario 1: restauración total desde backup completo

Es el método más rápido y seguro ante incidentes de pérdida de datos o migración hacia una nueva instancia.

1. Eliminación preventiva de la base de datos destino si ya existe:

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"
psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d postgres -c "DROP DATABASE IF EXISTS
↳ sig_db_unal_restaurada WITH (FORCE);"
```

DROP DATABASE

i Terminación forzada de sesiones concurrentes

La cláusula `WITH (FORCE)` asegura la terminación previa de cualquier sesión o conexión residual que haya quedado abierta contra `sig_db_unal_restaurada` durante la ejecución anterior, permitiendo la eliminación inmediata del catálogo.

2. Creación de la base de datos destino (vacía) en el clúster anfitrión:

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"
psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d postgres -c "CREATE DATABASE
↳ sig_db_unal_restaurada;"
```

CREATE DATABASE

3. Ejecución de la restauración integral con la utilidad binaria `pg_restore`:

Debido a que la extensión `pg_cron` no se encuentra instalada en el sistema anfitrión (host), `pg_restore` emite advertencias no bloqueantes y finaliza con un código de retorno 1. Por defecto, el motor de ejecución de Quarto/Knitr interpreta cualquier salida distinta de cero (`status != 0`) en un bloque `{bash}` como un fallo crítico, deteniendo la compilación del documento (`Execution halted`).

Para volcar la traza detallada en el reporte final sin interrumpir el renderizado ni desbordar los márgenes de página, se incorporan los siguientes mecanismos de control:

- **Directiva YAML `#| error: true`:** Instruye al motor Knitr para capturar las advertencias y flujos de error en el documento compilado sin suspender la ejecución del capítulo (ver Sección A.4).
- **Operador condicional `|| true`:** Fuerza a la subshell a retornar un código de salida 0 tras finalizar el comando, garantizando la continuidad del proceso de compilación.
- **Redirección y partición de línea (`2>&1 | fold -s -w 82`):** Redirige el flujo de error estándar (`stderr`, descriptor 2) hacia la salida estándar (`stdout`, descriptor 1) mediante `2>&1` para canalizar toda la

traza hacia la utilidad `fold`. El modificador `-w 82` restringe el ancho a 82 caracteres y `-s` asegura que la división ocurra en los espacios entre palabras, evitando cortes arbitrarios de texto y desbordamientos horizontales en la caja tipográfica del PDF.

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"
pg_restore -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d sig_db_unal_restaurada -v
↳ "${BACKUP_DIR}/sig_db_unal_docker_completo.backup" 2>&1 | fold -s -w 82 || true
```

```
pg_restore: connecting to database for restore
pg_restore: executing SELECT pg_catalog.set_config('search_path', '', false);
pg_restore: creating SCHEMA "base750"
pg_restore: creating EXTENSION "pg_cron"
pg_restore: while PROCESSING TOC:
pg_restore: from TOC entry 2; 3079 16387 EXTENSION pg_cron (no owner)
pg_restore: error: could not execute query: ERROR:  extension "pg_cron" is not
available
HINT:  The extension must first be installed on the system where PostgreSQL is
running.
Command was: CREATE EXTENSION IF NOT EXISTS pg_cron WITH SCHEMA pg_catalog;
```

```
pg_restore: creating COMMENT "EXTENSION pg_cron"
pg_restore: from TOC entry 6600; 0 0 COMMENT EXTENSION pg_cron
pg_restore: error: could not execute query: ERROR:  extension "pg_cron" does not
exist
Command was: COMMENT ON EXTENSION pg_cron IS 'Job scheduler for PostgreSQL';
```

```
pg_restore: creating SCHEMA "divpol"
pg_restore: creating SCHEMA "topology"
pg_restore: creating COMMENT "SCHEMA topology"
pg_restore: creating EXTENSION "hstore"
pg_restore: creating COMMENT "EXTENSION hstore"
pg_restore: creating EXTENSION "postgis"
pg_restore: creating COMMENT "EXTENSION postgis"
pg_restore: creating EXTENSION "pgrouting"
pg_restore: creating COMMENT "EXTENSION pgrouting"
pg_restore: creating EXTENSION "postgis_raster"
pg_restore: creating COMMENT "EXTENSION postgis_raster"
pg_restore: creating EXTENSION "postgis_topology"
pg_restore: creating COMMENT "EXTENSION postgis_topology"
pg_restore: creating FUNCTION "public.asbinary(public.geometry)"
pg_restore: creating FUNCTION "public.asbinary(public.geometry, text)"
pg_restore: creating FUNCTION "public.astext(public.geometry)"
```

```
pg_restore: creating FUNCTION "public.estimated_extent(text, text)"
pg_restore: creating FUNCTION "public.estimated_extent(text, text, text)"
pg_restore: creating FUNCTION "public.geomfromtext(text)"
pg_restore: creating FUNCTION "public.geomfromtext(text, integer)"
pg_restore: creating FUNCTION "public.ndims(public.geometry)"
pg_restore: creating FUNCTION "public.setsrid(public.geometry, integer)"
pg_restore: creating FUNCTION "public.srid(public.geometry)"
pg_restore: creating FUNCTION "public.st_asbinary(text)"
pg_restore: creating FUNCTION "public.st_astext(bytea)"
pg_restore: creating OPERATOR FAMILY "public.gist_geometry_ops"
pg_restore: creating OPERATOR CLASS "public.gist_geometry_ops"
pg_restore: creating TABLE "base750.hidro_lineas"
pg_restore: creating SEQUENCE "base750.hidro_lineas_gid_seq"
pg_restore: creating SEQUENCE OWNED BY "base750.hidro_lineas_gid_seq"
pg_restore: creating TABLE "base750.vias_lineas"
pg_restore: creating SEQUENCE "base750.vias_lineas_gid_seq"
pg_restore: creating SEQUENCE OWNED BY "base750.vias_lineas_gid_seq"
pg_restore: creating TABLE "base750.vias_puntos"
pg_restore: creating SEQUENCE "base750.vias_puntos_gid_seq"
pg_restore: creating SEQUENCE OWNED BY "base750.vias_puntos_gid_seq"
pg_restore: creating TABLE "base750.vias_texto"
pg_restore: creating SEQUENCE "base750.vias_texto_gid_seq"
pg_restore: creating SEQUENCE OWNED BY "base750.vias_texto_gid_seq"
pg_restore: creating TABLE "divpol.departamentos"
pg_restore: creating VIEW "divpol.capa_departamentos"
pg_restore: creating TABLE "divpol.municipios"
pg_restore: creating VIEW "divpol.capa_municipios"
pg_restore: creating SEQUENCE "divpol.departamentos_gid_seq"
pg_restore: creating SEQUENCE OWNED BY "divpol.departamentos_gid_seq"
pg_restore: creating SEQUENCE "divpol.municipios_gid_seq"
pg_restore: creating SEQUENCE OWNED BY "divpol.municipios_gid_seq"
pg_restore: creating TABLE "divpol.nacion"
pg_restore: creating SEQUENCE "divpol.nacion_gid_seq"
pg_restore: creating SEQUENCE OWNED BY "divpol.nacion_gid_seq"
pg_restore: creating DEFAULT "base750.hidro_lineas gid"
pg_restore: creating DEFAULT "base750.vias_lineas gid"
pg_restore: creating DEFAULT "base750.vias_puntos gid"
pg_restore: creating DEFAULT "base750.vias_texto gid"
pg_restore: creating DEFAULT "divpol.departamentos gid"
pg_restore: creating DEFAULT "divpol.municipios gid"
pg_restore: creating DEFAULT "divpol.nacion gid"
pg_restore: processing data for table "base750.hidro_lineas"
pg_restore: processing data for table "base750.vias_lineas"
```

```

pg_restore: processing data for table "base750.vias_puntos"
pg_restore: processing data for table "base750.vias_texto"
pg_restore: processing data for table "divpol.departamentos"
pg_restore: processing data for table "divpol.municipios"
pg_restore: processing data for table "divpol.nacion"
pg_restore: processing data for table "public.spatial_ref_sys"
pg_restore: processing data for table "topology.topology"
pg_restore: processing data for table "topology.layer"
pg_restore: executing SEQUENCE SET hidro_lineas_gid_seq
pg_restore: executing SEQUENCE SET vias_lineas_gid_seq
pg_restore: executing SEQUENCE SET vias_puntos_gid_seq
pg_restore: executing SEQUENCE SET vias_texto_gid_seq
pg_restore: executing SEQUENCE SET departamentos_gid_seq
pg_restore: executing SEQUENCE SET municipios_gid_seq
pg_restore: executing SEQUENCE SET nacion_gid_seq
pg_restore: executing SEQUENCE SET topology_id_seq
pg_restore: creating CONSTRAINT "base750.hidro_lineas hidro_lineas_pkey"
pg_restore: creating CONSTRAINT "base750.vias_lineas vias_lineas_pkey"
pg_restore: creating CONSTRAINT "base750.vias_puntos vias_puntos_pkey"
pg_restore: creating CONSTRAINT "base750.vias_texto vias_texto_pkey"
pg_restore: creating CONSTRAINT "divpol.departamentos departamentos_gid_key"
pg_restore: creating CONSTRAINT "divpol.departamentos departamentos_pkey"
pg_restore: creating CONSTRAINT "divpol.municipios municipios_gid_key"
pg_restore: creating CONSTRAINT "divpol.municipios municipios_pkey"
pg_restore: creating CONSTRAINT "divpol.nacion nacion_nac_nombre_key"
pg_restore: creating CONSTRAINT "divpol.nacion nacion_pkey"
pg_restore: creating INDEX "base750.hidro_lineas_the_geom_gist"
pg_restore: creating INDEX "base750.vias_lineas_the_geom_gist"
pg_restore: creating INDEX "base750.vias_puntos_the_geom_gist"
pg_restore: creating INDEX "base750.vias_texto_the_geom_gist"
pg_restore: creating INDEX "divpol.departamentos_the_geom_gist"
pg_restore: creating INDEX "divpol.municipios_the_geom_gist"
pg_restore: warning: errors ignored on restore: 2

```

Parámetros implementados:

- -v: Modo detallado (*verbose*) para monitorear el progreso de la reconstrucción de índices y tablas.

Advertencias en la restauración (errors ignored on restore: 2)

Durante la ejecución de la restauración, la consola puede reportar dos advertencias específicas asociadas a la extensión `pg_cron`:

1. ERROR: extension "pg_cron" is not available (al intentar ejecutar `CREATE EXTENSION IF NOT EXISTS pg_cron`).

2. ERROR: extension "pg_cron" does not exist (al intentar asociar el comentario DDL de la extensión).

Origen técnico: El clúster dockerizado de origen cuenta con la librería compilada de la extensión `pg_cron` (utilizada para la programación de tareas en segundo plano), la cual no se encuentra instalada por defecto en el motor relacional del sistema anfitrión (host).

Criterio de validación: Estas advertencias son **no bloqueantes** y pueden ignorarse con total seguridad. Su alcance se limita de forma exclusiva a las funciones del planificador de tareas. La totalidad de los esquemas cartográficos (`base750`, `divpol`), las extensiones espaciales (`postgis`, `postgis_raster`, `postgis_topology`), los registros tabulares, los catálogos OGC (`geometry_columns`, `spatial_ref_sys`) y los índices espaciales GiST se restauran de manera íntegra y funcional.

5.3.2 Escenario 2: restauración por componentes (estructura y datos)

Este método se utiliza si se requiere auditar o modificar la estructura DDL antes de cargar los datos o si se están integrando datos en un esquema ya existente.

1. Eliminación preventiva de la base de datos destino si ya existe en el clúster:

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"

# Cierre forzado de conexiones residuales y remoción del catálogo
psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d postgres -c "DROP DATABASE IF EXISTS
↳ sig_db_unal_restaurada WITH (FORCE);"
```

DROP DATABASE

2. Creación de la base de datos destino (vacía) en el clúster anfitrión:

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"

# Aprovisionamiento del contenedor relacional receptor
psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d postgres -c "CREATE DATABASE
↳ sig_db_unal_restaurada;"
```

CREATE DATABASE

3. Cargar la estructura DDL:

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"

# Inyección de directivas DDL: esquemas, extensiones, tablas, vistas e índices espaciales
psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d sig_db_unal_restaurada -f
↳ "${BACKUP_DIR}/sig_db_unal_docker_estructura.sql" 2>&1 | fold -s -w 82 || true
```

SET

SET

SET

SET

SET

SET

```
set_config
```

```
-----
```

```
(1 row)
```

```
SET
```

```
SET
```

```
SET
```

```
SET
```

```
CREATE SCHEMA
```

```
ALTER SCHEMA
```

```
psql:../backups_pg18/sig_db_unal_docker_estructura.sql:35: ERROR:  extension
```

```
"pg_cron" is not available
```

```
HINT:  The extension must first be installed on the system where PostgreSQL is  
running.
```

```
psql:../backups_pg18/sig_db_unal_docker_estructura.sql:42: ERROR:  extension
```

```
"pg_cron" does not exist
```

```
CREATE SCHEMA
```

```
ALTER SCHEMA
```

```
CREATE SCHEMA
```

```
ALTER SCHEMA
```

```
COMMENT
```

```
CREATE EXTENSION
```

```
COMMENT
```

```
CREATE EXTENSION
```

```
COMMENT
```

```
CREATE EXTENSION
```

```
COMMENT
```

```
CREATE EXTENSION
```

```
COMMENT
```

```
CREATE EXTENSION
```

```
COMMENT
```

```
CREATE FUNCTION
```

```
ALTER FUNCTION
```

```
CREATE FUNCTION
```

```
ALTER FUNCTION
```

```
CREATE FUNCTION
```

```
ALTER FUNCTION
```

```
CREATE FUNCTION
```

```
ALTER FUNCTION
```

```
CREATE FUNCTION
```

```
ALTER FUNCTION
```

```
CREATE FUNCTION
```

ALTER FUNCTION
CREATE FUNCTION
ALTER FUNCTION
CREATE FUNCTION
ALTER FUNCTION
CREATE FUNCTION
ALTER FUNCTION
CREATE FUNCTION
ALTER FUNCTION
CREATE FUNCTION
ALTER FUNCTION
CREATE FUNCTION
ALTER FUNCTION
CREATE OPERATOR FAMILY
ALTER OPERATOR FAMILY
ALTER OPERATOR FAMILY
CREATE OPERATOR CLASS
ALTER OPERATOR CLASS
SET
SET
CREATE TABLE
ALTER TABLE
CREATE SEQUENCE
ALTER SEQUENCE
ALTER SEQUENCE
CREATE TABLE
ALTER TABLE
CREATE SEQUENCE
ALTER SEQUENCE
ALTER SEQUENCE
CREATE TABLE
ALTER TABLE
CREATE SEQUENCE
ALTER SEQUENCE
ALTER SEQUENCE
CREATE TABLE
ALTER TABLE
CREATE SEQUENCE
ALTER SEQUENCE
ALTER SEQUENCE
CREATE TABLE
ALTER TABLE
CREATE VIEW

[illegible]

4. Cargar los datos desactivando temporalmente disparadores y restricciones:

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"

# Inserción masiva de tuplas con suspensión de llaves foráneas y disparadores
```

```
pg_restore -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d sig_db_unal_restaurada -a -v
↳ --disable-triggers "${BACKUP_DIR}/sig_db_unal_docker_datos.dump" 2>&1 | fold -s -w 82 || true
```

```
pg_restore: connecting to database for restore
pg_restore: executing SELECT pg_catalog.set_config('search_path', '', false);
pg_restore: disabling triggers for hidro_lineas
pg_restore: processing data for table "base750.hidro_lineas"
pg_restore: enabling triggers for hidro_lineas
pg_restore: disabling triggers for vias_lineas
pg_restore: processing data for table "base750.vias_lineas"
pg_restore: enabling triggers for vias_lineas
pg_restore: disabling triggers for vias_puntos
pg_restore: processing data for table "base750.vias_puntos"
pg_restore: enabling triggers for vias_puntos
pg_restore: disabling triggers for vias_texto
pg_restore: processing data for table "base750.vias_texto"
pg_restore: enabling triggers for vias_texto
pg_restore: disabling triggers for job
pg_restore: while PROCESSING TOC:
pg_restore: from TOC entry 6359; 0 16390 TABLE DATA job profe_unal
pg_restore: error: could not execute query: ERROR:  schema "cron" does not exist
Command was: ALTER TABLE cron.job DISABLE TRIGGER ALL;

pg_restore: processing data for table "cron.job"
pg_restore: error: could not execute query: ERROR:  schema "cron" does not exist
Command was: COPY cron.job (jobid, schedule, command, nodename, nodeport,
database, username, active, jobname) FROM stdin;
pg_restore: enabling triggers for job
pg_restore: error: could not execute query: ERROR:  schema "cron" does not exist
Command was: ALTER TABLE cron.job ENABLE TRIGGER ALL;

pg_restore: disabling triggers for job_run_details
pg_restore: from TOC entry 6361; 0 16417 TABLE DATA job_run_details profe_unal
pg_restore: error: could not execute query: ERROR:  schema "cron" does not exist
Command was: ALTER TABLE cron.job_run_details DISABLE TRIGGER ALL;

pg_restore: processing data for table "cron.job_run_details"
pg_restore: error: could not execute query: ERROR:  schema "cron" does not exist
Command was: COPY cron.job_run_details (jobid, runid, job_pid, database,
username, command, status, return_message, start_time, end_time) FROM stdin;
pg_restore: enabling triggers for job_run_details
pg_restore: error: could not execute query: ERROR:  schema "cron" does not exist
Command was: ALTER TABLE cron.job_run_details ENABLE TRIGGER ALL;
```

```

pg_restore: disabling triggers for departamentos
pg_restore: processing data for table "divpol.departamentos"
pg_restore: enabling triggers for departamentos
pg_restore: disabling triggers for municipios
pg_restore: processing data for table "divpol.municipios"
pg_restore: enabling triggers for municipios
pg_restore: disabling triggers for nacion
pg_restore: processing data for table "divpol.nacion"
pg_restore: enabling triggers for nacion
pg_restore: disabling triggers for spatial_ref_sys
pg_restore: processing data for table "public.spatial_ref_sys"
pg_restore: enabling triggers for spatial_ref_sys
pg_restore: disabling triggers for topology
pg_restore: processing data for table "topology.topology"
pg_restore: enabling triggers for topology
pg_restore: disabling triggers for layer
pg_restore: processing data for table "topology.layer"
pg_restore: enabling triggers for layer
pg_restore: executing SEQUENCE SET hidro_lineas_gid_seq
pg_restore: executing SEQUENCE SET vias_lineas_gid_seq
pg_restore: executing SEQUENCE SET vias_puntos_gid_seq
pg_restore: executing SEQUENCE SET vias_texto_gid_seq
pg_restore: executing SEQUENCE SET jobid_seq
pg_restore: from TOC entry 6590; 0 0 SEQUENCE SET jobid_seq profe_unal
pg_restore: error: could not execute query: ERROR:  relation "cron.jobid_seq"
does not exist
LINE 1: SELECT pg_catalog.setval('cron.jobid_seq', 1, false);
      ^
Command was: SELECT pg_catalog.setval('cron.jobid_seq', 1, false);

```

```

pg_restore: executing SEQUENCE SET runid_seq
pg_restore: from TOC entry 6591; 0 0 SEQUENCE SET runid_seq profe_unal
pg_restore: error: could not execute query: ERROR:  relation "cron.runid_seq"
does not exist
LINE 1: SELECT pg_catalog.setval('cron.runid_seq', 1, false);
      ^
Command was: SELECT pg_catalog.setval('cron.runid_seq', 1, false);

```

```

pg_restore: executing SEQUENCE SET departamentos_gid_seq
pg_restore: executing SEQUENCE SET municipios_gid_seq

```

```
pg_restore: executing SEQUENCE SET nacion_gid_seq
pg_restore: executing SEQUENCE SET topology_id_seq
pg_restore: warning: errors ignored on restore: 8
```

Parámetros implementados:

- **-a:** Restringe la operación exclusivamente a la inserción de registros (DML).
- **-v:** Activa la salida detallada en consola para auditar el progreso tabla por tabla.
- **--disable-triggers:** Optimiza la velocidad de inserción y suspende temporalmente las restricciones de integridad referencial (*foreign keys*).

⚠ Advertencias en la restauración desacoplada (errors ignored on restore: 8)

Durante la ejecución del paso DDL (paso 3), la consola reporta que la extensión **pg_cron** no se encuentra instalada en el motor del sistema anfitrión. En consecuencia, el paso DML (paso 4) genera 8 advertencias no bloqueantes asociadas de forma exclusiva a la imposibilidad de deshabilitar disparadores, copiar datos y calibrar secuencias sobre las tablas **cron.job** y **cron.job_run_details**.

Estas advertencias pueden ignorarse con total seguridad: la totalidad de las capas geográficas (**base750**, **divpol**), las extensiones PostGIS, las geometrías vectoriales y los índices espaciales GiST quedan restaurados de forma completa y funcional.

5. Sincronización de secuencias e identificadores autonuméricos:

Aunque el archivo binario incluye las directivas **setval** para las tablas respaldadas, en procedimientos de restauración desacoplada se debe ejecutar la siguiente rutina procedural en PL/pgSQL para recalibrar los generadores numéricos de todas las tablas registradas y prevenir fallos de clave duplicada (**duplicate key value**) en transacciones posteriores:

```
DO $$
DECLARE
    seq_record RECORD;
    max_id BIGINT;
BEGIN
    FOR seq_record IN
        SELECT s.relname AS seq_name, n.nspname AS schema_name, t.relname AS table_name, a.attname AS
            ↪ column_name
        FROM pg_class s
        JOIN pg_namespace n ON n.oid = s.relnamespace
        JOIN pg_depend d ON d.objid = s.oid
        JOIN pg_class t ON d.refobjid = t.oid
        JOIN pg_attribute a ON (d.refobjid, d.refobjsubid) = (a.attrelid, a.attnum)
        WHERE s.relkind = 'S'
    LOOP
        EXECUTE format('SELECT MAX(%I) FROM %I.%I', seq_record.column_name, seq_record.schema_name,
            ↪ seq_record.table_name) INTO max_id;
        IF max_id IS NOT NULL THEN
            EXECUTE format('SELECT setval('%I.%I', %L)', seq_record.schema_name, seq_record.seq_name,
            ↪ max_id + 1);
        END IF;
    END LOOP;
END $$;
```

5.4 Auditoría de integridad posterior a la restauración

Finalizada la reconstrucción del catálogo, se procede a validar la operatividad de las extensiones espaciales, la correcta inscripción de capas en los metadatos de la Open Geospatial Consortium (OGC) y la consistencia

volumétrica de las tablas en la base de datos restaurada.

Para facilitar el flujo de trabajo según el contexto pedagógico, los procedimientos se organizan en dos modalidades secuenciales independientes:

- **Modalidad 1: Auditoría automatizada mediante CLI (#| eval: true):** Bloques de ejecución secuencial desatendida desde la terminal integrada de Visual Studio Code, permitiendo la evaluación y renderizado directo de los resultados durante la compilación del documento.
- **Modalidad 2: Auditoría interactiva en sesión de psql (#| eval: false):** Flujo de trabajo continuo dentro de una única sesión interactiva del cliente de terminal, permitiendo autenticarse al inicio y ejecutar todas las sentencias SQL sin alternar contextos de conexión.

5.4.1 Auditoría automatizada desatendida mediante CLI

Esta secuencia ejecuta consultas directas mediante el modificador `-c`, evaluando el estado del socket, los controladores de PostGIS, el catálogo OGC y las métricas de almacenamiento.

1. Verificación del socket relacional y diagnóstico de conexión:

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"
psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d sig_db_unal_restaurada -c "\conninfo"
```

Connection Information	
Parameter	Value
Database	sig_db_unal_restaurada
Client User	postgres
Host	host.docker.internal
Host Address	192.168.65.254
Server Port	5432
Options	
Protocol Version	3.0
Password Used	true
GSSAPI Authenticated	false
Backend PID	2720
SSL Connection	false
Superuser	on
Hot Standby	off
(13 rows)	

2. Comprobación del núcleo PostGIS y enlace de librerías geoespaciales:

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"
psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d sig_db_unal_restaurada -x -P
↳ format=wrapped -P columns=90 -c "SELECT postgis_full_version();"

```

```

-[ RECORD 1 ]-----+-----
postgis_full_version | POSTGIS="3.6.2 3.6.2" [EXTENSION] PGSQL="180" GEOS="3.14.1dev-
CAPI.
                        |.-1.20.4" PROJ="8.2.1 NETWORK_ENABLED=OFF URL_ENDPOINT=https://cdn..
                        |.proj.org USER_WRITABLE_DIRECTORY=C:\WINDOWS\ServiceProfiles\Networ.
                        |.kService\AppData\Local\proj DATABASE_PATH=D:\Programs\PostgreSQL\1.
                        |.8\share\contrib\postgis-3.6\proj\proj.db" (compiled against PROJ 8.
                        |..2.1) GDAL="GDAL 3.9.2, released 2024/08/13" LIBXML="2.12.5" LIBJS.
                        |.ON="0.12" LIBPROTOBUF="1.2.1" WAGYU="0.5.0 (Internal)" TOPOLOGY RA.
                        |.STER

```

3. Inspección del catálogo formal de capas geométricas OGC:

```

export PGPASSWORD="${LOCAL_PGPASSWORD}"

query_geom="
SELECT
    f_table_schema AS esquema,
    f_table_name AS capa,
    f_geometry_column AS columna_geom,
    coord_dimension AS dimension,
    srid,
    type AS tipo_geometria
FROM geometry_columns
WHERE f_table_schema IN ('base750', 'divpol')
ORDER BY f_table_schema, f_table_name;
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d sig_db_unal_restaurada -x -P
    ↪ format=wrapped -P columns=90 -c "${query_geom}"

```

```

-[ RECORD 1 ]--+-+-----
esquema      | base750
capa         | hidro_lineas
columna_geom | the_geom
dimension    | 2
srid         | 21892
tipo_geometria | GEOMETRY
-[ RECORD 2 ]--+-+-----
esquema      | base750
capa         | vias_lineas
columna_geom | the_geom
dimension    | 2
srid         | 21892
tipo_geometria | GEOMETRY
-[ RECORD 3 ]--+-+-----
esquema      | base750
capa         | vias_puntos
columna_geom | the_geom
dimension    | 2
srid         | 21892
tipo_geometria | ST_Point

```

```

-[ RECORD 4 ]--+-----
esquema      | base750
capa         | vias_texto
columna_geom | the_geom
dimension    | 2
srid         | 21892
tipo_geometria | ST_Point
-[ RECORD 5 ]--+-----
esquema      | divpol
capa         | capa_departamentos
columna_geom | the_geom
dimension    | 2
srid         | 0
tipo_geometria | GEOMETRY
-[ RECORD 6 ]--+-----
esquema      | divpol
capa         | capa_municipios
columna_geom | the_geom
dimension    | 2
srid         | 0
tipo_geometria | GEOMETRY
-[ RECORD 7 ]--+-----
esquema      | divpol
capa         | departamentos
columna_geom | the_geom
dimension    | 2
srid         | 21892
tipo_geometria | GEOMETRY
-[ RECORD 8 ]--+-----
esquema      | divpol
capa         | municipios
columna_geom | the_geom
dimension    | 2
srid         | 21892
tipo_geometria | GEOMETRY

```

4. Auditoría de volumetría y conteo de registros en tablas de usuario:

```

export PGPASSWORD="${LOCAL_PGPASSWORD}"

query_conteo="
SELECT
    schemaname AS esquema,
    relname AS tabla,
    n_live_tup AS registros
FROM pg_stat_user_tables
WHERE schemaname IN ('base750', 'divpol')
ORDER BY schemaname, relname;
"

```

```
psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d sig_db_unal_restaurada -x -P
↳ format=wrapped -P columns=90 -c "${query_conteo}"
```

```
-[ RECORD 1 ]-----
esquema      | base750
tabla        | hidro_lineas
registros    | 124068
-[ RECORD 2 ]-----
esquema      | base750
tabla        | vias_lineas
registros    | 13399
-[ RECORD 3 ]-----
esquema      | base750
tabla        | vias_puntos
registros    | 6819
-[ RECORD 4 ]-----
esquema      | base750
tabla        | vias_texto
registros    | 6645
-[ RECORD 5 ]-----
esquema      | divpol
tabla        | departamentos
registros    | 32
-[ RECORD 6 ]-----
esquema      | divpol
tabla        | municipios
registros    | 1121
-[ RECORD 7 ]-----
esquema      | divpol
tabla        | nacion
registros    | 1
```

5.4.2 Auditoría interactiva en cliente psql

Esta modalidad permite abrir una única sesión en el motor relacional y ejecutar de manera consecutiva las directivas de inspección y consultas analíticas.

1. Apertura de la sesión interactiva en la base de datos restaurada:

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"
psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d sig_db_unal_restaurada
```

Si ya se dispone de una sesión activa de psql en otra base de datos del clúster, se ejecuta el cambio de contexto con el metacomando de conexión:

```
\c sig_db_unal_restaurada
```

2. Comprobación del estado y enlace de librerías de PostGIS:

```
SELECT postgis_full_version();
```

3. Inspección del catálogo formal de geometrías OGC:

```
SELECT
  f_table_schema AS esquema,
  f_table_name AS capa,
  f_geometry_column AS columna_geom,
  coord_dimension AS dimension,
  srid,
  type AS tipo_geometria
FROM geometry_columns
WHERE f_table_schema IN ('base750', 'divpol')
ORDER BY f_table_schema, f_table_name;
```

4. Auditoría de volumetría y conteo de registros por tabla:

```
SELECT
  schemaname AS esquema,
  relname AS tabla,
  n_live_tup AS registros
FROM pg_stat_user_tables
WHERE schemaname IN ('base750', 'divpol')
ORDER BY schemaname, relname;
```

5. Cierre de la sesión interactiva:

Para finalizar la interacción y retornar al intérprete de la terminal integrada de Visual Studio Code:

```
\q
```


Parte II

Fundamentos y diseño de bases de datos espaciales

Capítulo 6

Arquitectura de bases de datos espaciales y estándares OGC

6.0.1 Terminal integrada (psql)

Para la interacción interactiva mediante la consola de comandos o la terminal integrada de Visual Studio Code, se configuran las variables de entorno estándar de libpq para apuntar al clúster correspondiente. Ver el fundamento técnico en la Sección 4.3.

```
# 1. Parámetros para la instancia física en el sistema anfitrión (host local)
export LOCAL_PGHOST="host.docker.internal"
export LOCAL_PGPORT="5432"
export LOCAL_PGUSER="postgres"
export LOCAL_PGPASSWORD='geomatica2025'
export LOCAL_PGDATABASE="sig_db_unal"

# 2. Parámetros para la instancia dockerizada (red bridge interna)
export DOCKER_PGHOST="db-postgis"
export DOCKER_PGPORT="5432"
export DOCKER_PGUSER="profe_unal"
export DOCKER_PGPASSWORD='geomatica2025'
export DOCKER_PGDATABASE="sig_db_unal"

# 3. Asignación de variables estándar de libpq por defecto (apuntando al host local)
export PGHOST="${LOCAL_PGHOST}"
export PGPORT="${LOCAL_PGPORT}"
export PGUSER="${LOCAL_PGUSER}"
export PGPASSWORD="${LOCAL_PGPASSWORD}"
export PGDATABASE="${LOCAL_PGDATABASE}"

# 4. Verificación de endpoints configurados en la sesión
echo "Endpoint anfitrión (host): ${LOCAL_PGUSER}@${LOCAL_PGHOST}:${LOCAL_PGPORT}/${LOCAL_PGDATABASE}"
echo "Endpoint contenedor (docker):"
→  ${DOCKER_PGUSER}@${DOCKER_PGHOST}:${DOCKER_PGPORT}/${DOCKER_PGDATABASE}"
echo "Sesión vinculada por defecto: ${PGUSER}@${PGHOST}:${PGPORT}/${PGDATABASE}"
```

Endpoint anfitrión (host): postgres@host.docker.internal:5432/sig_db_unal

Endpoint contenedor (docker): profe_unal@db-postgis:5432/sig_db_unal

Sesión vinculada por defecto: postgres@host.docker.internal:5432/sig_db_unal

6.0.2 Python (SQLAlchemy, Pandas y GeoPandas)

Aprovisionamiento del entorno virtual en Conda:

Para la gestión de conexiones relacionales, ejecución de consultas SQL y manipulación de DataFrames alfanuméricos y espaciales en Python, se requiere las siguientes librerías disponibles en el contenedor del curso:

- **sqlalchemy**: Capa de abstracción de base de datos para la gestión de motores y grupos de conexiones (*connection pooling*).
- **psycopg2**: Controlador nativo (*driver*) en C para la comunicación bajo el protocolo cliente-servidor de PostgreSQL.
- **pandas**: Procesamiento y visualización de tuplas y metadatos tabulares relacionales.
- **geopandas**: Extensión geoespacial de Pandas que deserializa geometrías en WKB/WKT hacia estructuras GeoSeries y GeoDataFrame.
- **ipython-sql**: Extensión mágica (`%sql`, `%%sql`) para la ejecución declarativa directa de bloques SQL en celdas interactivas.

Instanciación global de motores y cadenas de conexión:

```
import os
from sqlalchemy import create_engine

# 1. Parámetros y motor para la instancia anfitriona (host local)
host_local = os.getenv("LOCAL_PGHOST", "host.docker.internal")
port_local = os.getenv("LOCAL_PGPORT", "5432")
user_local = os.getenv("LOCAL_PGUSER", "postgres")
pass_local = os.getenv("LOCAL_PGPASSWORD", "geomatica2025")
db_local = os.getenv("LOCAL_PGDATABASE", "sig_db_unal")

cadena_local = f"postgresql+psycopg2://{user_local}:{pass_local}@{host_local}:{port_local}/{db_local}"
engine_local = create_engine(cadena_local)

# 2. Parámetros y motor para la instancia en red interna (Docker)
host_docker = os.getenv("DOCKER_PGHOST", "db-postgis")
port_docker = os.getenv("DOCKER_PGPORT", "5432")
user_docker = os.getenv("DOCKER_PGUSER", "profe_unal")
pass_docker = os.getenv("DOCKER_PGPASSWORD", "geomatica2025")
db_docker = os.getenv("DOCKER_PGDATABASE", "sig_db_unal")

cadena_docker = f"postgresql+psycopg2://{user_docker}:{pass_docker}@{host_docker}:{port_docker}/{db_docker}"
engine_docker = create_engine(cadena_docker)

# Verificación de endpoints sin credenciales enmascaradas
print(
    "Motor local instanciado:\n "
    f"{engine_local.url.render_as_string(hide_password=False)}"
)
```

Motor local instanciado:

```
postgresql+psycopg2://postgres:geomatica2025@host.docker.internal:5432/sig_db_unal
```

```
print(
    "Motor docker instanciado:\n "
    f"{engine_docker.url.render_as_string(hide_password=False)}"
)
```

Motor docker instanciado:

```
postgresql+psycopg2://profe_unal:geomatica2025@db-postgis:5432/sig_db_unal
```

6.0.3 pgAdmin 4

i Entorno de ejecución interactivo: Las consultas y sentencias DDL/DML de este capítulo asumen que la sesión de terminal o el entorno de ejecución de Python disponen de las variables de conexión estándar debidamente configuradas. Si ejecuta los bloques de forma manual en la consola, verifique la asignación de parámetros descrita en la Sección 4.3.

6.1 Objetivos del capítulo

- Analizar los **fundamentos** conceptuales de las bases de datos y evaluar sus **ventajas** determinantes frente a los sistemas de archivos tradicionales.
- Comprender la **arquitectura** funcional, los **niveles de abstracción** y los subsistemas operativos de un sistema gestor de bases de datos (SGBD).
- Determinar el **modelo de datos y la arquitectura** de integración óptimos para soportar aplicaciones de sistemas de información geográfica (SIG).
- Examinar la implementación de los **estándares** internacionales del Open Geospatial Consortium (OGC) e ISO en motores relacionales extendidos.

6.2 De sistemas de archivos a bases de datos relacionales

El manejo de información en organizaciones y proyectos de geomática ha transitado históricamente desde esquemas basados en sistemas de archivos independientes hacia arquitecturas centralizadas administradas por **sistemas gestores de bases de datos (SGBD)** (*Database Management System*, DBMS).

Un sistema de archivos tradicional almacena los datos en estructuras cerradas dependientes de aplicaciones específicas (archivos planos, hojas de cálculo o formatos propietarios). Este enfoque presenta deficiencias estructurales críticas:

- **Redundancia e inconsistencia:** La duplicación descontrolada de archivos genera discrepancias en el estado de la información cuando se ejecutan actualizaciones parciales o no coordinadas.
- **Aislamiento y fragmentación de datos:** La dispersión de formatos y codificaciones heterogéneas impide la interoperabilidad y dificulta la formulación de consultas analíticas integradas.
- **Dependencia programa-datos:** Las aplicaciones se acoplan rígidamente a la estructura física de almacenamiento, lo que exige reescribir el software de consumo ante cualquier cambio en el formato de los archivos.
- **Vulnerabilidad transaccional y concurrencia deficiente:** Carencia de mecanismos ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad), lo que provoca condiciones de carrera, sobrescritura destructiva y corrupción de datos ante fallos del sistema o accesos simultáneos.
- **Costos elevados de mantenimiento y gobierno:** La ausencia de esquemas y catálogos normalizados fragmenta las políticas de seguridad, integridad y respaldo.

Frente a estas restricciones, una **base de datos** se define como una colección **integrada, estructurada y no redundante** de datos almacenados en soporte **persistente**, accesible **concurrentemente** por múltiples usuarios y aplicaciones mediante una estricta separación entre la representación **lógica** y la implementación **física**. Por su parte, el **sistema gestor de bases de datos (SGBD)** (*Database Management System*,

DBMS) constituye la infraestructura de **software** responsable de proveer las interfaces y mecanismos para la **definición, manipulación, optimización, persistencia y administración** integral de dicha información.

Cuando el motor relacional se extiende con soporte nativo para tipos **geométricos** y **geográficos**, predicados **topológicos** y métodos de **indexación multidimensional**, se conforma un **SGBD espacial** (*Spatial Database Management System*, SDBMS). En este texto se adoptan como sinónimos técnicos los términos **base de datos geográfica**, **base de datos espacial** o **GeoDB** para designar cualquier repositorio estructurado con soporte nativo de tipos espaciales. Por convención académica, el término comercial **geodatabase** se reserva exclusivamente para las estructuras de almacenamiento propietarias del ecosistema ESRI. Las definiciones formales, los invariantes del modelo y la terminología complementaria se consolidan en el Apéndice H.

Para comprender con rigor esta arquitectura, es necesario distinguir entre el plano **técnico** de habilitación y el principio **arquitectónico** fundamental. Extender el motor relacional con tipos geométricos, índices multidimensionales y predicados topológicos constituye el medio técnico instrumental. No obstante, la verdadera ruptura conceptual de un SGBD espacial radica en la **integración y centralización de esquemas**: la unificación de la geometría como un objeto de primera clase dentro del núcleo transaccional.

Como se sintetiza en la Figura 6.1, el motor gestor consolida en un repositorio común **atributos descriptivos**, **geometrías vectoriales tipadas** y **metadatos de referencia espacial**, gobernados por un **catálogo centralizado** y un **subsistema de integridad transaccional** accesible por múltiples clientes en red.

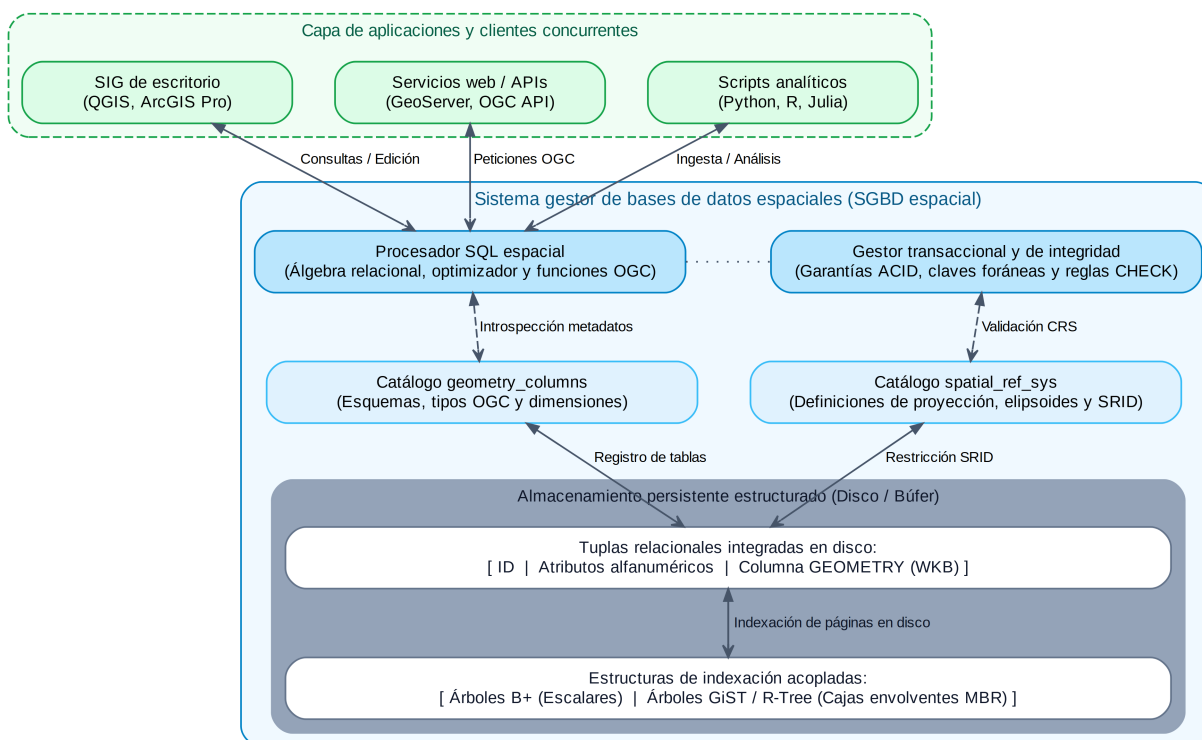


Figura 6.1: Centralización e integración de datos espaciales y alfanuméricos en un SGBD espacial.

6.2.1 Componentes y funciones de un SGBD

La arquitectura de un SGBD desacopla el almacenamiento físico de la interfaz lógica del usuario para garantizar concurrencia, rendimiento e integridad de la información.

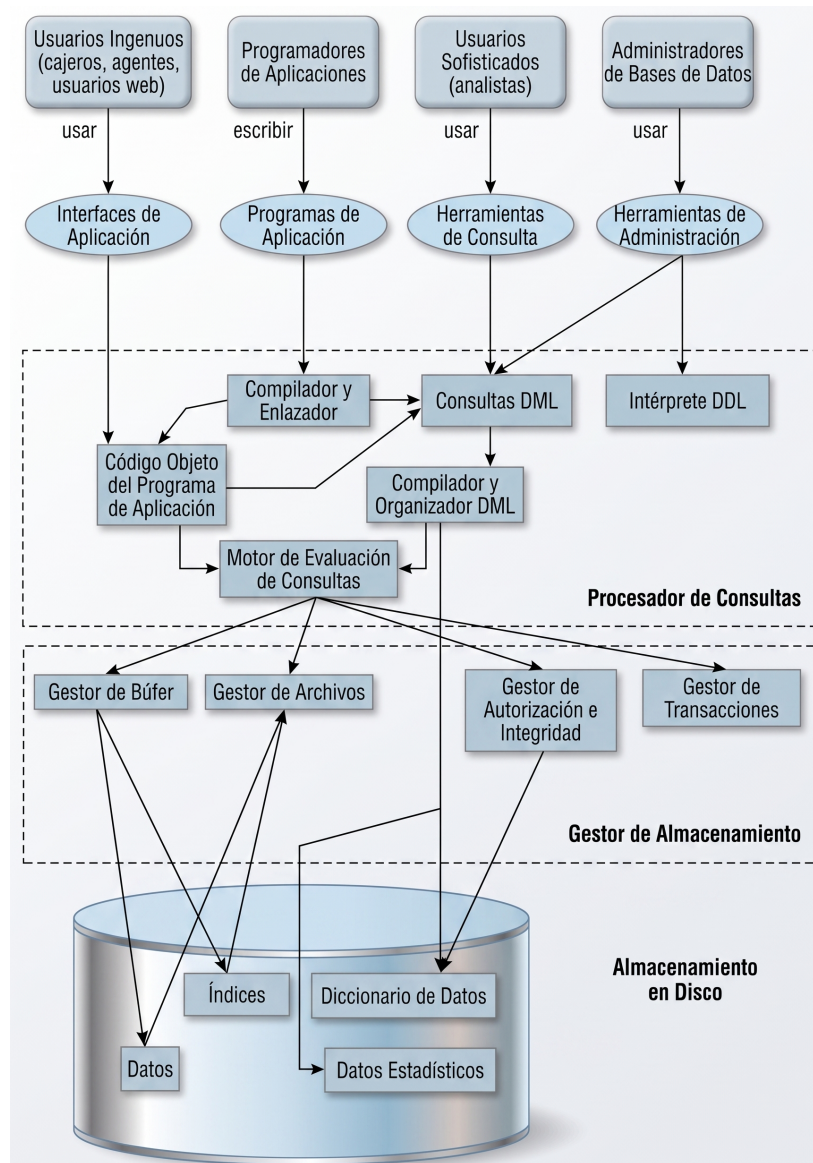


Figura 6.2: Componentes de un sistema de bases de datos, según Silberschatz, Korth y Sudarshan (2002).

En la Figura 6.2 se presenta la arquitectura clásica funcional de un SGBD estructurada en tres capas operativas fundamentales: el **procesador de consultas**, el **gestor de almacenamiento** y el **almacenamiento en disco**.

Procesador de consultas:

El **procesador de consultas** constituye el subsistema responsable de transformar instrucciones declarativas de alto nivel (ej. sentencias SQL donde se especifica qué información se requiere y no el algoritmo para buscarla) en planes algebraicos de ejecución de bajo costo, coordinando su despacho hacia el gestor de almacenamiento.

Las sentencias SQL ingresan a este procesador a través de dos vías fundamentales: como **SQL estático embebido**, donde las consultas forman parte integral del código fuente (ej. programas en C, C++ o COBOL) de un programa de aplicación (ej. un software GIS de escritorio o un sistema catastral desarrollado a medida) y son analizadas durante la fase de compilación y enlazado para generar código objeto ejecutable (ej. código binario de máquina vinculado a librerías cliente dinámicas `.dll` o `.so`); o como **SQL dinámico**, donde las sentencias se generan, concatenan o parametrizan al vuelo durante el tiempo de ejecución desde herramientas interactivas o interfaces de usuario. En ambos casos, el procesador debe interpretar la sintaxis, validar los esquemas contra el catálogo, optimizar el árbol de operadores relacionales y ejecutar las instrucciones físicas resultantes.

- **Intérprete DDL:** Procesa las sentencias de definición de datos y registra las definiciones correspondientes en el diccionario de datos (vínculo que la arquitectura conceptual formaliza directamente, aunque se omita la línea en el diagrama de Silberschatz).
- **Compilador y organizador DML:** Traduce las sentencias DML formuladas en lenguajes de consulta a un plan de evaluación de bajo nivel compuesto por operadores relacionales.
- **Motor de evaluación de consultas:** Ejecuta las instrucciones de bajo nivel generadas por el compilador DML interactuando directamente con el gestor de almacenamiento.
- **Compilador y enlazador de programas de aplicación:** Integra las llamadas a funciones DML embebidas en el código fuente con el motor de evaluación para generar código objeto ejecutable.

Gestor de almacenamiento:

El **gestor de almacenamiento** es el responsable de proveer la interfaz operativa entre los datos de bajo nivel almacenados en el medio secundario persistente y las instrucciones lógicas despachadas por el motor de evaluación de consultas. Su función cardinal radica en abstraer las complejidades del hardware y del sistema de archivos del sistema operativo, traduciendo las peticiones de tuplas, geometrías e índices en transferencias eficientes de entrada/salida (I/O) de bloques o páginas hacia la memoria RAM. Al mismo tiempo, este módulo coordina la sincronización de accesos concurrentes, salvaguarda la durabilidad y consistencia transaccional mediante bitácoras en disco, y valida que las operaciones de inserción, actualización o consulta cumplan con las directivas de seguridad y las restricciones de dominio e integridad referencial definidas en el catálogo.

- **Gestor de autorización e integridad:** Valida las restricciones de integridad declarativas y comprueba los privilegios de acceso de los usuarios a partir de los datos estadísticos y el diccionario.
- **Gestor de transacciones:** Garantiza que la base de datos permanezca en un estado consistente a pesar de fallos del sistema, coordinando la ejecución concurrente sin interferencias.
- **Gestor de archivos:** Asigna y gestiona el espacio en el almacenamiento secundario y las estructuras de datos usadas para representar los registros.
- **Gestor de búfer:** Controla la transferencia de páginas entre la memoria volátil intermedia (*buffer pool*) y el almacenamiento en disco persistente.

Estructuras de almacenamiento en disco:

Las **estructuras de almacenamiento** en disco constituyen el estrato de persistencia física no volátil del SGBD, encargado de alojar de forma permanente tanto los registros de información como las estructuras de control requeridas para el funcionamiento del motor. Este nivel organiza el almacenamiento secundario en bloques o páginas de tamaño fijo, desacoplando la representación lógica de las tablas de su almacenamiento físico subyacente. En este sustrato conviven no solo los datos crudos del usuario y las geometrías serializadas,

sino también los métodos de acceso acelerado (árboles B+, índices espaciales GiST o R-Tree), los catálogos del sistema que formalizan los esquemas e invariantes del modelo, y los metadatos estadísticos empleados por el optimizador de consultas para estimar costos de entrada y salida (I/O).

- **Datos:** Bloques físicos que alojan los registros de la base de datos.
- **Índices:** Estructuras de acceso rápido para acelerar la localización de tuplas específicas.
- **Diccionario de datos:** Almacén centralizado de metadatos sobre la estructura lógica y esquemas del sistema.
- **Datos estadísticos:** Información cuantitativa del catálogo sobre la distribución de las relaciones, empleada por el optimizador de consultas.

En la Tabla 6.1 se detalla la correspondencia entre los módulos de la arquitectura de Silberschatz, su nivel operativo y su responsabilidad fundamental dentro del motor de base de datos.

Tabla 6.1: Componentes principales del SGBD según Silberschatz y sus responsabilidades funcionales.

Componente	Nivel funcional	Responsabilidad principal
Intérprete DDL	Procesamiento lógico	Procesamiento de sentencias de definición y actualización de metadatos.
Compilador y organizador DML	Procesamiento lógico	Traducción de consultas de alto nivel a planes de ejecución evaluables.
Motor de evaluación de consultas	Ejecución lógica	Orquestación y despacho de instrucciones de bajo nivel al motor de almacenamiento.
Gestor de autorización e integridad	Almacenamiento / Control	Validación de restricciones declarativas y verificación de permisos.
Gestor de transacciones	Transaccional	Preservación de propiedades ACID y control de operaciones simultáneas.
Gestor de búfer	Almacenamiento físico	Paginación y caché de bloques de memoria entre disco y RAM.
Gestor de archivos	Almacenamiento físico	Asignación de espacio físico y direccionamiento de bloques de registros e índices.

Como marco comparativo, Elmasri y Navathe (2007) plantean una descomposición modular con mayor detalle en los subsistemas de ejecución de transacciones y el procesador de base de datos en tiempo de ejecución (*runtime database processor*), tal como se ilustra en la Figura 6.3.

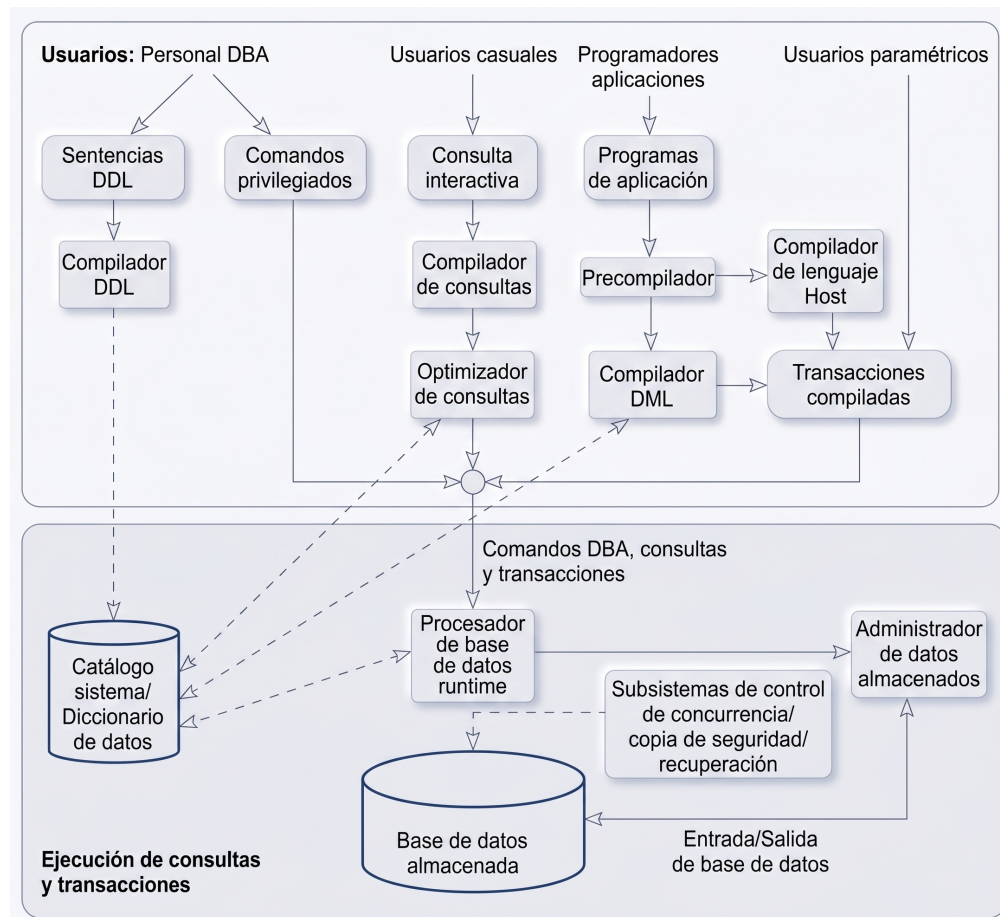


Figura 6.3: Arquitectura de componentes y ejecución de transacciones, según Elmasri y Navathe (2007).

6.2.2 Topología cliente-servidor y tránsito en la pila de red

Los sistemas gestores de bases de datos objeto-relacionales operan primordialmente bajo una **arquitectura cliente-servidor**. En este esquema, el motor de base de datos se ejecuta como un proceso demonio en segundo plano (*daemon process*, tal como el ejecutable **postgres**) aislado en su propio espacio de direcciones de memoria virtual, a la escucha de solicitudes entrantes a través de un puerto de red (por convención, el puerto TCP 5432) o un socket de dominio UNIX local.

Cualquier interacción ejecutada desde una aplicación cliente (un SIG de escritorio como QGIS, un servicio web como GeoServer o un script analítico en Python, R o Julia) requiere atravesar la frontera de aislamiento del sistema operativo mediante mecanismos de comunicación interprocesos (IPC) o protocolos de red. Al contrastar este flujo con el modelo de referencia **OSI** (*Open Systems Interconnection*), cada consulta SQL y cada conjunto de geometrías recuperadas deben franquear las siete capas funcionales:

- **Capa 7 (Aplicación):** La aplicación cliente formula la petición y el motor la procesa mediante un protocolo especializado a nivel de base de datos (como el protocolo *Frontend/Backend v3.0* de PostgreSQL o especificaciones HTTP/REST en OGC API).
- **Capa 6 (Presentación):** Se resuelve la serialización y codificación de los datos. En bases de datos espaciales, esto implica la traducción bidireccional entre las estructuras en memoria de librerías de cálculo (como GEOS) y formatos de intercambio estándar sobre la red (*Well-Known Binary* - WKB o *Extended WKB*), verificando la ordenación de bytes (*endianness*) y el juego de caracteres (UTF-8).
- **Capa 5 (Sesión):** Controla el establecimiento, autenticación criptográfica (ej. SCRAM-SHA-256), mantenimiento y cierre de la sesión de trabajo, administrando variables de entorno dinámicas como el `search_path` o el tiempo de espera transaccional.
- **Capa 4 (Transporte):** Segmentación y control de flujo mediante el protocolo **TCP**. Cada paquete se asocia unívocamente a un puerto de origen efímero en el cliente y al **puerto de destino** del gestor (5432), garantizando la entrega ordenada, libre de errores y con confirmación de recepción (*handshake* y ACKs).
- **Capa 3 (Red):** Empaquetamiento lógico y direccionamiento mediante datagramas **IP**. Define el enrutamiento entre interfaces físicas heterogéneas, redes locales corporativas o puentes de red virtualizados en contenedores Docker (`db-postgis`, `host.docker.internal`).
- **Capa 2 (Enlace de datos):** Estructuración en tramas (*frames*) Ethernet a nivel de direcciones físicas MAC o mediante controladores virtuales de bucle invertido (*loopback interface* `lo0` / `127.0.0.1`).
- **Capa 1 (Física):** Transmisión binaria de señales eléctricas, ópticas o electromagnéticas sobre el medio guiado, o bien transferencias de ráfagas a través del bus de datos y controladores del sistema operativo cuando cliente y servidor coexisten en el mismo equipo físico.

Esta separación física y lógica impone que cualquier intercambio de tuplas espaciales demande ciclos continuos de serialización, llamadas al sistema (*syscalls*) y cambios de contexto, lo cual representa el costo operativo necesario para garantizar concurrencia multiusuario, control centralizado y gobierno estricto de los datos.

6.2.3 Importancia y características de los SGBD

En la infraestructura tecnológica contemporánea, los SGBD constituyen el núcleo operativo para garantizar la persistencia, disponibilidad e integridad de los activos de información. Su arquitectura supera la fragilidad intrínseca de los sistemas de archivos mediante la implementación de niveles formales de abstracción de datos

(esquemas físico, lógico y de vistas, formalizados en la Sección 6.2.5), lo que garantiza el desacoplamiento estricto entre las aplicaciones cliente y las modificaciones en las estructuras de almacenamiento subyacentes.

Las propiedades funcionales y arquitectónicas que distinguen a un SGBD frente a soluciones ad-hoc (a la medida) comprenden:

- **Independencia lógica y física:** Capacidad de modificar la organización física de los archivos o el diseño conceptual de las tablas sin requerir la reescritura de las aplicaciones que consumen la base de datos.
- **Soporte de tipado abstracto y dominios complejos:** Definición rigurosa de tipos primitivos (escalares, numéricos, temporales) y estructurados (JSON, tipos geométricos/geográficos OGC, grandes objetos binarios o BLOB).
- **Procesamiento declarativo y optimización de consultas:** Interpretación de sentencias en SQL estándar mediante un optimizador basado en costos (CBO), el cual formula planes de ejecución eficientes a partir de la distribución estadística de los datos.
- **Mecanismos de concurrencia y garantías ACID:** Aislamiento de transacciones concurrentes para evitar anomalías como lecturas sucias, actualizaciones perdidas y lecturas no repetibles.
- **Estructuras de acceso optimizado:** Implementación de índices univariados y multidimensionales (árboles B+, GiST, SP-GiST, R-Trees) que reducen exponencialmente los costos de entrada/salida (I/O) en operaciones de filtrado y agregación.
- **Tolerancia a fallos y recuperación:** Persistencia transaccional apoyada en registros de auditoría y bitácoras de operaciones (*Write-Ahead Logging*), asegurando la reconstrucción del estado consistente del sistema ante interrupciones no programadas de energía o caídas del kernel.
- **Seguridad y auditoría granular:** Control de accesos basado en roles (RBAC), mecanismos de autenticación criptográfica, cifrado en reposo y en tránsito, y políticas de seguridad a nivel de fila (*Row-Level Security*).

6.2.4 Perfiles de usuario y responsabilidades del administrador

La arquitectura canónica formaliza cuatro categorías de usuarios en función de su nivel de interacción técnica y el grado de abstracción con el que operan:

- **Usuarios ingenuos o paramétricos:** Interactúan con el sistema de forma indirecta mediante interfaces de usuario precompiladas y programas de aplicación (formularios web, aplicaciones móviles o terminales transaccionales), ejecutando operaciones predefinidas sin conocer la sintaxis SQL ni el modelo de datos subyacente.
- **Programadores de aplicaciones:** Desarrollan artefactos de software en lenguajes anfitriones (Python, Java, C++, Julia), embebiendo sentencias DML o invocando interfaces de programación (ODBC, JDBC, APIs nativas) para conectar las aplicaciones con el motor de base de datos.
- **Usuarios sofisticados o analistas:** Interactúan directamente con el procesador de consultas mediante herramientas interactivas, consolas SQL o plataformas de ciencia de datos, formulando consultas analíticas complejas y extrayendo conjuntos de datos sin intermediación de código de aplicación estático.
- **Usuarios especializados:** Diseñan e implementan sistemas de bases de datos no tradicionales, tales como motores de bases de conocimiento, procesadores analíticos multidimensionales o sistemas de información espacial avanzados.

Dentro de este ecosistema, el **administrador de bases de datos (DBA)** representa la figura de control centralizado encargada del ciclo de vida operativo, el rendimiento y la seguridad del sistema gestor. Sus responsabilidades comprenden:

- **Definición del esquema conceptual y físico:** Redacción de especificaciones en lenguaje DDL para la creación del modelo relacional, las cuales el compilador DDL interpreta y almacena en el diccionario de datos.
- **Selección de métodos de acceso y estructuras de almacenamiento:** Definición de la topología de almacenamiento físico, asignación de espacios de tablas (*tablespaces*), tamaño de páginas de disco y diseño de índices primarios y secundarios.
- **Modificación y afinamiento físico (*tuning*):** Ajuste dinámico de parámetros del gestor de búferes, memoria intermedia y reestructuración periódica de índices para mitigar cuellos de botella según el comportamiento de la carga de trabajo.
- **Gestión de autorizaciones y directivas de seguridad:** Creación de roles de usuario, asignación granular de privilegios sobre esquemas o tablas, y administración de credenciales bajo el principio de mínimo privilegio.
- **Supervisión de restricciones de integridad:** Especificación de reglas declarativas (claves primarias, integridad referencial, restricciones de dominio) para que el gestor de integridad impida estados inconsistentes.
- **Mantenimiento preventivo, respaldo y contingencia:** Definición de políticas de copias de seguridad (completas, incrementales y continuas), monitorización del almacenamiento secundario y ejecución periódica de simulacros de recuperación ante desastres.

6.2.5 Niveles de abstracción e independencia de datos

Un sistema gestor de bases de datos abstrae la complejidad del almacenamiento secundario para presentar una representación simplificada de la información. La arquitectura estándar de tres niveles (ANSI/SPARC) formaliza este principio mediante capas operativas independientes, como se ilustra en la Figura 6.4.

Las tres capas estructuran responsabilidades específicas:

1. **Nivel físico o interno:** Describe la organización técnica en que los registros se estructuran en el disco: distribución de páginas de datos, métodos de compresión, rutas de acceso físicas y estructuras de indexación secundarias (árboles B+ o árboles R/GiST en contextos espaciales).
2. **Nivel conceptual o lógico:** Modela la estructura integral del dominio de datos con independencia de los detalles del hardware de almacenamiento. En este nivel se definen las entidades, atributos, dominios (incluyendo tipos geométricos OGC), restricciones de integridad (**CHECK**, claves primarias, llaves foráneas) y relaciones lógicas entre conjuntos de datos.
3. **Nivel de visión o externo:** Define representaciones parciales y personalizadas del esquema lógico orientadas a distintos perfiles de usuario, roles operativos o módulos de software mediante vistas relacionales (**VIEW**) o políticas de seguridad a nivel de filas y columnas.

En la Tabla 6.2 se sintetizan las responsabilidades de cada estrato arquitectónico, los perfiles de usuario involucrados y su manifestación técnica en motores geoespaciales.

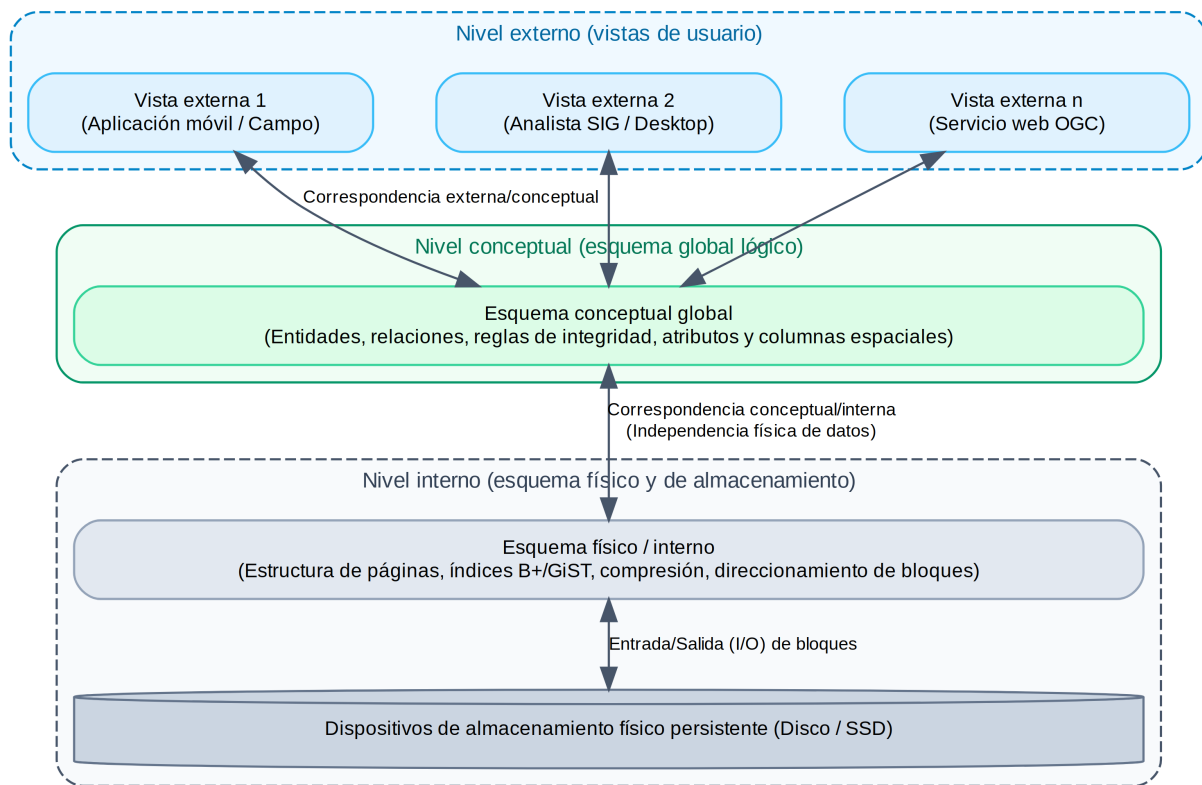


Figura 6.4: Arquitectura ANSI/SPARC de tres niveles y correspondencias de independencia de datos.

Tabla 6.2: Niveles de abstracción arquitectónica y su instanciación técnica.

Nivel de abstracción	Audiencia principal	Mecanismos de implementación	Instanciación espacial (SGBD espacial)
Visión (externo)	Usuarios finales y analistas	Vistas SQL (VIEW, MATERIALIZED VIEW), permisos de rol.	Capas filtradas por cobertura territorial o temática.
Conceptual (lógico)	Administrador de datos y diseñador	Esquemas DDL, restricciones relacionales, tipos de datos.	Tipos de datos espaciales (GEOMETRY, GEOGRAPHY, SRID).
Físico (interno)	DBA y motor de almacenamiento	Asignación de páginas en disco, bloques físicos, WAL.	Codificación binaria (WKB) e índices espaciales (GiST, SP-GiST).

La separación desacoplada entre estos estratos garantiza dos propiedades operativas fundamentales:

- **Independencia física de datos:** Capacidad de reestructurar la disposición de almacenamiento, cambiar de medio físico, modificar el particionamiento de tablas o crear nuevos índices en disco sin requerir ajustes en el esquema conceptual ni reescribir las sentencias de consulta de las aplicaciones cliente.
- **Independencia lógica de datos:** Capacidad de expandir o modificar el esquema conceptual (añadir nuevas columnas, entidades o relocalizar tablas) sin alterar las aplicaciones que consumen vistas externas preexistentes, siempre que estas no dependan de los elementos modificados.

6.3 Evolución de modelos de datos y la dimensión espacial

A lo largo del desarrollo de los sistemas de información, los modelos de bases de datos se han estructurado en torno a paradigmas que responden tanto a las limitaciones del hardware de cada época como a la complejidad semántica de los datos:

- **Modelos basados en registros y punteros físicos (Tipo 1):** Dominantes en las décadas de 1950 y 1960 (modelos jerárquicos y de red), donde las asociaciones entre entidades se implementaban mediante enlaces físicos de memoria o apuntadores en disco.
- **Modelo relacional (Tipo 2):** Introducido por E. F. Codd (Codd (1970)), fundamentado en la teoría de conjuntos y la lógica de predicados de primer orden. La información se modela en relaciones (tablas compuestas por tuplas y atributos atómicos), y las vinculaciones se realizan exclusivamente a través de valores escalares compartidos (claves primarias y foráneas).
- **Modelos objeto-relacionales y orientados a objetos (Tipo 3):** Formulados a partir de 1990 para superar la incapacidad del modelo relacional plano al manejar datos no atómicos. Introducen encapsulamiento, herencia, métodos y soporte nativo para tipos abstractos de datos (ADT), habilitando la persistencia e indexación de geometrías espaciales y tipos de datos multidimensionales.
- **Modelos no relacionales y de grafos (NoSQL):** Consolidados a partir de 2010 para resolver cuellos de botella de escalabilidad horizontal y rigidez de esquemas. Reemplazan la normalización formal por esquemas flexibles de lectura (*schema-on-read*) divididos en cuatro familias: documentales

(JSON/BSON), clave-valor, familias de columnas (*wide-column*) y grafos de propiedades, estos últimos formalizados mediante el estándar internacional ISO/IEC 39075:2024 (GQL).

- **Evolución contemporánea (NewSQL, multimodelo y bases de datos vectoriales):** Desde 2012, el modelo relacional tradicional derivó hacia sistemas distribuidos elásticos con consistencia transaccional estricta mediante algoritmos de consenso (NewSQL). De forma paralela, surgieron *motores convergentes multimodelo* y *bases de datos vectoriales* optimizadas para búsqueda de similitud semántica mediante grafos de mundo pequeño (*HNSW*) sobre representaciones densas de alta dimensionalidad (*embeddings*).

En la Figura 6.5 se resume la trayectoria evolutiva de estos paradigmas, destacando la bifurcación entre sistemas orientados a transacciones ACID y soluciones diseñadas para escalabilidad distribuida o cómputo multidimensional.

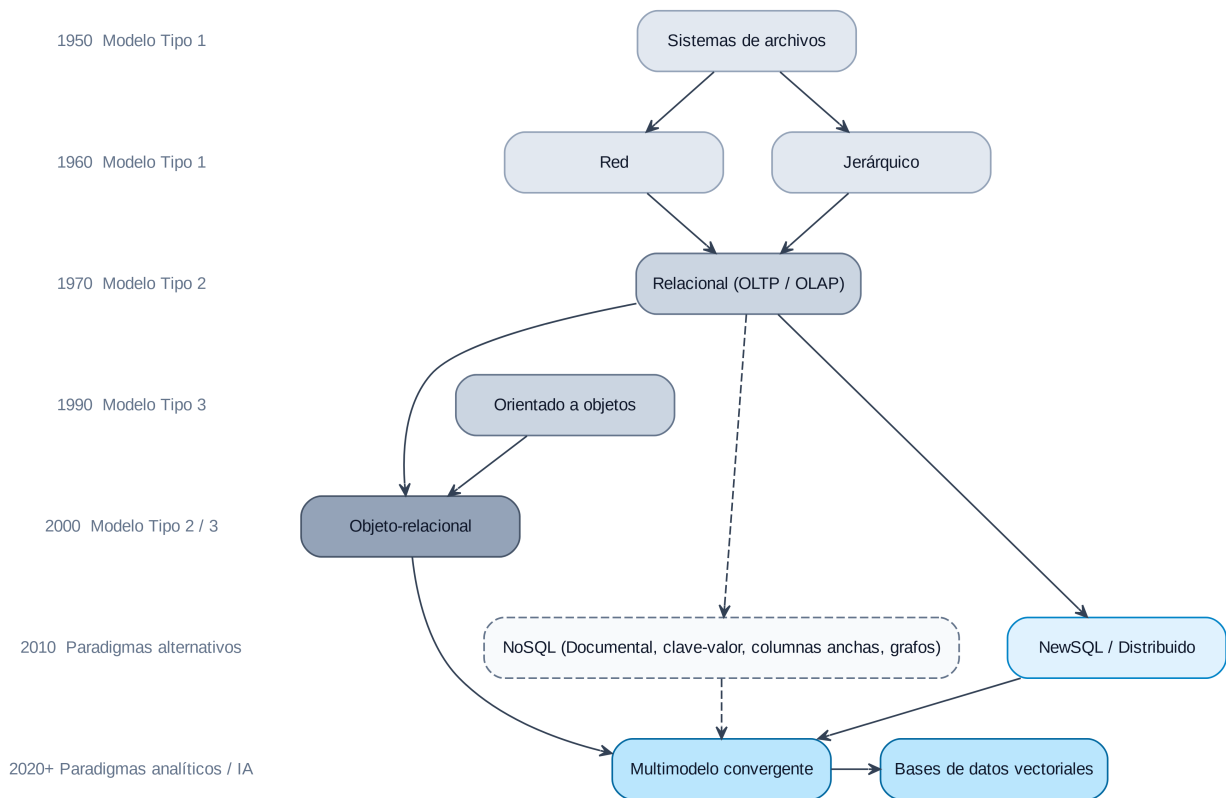


Figura 6.5: Cronología de modelos de bases de datos con alineación temporal formal.

En la Tabla 6.3 se sistematizan los motores de bases de datos representativos de la disciplina, estructurados conforme a los paradigmas arquitectónicos presentados.

Tabla 6.3: Clasificación de sistemas gestores de bases de datos según su cronología formal.

Modelo arquitectónico	Época / Taxonomía	Sistemas y motores representativos	Mecanismo de estructuración predominante
Sistemas de archivos y métodos de acceso	Década de 1950 (Tipo 1)	IBM ISAM, VSAM, archivos secuenciales	Registros mapeados por <i>offset</i> físico en disco o cinta.
Modelo jerárquico	Década de 1960 (Tipo 1)	IBM IMS (<i>Information Management System</i>)	Estructuras en árbol con relaciones 1:N mediante punteros físicos padre-hijo.
Modelo en red	Década de 1960 (Tipo 1)	IDS, Cullinet IDMS (estándar CODASYL)	Grafos de registros con relaciones N:M gestionadas mediante conjuntos (<i>sets</i>) y apuntadores en anillo.
Modelo relacional transaccional (OLTP)	Década de 1970 (Tipo 2)	IBM System R, Ingres, IBM DB2, MySQL, SQLite	Tablas estructuradas por filas con tuplas y atributos atómicos, gobernadas por álgebra relacional y cumplimiento ACID.
Orientado a objetos (OODBMS)	Década de 1990 (Tipo 3)	ObjectStore, db4o, Versant, Objectivity/DB	Persistencia directa de instancias del lenguaje anfitrión con encapsulamiento, identidad propia (OID) y métodos.
Objeto-relacional (ORDBMS)	Década de 2000 (Tipo 2/3)	PostgreSQL (+ PostGIS), Oracle (v8i+), IBM Informix	Núcleo relacional con soporte nativo de tipos abstractos (ADT), polimorfismo, herencia de tablas e índices extensibles (GiST/SP-GiST).
NoSQL: Documental	Década de 2010	MongoDB, Apache CouchDB	Registros semiestructurados jerárquicos (JSON, BSON) con esquema dinámico (<i>schema-on-read</i>).

Modelo arquitectónico	Época / Taxonomía	Sistemas y motores representativos	Mecanismo de estructuración predominante
NoSQL: Grafos de propiedades	Década de 2010	Neo4j, Amazon Neptune, Memgraph	Nodos, aristas y propiedades tipadas optimizados para recorridos de relaciones complejas sin operaciones de combinación (<i>joins</i>).
NoSQL: Clave-valor	Década de 2010	Redis, Amazon DynamoDB, Memcached	Almacén asociativo indexado por <i>hash</i> unidimensional de muy baja latencia en memoria o disco.
NoSQL: Familias de columnas (<i>wide-column</i>)	Década de 2010	Apache Cassandra, Apache HBase, ScyllaDB	Tablas multidimensionales dispersas y particionadas horizontalmente, optimizadas para escrituras masivas distribuidas.
NewSQL / SQL distribuido	Década de 2010 (2012+)	Google Cloud Spanner, CockroachDB, YugabyteDB, TiDB	Álgebra relacional estándar y transaccionalidad ACID global sobre clústeres elásticos mediante protocolos de consenso (Paxos/Raft).
Modelo relacional analítico columnar (OLAP)	Década de 2010 (2019+)	DuckDB, ClickHouse, MonetDB	Álgebra relacional y sintaxis SQL estándar implementada sobre almacenamiento columnar y ejecución vectorial en memoria/CPU (SIMD).
Multimodelo nativo	Década de 2010 (2015+)	ArangoDB, Microsoft Azure Cosmos DB	Motores concebidos de origen para procesar de forma unificada grafos, documentos, clave-valor y estructuras tabulares.

Modelo arquitectónico	Época / Taxonomía	Sistemas y motores representativos	Mecanismo de estructuración predominante
Bases de datos vectoriales	Década de 2020 (2020+)	Milvus, Qdrant, Pinecone, Weaviate, Chroma	Almacenamiento e indexación de <i>embeddings</i> multidimensionales mediante grafos HNSW o cuantización vectorial para cálculo de distancias métricas.

6.3.1 Coexistencia paradigmática y persistencia políglota

La trayectoria histórica de los modelos de datos no representa una secuencia de sustitución u obsolescencia tecnológica. El principio de **persistencia políglota** establece que las organizaciones deben seleccionar motores heterogéneos según la naturaleza del problema y las demandas del procesamiento.

Un motor relacional objeto-relacional (ORDBMS) con cumplimiento estricto de propiedades ACID continúa siendo el componente irremplazable en sistemas operacionales y catastrales donde la consistencia de las mutaciones concurrentes no es negociable. En contraste, las arquitecturas distribuidas, columnares o vectoriales resuelven problemas analíticos, elásticos o de inteligencia artificial que desbordarían los mecanismos de un motor relacional tradicional.

En la Tabla 6.4 se contrastan los objetivos operativos, compromisos de diseño y el rol contemporáneo de cada paradigma dentro del ecosistema geoespacial.

Tabla 6.4: Objetivos operativos, compromisos arquitectónicos y vigencia de los modelos de datos en entornos geoespaciales.

Paradigma de datos	Objetivo operativo primordial	Compromisos de diseño (<i>trade-offs</i>)	Rol vigente en geomática y sistemas espaciales
Relacional transaccional y objeto-relacional (OLTP / ORDBMS)	Integridad transaccional estricta (ACID), soporte de esquemas complejos normalizados y consultas ad-hoc expresivas.	Escalabilidad horizontal compleja de escritura; sobrecarga de sincronización de páginas en disco.	Núcleo institucional primario: Catastro, registro predial, redes de infraestructura y repositorios corporativos gobernados por PostGIS u Oracle Spatial.
Relacional analítico columnar (OLAP)	Agregaciones masivas, análisis temporal y escaneos exhaustivos de grandes volúmenes proyectando pocos atributos.	Ineficiente para mutaciones puntuales en tiempo real (UPDATE/DELETE fila a fila); ausencia de transacciones concurrentes complejas.	Analítica espacial local y cubos de datos: Procesamiento vectorial embebido de series de tiempo y nubes de puntos mediante DuckDB o ClickHouse.

Paradigma de datos	Objetivo operativo primordial	Compromisos de diseño (<i>trade-offs</i>)	Rol vigente en geomática y sistemas espaciales
NoSQL: Documental y clave-valor	Baja latencia, esquemas dinámicos flexibles (<i>schema-on-read</i>) y alta disponibilidad con escalado horizontal sencillo.	Ausencia de relaciones formales (<i>joins</i> costosos o inexistentes); eventual consistencia de datos (modelo BASE).	Caché y microservicios cartográficos: Almacenamiento rápido de metadatos no estructurados, catálogos GeoJSON y caché de teselas web (Redis, MongoDB).
NoSQL: Grafos de propiedades	Recorrido recursivo de relaciones densas y complejas sin operaciones costosas de combinación (<i>index-free adjacency</i>).	Ineficiente para escaneos masivos tabulares o agregaciones globales sobre atributos escalares.	Topología y conectividad espacial: Análisis de redes de transporte, flujo en cuencas hidrográficas y trazabilidad de suministro eléctrico (Neo4j).
NewSQL / SQL distribuido	Consistencia transaccional ACID estricta distribuida sobre clústeres elásticos globales mediante protocolos de consenso.	Mayor latencia en transacciones trans-nodo por coordinación de quórum; mayor complejidad de infraestructura.	Sistemas espaciales de escala planetaria: Plataformas con concurrencia masiva global y demanda de disponibilidad ininterrumpida (CockroachDB, Spanner).
Bases de datos vectoriales	Búsqueda aproximada de vecinos más cercanos (ANN) por similitud métrica sobre espacios densos de alta dimensionalidad.	No soportan lógica booleana relacional estándar ni predicados topológicos OGC exactos sobre geometrías físicas.	Búsqueda semántica en observación de la Tierra: Recuperación de teselas satelitales por contenido visual e indexación de <i>embeddings</i> espectrales (Milvus, pgvector).

6.3.2 La dimensión espacial como catalizador del modelo objeto-relacional

La adopción de modelos de bases de datos en geomática estuvo condicionada por la complejidad intrínseca de los datos geográficos frente a los datos alfanuméricos tradicionales. Esta complejidad se manifiesta en cuatro dimensiones críticas:

- **Multidimensionalidad:** Las entidades espaciales poseen extensión geométrica en dos, tres o cuatro dimensiones (coordenadas X , Y , elevación Z , y valores de medición lineal M).

- **Topología y continuidad:** Los objetos en el territorio exhiben relaciones espaciales continuas (adyacencia, contención, solapamiento, conectividad) que no se reducen a operadores de comparación escalar ($=$, $<$, $>$).
- **Tipologías heterogéneas:** Coexisten entidades vectoriales discretas (puntos, líneas, polígonos) y campos continuos (matrices ráster y mallas irregulares).
- **Ineficiencia de árboles de búsqueda unidimensionales:** Los árboles B y B+ son incapaces de indexar eficazmente cajas envolventes mínimas (*Bounding Boxes*) o rangos multidimensionales.

Como se sintetiza en la Figura 6.6, la evolución arquitectónica respondió a la necesidad de gestionar simultáneamente atributos escalares simples y geometrías dotadas de comportamiento topológico.

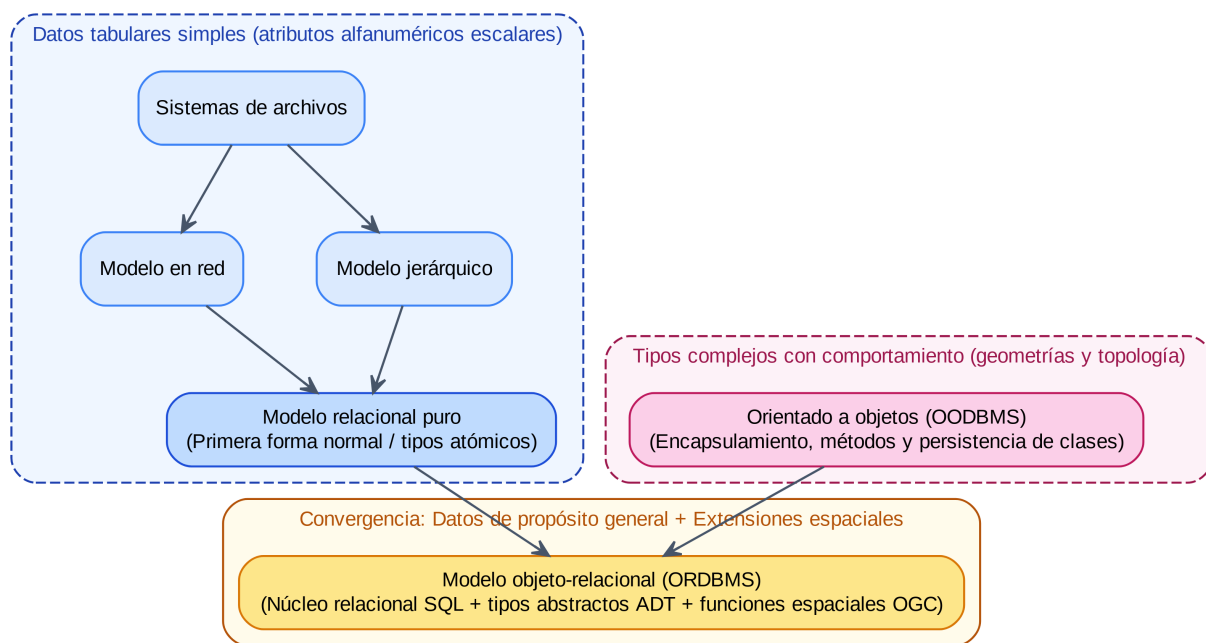


Figura 6.6: Convergencia de requerimientos de datos hacia el modelo objeto-relacional.

El modelo relacional tradicional garantizaba integridad referencial, consistencia transaccional y optimización sobre atributos atómicos escalares, pero carecía de mecanismos nativos para operar geometrías complejas sin desarticularlas en colecciones de coordenadas normalizadas. Por su parte, los sistemas orientados a objetos (OODBMS) permitían modelar geometrías y operadores topológicos como métodos de clase, pero sacrificaban la madurez del álgebra relacional, el lenguaje declarativo SQL y la eficiencia de los optimizadores basados en costos.

El modelo objeto-relacional resolvió esta encrucijada al extender el motor relacional con tipos de datos abstractos (ADT), índices espaciales multidimensionales (árboles R sobre infraestructura GiST) y predicados topológicos evaluables dentro del planificador de consultas.

6.3.3 Consideraciones arquitectónicas: extensibilidad y motores de cómputo analítico

Al analizar el ecosistema de datos espaciales y analíticos, es necesario distinguir entre la arquitectura de núcleo de un SGBD, los mecanismos de convergencia multimodelo por extensibilidad y los motores de procesamiento distribuido sobre lagos de datos:

- **Estatus arquitectónico de PostgreSQL y su convergencia multimodelo:** PostgreSQL es en su diseño de núcleo un sistema objeto-relacional (ORDBMS). La extensibilidad provista por su arquitectura permitió implementar soporte para datos semiestructurados (JSONB), tipos espaciales avanzados e índices multidimensionales mediante PostGIS, y búsqueda por similitud vectorial mediante `pgvector`. En sus versiones beta más recientes (como PostgreSQL 19), esta convergencia se amplía de forma nativa al incorporar consultas de grafos de propiedades mediante el estándar SQL/PGQ y capacidades avanzadas de optimización y asesoría de planes (`pg_plan_advice`). De este modo, aunque su fundamento teórico no es multimodelo nativo, opera en la práctica como un sustrato convergente altamente versátil para cargas de trabajo heterogéneas.
- **Diferenciación funcional de DuckDB frente a la arquitectura cliente-servidor:** DuckDB no plantea un nuevo modelo algebraico de datos, sino una reconfiguración radical de la arquitectura de ejecución y enlace. Se clasifica dentro del modelo relacional (Tipo 2), pero sustituye el almacenamiento tradicional orientado a filas (*row-oriented*) por almacenamiento columnar vectorial en memoria. Su verdadera ruptura radica en su modelo de despliegue embebido (*in-process*): el motor se compila como una biblioteca directamente vinculada dentro del mismo espacio de direcciones del proceso anfitrión (Python, R, Julia o C++). Al suprimir la arquitectura cliente-servidor, el motor elimina la totalidad del tránsito por la pila de protocolos OSI (puertos TCP, llamadas al sistema, sockets y serialización de geometrías a WKB), permitiendo el intercambio directo de punteros en memoria RAM mediante estándares columnares como Apache Arrow.
- **Distinción entre SGBD y motores de cómputo analítico (Apache Sedona):** Soluciones como Apache Sedona (anteriormente GeoSpark) no constituyen sistemas gestores de bases de datos (SGBD). Carecen de motores de almacenamiento persistente propios, gestores de transacciones ACID, subsistemas de bitácora (*Write-Ahead Logging*) y gestores de búfer en disco. Sedona es un **motor de procesamiento espacial distribuido** que actúa como capa analítica sobre entornos de cómputo en clúster (Apache Spark y Apache Flink), permitiendo evaluar predicados espaciales e indexación en memoria sobre archivos externos (GeoParquet, Shapefiles o capas de lagos de datos).

En la Tabla 6.5 se contrastan las implicaciones operativas y de ingeniería entre el modelo cliente-servidor tradicional y el paradigma analítico embebido.

Tabla 6.5: Comparación arquitectónica entre la topología cliente-servidor y el modelo de despliegue embebido.

Dimensión de diseño	Arquitectura cliente-servidor (PostgreSQL / PostGIS)	Arquitectura embebida <i>in-process</i> (DuckDB / SQLite)
Espacio de memoria	Desacoplado; espacios de direcciones virtuales aislados entre cliente y servidor.	Compartido; el motor reside dentro del mismo espacio de direcciones del ejecutable anfitrión.
Mecanismo de enlace	Sockets de red TCP/IP (puerto 5432) o sockets de dominio UNIX locales.	Vinculación estática o dinámica de bibliotecas (C ABI, <code>.so</code> , <code>.dll</code>).
Tránsito por pila OSI	Recorrido íntegro de capas 1 a 7 (encapsulamiento de tramas, paquetes y segmentos).	Nulo; supresión total de las capas de transporte, red y enlace del modelo OSI.
Serialización de geometrías	Obligatoria; conversión continua entre objetos C en memoria y flujos binarios WKB de red.	Desacoplada de red; intercambio de arreglos contiguos en memoria (Apache Arrow / punteros directos).
Latencia por llamada	Elevada por sobrecarga de sockets, llamadas al sistema (<i>syscalls</i>) y cambios de contexto.	Mínima; invocación directa de funciones por interfaz binaria en memoria RAM.
Perfil operativo idóneo	Sistemas transaccionales concurrentes (OLTP), catastro multiusuario y control de acceso RBAC.	Procesamiento analítico local masivo (OLAP), transformaciones ETL intermedias y ciencia de datos.

6.4 Arquitecturas para aplicaciones de sistemas de información geográfica

Históricamente, la vinculación entre los sistemas de información geográfica (SIG) y los sistemas de gestión de datos ha transitado por tres generaciones arquitectónicas consolidadas dentro del paradigma relacional y objeto-relacional (ORDBMS). Esta taxonomía describe la evolución del acoplamiento entre la geometría, los atributos descriptivos y el motor de ejecución en entornos transaccionales operacionales (OLTP). Si bien los paradigmas analíticos contemporáneos (como motores columnares embebidos, cómputo distribuido sobre clústeres, o almacenes orientados a grafos y NoSQL) responden a demandas analíticas de frontera que se abordan en los proyectos de investigación al cierre del curso, el modelo integrado objeto-relacional constituye la base conceptual y operativa sobre la cual se fundamenta la integridad, concurrencia y estandarización de los datos geoespaciales corporativos.

En la Figura 6.7 se ilustra esta evolución conceptual desde esquemas propietarios aislados hasta la centralización transaccional en motores espaciales integrados.

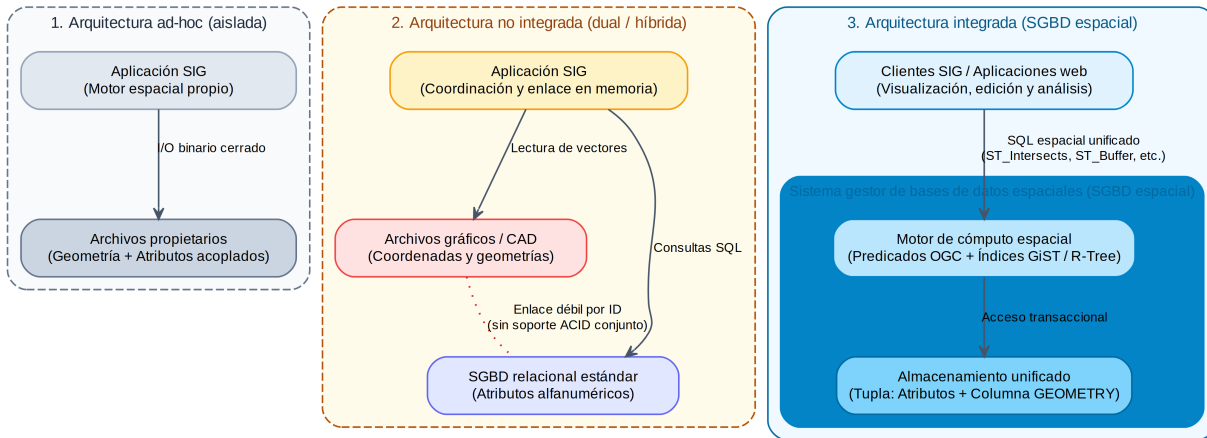


Figura 6.7: Evolución de las arquitecturas de integración entre SIG y bases de datos.

6.4.1 Arquitectura ad-hoc

En esta configuración inicial, los paquetes de software SIG implementan formatos binarios cerrados y algoritmos propietarios de cálculo geométrico ejecutados directamente sobre el sistema de archivos:

- **Ventajas:** Alto rendimiento y eficiencia computacional para operaciones cartográficas locales en estaciones monousuario, al carecer de sobrecarga (*overhead*) por capas de red o bloqueos transaccionales.
- **Limitaciones:** Nula extensibilidad hacia nuevas aplicaciones, imposibilidad de compartir datos fuera del entorno de origen, ausencia de control de concurrencia y proliferación de información redundante e inconsistente.

6.4.2 Arquitectura no integrada (dual o híbrida)

Este esquema desacopla el almacenamiento en dos repositorios independientes: los atributos temáticos se delegan a un SGBD relacional estándar, mientras que las geometrías continúan gestionándose mediante el sistema de archivos tradicional:

- **Ventajas:** Incorpora las capacidades analíticas, de seguridad y de consulta declarativa del SGBD para la fracción alfanumérica de la información.
- **Limitaciones:** La responsabilidad de integrar y sincronizar la geometría con la tabla recae exclusivamente en la capa de aplicación (*application programs*), lo que genera un rendimiento inferior al de las soluciones ad-hoc, sobrecarga de memoria en el cliente y riesgo permanente de inconsistencia transaccional (huérfanos espaciales o tablas sin geometría asociada).

6.4.3 Arquitectura integrada (SGBD espacial)

Representa el estándar contemporáneo, donde el sistema gestor de bases de datos asume tanto la geometría como los atributos descriptivos dentro de un mismo motor estructurado:

- **Reutilización y extensibilidad:** Desacopla los datos espaciales de los programas SIG clientes, permitiendo que múltiples aplicaciones heterogéneas (escritorio, servicios web OGC, pipelines en Python/R/Julia) consuman y actualicen las mismas capas de forma concurrente.

- **Procesamiento en el servidor:** Las funciones analíticas, el cálculo de relaciones topológicas y la indexación multidimensional se ejecutan directamente en el motor de la base de datos, optimizando el ancho de banda al retornar únicamente el subconjunto filtrado.
- **Compromiso de rendimiento (*trade-off*):** Aunque puede presentar un costo operativo marginalmente superior frente a algoritmos locales ad-hoc debido a la gestión de transacciones ACID y la serialización SQL, dicho margen queda plenamente compensado por la integridad estructural, la escalabilidad y la interoperabilidad institucional de los datos.

6.4.4 Distribución de responsabilidades entre el SIG y el SGBD objeto-relacional

La adopción de una arquitectura integrada formaliza un esquema multicapa donde se dividen estrictamente las tareas entre la aplicación cliente y el motor gestor de bases de datos, tal como se esquematiza en la Figura 6.8.

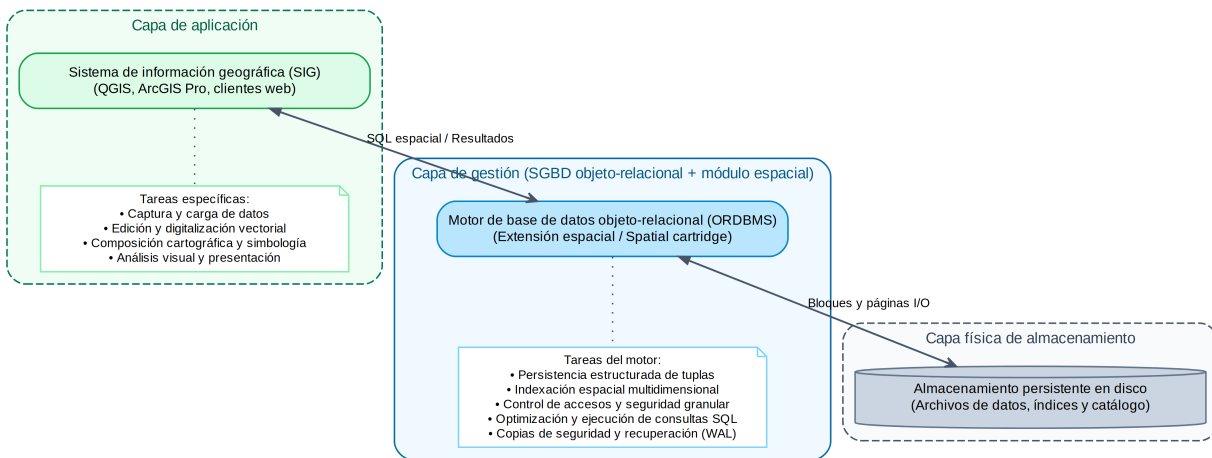


Figura 6.8: Distribución de capas y responsabilidades entre el software SIG y el SGBD objeto-relacional.

Bajo este modelo de distribución operativa se derivan ventajas determinantes:

- **Descarga computacional de la aplicación cliente:** Al incorporar un **módulo espacial** (*spatial extension* o *cartridge*) directamente acoplado al núcleo del motor, los algoritmos geométricos computacionalmente intensivos (cálculo de áreas, uniones espaciales, generación de zonas de influencia o evaluación de relaciones topológicas) se procesan dentro del servidor.
- **Unificación de datos convencionales y espaciales:** El SGBD administra atributos alfanuméricos y objetos geométricos bajo un mismo esquema declarativo, garantizando que las reglas de integridad referencial, directivas de seguridad y restricciones de dominio apliquen uniformemente sobre toda la entidad.
- **Simplificación del ciclo de vida del software:** Las aplicaciones SIG se liberan de la responsabilidad de implementar motores de indexación propios o controladores de concurrencia a nivel de archivo. Esto reduce la complejidad del código cliente y previene inconsistencias por accesos simultáneos.

6.5 PostgreSQL y la extensión PostGIS

PostgreSQL es un sistema gestor de bases de datos objeto-relacional (ORDBMS) de código abierto, diseñado con una arquitectura extensible que permite incorporar nuevos tipos de datos, funciones definidas por el usuario en múltiples lenguajes (C, PL/pgSQL, Python) y métodos de acceso indexados especializados.

PostGIS es la extensión espacial oficial de PostgreSQL. Transforma el motor relacional en un SGBD espacial completo al incorporar:

- Tipos de datos espaciales geométricos vectoriales (**geometry**), geodésicos (**geography**), matriciales (**raster**) y estructuras topológicas (**topology**).
- Funciones de análisis espacial conformes a las especificaciones internacionales de la OGC.
- Soporte para métodos de indexación espacial mediante árboles de búsqueda generalizada (GiST, *Generalized Search Tree*) y árboles R.

i Instrucciones específicas para el sistema anfitrión (Windows)

Cuando se administra PostgreSQL de forma nativa sobre el sistema anfitrión Windows, el servicio del clúster puede gestionarse desde la consola de servicios (**services.msc**), identificando la entrada **postgresql-x64-<version>**. Alternativamente, desde una terminal de PowerShell con privilegios administrativos, se puede consultar el estado del servicio mediante:

```
Get-Service -Name postgresql*
```

Para asegurar que las utilidades de línea de comandos (**psql**, **pg_dump**, **pg_restore**) respondan globalmente en la consola del sistema anfitrión, la ruta binaria del motor (por ejemplo, **C:\Program Files\PostgreSQL\<version>\bin**) debe estar registrada en la variable de entorno del sistema **Path**.

6.5.1 Verificación de PostGIS en el sistema anfitrión (Windows)

Para comprobar que el entorno espacial se encuentra habilitado y operativo en la base de datos de trabajo **sig_db_unal**, se consulta la firma completa de dependencias compiladas contra el motor relacional:

6.5.2 Terminal (psql)

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"
# Inspección de versión y librerías geoespaciales vinculadas a PostGIS
psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-x -P format=wrapped -P columns=90 \
-c "SELECT postgis_full_version();"

```

```
-[ RECORD 1 ]-----+-----
postgis_full_version | POSTGIS="3.6.2 3.6.2" [EXTENSION] PGSQL="180" GEOS="3.14.1dev-
CAPI.
```

```
|-1.20.4" PROJ="8.2.1 NETWORK_ENABLED=OFF URL_ENDPOINT=https://cdn..
|.proj.org USER_WRITABLE_DIRECTORY=C:\WINDOWS\ServiceProfiles\Networ.
```

```
|.kService\AppData\Local\proj DATABASE_PATH=D:\Programs\PostgreSQL\1.
|.8\share\contrib\postgis-3.6\proj\proj.db" (compiled against PROJ 8.
|.2.1) LIBXML="2.12.5" LIBJSON="0.12" LIBPROTOBUF="1.2.1" WAGYU="0..
|.5.0 (Internal)"
```

6.5.3 Python (SQLAlchemy)

```
import textwrap
from sqlalchemy import text

# Ejecución de la consulta utilizando el motor instanciado para Docker
with engine_docker.connect() as conn:
    version_postgis = conn.execute(text("SELECT postgis_full_version();")).scalar()
    print(textwrap.fill(version_postgis, width=90))
```

```
POSTGIS="3.6.1 f533623" [EXTENSION] PGSQL="180" GEOS="3.13.1-CAPI-1.19.2" PROJ="9.6.0
NETWORK_ENABLED=OFF URL_ENDPOINT=https://cdn.proj.org
USER_WRITABLE_DIRECTORY=/var/lib/postgresql/.local/share/proj
DATABASE_PATH=/usr/share/proj/proj.db" (compiled against PROJ 9.6.0) GDAL="GDAL 3.10.3,
released 2025/04/01" LIBXML="2.9.14" LIBJSON="0.18" LIBPROTOBUF="1.5.1" WAGYU="0.5.0
(Internal)" TOPOLOGY RASTER
```

6.5.4 Python (Pandas)

```
import textwrap
import pandas as pd

# Lectura directa a DataFrame mediante el motor de Docker
query = "SELECT postgis_full_version();"
df_version = pd.read_sql(query, con=engine_docker)

# Extracción y particionamiento a 90 columnas
cadena_version = df_version.iloc[0, 0]
print(textwrap.fill(cadena_version, width=90))
```

```
POSTGIS="3.6.1 f533623" [EXTENSION] PGSQL="180" GEOS="3.13.1-CAPI-1.19.2" PROJ="9.6.0
NETWORK_ENABLED=OFF URL_ENDPOINT=https://cdn.proj.org
USER_WRITABLE_DIRECTORY=/var/lib/postgresql/.local/share/proj
DATABASE_PATH=/usr/share/proj/proj.db" (compiled against PROJ 9.6.0) GDAL="GDAL 3.10.3,
released 2025/04/01" LIBXML="2.9.14" LIBJSON="0.18" LIBPROTOBUF="1.5.1" WAGYU="0.5.0
(Internal)" TOPOLOGY RASTER
```

El resultado de esta función expone las librerías base sobre las cuales opera el motor espacial:

- **GEOS** (*Geometry Engine Open Source*): Provee las primitivas de cálculo computacional y evaluación de predicados topológicos.
- **PROJ**: Motor de transformación y reproyección entre sistemas de referencia de coordenadas.
- **GDAL** (*Geospatial Data Abstraction Library*): Librería de traducción y soporte para formatos ráster y vectoriales.
- **LIBXML2**: Procesador de serializaciones XML/GML.

6.6 Estándar OGC Simple Features for SQL (ISO 19125)

El Open Geospatial Consortium (OGC) y la Organización Internacional de Normalización (ISO/TC 211) formularon la especificación *Simple Feature Access* (SFA) —conocida comúnmente en entornos relacionales como *Simple Features for SQL* (SFS)—, estandarizada bajo la norma ISO 19125. Esta especificación se estructura formalmente en dos partes complementarias: la Parte 1 (*Common Architecture*), que define el modelo conceptual de geometrías, la jerarquía de tipos (`Point`, `Curve`, `Surface`, `GeometryCollection`) y su vinculación con sistemas de referencia de coordenadas ([Open Geospatial Consortium, 2011](#)); y la Parte 2 (*SQL Option*), que especifica la implementación técnica, el esquema relacional y las funciones en SQL requeridas para almacenar, manipular y consultar dichas colecciones geométricas en motores de bases de datos relacionales ([Open Geospatial Consortium, 2010](#)).

6.6.1 Jerarquía de tipos geométricos OGC

El estándar define una taxonomía orientada a objetos en la que la clase abstracta raíz es `Geometry`. En la Figura 6.9 se presenta el diagrama de clases conforme a la especificación formal ISO 19125 / OGC SFA.

Para la interpretación de la Figura 6.9 se deben considerar las siguientes convenciones de la notación UML:

- **Clases abstractas** (**«Abstract»**): Entidades conceptuales que no admiten instanciación directa (`Geometry`, `Curve`, `Surface`, `MultiSurface`, `MultiCurve`). Definen contratos de métodos y propiedades que heredan las clases concretas.
- **Generalización** ($\triangle$): Relación de herencia (*es-un*). La flecha orientada hacia la superclase indica que la subclase adquiere sus atributos, operaciones y restricciones topológicas.
- **Agregación** ($\diamond$): Relación estructural (*compuesto-por*). El rombo ubicado en el extremo de la clase contenedora indica que esta se compone de una colección de elementos de la clase vinculada.
- **Multiplicidad**: Restricción cardinal de las asociaciones; por ejemplo, `2..*` denota la obligatoriedad de al menos dos vértices para instanciar un `LineString`, mientras que `1..*` exige al menos un anillo delimitador para un `Polygon`.

6.6.2 Representación espacial de primitivas y colecciones

Cada entidad definida en el estándar posee una signatura geométrica y una dimensión topológica intrínseca. En la Figura 6.10 se presenta la morfología de las geometrías fundamentales y sus agrupaciones.

En la Tabla 6.6 se detallan las características topológicas y dimensionales de los tipos definidos en la norma.

Tabla 6.6: Primitivas y colecciones geométricas del estándar OGC.

Tipo geométrico	Dimensión topológica	Descripción formal OGC
<code>Point</code>	0	Posición geométrica aislada descrita por un único par ordenado (X, Y) .
<code>LineString</code>	1	Curva lineal interpolada entre dos o más vértices.

Tipo geométrico	Dimensión topológica	Descripción formal OGC
LinearRing	1	LineString cerrada y no autosecante (define límites poligonales).
Polygon	2	Superficie planar delimitada por un anillo exterior y cero o más interiores.
MultiPoint	0	Agrupación no conexa de puntos independientes.
MultiLineString	1	Conjunto estructurado de cadenas de líneas.
MultiPolygon	2	Conjunto conexo o disjunto de polígonos.
GeometryCollection	Variable	Agrupación heterogénea arbitraria de entidades geométricas.

6.6.3 Serialización y formatos estándar: WKT y WKB

Para garantizar el intercambio de geometrías entre sistemas heterogéneos, el estándar OGC define dos formatos de interoperabilidad:

6.6.3.1 Formato Well-Known Text (WKT)

WKT es una codificación en texto plano legible por humanos que define la estructura del tipo geométrico y sus secuencias de coordenadas:

- POINT(1000000 1000000)
- LINESTRING(1000000 1000000, 1050000 1050000, 1100000 1020000)
- POLYGON((1000000 1000000, 1100000 1000000, 1100000 1100000, 1000000 1100000, 1000000 1000000))

En superficies poligonales, el primer anillo define el perímetro exterior, mientras que los anillos sucesivos delimitan perforaciones interiores:

```
POLYGON((0 0, 10 0, 10 10, 0 10, 0 0), (2 2, 4 2, 4 4, 2 4, 2 2))
```

PostGIS extiende el estándar WKT incorporando componentes tridimensionales y de medición lineal: POINT Z, POINT M y POINT ZM.

6.6.3.2 Formato Well-Known Binary (WKB) y EWKB

WKB es una representación binaria estructurada en secuencias de bytes (*byte arrays*), optimizada para la transferencia en red y el almacenamiento interno en páginas de disco:

- Incorpora indicadores de ordenamiento de bytes (*Endianness*: Big-Endian o Little-Endian).

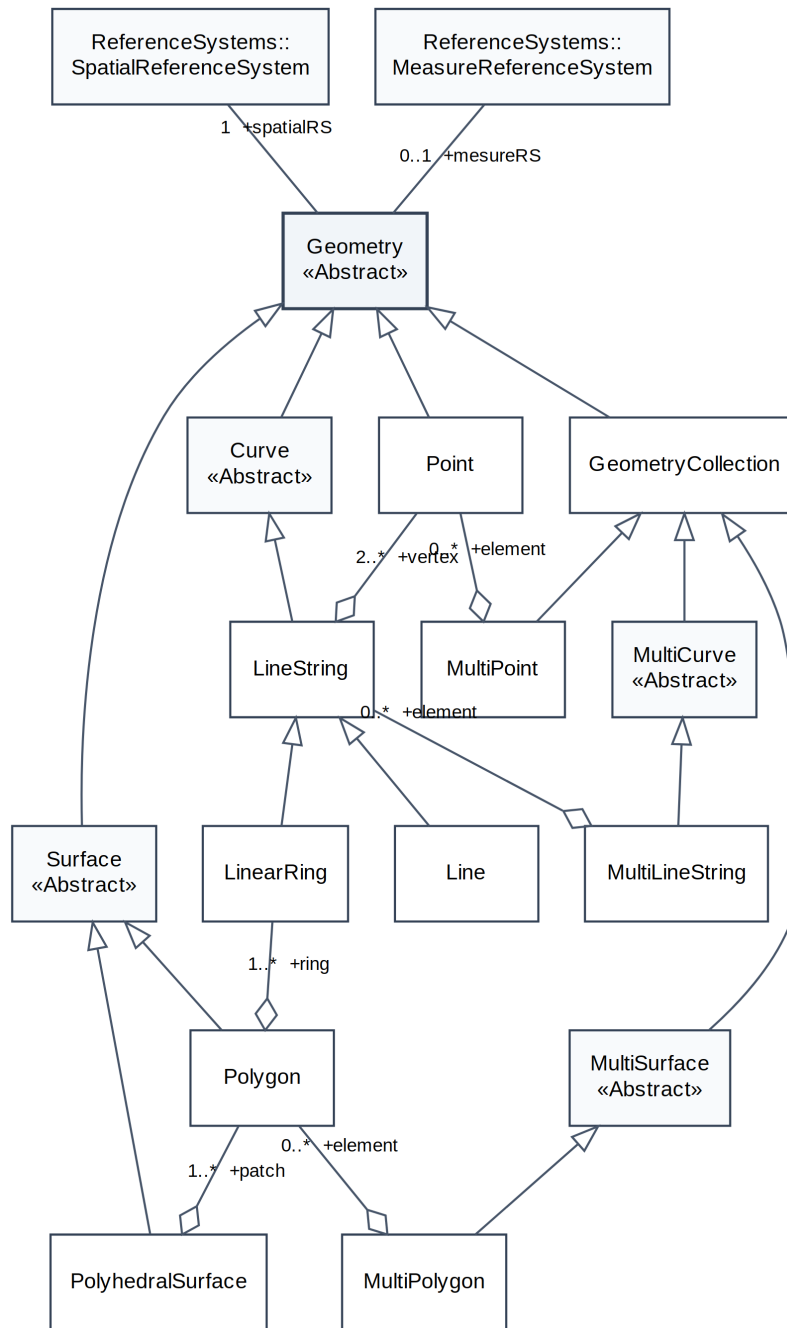


Figura 6.9: Diagrama de clases UML para la jerarquía de geometrías según ISO 19125 / OGC Simple Feature Access.

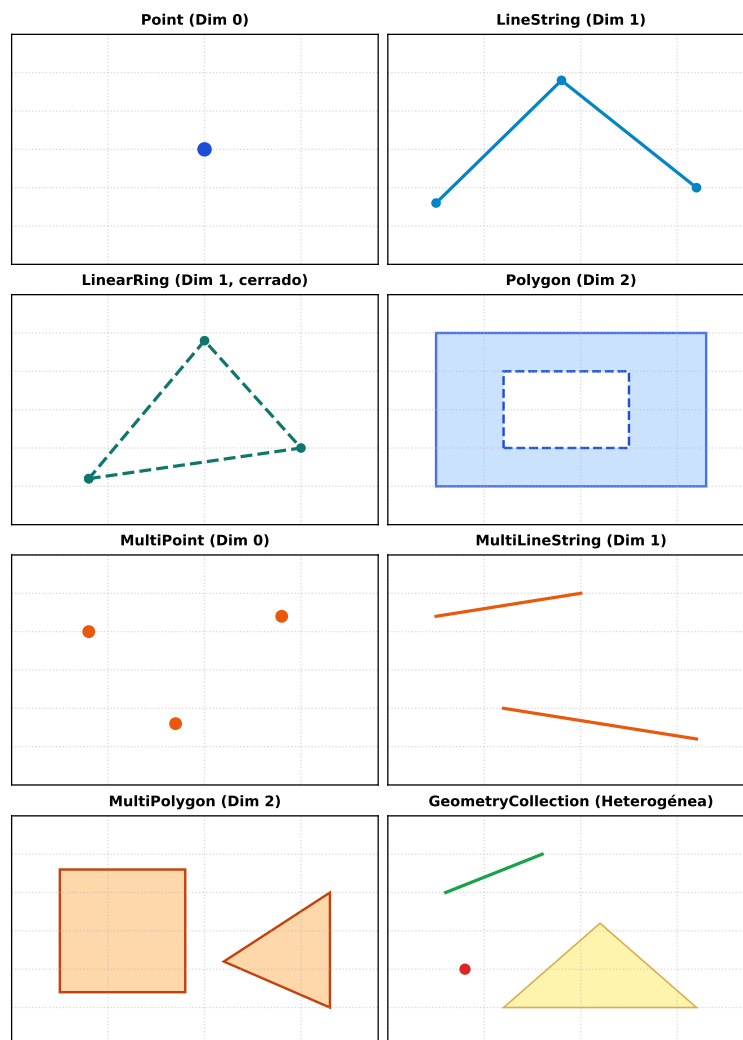


Figura 6.10: Instanciación espacial y morfología de tipos geométricos según el estándar OGC.

- Define identificadores numéricos que codifican el tipo de geometría.
- Preserva la precisión numérica en coma flotante de doble precisión (IEEE 754) para cada coordenada.

PostGIS implementa el formato extendido **EWKB** (*Extended Well-Known Binary*), el cual embebe directamente el identificador del sistema de referencia espacial (SRID) en la cabecera binaria del objeto geométrico.

6.6.4 Representación espacial de primitivas y colecciones en PostGIS

Para verificar la persistencia física de las geometrías OGC y evaluar el intercambio de datos mediante los formatos canónicos WKT y WKB directamente en el motor relacional del servidor local, el flujo de trabajo se estructura en dos fases complementarias:

1. **Persistencia relacional en PostGIS:** Se provisiona una tabla polimórfica (`public.muestras_geometrias_ogc`) con una columna geométrica sin modificadores de tipo restrictivos (*typmod*), permitiendo almacenar las ocho primitivas y colecciones fundamentales mediante constructores estándar `ST_GeomFromText`. Conforme a la gramática formal de la norma ISO 19125-2, el tipo `LinearRing` carece de un constructor textual WKT autónomo en el estándar SQL; por consiguiente, se persiste como una curva `LineString` cerrada con coincidencia topológica estricta entre el vértice inicial y final.
2. **Extracción binaria y renderizado en Python:** El entorno ejecuta una consulta relacional solicitando la codificación canónica *Well-Known Binary* (`ST_AsBinary`), decodifica los flujos de bytes mediante `shapely.wkb` y genera la matriz gráfica comparativa.

El siguiente bloque de instrucciones DDL y DML ejecuta el aprovisionamiento y poblamiento de la relación en el clúster PostgreSQL:

- **Eliminación condicional previa:** Suprime la tabla `public.muestras_geometrias_ogc` si existe en el esquema (`DROP TABLE IF EXISTS`), garantizando la idempotencia del script ante múltiples ejecuciones.
- **Definición del esquema de la tabla:** Crea la relación especificando una clave primaria serial, identificadores de ordenamiento, atributos descriptivos y una columna espacial de tipo genérico `geometry` sin restricción de subtipo (*typmod*).
- **Inserción de atributos y geometrías:** Registra los datos alfanuméricos e instancia físicamente cada objeto geométrico a partir de literales WKT mediante la función constructora `ST_GeomFromText`.

6.6.5 Terminal (psql)

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"

query_muestras="
SET client_min_messages = warning;
DROP TABLE IF EXISTS public.muestras_geometrias_ogc;
RESET client_min_messages;

CREATE TABLE public.muestras_geometrias_ogc (
  id serial PRIMARY KEY,
  orden integer NOT NULL,
  nombre varchar(30) NOT NULL,
  descripcion varchar(50) NOT NULL,
  geom geometry NOT NULL
);

INSERT INTO public.muestras_geometrias_ogc (orden, nombre, descripcion, geom) VALUES
(1, 'Point', 'Point (Dim 0)',
  ST_GeomFromText('POINT(2 1.5)'),
```

```

(2, 'LineString', 'LineString (Dim 1)',
  ST_GeomFromText('LINESTRING(0.5 0.8, 1.8 2.4, 3.2 1.0)')),
(3, 'LinearRing', 'LinearRing (Dim 1, cerrado)',
  ST_GeomFromText('LINESTRING(0.8 0.6, 3.0 1.0, 2.0 2.4, 0.8 0.6)')),
(4, 'Polygon', 'Polygon (Dim 2)',
  ST_GeomFromText('POLYGON((0.5 0.5, 3.3 0.5, 3.3 2.5, 0.5 2.5, 0.5 0.5), (1.2 1.0, 2.5 1.0, 2.5 2.0,
  ↪ 1.2 2.0, 1.2 1.0)'))),
(5, 'MultiPoint', 'MultiPoint (Dim 0)',
  ST_GeomFromText('MULTIPOINT(0.8 2.0, 1.7 0.8, 2.8 2.2)')),
(6, 'MultiLineString', 'MultiLineString (Dim 1)',
  ST_GeomFromText('MULTILINESTRING((0.5 2.2, 2.0 2.5), (1.2 1.0, 3.2 0.6)'))),
(7, 'MultiPolygon', 'MultiPolygon (Dim 2)',
  ST_GeomFromText('MULTIPOLYGON(((0.5 0.7, 1.8 0.7, 1.8 2.3, 0.5 2.3, 0.5 0.7)), ((2.2 1.1, 3.3 0.5, 3.3
  ↪ 2.0, 2.2 1.1)')))),
(8, 'GeometryCollection', 'GeometryCollection (Heterogénea)',
  ST_GeomFromText('GEOMETRYCOLLECTION(POLYGON((1.2 0.5, 3.2 0.5, 2.2 1.6, 1.2 0.5)), LINESTRING(0.6 2.0,
  ↪ 1.6 2.5), POINT(0.8 1.0)')));
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-c "${query_muestras}"

```

SET

DROP TABLE

RESET

CREATE TABLE

INSERT 0 8

6.6.6 Python (SQLAlchemy)

```

from sqlalchemy import text

query_muestras = """
SET client_min_messages = warning;
DROP TABLE IF EXISTS public.muestras_geometrias_ogc;
RESET client_min_messages;

CREATE TABLE public.muestras_geometrias_ogc (
  id serial PRIMARY KEY,
  orden integer NOT NULL,
  nombre varchar(30) NOT NULL,
  descripcion varchar(50) NOT NULL,
  geom geometry NOT NULL
);

INSERT INTO public.muestras_geometrias_ogc (orden, nombre, descripcion, geom) VALUES
(1, 'Point', 'Point (Dim 0)',
  ST_GeomFromText('POINT(2 1.5)')),
(2, 'LineString', 'LineString (Dim 1)',
  ST_GeomFromText('LINESTRING(0.5 0.8, 1.8 2.4, 3.2 1.0)')),
(3, 'LinearRing', 'LinearRing (Dim 1, cerrado)',
  ST_GeomFromText('LINESTRING(0.8 0.6, 3.0 1.0, 2.0 2.4, 0.8 0.6)')),
(4, 'Polygon', 'Polygon (Dim 2)',
  ST_GeomFromText('POLYGON((0.5 0.5, 3.3 0.5, 3.3 2.5, 0.5 2.5, 0.5 0.5), (1.2 1.0, 2.5 1.0, 2.5 2.0,
  ↪ 1.2 2.0, 1.2 1.0)'))),
(5, 'MultiPoint', 'MultiPoint (Dim 0)',
  ST_GeomFromText('MULTIPOINT(0.8 2.0, 1.7 0.8, 2.8 2.2)')),
(6, 'MultiLineString', 'MultiLineString (Dim 1)',
  ST_GeomFromText('MULTILINESTRING((0.5 2.2, 2.0 2.5), (1.2 1.0, 3.2 0.6)'))),
(7, 'MultiPolygon', 'MultiPolygon (Dim 2)',
  ST_GeomFromText('MULTIPOLYGON(((0.5 0.7, 1.8 0.7, 1.8 2.3, 0.5 2.3, 0.5 0.7)), ((2.2 1.1, 3.3 0.5, 3.3
  ↪ 2.0, 2.2 1.1)')))),
(8, 'GeometryCollection', 'GeometryCollection (Heterogénea)',
  ST_GeomFromText('GEOMETRYCOLLECTION(POLYGON((1.2 0.5, 3.2 0.5, 2.2 1.6, 1.2 0.5)), LINESTRING(0.6 2.0,
  ↪ 1.6 2.5), POINT(0.8 1.0)')));
"""

# Ejecución transaccional atómica sobre el motor anfitrión previamente instanciado
with engine_local.begin() as conn:
    conn.execute(text(query_muestras))
    print("Tabla public.muestras_geometrias_ogc aprovisionada y poblada exitosamente.")

```

<sqlalchemy.engine.cursor.CursorResult object at 0x73009f709a20>

Tabla public.muestras_geometrias_ogc aprovisionada y poblada exitosamente.

Una vez confirmada la inserción en el motor, el bloque subsecuente recupera las geometrías en WKB, reconstruye los objetos topológicos e ilustra la morfología de cada tipo en Python (Figura 6.11), R (Figura 6.13) y Julia (Figura 6.14):

6.6.7 Python (Shapely)

```
# =====
# 0. Módulos y bibliotecas requeridas
# =====
# 'os': Módulo estándar para interactuar con el sistema operativo;
# permite recuperar variables de entorno del sistema (credenciales seguras).
import os

# 'psycopg2': Controlador canónico (driver) de PostgreSQL para Python;
# gestiona el protocolo de red, la conexión y la ejecución de sentencias SQL.
import psycopg2

# 'matplotlib.pyplot': Biblioteca estándar de graficación técnica y científica;
# provee las funciones para construir y configurar lienzos vectoriales 2D.
import matplotlib.pyplot as plt

# 'shapely.wkb': Submódulo de la biblioteca geoespacial Shapely (basada en GEOS);
# permite deserializar cadenas binarias en formato Well-Known Binary (WKB)
# y reconstruirlas como objetos geométricos espaciales manipulables en memoria.
from shapely import wkb

# -----
# 1. Conexión a la base de datos relacional
# -----
conn = psycopg2.connect(
    host=os.environ.get("LOCAL_PGHOST"),
    port=os.environ.get("LOCAL_PGPORT"),
    dbname=os.environ.get("LOCAL_PGDATABASE"),
    user=os.environ.get("LOCAL_PGUSER"),
    password=os.environ.get("LOCAL_PGPASSWORD")
)

# -----
# 2. Extracción de entidades en formato binario estándar (WKB)
# -----
cursor = conn.cursor()
cursor.execute("""
    SELECT orden, nombre, descripcion, ST_AsBinary(geom) AS wkb
    FROM public.muestras_geometrias_ogc
    ORDER BY orden ASC;
""")
registros = cursor.fetchall()
cursor.close()
conn.close()

# -----
# 3. Configuración del lienzo gráfico (Matplotlib)
# -----
fig, axes = plt.subplots(4, 2, figsize=(6.5, 9.0), constrained_layout=True)
axes_flat = axes.flatten()

# -----
# 4. Función de despacho polimórfico para renderizado geométrico
# -----
def graficar_geometria(ax, geom, nombre):
    if geom.geom_type == 'Point':
        ax.plot(geom.x, geom.y, 'o', color='#1d4ed8', markersize=8)

    elif geom.geom_type == 'LineString':
        x, y = geom.xy
        if nombre == 'LinearRing':
            ax.plot(x, y, '--o', color='#0f766e', linewidth=2, markersize=5)
        else:
            ax.plot(x, y, '-o', color='#0284c7', linewidth=2, markersize=5)

    elif geom.geom_type == 'Polygon':
        ax.fill(*geom.exterior.xy, color='#bfd8fe', alpha=0.8, edgecolor='#1d4ed8', linewidth=1.5)
        for interior in geom.interiors:
            ax.fill(*interior.xy, color='#ffffff', edgecolor='#1d4ed8', linewidth=1.5, linestyle='--')
```

```

elif geom.geom_type == 'MultiPoint':
    for pt in geom.geoms:
        ax.plot(pt.x, pt.y, 'o', color='#ea580c', markersize=7)

elif geom.geom_type == 'MultiLineString':
    for line in geom.geoms:
        ax.plot(*line.xy, '-', color='#ea580c', linewidth=2)

elif geom.geom_type == 'MultiPolygon':
    for poly in geom.geoms:
        ax.fill(*poly.exterior.xy, color='#fed7aa', edgecolor='#c2410c', linewidth=1.5)

elif geom.geom_type == 'GeometryCollection':
    for part in geom.geoms:
        if part.geom_type == 'Polygon':
            ax.fill(*part.exterior.xy, color='#fef08a', edgecolor='#ca8a04', alpha=0.7)
        elif part.geom_type == 'LineString':
            ax.plot(*part.xy, '-', color='#16a34a', linewidth=2)
        elif part.geom_type == 'Point':
            ax.plot(part.x, part.y, 'o', color='#dc2626', markersize=6)

# -----
# 5. Iteración, deserialización binaria y graficación
# -----
for idx, (orden, nombre, descripcion, binario_wkb) in enumerate(registros):
    ax = axes_flat[idx]
    geometria = wkb.loads(bytes(binario_wkb))
    graficar_geometria(ax, geometria, nombre)
    ax.set_title(descripcion, fontsize=9, weight='bold')
    _ = ax.set_xlim(0, 3.8)
    _ = ax.set_ylim(0, 3.0)
    ax.grid(True, linestyle=':', alpha=0.5)
    ax.tick_params(left=False, bottom=False, labelleft=False, labelbottom=False)

plt.show()

```

6.6.8 Python (GeoPandas)

```

# =====
# 0. Módulos y bibliotecas requeridas
# =====
# 'geopandas': Extensión geoespacial para estructuras tabulares DataFrame;
# abstrae la deserialización de columnas espaciales relacionales directamente
# hacia objetos geométricos vectoriales GeoSeries mediante GEOS/Shapely.
import geopandas as gpd

# 'sqlalchemy.text': Envoltorio declarativo de consultas ejecutables en SQLAlchemy 2.0.
from sqlalchemy import text

# 'matplotlib.pyplot': Biblioteca estándar para renderizado y control de lienzo 2D.
import matplotlib.pyplot as plt

# -----
# 1. Extracción relacional directa hacia GeoDataFrame (motor local)
# -----
query_muestras = """
SELECT orden, nombre, descripcion, geom
FROM public.muestras_geometrias_ogc
ORDER BY orden ASC;
"""

with engine_local.connect() as conn:
    gdf = gpd.read_postgis(text(query_muestras), con=conn, geom_col="geom")

# -----
# 2. Configuración del lienzo gráfico (Matplotlib)
# -----
fig, axes = plt.subplots(4, 2, figsize=(6.5, 9.0), constrained_layout=True)
axes_flat = axes.flatten()

# -----
# 3. Función de despacho polimórfico para renderizado de geometrías
# -----
def graficar_geometria_gpd(ax, geom, nombre):
    if geom.geom_type == 'Point':
        ax.plot(geom.x, geom.y, 'o', color='#1d4ed8', markersize=8)

    elif geom.geom_type == 'LineString':
        x, y = geom.xy
        if nombre == 'LinearRing':

```

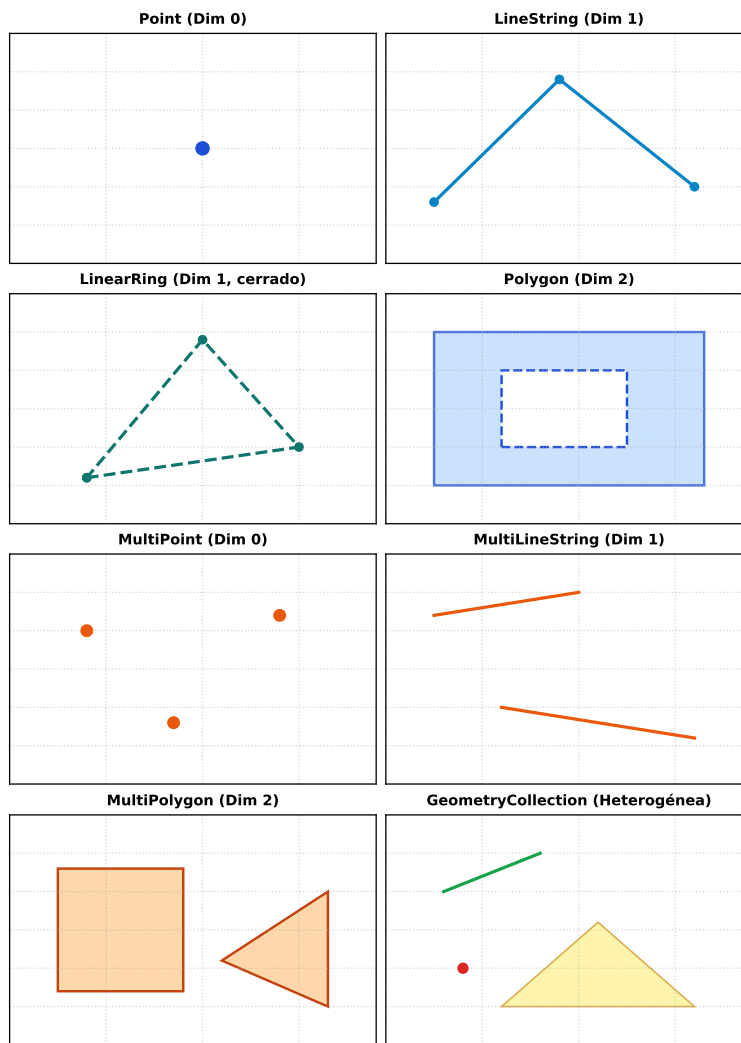


Figura 6.11: Instanciación espacial y morfología de tipos geométricos según el estándar OGC en Python con Shapely.

```

        ax.plot(x, y, '--o', color='#0f766e', linewidth=2, markersize=5)
    else:
        ax.plot(x, y, '-o', color='#0284c7', linewidth=2, markersize=5)

    elif geom.geom_type == 'Polygon':
        ax.fill(*geom.exterior.xy, color='#bfbdbfe', alpha=0.8, edgecolor='#1d4ed8', linewidth=1.5)
        for interior in geom.interiors:
            ax.fill(*interior.xy, color='ffffff', edgecolor='#1d4ed8', linewidth=1.5, linestyle='--')

    elif geom.geom_type == 'MultiPoint':
        for pt in geom.geoms:
            ax.plot(pt.x, pt.y, 'o', color='#ea580c', markersize=7)

    elif geom.geom_type == 'MultiLineString':
        for line in geom.geoms:
            ax.plot(*line.xy, '-', color='#ea580c', linewidth=2)

    elif geom.geom_type == 'MultiPolygon':
        for poly in geom.geoms:
            ax.fill(*poly.exterior.xy, color='#fed7aa', edgecolor='#c2410c', linewidth=1.5)

    elif geom.geom_type == 'GeometryCollection':
        for part in geom.geoms:
            if part.geom_type == 'Polygon':
                ax.fill(*part.exterior.xy, color='#fef08a', edgecolor='#ca8a04', alpha=0.7)
            elif part.geom_type == 'LineString':
                ax.plot(*part.xy, '-', color='#16a34a', linewidth=2)
            elif part.geom_type == 'Point':
                ax.plot(part.x, part.y, 'o', color='#dc2626', markersize=6)

# -----
# 4. Iteración y graficación desde el GeoDataFrame
# -----
for idx, fila in gdf.iterrows():
    ax = axes_flat[idx]
    graficar_geometria_gpd(ax, fila.geom, fila["nombre"])
    ax.set_title(fila["descripcion"], fontsize=9, weight='bold')
    _ = ax.set_xlim(0, 3.8)
    _ = ax.set_ylim(0, 3.0)
    ax.grid(True, linestyle=':', alpha=0.5)
    ax.tick_params(left=False, bottom=False, labelleft=False, labelbottom=False)

plt.show()

```

6.6.9 R

```

# =====
# 0. Módulos y bibliotecas requeridas
# =====
# 'DBI': Interfaz unificada de conexión a bases de datos relacionales en R.
library(DBI)

# 'RPostgres': Controlador optimizado en C++ para comunicación nativa con PostgreSQL.
library(RPostgres)

# 'sf': Biblioteca estándar para geometrías vectoriales bajo Simple Features;
# enlaza con GDAL, GEOS y PROJ, provyendo deserialización nativa de WKB.
library(sf)

# -----
# 1. Conexión a la base de datos relacional
# -----
conn <- dbConnect(
  RPostgres::Postgres(),
  host = Sys.getenv("LOCAL_PGHOST"),
  port = as.integer(Sys.getenv("LOCAL_PGPORT")),
  dbname = Sys.getenv("LOCAL_PGDATABASE"),
  user = Sys.getenv("LOCAL_PGUSER"),
  password = Sys.getenv("LOCAL_PGPASSWORD")
)

# -----
# 2. Extracción de entidades en formato binario estándar (WKB)
# -----
query <- "
  SELECT orden, nombre, descripcion, ST_AsBinary(geom) AS wkb
  FROM public.muestras_geometrias_ogc
  ORDER BY orden ASC;
"
registros <- dbGetQuery(conn, query)

```

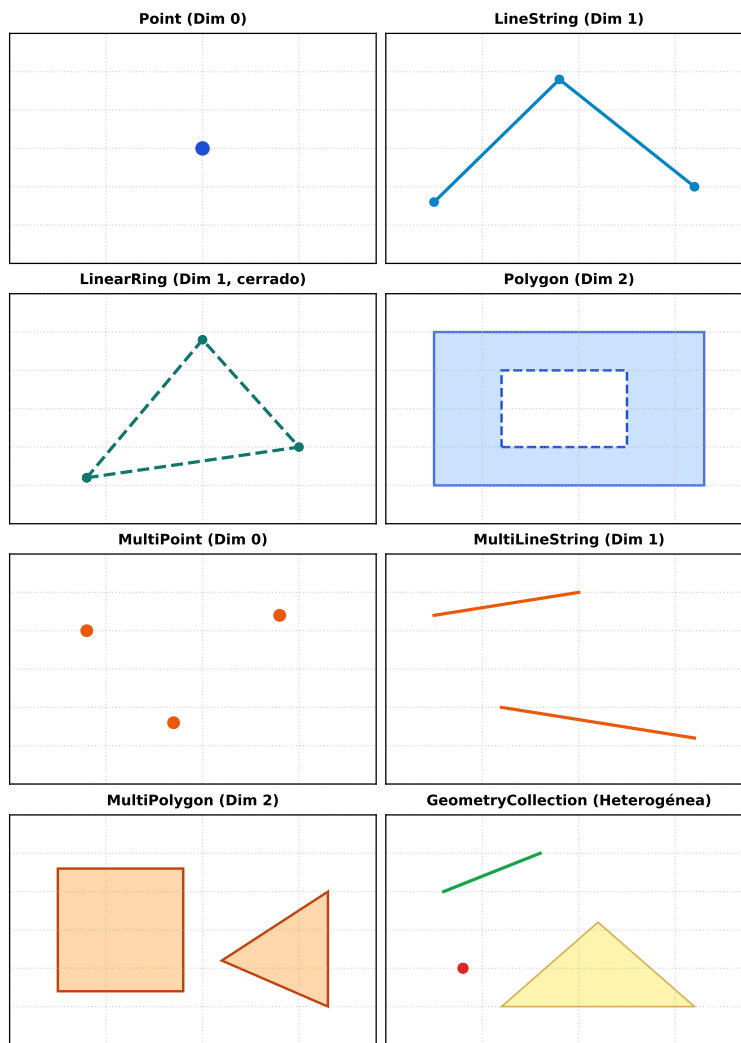


Figura 6.12: Instanciación espacial y morfología de tipos geométricos según el estándar OGC en Python con GeoPandas.

```

dbDisconnect(conn)

# Conversión del vector binario WKB a objetos de clase 'sfc' (Simple Feature Column)
class(registros$wkb) <- "WKB"
geometrias <- st_as_sfc(registros$wkb, EWKB = FALSE)

# -----
# 3. Configuración del lienzo gráfico (Base R Graphics)
# -----
# Márgenes calibrados por celda (inferior, izquierdo, superior, derecho)
par(mfrow = c(4, 2), mar = c(1.8, 1.8, 2.2, 1.2), oma = c(0, 0, 0, 0))

# -----
# 4. Función de despacho polimórfico para renderizado geométrico
# -----
graficar_geometria_r <- function(geom, nombre, descripcion) {
  # Inicialización del marco cartesiano libre de distorsión por relación de aspecto fija
  plot.new()
  plot.window(xlim = c(0, 3.8), ylim = c(0, 3.0))

  # Cuadrícula homogénea de 4 columnas x 3 filas con estilo punteado
  grid(nx = 4, ny = 3, lty = 3, col = "gray75")
  box(col = "#334155", lwd = 1.0)
  title(main = descripcion, cex.main = 1.05, font.main = 2, line = 0.6)

  # Renderizado condicional según el tipo primitivo
  if (inherits(geom, "POINT")) {
    points(geom[1], geom[2], pch = 19, col = "#1d4ed8", cex = 2.2)
  } else if (inherits(geom, "LINESTRING")) {
    if (nombre == "LinearRing") {
      lines(geom[, 1], geom[, 2], col = "#0f766e", lty = 2, lwd = 2.5)
      points(geom[, 1], geom[, 2], pch = 19, col = "#0f766e", cex = 1.2)
    } else {
      lines(geom[, 1], geom[, 2], col = "#0284c7", lwd = 2.5)
      points(geom[, 1], geom[, 2], pch = 19, col = "#0284c7", cex = 1.2)
    }
  } else if (inherits(geom, "POLYGON")) {
    # Anillo exterior
    polygon(geom[[1]][, 1], geom[[1]][, 2], col = "#bfd8b1", border = "#1d4ed8", lwd = 2.0)
    # Huecos interiores (islas)
    if (length(geom) > 1) {
      for (h in 2:length(geom)) {
        polygon(geom[[h]][, 1], geom[[h]][, 2], col = "white", border = "#1d4ed8", lty = 2, lwd = 2.0)
      }
    }
  } else if (inherits(geom, "MULTIPOINT")) {
    points(geom[, 1], geom[, 2], pch = 19, col = "#ea580c", cex = 1.8)
  } else if (inherits(geom, "MULTILINESTRING")) {
    for (linea in geom) {
      lines(linea[, 1], linea[, 2], col = "#ea580c", lwd = 2.5)
    }
  } else if (inherits(geom, "MULTIPOLYGON")) {
    for (poligono in geom) {
      polygon(poligono[[1]][, 1], poligono[[1]][, 2], col = "#fed7aa", border = "#c2410c", lwd = 2.0)
    }
  } else if (inherits(geom, "GEOMETRYCOLLECTION")) {
    for (elem in geom) {
      if (inherits(elem, "POLYGON")) {
        polygon(elem[[1]][, 1], elem[[1]][, 2], col = "#fef08a", border = "#ca8a04", lwd = 2.0)
      } else if (inherits(elem, "LINESTRING")) {
        lines(elem[, 1], elem[, 2], col = "#16a34a", lwd = 2.5)
      } else if (inherits(elem, "POINT")) {
        points(elem[1], elem[2], pch = 19, col = "#dc2626", cex = 1.8)
      }
    }
  }
}

# -----
# 5. Iteración y graficación
# -----
for (i in seq_len(nrow(registros))) {
  graficar_geometria_r(geometrias[[i]], registros$nombre[i], registros$descripcion[i])
}

```

Linking to GEOS 3.12.1, GDAL 3.8.4, PROJ 9.4.0; sf_use_s2() is TRUE

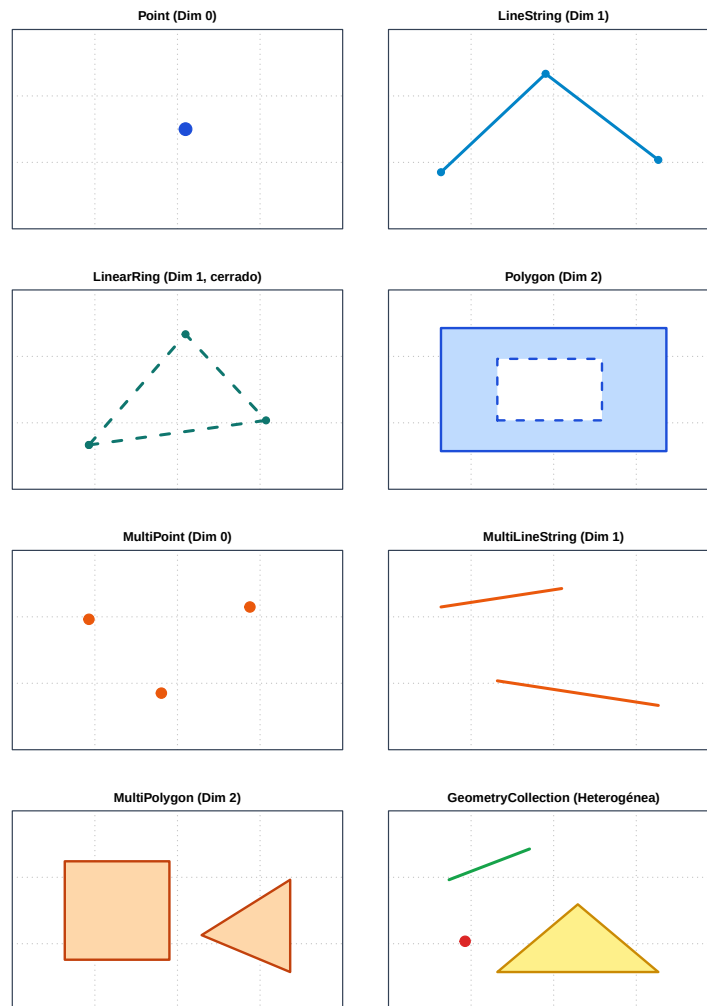


Figura 6.13: Instanciación espacial y morfología de tipos geométricos según el estándar OGC en R.

6.6.10 Julia

Starting Julia ...

```
julia>
```

```
ENV["GKSwtstype"] = "100"
"100"
```

```
julia>
```

```
using LibPQ
```

```
julia>
```

```
using DataFrames
```

```
julia>
```

```
using Plots
```

```
julia>
```

```
gr()
```

```
Plots.GRBackend()
```

```
julia>
```

```
try
```

```
    host = get(ENV, "LOCAL_PGHOST", "db-postgis")
    port = get(ENV, "LOCAL_PGPORT", "5432")
    db    = get(ENV, "LOCAL_PGDATABASE", "sig_db_unal")
    user = get(ENV, "LOCAL_PGUSER", "profe_unal")
    pass = get(ENV, "LOCAL_PGPASSWORD", "geomatica2025")
```

```
    cadena_conexion = "host=$(host) port=$(port) dbname=$(db) user=$(user) password=$(pass)"
    )"
```

```
    canal = LibPQ.Connection(cadena_conexion)
```

```
    query = "SELECT orden, nombre, descripcion, ST_AsText(geom) AS wkt FROM public.muestras_geometrias_ogc ORDER BY orden ASC;"
```

```
    df = DataFrame(execute(canal, query))
```

```
    close(canal)
```

```
# Configuración base balanceada para cada celda
```

```
cfg(tit) = (
```

```
    xlims = (0, 3.8),
    ylims = (0, 3.0),
    title = tit,
    titlefontsize = 11,
    framestyle = :box,
    grid = true,
    gridstyle = :dot,
    gridalpha = 0.5,
```

```

        xticks = (0.5:1.0:3.5, ["", "", "", ""]),
        yticks = (0.5:1.0:2.5, ["", "", ""]),
        legend = false,
        top_margin = 4Plots.mm,
        bottom_margin = 2Plots.mm
    )

# 1. Point
local p1 = plot(; cfg(df.descripcion[1])...)
scatter!(p1, [2.0], [1.5], color = "#1d4ed8", markersize = 10)

# 2. LineString
local p2 = plot(; cfg(df.descripcion[2])...)
plot!(p2, [0.5, 1.8, 3.2], [0.8, 2.4, 1.0], color = "#0284c7", linewidth = 2.5, marker
= :circle, markersize = 6)

# 3. LinearRing
local p3 = plot(; cfg(df.descripcion[3])...)
plot!(p3, [0.8, 3.0, 2.0, 0.8], [0.6, 1.0, 2.4, 0.6], color = "#0f766e", linestyle = :
dash, linewidth = 2.5, marker = :circle, markersize = 6)

# 4. Polygon
local p4 = plot(; cfg(df.descripcion[4])...)
plot!(p4, Shape([0.5, 3.3, 3.3, 0.5], [0.5, 0.5, 2.5, 2.5]), fillcolor = "#bfd8bf", fi
llalpha = 0.8, linecolor = "#1d4ed8", linewidth = 2)
plot!(p4, Shape([1.2, 2.5, 2.5, 1.2], [1.0, 1.0, 2.0, 2.0]), fillcolor = :white, linec
olor = "#1d4ed8", linestyle = :dash, linewidth = 2)

# 5. MultiPoint
local p5 = plot(; cfg(df.descripcion[5])...)
scatter!(p5, [0.8, 1.7, 2.8], [2.0, 0.8, 2.2], color = "#ea580c", markersize = 8)

# 6. MultiLineString
local p6 = plot(; cfg(df.descripcion[6])...)
plot!(p6, [0.5, 2.0], [2.2, 2.5], color = "#ea580c", linewidth = 2.5)
plot!(p6, [1.2, 3.2], [1.0, 0.6], color = "#ea580c", linewidth = 2.5)

# 7. MultiPolygon
local p7 = plot(; cfg(df.descripcion[7])...)
plot!(p7, Shape([0.5, 1.8, 1.8, 0.5], [0.7, 0.7, 2.3, 2.3]), fillcolor = "#fed7aa", li
necolor = "#c2410c", linewidth = 2)
plot!(p7, Shape([2.2, 3.3, 3.3, 2.2], [1.1, 0.5, 2.0, 1.1]), fillcolor = "#fed7aa", li
necolor = "#c2410c", linewidth = 2)

# 8. GeometryCollection
local p8 = plot(; cfg(df.descripcion[8])...)
plot!(p8, Shape([1.2, 3.2, 2.2], [0.5, 0.5, 1.6]), fillcolor = "#fef08a", linecolor =

```

```

"#ca8a04", linewidth = 2)
plot!(p8, [0.6, 1.6], [2.0, 2.5], color = "#16a34a", linewidth = 2.5)
scatter!(p8, [0.8], [1.0], color = "#dc2626", markersize = 8)

plot(
    p1, p2, p3, p4, p5, p6, p7, p8,
    layout = (4, 2),
    size = (700, 950)
)

catch fallo_proceso
println("Error durante la extracción o graficación en Julia: ", fallo_proceso)

plot(
    title = "Fallo en ejecución de bloque espacial",
    framestyle = :none,
    annotations = (0.5, 0.5, text("Error: " * string(fallo_proceso), 10, :red, :center
    )),
    legend = false,
    size = (700, 950)
)
end

```

6.7 Catálogos de metadatos espaciales

En un motor relacional espacial, el clúster administra la información cartográfica y el esquema de tipos geométricos siguiendo las especificaciones normativas de la norma ISO 19125-2 / OGC Simple Feature Access - Part 2: SQL Option ([Open Geospatial Consortium, 2010](#)). El estándar define dos estructuras de metadatos cardinales para la interoperabilidad entre motores de bases de datos y clientes SIG (Figura 3 de OGC 06-104r4):

- **spatial_ref_sys**: Diccionario canónico de sistemas de referencia de coordenadas (CRS). Almacena las proyecciones, elipsoides, datums y parámetros de transformación matemática.
- **geometry_columns**: Inventario descriptivo de columnas espaciales. Registra la relación entre tablas de entidades, nombres de columnas, dimensiones de coordenadas y el SRID asignado. Conforme a la cláusula 7.1.1 de la norma, los motores pueden implementar este catálogo como una vista dinámica sobre el diccionario del sistema en lugar de una tabla física base.

6.7.1 El catálogo spatial_ref_sys

El catálogo **spatial_ref_sys** materializa la especificación formal descrita en la cláusula 7.1.2 de la norma OGC SFA Parte 2, la cual exige la persistencia de los identificadores y definiciones WKT de los marcos geodésicos:

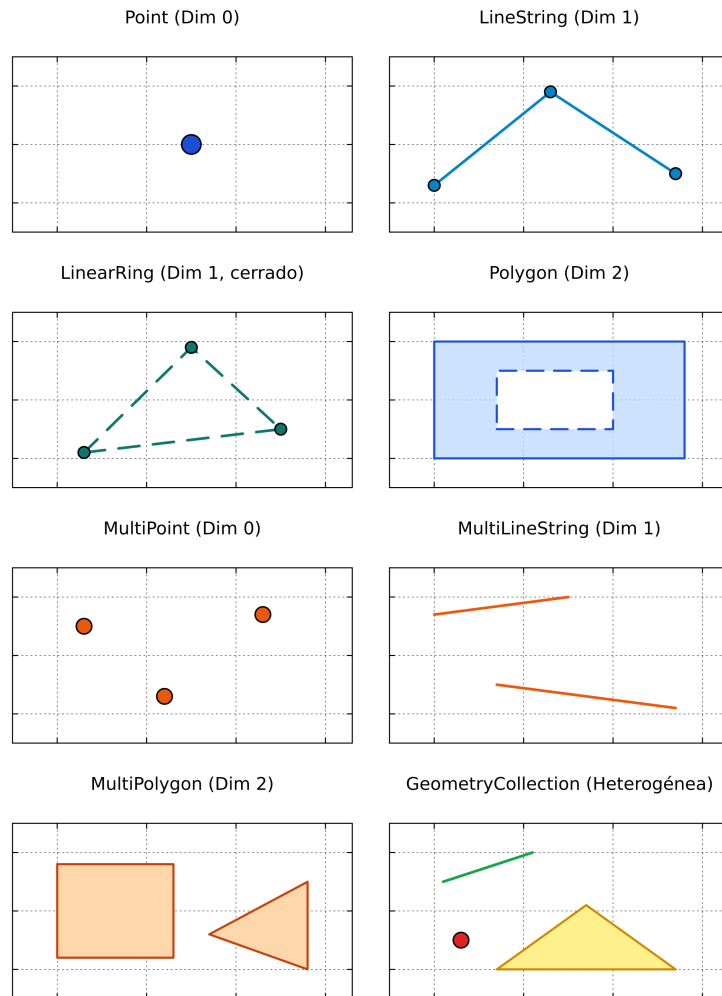


Figura 6.14: Instanciación espacial y morfología de tipos geométricos según el estándar OGC en Julia.

6.7.2 Terminal (psql)

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"

query_cols="
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'spatial_ref_sys';
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-P format=wrapped -P columns=90 \
-c "${query_cols}"
```

column_name	data_type
srid	integer
auth_name	character varying
auth_srid	integer
srtext	character varying
proj4text	character varying

(5 rows)

6.7.3 Python (SQLAlchemy)

```
from sqlalchemy import text

query_cols = """
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'spatial_ref_sys';
"""

with engine_local.connect() as conn:
    resultado = conn.execute(text(query_cols)).fetchall()
    print(f'column_name:<20> | {'data_type'}')
    print("-" * 45)
    for fila in resultado:
        print(f'fila.column_name:<20> | {'data_type'}')
```

column_name	data_type
srid	integer
auth_name	character varying
auth_srid	integer
srtext	character varying
proj4text	character varying

6.7.4 Python (Pandas)

```
import pandas as pd

query_cols = """
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'spatial_ref_sys';
```

```

"""
df_columnas = pd.read_sql(query_cols, con=engine_local)
df_columnas

```

	column_name	data_type
0	srid	integer
1	auth_name	character varying
2	auth_srid	integer
3	srtext	character varying
4	proj4text	character varying

Sus columnas clave comprenden:

- **srid**: Identificador entero del sistema de referencia espacial (habitualmente alineado con el estándar internacional EPSG).
- **auth_name**: Nombre del organismo emisor de la norma (comúnmente 'EPSG').
- **auth_srid**: Código numérico asignado por la autoridad cartográfica.
- **srtext**: Definición formal en formato WKT CRS (*Well-Known Text of Spatial Reference Systems*), requerida por clientes SIG y bibliotecas de procesamiento geométrico.
- **proj4text**: Cadena de inicialización de parámetros empleada por el motor de transformación cartográfica PROJ.

Al consultar las definiciones preinstaladas para Colombia, el motor lista los marcos geodésicos históricos y las fajas Gauss-Krüger tradicionales:

6.7.5 Terminal (psql)

```

export PGPASSWORD="${DOCKER_PG_PASSWORD}"

query_sistemas_colombia="
SELECT
    srid,
    auth_name,
    split_part(srtext, '\n', 2) AS nombre_sistema
FROM spatial_ref_sys
WHERE srtext ILIKE '%Colombia%' OR srtext ILIKE '%MAGNA-SIRGAS%'
ORDER BY srid;
"

psql -h ${DOCKER_PGHOST} -p ${DOCKER_PGPORT} -U ${DOCKER_PGUSER} -d ${DOCKER_PGDATABASE} \
-P format=wrapped -P columns=90 \
-c "${query_sistemas_colombia}"

```

srid	auth_name	nombre_sistema
3114	EPSG	MAGNA-SIRGAS / Colombia Far West zone
3115	EPSG	MAGNA-SIRGAS / Colombia West zone
3116	EPSG	MAGNA-SIRGAS / Colombia Bogota zone
3117	EPSG	MAGNA-SIRGAS / Colombia East Central zone
3118	EPSG	MAGNA-SIRGAS / Colombia East zone
4686	EPSG	MAGNA-SIRGAS
4996	EPSG	MAGNA-SIRGAS

```

4997 | EPSG      | MAGNA-SIRGAS
21891 | EPSG      | Bogota 1975 / Colombia West zone (deprecated)
21892 | EPSG      | Bogota 1975 / Colombia Bogota zone (deprecated)
21893 | EPSG      | Bogota 1975 / Colombia East Central zone (deprecated)
21894 | EPSG      | Bogota 1975 / Colombia East (deprecated)
21896 | EPSG      | Bogota 1975 / Colombia West zone
21897 | EPSG      | Bogota 1975 / Colombia Bogota zone
21898 | EPSG      | Bogota 1975 / Colombia East Central zone
21899 | EPSG      | Bogota 1975 / Colombia East
102231 | ESRI     | Colombia_West_West_Zone
102233 | ESRI     | MAGNA_Ciudad_Bogota
(18 rows)

```

6.7.6 Python (SQLAlchemy)

```

from sqlalchemy import text

query_sistemas_colombia = """
SELECT
    srid,
    auth_name,
    split_part(srtext, ' ', 2) AS nombre_sistema
FROM spatial_ref_sys
WHERE srtext ILIKE '%Colombia%' OR srtext ILIKE '%MAGNA-SIRGAS%'
ORDER BY srid;
"""

with engine_docker.connect() as conn:
    resultado = conn.execute(text(query_sistemas_colombia)).fetchall()
    print(f"{'srid':<8} | {'auth_name':<10} | {'nombre_sistema'}")
    print("-" * 90)
    for fila in resultado:
        nombre = fila.nombre_sistema if len(fila.nombre_sistema) <= 65 else fila.nombre_sistema[:62] +
        "..."
        print(f"{'srid':<8} | {'auth_name':<10} | {'nombre':<65}")

```

srid	auth_name	nombre_sistema
3114	EPSG	MAGNA-SIRGAS / Colombia Far West zone
3115	EPSG	MAGNA-SIRGAS / Colombia West zone
3116	EPSG	MAGNA-SIRGAS / Colombia Bogota zone
3117	EPSG	MAGNA-SIRGAS / Colombia East Central zone
3118	EPSG	MAGNA-SIRGAS / Colombia East zone
4686	EPSG	MAGNA-SIRGAS
4996	EPSG	MAGNA-SIRGAS
4997	EPSG	MAGNA-SIRGAS
21891	EPSG	Bogota 1975 / Colombia West zone (deprecated)
21892	EPSG	Bogota 1975 / Colombia Bogota zone (deprecated)
21893	EPSG	Bogota 1975 / Colombia East Central zone (deprecated)
21894	EPSG	Bogota 1975 / Colombia East (deprecated)
21896	EPSG	Bogota 1975 / Colombia West zone
21897	EPSG	Bogota 1975 / Colombia Bogota zone

```

21898      | EPSG          | Bogota 1975 / Colombia East Central zone
21899      | EPSG          | Bogota 1975 / Colombia East
102231     | ESRI          | Colombia_West_West_Zone
102233     | ESRI          | MAGNA_Ciudad_Bogota

```

6.7.7 Pvthon (Pandas)

```

import pandas as pd
from sqlalchemy import text

# Control estricto de visualización para terminal y celdas de texto
pd.set_option("display.width", 90)
pd.set_option("display.max_columns", None)
pd.set_option("display.max_colwidth", 60)

query_sistemas_colombia = """
SELECT
    srid,
    auth_name,
    split_part(srtext, ' ', 2) AS nombre_sistema
FROM spatial_ref_sys
WHERE srtext ILIKE '%Colombia%' OR srtext ILIKE '%MAGNA-SIRGAS%'
ORDER BY srid;
"""

with engine_docker.connect() as conn:
    df_sistemas = pd.read_sql(text(query_sistemas_colombia), con=conn)

df_sistemas

```

	srid	auth_name	nombre_sistema
0	3114	EPSG	MAGNA-SIRGAS / Colombia Far West zone
1	3115	EPSG	MAGNA-SIRGAS / Colombia West zone
2	3116	EPSG	MAGNA-SIRGAS / Colombia Bogota zone
3	3117	EPSG	MAGNA-SIRGAS / Colombia East Central zone
4	3118	EPSG	MAGNA-SIRGAS / Colombia East zone
5	4686	EPSG	MAGNA-SIRGAS
6	4996	EPSG	MAGNA-SIRGAS
7	4997	EPSG	MAGNA-SIRGAS
8	21891	EPSG	Bogota 1975 / Colombia West zone (deprecated)
9	21892	EPSG	Bogota 1975 / Colombia Bogota zone (deprecated)
10	21893	EPSG	Bogota 1975 / Colombia East Central zone (deprecated)
11	21894	EPSG	Bogota 1975 / Colombia East (deprecated)
12	21896	EPSG	Bogota 1975 / Colombia West zone
13	21897	EPSG	Bogota 1975 / Colombia Bogota zone
14	21898	EPSG	Bogota 1975 / Colombia East Central zone
15	21899	EPSG	Bogota 1975 / Colombia East
16	102231	ESRI	Colombia_West_West_Zone
17	102233	ESRI	MAGNA_Ciudad_Bogota

⚠ Ausencia del código EPSG 9377 en volcados iniciales de spatial_ref_sys

La tabla `spatial_ref_sys` es una tabla relacional física en disco poblada a partir de los scripts iniciales de PostGIS o volcados históricos (como `init_unal_spatialdb.sql`). A diferencia del software de escritorio moderno o la biblioteca PROJ (que actualizan sus parámetros en cada compilación contra `proj.db`), el catálogo relacional en PostgreSQL es estático y no se actualiza de forma automática.

El sistema oficial de Colombia con Origen Nacional Único fue adoptado por el Instituto Geográfico Agustín Codazzi mediante la Resolución IGAC 471 de 2020 e indexado en el registro internacional bajo el código **EPSG:9377**. Si el clúster se inicializó con un esquema previo a dicha norma, la tupla no existirá en disco.

Aunque el tipado declarativo de PostgreSQL permite crear columnas con `geometry(Point, 9377)` sin validar la tabla, cualquier operación analítica de cálculo o reproyección cartográfica (`ST_Transform`) fallará en tiempo de ejecución si el registro no reside en `spatial_ref_sys`.

Como labor fundamental de administración de bases de datos (DBA), el registro oficial debe ser incorporado al catálogo. **Esta instrucción DML solo requiere ejecutarse una única vez en la base de datos**, pues la cláusula `ON CONFLICT` garantiza su persistencia permanente sin duplicación:

6.7.8 Terminal (psql)

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"

query_insert_9377="
INSERT INTO spatial_ref_sys (srid, auth_name, auth_srid, srtext, proj4text)
VALUES (
    9377,
    'EPSG',
    9377,
    'PROJCS["MAGNA-SIRGAS / Origen-Nacional",GEOGCS["MAGNA-SIRGAS",DATUM["Marco_Geocentrico_Nacional_de_Referencia",SPHEROID["GRS
1980",6378137,298.257222101,AUTHORITY["EPSG","7019"]],AUTHORITY["EPSG","6686"]],PRIMEM["Greenwich",0,AUTHORITY["EPSG","8901"]],UNIT["degree",0.0174532925199433,AUTHORITY["EPSG","9122"]],AUTHORITY["EPSG","4686"]],PROJECTION["Transverse_Mercator"],PARAMETER["latitude_of_origin",4],PARAMETER["central_meridian",-73],PARAMETER["scale_factor",0.9992],PARAMETER["false_easting",5000000],PARAMETER["false_northing",2000000],UNIT["metre",1,AUTHORITY["EPSG","9001"]],AUTHORITY["EPSG","9377"]]',
    '+proj=tmerc +lat_0=4 +lon_0=-73 +k=0.9992 +x_0=5000000 +y_0=2000000 +ellps=GRS80 +tows84=0,0,0,0,0,0 +units=m +no_defs +type=crs'
)
ON CONFLICT (srid) DO UPDATE
SET srtext = EXCLUDED.srtext, proj4text = EXCLUDED.proj4text;
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-c "${query_insert_9377}"
```

```
INSERT 0 1
```

6.7.9 Python (SQLAlchemy)

```
from sqlalchemy import text

query_insert_9377 = """
INSERT INTO spatial_ref_sys (srid, auth_name, auth_srid, srtext, proj4text)
VALUES (
    9377,
    'EPSG',
```

```

9377,
'PROJCS["MAGNA-SIRGAS /
→ Origen-Nacional",GEOGCS["MAGNA-SIRGAS",DATUM["Marco_Geocentrico_Nacional_de_Referencia",SPHEROID["GRS
→ 1980",6378137,298.257222101,AUTHORITY["EPSG","7019"]],AUTHORITY["EPSG","6686"]],PRIMEM["Greenwich",0,AUTHORITY["EPSG","890
'proj=tmerc +lat_0=4 +lon_0=-73 +k=0.9992 +x_0=5000000 +y_0=2000000 +ellps=GRS80
→ +towgs84=0,0,0,0,0,0,0 +units=m +no_defs +type=crs'
)
ON CONFLICT (srid) DO UPDATE
SET srtext = EXCLUDED.srtext, proj4text = EXCLUDED.proj4text;
"""

with engine_local.begin() as conn:
    conn.execute(text(query_insert_9377))
    print("Registro EPSG:9377 incorporado/actualizado exitosamente en spatial_ref_sys.")

```

<sqlalchemy.engine.cursor.CursorResult object at 0x73009d1cb8c0>

Registro EPSG:9377 incorporado/actualizado exitosamente en spatial_ref_sys.

Mediante la cláusula `ON CONFLICT (srid) DO UPDATE`, la sentencia formaliza una operación atómica de tipo **UPSERT**, confiriendo **idempotencia** a la ejecución:

- **Primera compilación:** Al no existir el identificador 9377, el motor ejecuta la inserción estándar (`INSERT`) en el catálogo físico.
- **Compilaciones sucesivas:** El motor detecta la colisión sobre el índice de la clave primaria (`srid`) y, en lugar de abortar la transacción por infracción de unicidad (`duplicate key value violates unique constraint`), actualiza los parámetros sobre la tupla preexistente.

Este diseño garantiza que los ciclos continuos de renderizado en Quarto mantengan el catálogo en un estado consistente y libre de registros redundantes, sin interrumpir el flujo de publicación del texto técnico.

Una vez registrada la tupla en disco, la inspección detallada de los sistemas cartográficos de referencia para el país expone sus parámetros elipsoidales y de proyección:

6.7.10 Terminal (psql)

```

export PGPASSWORD="${LOCAL_PGPASSWORD}"

query_colombia="
SELECT srid, auth_name, auth_srid, srtext
FROM spatial_ref_sys
WHERE srid IN (4326, 3116, 9377)
ORDER BY srid;
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-x -P format=wrapped -P columns=90 \
-c "${query_colombia}"

```

```

-[ RECORD 1 ]-----
srid      | 3116
auth_name | EPSG
auth_srid | 3116
srtext    | PROJCS["MAGNA-SIRGAS / Colombia Bogota zone",GEOGCS["MAGNA-SIRGAS",DATUM["Mar.
| .co_Geocentrico_Nacional_de_Referencia",SPHEROID["GRS 1980",6378137,298.257222.
| .101,AUTHORITY["EPSG","7019"]],TOWGS84[0,0,0,0,0,0,0],AUTHORITY["EPSG","6686"].
| .],PRIMEM["Greenwich",0,AUTHORITY["EPSG","8901"]],UNIT["degree",0.017453292519.
| .9433,AUTHORITY["EPSG","9122"]],AUTHORITY["EPSG","4686"]],PROJECTION["Transver.

```

```

        |.se_Mercator"],PARAMETER["latitude_of_origin",4.596200416666666],PARAMETER["ce.
        |.ntral_meridian",-74.07750791666666],PARAMETER["scale_factor",1],PARAMETER["fa.
        |.lse_easting",1000000],PARAMETER["false_northing",1000000],UNIT["metre",1,AUTH.
        |.ORITY["EPSG","9001"]],AUTHORITY["EPSG","3116"]]
-[ RECORD 2 ]-----
srid      | 4326
auth_name | EPSG
auth_srid | 4326
srtext    | GEOGCS["WGS 84",DATUM["WGS_1984",SPHEROID["WGS 84",6378137,298.257223563,AUTH.
        |.ORITY["EPSG","7030"]],AUTHORITY["EPSG","6326"]],PRIMEM["Greenwich",0,AUTHORIT.
        |.Y["EPSG","8901"]],UNIT["degree",0.0174532925199433,AUTHORITY["EPSG","9122"]],.
        |.AUTHORITY["EPSG","4326"]]
-[ RECORD 3 ]-----
srid      | 9377
auth_name | EPSG
auth_srid | 9377
srtext    | PROJCS["MAGNA-SIRGAS / Origen-Nacional",GEOGCS["MAGNA-SIRGAS",DATUM["Marco_Ge.
        |.ocentrico_Nacional_de_Referencia",SPHEROID["GRS 1980",6378137,298.257222101,A.
        |.UTHORITY["EPSG","7019"]],AUTHORITY["EPSG","6686"]],PRIMEM["Greenwich",0,AUTHO.
        |.RITY["EPSG","8901"]],UNIT["degree",0.0174532925199433,AUTHORITY["EPSG","9122".
        |.]],AUTHORITY["EPSG","4686"]],PROJECTION["Transverse_Mercator"],PARAMETER["lat.
        |.itude_of_origin",4],PARAMETER["central_meridian",-73],PARAMETER["scale_factor.
        |.",0.9992],PARAMETER["false_easting",5000000],PARAMETER["false_northing",20000.
        |.00],UNIT["metre",1,AUTHORITY["EPSG","9001"]],AUTHORITY["EPSG","9377"]]

```

6.7.11 Python (SQLAlchemy)

```

import textwrap
from sqlalchemy import text

query_colombia = """
SELECT srid, auth_name, auth_srid, srtext
FROM spatial_ref_sys
WHERE srid IN (4326, 3116, 9377)
ORDER BY srid;
"""

with engine_local.connect() as conn:
    registros = conn.execute(text(query_colombia)).fetchall()
    for idx, r in enumerate(registros, start=1):
        print(f"-[ RECORD {idx} ]" + "-" * 75)
        print(f"srid      | {r.srid}")
        print(f"auth_name | {r.auth_name}")
        print(f"auth_srid | {r.auth_srid}")
        srtext_ajustado = textwrap.fill(r.srtext, width=76, subsequent_indent=" " * 12)
        print(f"srtext    | {srtext_ajustado}\n")

```

```

-[ RECORD 1 ]-----
srid      | 3116
auth_name | EPSG
auth_srid | 3116
srtext    | PROJCS["MAGNA-SIRGAS / Colombia Bogota zone",GEOGCS["MAGNA-

```

```

        SIRGAS",DATUM["Marco_Geocentrico_Nacional_de_Referencia",SPHEROID["GRS 1980",6378137,298.257222101,AUTHORITY["EPSG","7019"]],TOWGS84[0,0,0,0,0,0,0],AUTHORITY["EPSG","6686"]],PRIMEM["Greenwich",0,AUTHORITY["EPSG","8901"]],UNIT["degree",0.0174532925199433,AUTHORITY["EPSG","9122"]],AUTHORITY["EPSG","4686"]],PROJECTION["Transverse_Mercator"],PARAMETER["latitude_of_origin",4.596200416666666],PARAMETER["central_meridian",-74.07750791666666],PARAMETER["scale_factor",1],PARAMETER["false_easting",1000000],PARAMETER["false_northing",1000000],UNIT["metre",1,AUTHORITY["EPSG","9001"]],AUTHORITY["EPSG","3116"]]

-[ RECORD 2 ]-----
srid      | 4326
auth_name | EPSG
auth_srid | 4326
srtext    | GEOGCS["WGS 84",DATUM["WGS_1984",SPHEROID["WGS 84",6378137,298.257223563,AUTHORITY["EPSG","7030"]],AUTHORITY["EPSG","6326"]],PRIMEM["Greenwich",0,AUTHORITY["EPSG","8901"]],UNIT["degree",0.0174532925199433,AUTHORITY["EPSG","9122"]],AUTHORITY["EPSG","4326"]]

-[ RECORD 3 ]-----
srid      | 9377
auth_name | EPSG
auth_srid | 9377
srtext    | PROJCS["MAGNA-SIRGAS / Origen-Nacional",GEOGCS["MAGNA-SIRGAS",DATUM["Marco_Geocentrico_Nacional_de_Referencia",SPHEROID["GRS 1980",6378137,298.257222101,AUTHORITY["EPSG","7019"]],AUTHORITY["EPSG","6686"]],PRIMEM["Greenwich",0,AUTHORITY["EPSG","8901"]],UNIT["degree",0.0174532925199433,AUTHORITY["EPSG","9122"]],AUTHORITY["EPSG","4686"]],PROJECTION["Transverse_Mercator"],PARAMETER["latitude_of_origin",4],PARAMETER["central_meridian",-73],PARAMETER["scale_factor",0.9992],PARAMETER["false_easting",5000000],PARAMETER["false_northing",2000000],UNIT["metre",1,AUTHORITY["EPSG","9001"]],AUTHORITY["EPSG","9377"]]

```

6.7.12 Python (Pandas)

```

import pandas as pd

# Configuración del ancho máximo tabular a 90 columnas
pd.set_option("display.width", 90)
pd.set_option("display.max_columns", None)
pd.set_option("display.max_colwidth", 45)

query_colombia = """
SELECT srid, auth_name, auth_srid, srtext
FROM spatial_ref_sys
WHERE srid IN (4326, 3116, 9377)

```

```
ORDER BY srid;
"""

df_colombia = pd.read_sql(query_colombia, con=engine_local)
df_colombia
```

	srid	auth_name	auth_srid	srtext
0	3116	EPSG	3116	PROJCS["MAGNA-SIRGAS / Colombia Bogota zo...
1	4326	EPSG	4326	GEOGCS["WGS 84",DATUM["WGS_1984",SPHEROID...
2	9377	EPSG	9377	PROJCS["MAGNA-SIRGAS / Origen-Nacional",G...

Entre estos identificadores fundamentales destacan:

- **SRID 4326:** Sistema geográfico global WGS 84 (coordenadas angulares en grados decimales de latitud y longitud).
- **SRID 3116:** Proyección MAGNA-SIRGAS / Colombia Bogotá (proyección conforme Gauss-Krüger sobre meridiano central 74°04'39.0285" W, con falso este de 1 000 000 m y falso norte de 1 000 000 m).
- **SRID 9377:** Proyección MAGNA-SIRGAS / Origen Nacional Único (sistema oficial conforme a la Resolución IGAC 471 de 2020, con origen en 4° N, 73° W, factor de escala $k_0 = 0.9992$, falso este de 5 000 000 m y falso norte de 2 000 000 m).

6.7.13 La vista de catálogo geometry_columns

La especificación OGC SFA Parte 2 (cláusula 7.1.3) establece que toda relación que implemente atributos geométricos debe registrar sus metadatos descriptivos en el catálogo GEOMETRY_COLUMNS ([Open Geospatial Consortium, 2010](#)). Esta estructura provee a las aplicaciones clientes y motores de análisis la información indispensable para validar la dimensionalidad, el tipo de primitiva y el sistema de referencia de coordenadas sin necesidad de escanear la totalidad de las tuplas de la tabla.

En implementaciones tempranas de bases de datos espaciales, este catálogo operaba como una tabla física estática cuyo registro exigía procedimientos imperativos como `AddGeometryColumn()` o disparadores (*triggers*). A partir de PostGIS 2.0, y en concordancia con la cláusula 7.1.1 de la norma, `geometry_columns` se materializa como una vista del sistema construida dinámicamente por introspección sobre los catálogos relacionales de PostgreSQL (`pg_class`, `pg_attribute`, `pg_type`, `pg_namespace` y `pg_constraint`).

6.7.14 Terminal (psql)

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"

query_geom_cols="
SELECT f_table_schema, f_table_name, f_geometry_column, coord_dimension, srid, type
FROM geometry_columns
LIMIT 10;
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-P format=wrapped -P columns=90 \
-c "${query_geom_cols}"
```

f_table_schema	f_table_name	f_geometry_column	coord_dimension	srid	type
base750	hidro_lineas	the_geom	2	21892	GEOMETRY

base750		vias_lineas		the_geom		2		21892		GEOMETRY
base750		vias_puntos		the_geom		2		21892		ST_Point
base750		vias_texto		the_geom		2		21892		ST_Point
divpol		departamentos		the_geom		2		21892		GEOMETRY
divpol		municipios		the_geom		2		21892		GEOMETRY
divpol		capa_departame.		the_geom		2		21892		GEOMETRY
		.ntos								
divpol		capa_municipio.		the_geom		2		21892		GEOMETRY
		.s								
public		infraestructur.		geom		2		9377		POINT
		.a_referencia								
public		muestras_geome.		geom		2		0		GEOMETRY
		.trias_ogc								

(10 rows)

6.7.15 Python (SQLAlchemy)

```
from sqlalchemy import text

query_geom_cols = """
SELECT f_table_schema, f_table_name, f_geometry_column, coord_dimension, srid, type
FROM geometry_columns
LIMIT 10;
"""

with engine_local.connect() as conn:
    resultado = conn.execute(text(query_geom_cols)).fetchall()
    print(f"{'schema':<12} | {'table_name':<20} | {'geom_col':<16} | {'dim':<4} | {'srid':<7} | {'type'}")
    print("-" * 88)
    for r in resultado:
        print(f"{r.f_table_schema:<12} | {r.f_table_name:<20} | {r.f_geometry_column:<16} |
              ↪ {r.coord_dimension:<4} | {r.srid:<7} | {r.type}")
```

schema		table_name		geom_col		dim		srid		type

base750		hidro_lineas		the_geom		2		21892		GEOMETRY
base750		vias_lineas		the_geom		2		21892		GEOMETRY
base750		vias_puntos		the_geom		2		21892		ST_Point
base750		vias_texto		the_geom		2		21892		ST_Point
divpol		departamentos		the_geom		2		21892		GEOMETRY
divpol		municipios		the_geom		2		21892		GEOMETRY
divpol		capa_departamentos		the_geom		2		21892		GEOMETRY
divpol		capa_municipios		the_geom		2		21892		GEOMETRY
public		infraestructura_referencia		geom		2		9377		POINT
public		muestras_geometrias_ogc		geom		2		0		GEOMETRY

6.7.16 Python (Pandas)

```
import pandas as pd
```

```
# Calibración del ancho y columnas para salida tabular estricta de 90 caracteres
pd.set_option("display.width", 90)
pd.set_option("display.max_columns", None)
pd.set_option("display.max_colwidth", 25)

query_geom_cols = """
SELECT f_table_schema, f_table_name, f_geometry_column, coord_dimension, srid, type
FROM geometry_columns
LIMIT 10;
"""

df_geom_cols = pd.read_sql(query_geom_cols, con=engine_local)
df_geom_cols
```

	f_table_schema	f_table_name	f_geometry_column	coord_dimension	srid	\
0	base750	hidro_lineas	the_geom	2	21892	
1	base750	vias_lineas	the_geom	2	21892	
2	base750	vias_puntos	the_geom	2	21892	
3	base750	vias_texto	the_geom	2	21892	
4	divpol	departamentos	the_geom	2	21892	
5	divpol	municipios	the_geom	2	21892	
6	divpol	capa_departamentos	the_geom	2	21892	
7	divpol	capa_municipios	the_geom	2	21892	
8	public	infraestructura_refer...	geom	2	9377	
9	public	muestras_geometrias_ogc	geom	2	0	

	type
0	GEOMETRY
1	GEOMETRY
2	ST_Point
3	ST_Point
4	GEOMETRY
5	GEOMETRY
6	GEOMETRY
7	GEOMETRY
8	POINT
9	GEOMETRY

Los campos descriptivos obligatorios según la norma OGC son:

- **f_table_schema**: Esquema relacional donde reside la tabla (ej. public, base750, divpol).
- **f_table_name**: Nombre de la tabla que contiene la capa espacial.
- **f_geometry_column**: Nombre de la columna que almacena los objetos espaciales (convencionalmente geom o the_geom).
- **coord_dimension**: Dimensión de las coordenadas almacenadas (2 para XY, 3 para XYZ/XYM, o 4 para XYZM).
- **srid**: Identificador entero del sistema de referencia espacial asociado.
- **type**: Restricción del tipo geométrico (POINT, LINESTRING, POLYGON, o GEOMETRY si admite tipos mixtos).

Para realizar una auditoría integral de la base de datos pedagógica sig_db_unal, se combina la vista geometry_columns con la tabla spatial_ref_sys. Esta consulta consolida los marcos de referencia en uso

real, identificando las capas cartográficas existentes por esquema y detectando capas sin proyección asignada (srid: 0):

6.7.17 Terminal (psql)

```
export PGPASSWORD="${DOCKER_PGPASSWORD}"

query_crs_en_uso="
SELECT
    g.srid,
    COALESCE(s.auth_name, 'LOCAL') AS autoridad,
    COALESCE(split_part(s.srtext, '\', 2), 'Sin CRS asignado (Plano / Indefinido)') AS descripcion_crs,
    count(*) AS numero_de_capas,
    string_agg(g.f_table_schema || '.' || g.f_table_name, ', ') AS capas_asociadas
FROM geometry_columns g
LEFT JOIN spatial_ref_sys s ON g.srid = s.srid
GROUP BY g.srid, s.auth_name, s.srtext
ORDER BY g.srid;
"

psql -h ${DOCKER_PGHOST} -p ${DOCKER_PGPORT} -U ${DOCKER_PGUSER} -d ${DOCKER_PGDATABASE} \
-x -P format=wrapped -P columns=90 \
-c "${query_crs_en_uso}"
```

```
-[ RECORD 1 ]---+-----
srid          | 0
autoridad     | LOCAL
descripcion_crs | Sin CRS asignado (Plano / Indefinido)
numero_de_capas | 2
capas_asociadas | divpol.capa_departamentos, divpol.capa_municipios

-[ RECORD 2 ]---+-----
srid          | 21892
autoridad     | EPSG
descripcion_crs | Bogota 1975 / Colombia Bogota zone (deprecated)
numero_de_capas | 6
capas_asociadas | base750.vias_puntos, base750.vias_texto, base750.hidro_lineas, divpol.m.
                  | .unicipios, divpol.departamentos, base750.vias_lineas
```

6.7.18 Python (SQLAlchemy)

```
import textwrap
from sqlalchemy import text

query_crs_en_uso = """
SELECT
    g.srid,
    COALESCE(s.auth_name, 'LOCAL') AS autoridad,
    COALESCE(split_part(s.srtext, '\', 2), 'Sin CRS asignado (Plano / Indefinido)') AS descripcion_crs,
    count(*) AS numero_de_capas,
    string_agg(g.f_table_schema || '.' || g.f_table_name, ', ') AS capas_asociadas
FROM geometry_columns g
LEFT JOIN spatial_ref_sys s ON g.srid = s.srid
GROUP BY g.srid, s.auth_name, s.srtext
ORDER BY g.srid;
"""

with engine_docker.connect() as conn:
    registros = conn.execute(text(query_crs_en_uso)).fetchall()
    for idx, r in enumerate(registros, start=1):
        print(f"-[ RECORD {idx} ]" + "-" * 75)
        print(f"srid          | {r.srid}")
```

```

print(f"autoridad      | {r.autoridad}")
desc_crs = textwrap.fill(r.descripcion_crs, width=70, subsequent_indent=" " * 18)
print(f"descripcion_crs | {desc_crs}")
print(f"numero_de_capas | {r.numero_de_capas}")
capas = textwrap.fill(r.capas_asociadas, width=70, subsequent_indent=" " * 18)
print(f"capas_asociadas | {capas}\n")

```

```

-[ RECORD 1 ]-----
srid          | 0
autoridad     | LOCAL
descripcion_crs | Sin CRS asignado (Plano / Indefinido)
numero_de_capas | 2
capas_asociadas | divpol.capa_departamentos, divpol.capa_municipios

-[ RECORD 2 ]-----
srid          | 21892
autoridad     | EPSG
descripcion_crs | Bogota 1975 / Colombia Bogota zone (deprecated)
numero_de_capas | 6
capas_asociadas | base750.vias_puntos, base750.vias_texto, base750.hidro_lineas,
                  divpol.municipios, divpol.departamentos,
                  base750.vias_lineas

```

6.7.19 Python (Pandas)

```

import pandas as pd

# Control de ancho y corte de columnas para salida tabular acotada a 90 caracteres
pd.set_option("display.width", 90)
pd.set_option("display.max_columns", None)
pd.set_option("display.max_colwidth", 32)

query_crs_en_uso = """
SELECT
    g.srid,
    COALESCE(s.auth_name, 'LOCAL') AS autoridad,
    COALESCE(split_part(s.srtext, ' ', 2), 'Sin CRS asignado (Plano / Indefinido)') AS descripcion_crs,
    count(*) AS numero_de_capas,
    string_agg(g.f_table_schema || ' ' || g.f_table_name, ' ' AS capas_asociadas
FROM geometry_columns g
LEFT JOIN spatial_ref_sys s ON g.srid = s.srid
GROUP BY g.srid, s.auth_name, s.srtext
ORDER BY g.srid;
"""

df_crs_en_uso = pd.read_sql(query_crs_en_uso, con=engine_docker)
df_crs_en_uso

```

```

      srid autoridad      descripcion_crs  numero_de_capas \
0         0      LOCAL Sin CRS asignado (Plano / In...      2
1    21892      EPSG  Bogota 1975 / Colombia Bogot...      6

      capas_asociadas
0  divpol.capa_departamentos, d...
1  base750.vias_puntos, base750...

```

6.7.20 Asignación de identificador espacial en las vistas del esquema

Al examinar los resultados de la consulta cruzada anterior, se constata que las relaciones `capa_departamentos` y `capa_municipios` del esquema `divpol` reportan `srId: 0`. Esto ocurre porque ambos objetos son vistas relacionales (consultas almacenadas) en las que la columna geométrica fue proyectada sin un identificador de referencia explícito, lo que ocasiona que los clientes SIG interpreten la proyección como indefinida.

Aunque los fundamentos formales de DDL y DML para el diseño avanzado de vistas espaciales se estudiarán detalladamente en capítulos posteriores, es indispensable regularizar el catálogo desde esta primera sesión. La solución consiste en redefinir ambas vistas forzando la conversión explícita de la columna geométrica al sistema de coordenadas de la cartografía base de la base de datos `sig_db_unal`, es decir EPSG:21892, *Bogotá 1975 / Colombia Bogotá zone* (asociado al elipsoide Internacional 1924 / Hayford y datum Bogotá 1975):

6.7.21 Terminal (psql)

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"

# Redefinición de vistas con asignación explícita de SRID 21892
query_recreacion_vistas="
DROP VIEW IF EXISTS divpol.capa_departamentos;
CREATE VIEW divpol.capa_departamentos AS
SELECT
    d.gid,
    d.gid AS oid,
    d.dep_id AS id_dane,
    d.dep_nombre AS departamento,
    d.the_geom::geometry(Geometry, 21892) AS the_geom
FROM divpol.departamentos d
WHERE d.the_geom IS NOT NULL
ORDER BY d.dep_id;

DROP VIEW IF EXISTS divpol.capa_municipios;
CREATE VIEW divpol.capa_municipios AS
SELECT
    m.gid,
    m.gid AS oid,
    m.mun_id AS id_dane,
    d.dep_id AS dep_codigo_dane,
    initcap(d.dep_nombre::text) AS departamento,
    m.mun_nombre AS municipio,
    m.the_geom::geometry(Geometry, 21892) AS the_geom
FROM divpol.municipios m
LEFT JOIN divpol.departamentos d ON d.dep_id::text = m.dep_id::text
WHERE m.the_geom IS NOT NULL
ORDER BY m.mun_id;
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-c "${query_recreacion_vistas}"
```

```
DROP VIEW
CREATE VIEW
DROP VIEW
CREATE VIEW
```

6.7.22 Python (SQLAlchemy)

```
from sqlalchemy import text

query_recreacion_vistas = """
DROP VIEW IF EXISTS divpol.capa_departamentos;
CREATE VIEW divpol.capa_departamentos AS
SELECT
```

```

d.gid,
d.gid AS oid,
d.dep_id AS id_dane,
d.dep_nombre AS departamento,
d.the_geom::geometry(Geometry, 21892) AS the_geom
FROM divpol.departamentos d
WHERE d.the_geom IS NOT NULL
ORDER BY d.dep_id;

DROP VIEW IF EXISTS divpol.capa_municipios;
CREATE VIEW divpol.capa_municipios AS
SELECT
m.gid,
m.gid AS oid,
m.mun_id AS id_dane,
d.dep_id AS dep_codigo_dane,
initcap(d.dep_nombre::text) AS departamento,
m.mun_nombre AS municipio,
m.the_geom::geometry(Geometry, 21892) AS the_geom
FROM divpol.municipios m
LEFT JOIN divpol.departamentos d ON d.dep_id::text = m.dep_id::text
WHERE m.the_geom IS NOT NULL
ORDER BY m.mun_id;
"""

with engine_local.begin() as conn:
    conn.execute(text(query_recreacion_vistas))
    print(
        "Vistas divpol.capa_departamentos y divpol.capa_municipios\n"
        "recreadas exitosamente con SRID 21892."
    )

```

<sqlalchemy.engine.cursor.CursorResult object at 0x73009d1cbe70>

Vistas divpol.capa_departamentos y divpol.capa_municipios

recreadas exitosamente con SRID 21892.

Tras la ejecución del bloque anterior, se audita nuevamente el catálogo `geometry_columns` para validar la correcta inscripción del identificador espacial:

6.7.23 Terminal (psql)

```

export PGPASSWORD="${LOCAL_PGPASSWORD}"

query_verif_vistas="
SELECT
f_table_schema,
f_table_name,
f_geometry_column,
coord_dimension,
srid,
type
FROM geometry_columns
WHERE f_table_schema = 'divpol'
  AND f_table_name IN ('capa_departamentos', 'capa_municipios');
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-P format=wrapped -P columns=90 \
-c "${query_verif_vistas}"

```

f_table_schema	f_table_name	f_geometry_column	coord_dimension	srid	type
divpol	capa_departame.	the_geom	2	21892	GEOMETRY
	.ntos				
divpol	capa_municipio.	the_geom	2	21892	GEOMETRY
	.s				

(2 rows)

6.7.24 Python (SQLAlchemy)

```

from sqlalchemy import text

query_verif_vistas = """
SELECT
    f_table_schema,
    f_table_name,
    f_geometry_column,
    coord_dimension,
    srid,
    type
FROM geometry_columns
WHERE f_table_schema = 'divpol'
    AND f_table_name IN ('capa_departamentos', 'capa_municipios');
"""

with engine_local.connect() as conn:
    resultado = conn.execute(text(query_verif_vistas)).fetchall()
    print(f"{'schema':<10} | {'table_name':<20} | {'geom_column':<15} | {'dim':<4} | {'srid':<6} | {'type':<10}")
    print("-" * 75)
    for r in resultado:
        print(f"{r.f_table_schema:<10} | {r.f_table_name:<20} | {r.f_geometry_column:<15} | {r.coord_dimension:<4} | {r.srid:<6} | {r.type:<10}")

```

schema	table_name	geom_column	dim	srid	type
divpol	capa_departamentos	the_geom	2	21892	GEOMETRY
divpol	capa_municipios	the_geom	2	21892	GEOMETRY

6.7.25 Python (Pandas)

```

import pandas as pd

# Calibración de parámetros de visualización para salida tabular de 90 caracteres
pd.set_option("display.width", 90)
pd.set_option("display.max_columns", None)
pd.set_option("display.max_colwidth", 25)

query_verif_vistas = """
SELECT
    f_table_schema,
    f_table_name,
    f_geometry_column,
    coord_dimension,
    srid,
    type
FROM geometry_columns
WHERE f_table_schema = 'divpol'
    AND f_table_name IN ('capa_departamentos', 'capa_municipios');
"""

df_vistas_srid = pd.read_sql(query_verif_vistas, con=engine_local)
df_vistas_srid

```

	f_table_schema	f_table_name	f_geometry_column	coord_dimension	srid	type
0	divpol	capa_departamentos	the_geom	2	21892	GEOMETRY
1	divpol	capa_municipios	the_geom	2	21892	GEOMETRY

El catálogo reporta ahora el identificador 21892, garantizando la interoperabilidad con clientes de escritorio y servicios web.

💡 Persistencia del catálogo mediante respaldo

Tras incorporar el sistema EPSG:9377 en el catálogo `spatial_ref_sys` y corregir el tipado de las vistas del esquema `divpol`, el catálogo relacional alcanza un estado consistente con el estándar OGC.

Para preservar estas modificaciones frente a eventuales destrucciones del volumen persistente, fallos del contenedor o reinicializaciones del clúster local, se recomienda generar un nuevo respaldo de la base de datos `sig_db_unal` aplicando los procedimientos documentados en la [Capítulo 5](#). Disponer de un archivo de volcado actualizado (en formato SQL plano o binario personalizado `.dump`) permite reconstruir la base de datos con las definiciones espaciales saneadas, eliminando la necesidad de reejecutar manualmente las sentencias de remediación en futuras restauraciones.

6.8 Verificación práctica en la consola integrada

A continuación se demuestra la secuencia operativa para comprobar la disponibilidad de la extensión espacial, aprovisionar una tabla pedagógica con tipado geométrico estricto, insertar una entidad mediante sintaxis WKT y auditar su catalogación automática:

6.8.1 Terminal (psql)

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"

# 1. Inspección de extensiones espaciales activas
psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-P format=wrapped -P columns=90 \
-c "SELECT extname, extversion FROM pg_extension WHERE extname = 'postgis';"

# 2. Creación e inserción pedagógica (con limpieza previa para asegurar idempotencia)
query_ddl="
DROP TABLE IF EXISTS public.infraestructura_referencia;

CREATE TABLE public.infraestructura_referencia (
  id serial PRIMARY KEY,
  codigo varchar(10) NOT NULL UNIQUE,
  descripcion text,
  geom geometry(Point, 9377) NOT NULL
);

INSERT INTO public.infraestructura_referencia (codigo, descripcion, geom)
VALUES (
  'EST-001',
  'Estación de monitoreo central',
  ST_GeomFromText('POINT(5000000 2000000)', 9377)
);
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-c "${query_ddl}"

# 3. Auditoría de registro automático en geometry_columns
query_audit="
SELECT f_table_schema, f_table_name, f_geometry_column, coord_dimension, srid, type
FROM geometry_columns
WHERE f_table_name = 'infraestructura_referencia';
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-P format=wrapped -P columns=90 \
-c "${query_audit}"
```

```
extname | extversion
-----+-----
postgis | 3.6.2
```

(1 row)

DROP TABLE

CREATE TABLE

INSERT 0 1

f_table_schema	f_table_name	f_geometry_column	coord_dimension	srid	type
public	infraestructura_ref.	geom	2	9377	POIN.
	referencia				.T

(1 row)

6.8.2 Python (SQLAlchemy)

```
from sqlalchemy import text

# 1. Inspección de extensiones espaciales activas
with engine_local.connect() as conn:
    ext = conn.execute(text("SELECT extname, extversion FROM pg_extension WHERE extname =
    ↪ 'postgis';")).fetchone()
    print(f"Extensión activa: {ext.extname} | Versión: {ext.extversion}\n")
```

Extensión activa: postgis | Versión: 3.6.2

```
# 2. Creación e inserción pedagógica (transacción atómica)
query_ddl = """
DROP TABLE IF EXISTS public.infraestructura_referencia;

CREATE TABLE public.infraestructura_referencia (
    id serial PRIMARY KEY,
    codigo varchar(10) NOT NULL UNIQUE,
    descripcion text,
    geom geometry(Point, 9377) NOT NULL
);

INSERT INTO public.infraestructura_referencia (codigo, descripcion, geom)
VALUES (
    'EST-001',
    'Estación de monitoreo central',
    ST_GeomFromText('POINT(5000000 2000000)', 9377)
);
"""

with engine_local.begin() as conn:
    conn.execute(text(query_ddl))
    print("Tabla public.infraestructura_referencia aprovisionada e inserción confirmada.\n")
```

<sqlalchemy.engine.cursor.CursorResult object at 0x73009d1cbf50>

Tabla public.infraestructura_referencia aprovisionada e inserción confirmada.

```
# 3. Auditoría de registro automático en geometry_columns
query_audit = """
SELECT f_table_schema, f_table_name, f_geometry_column, coord_dimension, srid, type
FROM geometry_columns
WHERE f_table_name = 'infraestructura_referencia';
"""

with engine_local.connect() as conn:
    resultado = conn.execute(text(query_audit)).fetchall()
    print(f"{'schema':<10} | {'table_name':<28} | {'geom_column':<12} | {'dim':<4} | {'srid':<6} |
    ↪ {'type':<10}")
    print("-" * 75)
```

```
for r in resultado:
    print(f"{r.f_table_schema:<10} | {r.f_table_name:<28} | {r.f_geometry_column:<12} |
          ↳ {r.coord_dimension:<4} | {r.srid:<6} | {r.type}")
```

schema	table_name	geom_column	dim	srid	type
public	infraestructura_referencia	geom	2	9377	POINT

6.8.3 Python (Pandas)

```
import pandas as pd
from sqlalchemy import text

# Configuración del ancho máximo tabular a 90 columnas
pd.set_option("display.width", 90)
pd.set_option("display.max_columns", None)
pd.set_option("display.max_colwidth", 30)

# 1. Inspección de extensiones espaciales activas
df_ext = pd.read_sql(
    "SELECT extname, extversion FROM pg_extension WHERE extname = 'postgis';",
    con=engine_local
)
print("Estado de la extensión espacial:")
```

Estado de la extensión espacial:

```
print(df_ext.to_string(index=False))
```

```
extname extversion
postgis      3.6.2
```

```
print("\n" + "-" * 60 + "\n")
```

```
# 2. Creación e inserción pedagógica mediante el motor transaccional
query_ddl = """
DROP TABLE IF EXISTS public.infraestructura_referencia;

CREATE TABLE public.infraestructura_referencia (
    id serial PRIMARY KEY,
    codigo varchar(10) NOT NULL UNIQUE,
    descripcion text,
    geom geometry(Point, 9377) NOT NULL
);

INSERT INTO public.infraestructura_referencia (codigo, descripcion, geom)
VALUES (
    'EST-001',
    'Estación de monitoreo central',
    ST_GeomFromText('POINT(5000000 2000000)', 9377)
);
"""

with engine_local.begin() as conn:
    conn.execute(text(query_ddl))
```

<sqlalchemy.engine.cursor.CursorResult object at 0x73009cd3c280>

```
# 3. Auditoría de registro automático en geometry_columns
query_audit = """
SELECT f_table_schema, f_table_name, f_geometry_column, coord_dimension, srid, type
FROM geometry_columns
WHERE f_table_name = 'infraestructura_referencia';
"""

df_audit = pd.read_sql(query_audit, con=engine_local)
df_audit
```

```
f_table_schema      f_table_name f_geometry_column  coord_dimension  srid  \
0      public  infraestructura_referencia      geom      2  9377

type
0  POINT
```

La consulta a `geometry_columns` reporta de forma automática la dimensión 2, el SRID 9377 y el tipo de dato POINT, validando la interoperabilidad del clúster con clientes SIG externos.

6.9 Articulación con los proyectos de investigación

Los fundamentos examinados a lo largo de este capítulo —la transición de archivos a repositorios transaccionales, la división modular del SGBD, la sobrecarga del tránsito por la pila de protocolos OSI en la arquitectura cliente-servidor y la centralización de geometrías tipadas bajo estándares OGC— constituyen la línea base teórica sobre la cual se estructuran los proyectos de investigación aplicada del Eje 4.

Cada tecnología emergente abordada en las semanas 11 a 16 no surge para reemplazar el núcleo relacional con garantías ACID, sino para mitigar una tensión arquitectónica específica cuando el volumen, la frecuencia de ingesta o la topología de las consultas desbordan los mecanismos tradicionales del motor objeto-relacional.

En la Tabla 6.7 se formaliza la correspondencia entre los componentes conceptuales desarrollados en esta sesión introductoria y las hipótesis experimentales que cada equipo de investigación defenderá en los seminarios finales.

Tabla 6.7: Matriz de articulación conceptual entre los fundamentos del capítulo y los proyectos finales de investigación.

Proyecto y tecnología (Eje 4)	Eje conceptual abordado en el capítulo	Tensión arquitectónica evaluada	Objeto del taller y contraste experimental
Tema 01: DuckDB espacial y GeoParquet	Topología cliente-servidor, almacenamiento columnar (OLAP) y modelo embebido (<i>in-process</i>).	La sobrecarga de red por sockets TCP, llamadas al sistema (<i>syscalls</i>) y la ineficiencia del <i>heap</i> <i>storage</i> de 8 KB en escaneos masivos.	Cuantificar la ganancia computacional de suprimir la pila OSI y evaluar la delegación de filtros (<i>predicate</i> <i>pushdown</i>) directamente sobre archivos GeoParquet en almacenamiento de nube.

Proyecto y tecnología (Eje 4)	Eje conceptual abordado en el capítulo	Tensión arquitectónica evaluada	Objeto del taller y contraste experimental
Tema 02: Apache Sedona y PySpark	Distinción entre SGBD acoplado y motor de cómputo analítico distribuido.	El límite físico de escalabilidad vertical (<i>scale-up</i>) del <i>buffer pool</i> local al ejecutar cruces espaciales (<i>Spatial Joins</i>) masivos.	Evaluar el particionamiento multidimensional distribuido (<i>Quad-Tree</i> / <i>KDB-Tree</i>) y la optimización de grafos acíclicos dirigidos (DAG) para mitigar el trasiego de datos (<i>data shuffling</i>) en clúster.
Tema 03: Mallas discretas globales (Uber H3 y Google S2)	Indexación multidimensional continua (GiST / R-Tree) y cajas mínimas delimitadoras (MBR).	La ineficiencia y solapamiento de árboles espaciales continuos frente a telemetría masiva de alta densidad.	Contrastar la indexación geométrica planar continua con índices escalares discretos unidimensionales generados por curvas de llenado del espacio, evaluando la distorsión métrica y la velocidad de agregación temporal.
Tema 04: Neo4j Spatial frente a pgRouting	Modelos NoSQL de grafos de propiedades (ISO GQL) frente al modelo objeto-relacional normalizado.	El costo algorítmico exponencial de evaluar relaciones topológicas continuas mediante operaciones de combinación relacional (<i>joins</i>) indexadas.	Medir la latencia en algoritmos de ruteo dinámico (Dijkstra / A) <i>aprovechando la adyacencia libre de índices</i> (index-free adjacency*) en grafos nativos frente a la indexación relacional en PostGIS.
Tema 05: Elasticsearch / OpenSearch frente a MongoDB espacial	Esquemas dinámicos en lectura (<i>schema-on-read</i>) y transaccionalidad BASE frente a garantías ACID estrictas.	La rigidez del esquema relacional en escritura (<i>schema-on-write</i>) ante flujos desestructurados de eventos y telemetría de sensores IoT en tiempo real.	Evaluar la tasa de ingestión concurrente masiva y la velocidad de filtrado geoespacial (<i>geo-fencing</i>) utilizando árboles BKD y mallas 2dsphere.

Proyecto y tecnología (Eje 4)	Eje conceptual abordado en el capítulo	Tensión arquitectónica evaluada	Objeto del taller y contraste experimental
Tema 06: Programación procedural y extensiones nativas en Rust (PGRX)	Serialización de geometrías (WKB/EWKB), extensibilidad de bajo nivel y ejecución compilada en el motor.	La penalización de rendimiento derivada de la deserialización de flujos binarios entre el espacio de usuario y las interfaces dinámicas en lenguajes interpretados.	Desarrollar rutinas analíticas espaciales compiladas directamente contra la memoria interna del servidor (GSERIALIZED), midiendo el balance entre seguridad de memoria y aceleración computacional sin salir del proceso gestor.

Comprender esta correspondencia permite concebir la infraestructura geoespacial corporativa no como una tecnología aislada, sino como un ecosistema articulado bajo el principio de **persistencia políglota**: el motor relacional espacial opera como el repositorio maestro transaccional que salvaguarda la integridad y la topología legal de los datos, mientras que las arquitecturas especializadas resuelven cuellos de botella de escala, conectividad analítica y latencia masiva.

i Consulta terminológica y acrónimos: El catálogo formal de siglas técnicas (OGC, ISO, ACID, MVCC), junto con la definición de los invariantes del modelo objeto-relacional y las estructuras de indexación multidimensional, se encuentra disponible para consulta transversal en la [Apéndice H](#).

Capítulo 7

Modelado conceptual, diseño lógico e implementación física espacial

7.1 Fases en el ciclo de vida de una base de datos espacial

Una base de datos se define formalmente como una colección integrada y estructurada de datos interrelacionados, almacenados sin redundancias perjudiciales, cuya finalidad es servir concurrentemente a múltiples aplicaciones y perfiles de usuario de forma independiente a los programas que gestionan su procesamiento. En los sistemas de información geográfica (SIG), el diseño de una base de datos espacial no se reduce a la digitalización o conversión directa de capas cartográficas aisladas, sino que constituye un proceso metodológico de modelado de datos e ingeniería de software. Siguiendo el marco metodológico planteado por Yeung y Hall en *Spatial Database Systems: Design, Implementation, and Project Management*, el desarrollo de una infraestructura de datos espaciales debe transformar las demandas y dinámicas del territorio en un repositorio operacional eficiente, coherente y mantenible.

7.1.1 De la abstracción territorial al esquema operacional

El punto de partida ineludible en el desarrollo de una base de datos espacial son los **requerimientos de información** y las **reglas operativas** de la organización o del problema territorial en estudio. La base de datos no debe estructurarse a partir de los formatos de archivo preexistentes (como capas Shapefile o GeoPackage dispersas), sino desde la **identificación** rigurosa de las **entidades** del mundo real, sus **propiedades** espaciales y alfanuméricas, y las **relaciones** topológicas y funcionales que las articulan.

El espacio geográfico presenta una continuidad multidimensional compleja. Para gestionar esta complejidad, el diseño descompone la realidad en **niveles progresivos de abstracción**, filtrando el detalle innecesario para consolidar un modelo estructurado capaz de responder a consultas analíticas, procesos de ordenamiento territorial y transacciones concurrentes.

El ciclo de desarrollo recorre etapas secuenciales e iterativas:

- **Levantamiento y análisis de requerimientos:** Identificación formal de los problemas espaciales a resolver, los actores involucrados, los flujos de información y los límites del sistema.

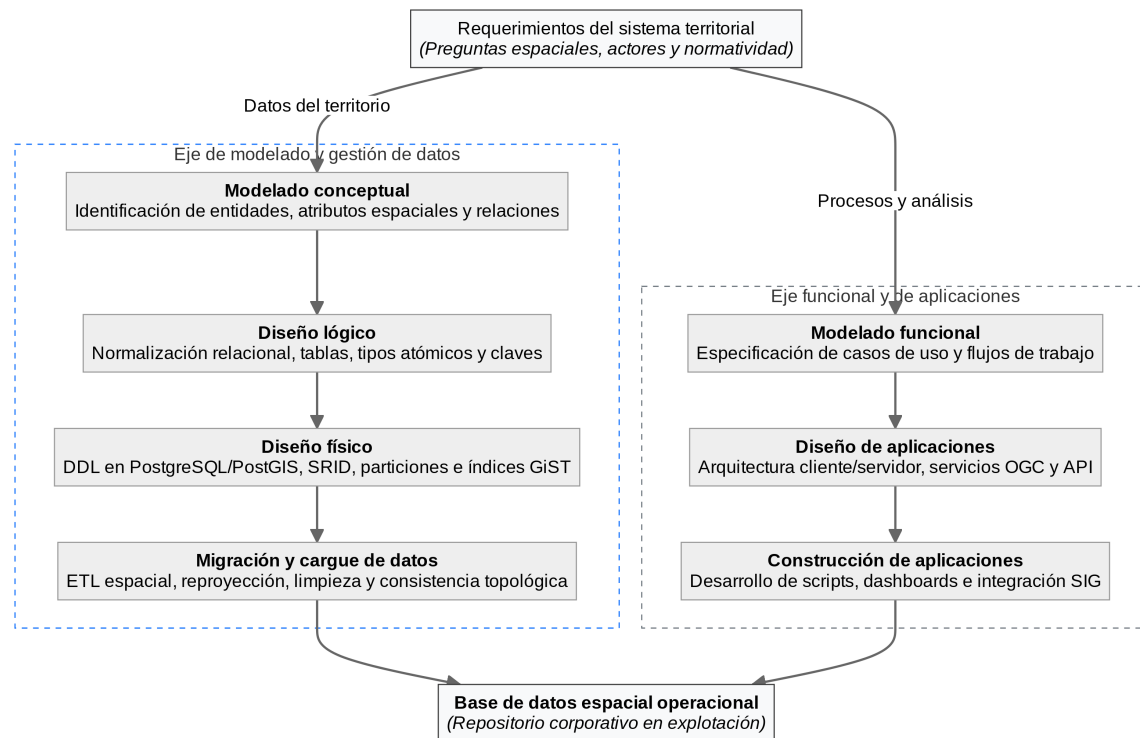


Figura 7.1: Fases en el desarrollo de una base de datos espacial operacional.

- **Modelado conceptual:** Abstracción del dominio territorial en entidades, atributos y relaciones, sin vincularse a plataformas tecnológicas o gestores de bases de datos específicos.
- **Diseño lógico:** Traducción de las estructuras conceptuales al paradigma relacional u objeto-relacional mediante tablas, columnas tipificadas y restricciones de integridad referencial.
- **Diseño e implementación física:** Materialización del esquema lógico en un motor de base de datos (PostgreSQL/PostGIS) utilizando sentencias DDL, configuración de tipos geométricos, SRID, particiones e índices espaciales.
- **Construcción, migración y explotación:** Carga inicial de datos, auditoría de consistencia topológica, definición de vistas analíticas e integración con clientes GIS y servicios web geográficos.

7.1.2 Niveles de modelado: conceptual, lógico y físico

Para asegurar la independencia entre las reglas de negocio del territorio y los mecanismos computacionales de almacenamiento, el proceso de diseño se articula en tres niveles de abstracción:

- **Modelo conceptual:** Corresponde a la fase con el nivel de abstracción más elevado. Su objetivo es identificar las **clases** de objetos del territorio, también llamados **entidades** relevantes para el sistema (por ejemplo: municipios, departamentos, cuencas, redes de transporte o zonas de reserva) y las **relaciones** que existen entre ellas. Este modelo es completamente independiente de cualquier consideración de hardware, software o sistema de gestión de bases de datos (SGBD/DBMS). Se formaliza comúnmente mediante el modelo **Entidad-Relación (E-R)** (donde se utiliza el término *entidades*) o **diagramas de clases** (donde se utiliza el término *clases*) del Lenguaje Unificado de Modelado (UML).

- **Diseño lógico:** Traduce las entidades y relaciones del modelo conceptual a las **estructuras canónicas** del paradigma relacional. En esta etapa, las **entidades** se transforman en **relaciones (tablas)**, los **atributos** en **columnas** con tipos de datos atómicos, los **identificadores únicos** en **claves primarias (PK)**, y las **asociaciones** en **claves foráneas (FK)** o **tablas de asociación** para cardinalidades muchos a muchos ($N : M$). Aunque este modelo estandariza la integridad de los datos, permanece desacoplado de las extensiones internas o particularidades de un motor específico.

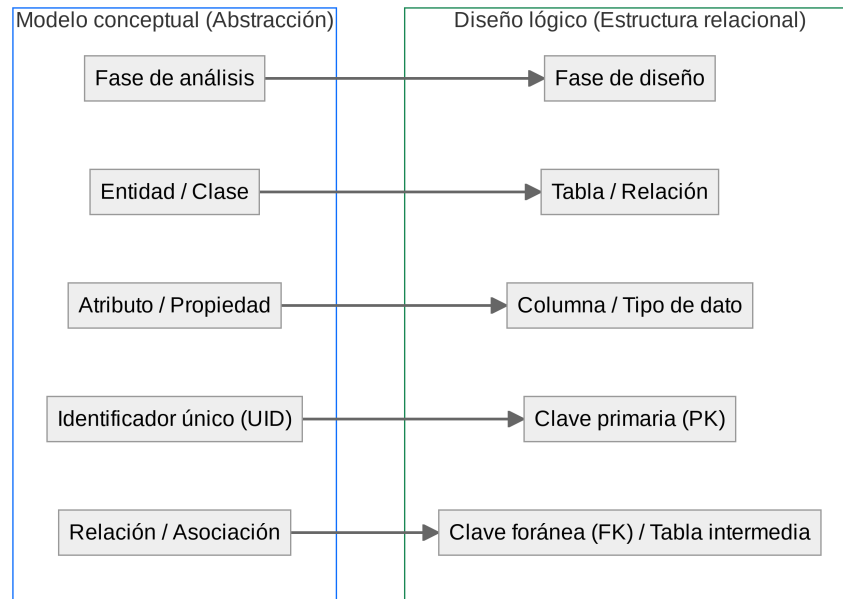


Figura 7.2: Correspondencia entre los componentes del modelo conceptual y el diseño lógico relacional.

- **Diseño físico:** Implementa el diseño lógico sobre un sistema gestor de bases de datos concreto, como PostgreSQL con la extensión PostGIS. Implica la codificación del esquema mediante sentencias **DDL** (SQL estándar y dialectos propios del motor), la selección de **tipos de datos** nativos para coordenadas y geometrías (**geometry**, SRID específicos como el estándar actual EPSG:9377 o el legado EPSG:21892), la creación de **secuencias** para identificadores, la parametrización de **restricciones** (CHECK, UNIQUE, NOT NULL) y el diseño de **índices** espaciales (GiST). Asimismo, contempla decisiones de bajo nivel de almacenamiento físico: asignación de **espacios de tablas** (*tablespaces*), **tamaño de bloques**, estrategias de **compresión** y **configuración de memoria** de búfer para optimizar el rendimiento de las consultas espaciales.

7.1.3 Hoja de ruta taxonómica y conceptual

Para garantizar el rigor metodológico en la transición desde la abstracción territorial hasta la implementación en el motor de base de datos, el presente capítulo formaliza progresivamente los siguientes bloques conceptuales:

1. **Fundamentos de modelado y abstracción territorial:** Entidades, atributos (espaciales y alfanuméricos), identificadores únicos, relaciones y opcionalidad.
2. **Paradigma relacional e integridad de datos:** Tablas (relaciones), tuplas, claves primarias (PK), claves foráneas (FK), normalización y restricciones de dominio.
3. **Evolución del modelado:** Transición del modelo Entidad-Relación (E-R) tradicional a los diagramas

de clases del Lenguaje Unificado de Modelado (UML).

4. **Computación geoespacial:** Tipos de datos espaciales abstractos (*geometry*), sistemas de referencia (SRID) e índices espaciales.

7.2 Modelado conceptual y abstracción territorial (modelo E-R)

El modelado conceptual constituye la fase de **mayor nivel de abstracción** dentro del ciclo de desarrollo de una base de datos espacial. Su objetivo primordial es identificar los **elementos de importancia** para el territorio y la organización, abstrayéndose por completo de las restricciones de hardware, plataformas tecnológicas o sistemas de gestión de bases de datos (SGBD).

En esta etapa se genera un modelo formal a partir de los requerimientos del sistema territorial, identificando las **entidades** críticas y las **relaciones lógicas** y espaciales que las articulan. La calidad del modelado conceptual determina la estabilidad, extensibilidad y consistencia de las fases subsiguientes de diseño lógico e implementación física.

La técnica estándar en esta etapa es el **modelo Entidad-Relación (E-R)**, el cual representa formalmente los datos y las dinámicas del sistema. Un modelo E-R formal se compone de dos elementos interdependientes:

- **Diagrama E-R:** Representación gráfica estructurada de las **entidades**, **atributos**, **identificadores** y **asociaciones**.
- **Documentación de soporte:** Especificación descriptiva de **dominios de valores**, **reglas** operativas de negocio, **restricciones** de integridad y **variables derivadas**.

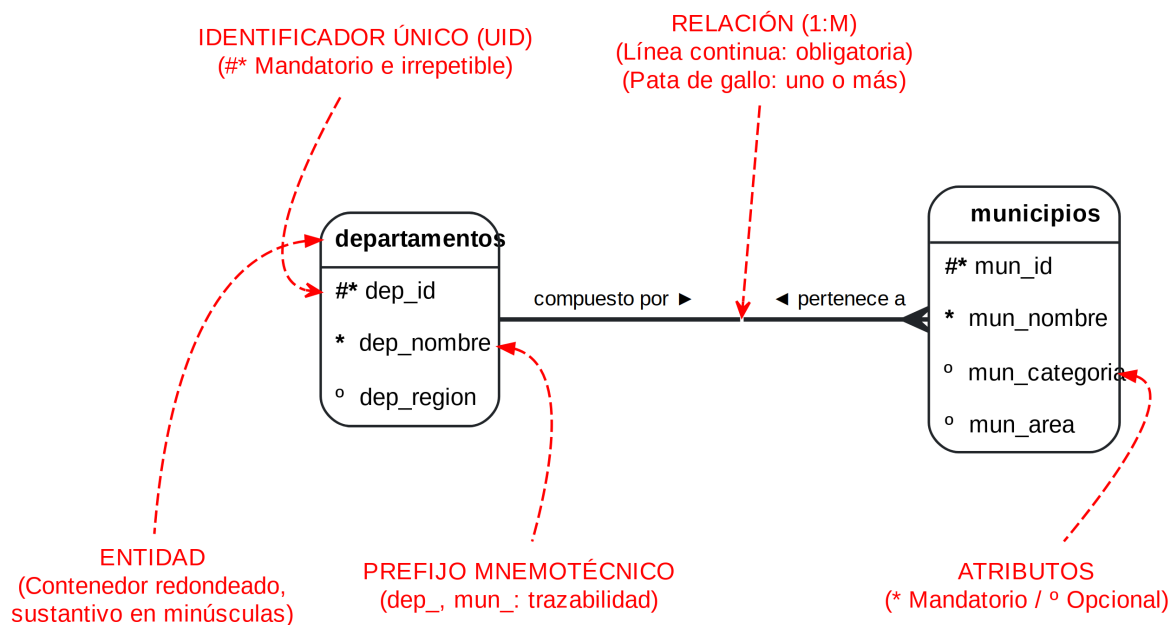


Figura 7.3: Componentes estructurales y reglas de anotación formal en un diagrama Entidad-Relación (1:M).

7.2.1 Modelado de entidades

Una entidad representa un **objeto**, **fenómeno** o **concepto** del mundo real sobre el cual la organización necesita registrar y mantener información estructurada. En el análisis de requerimientos territoriales, las entidades se identifican sistemáticamente a partir de **sustantivos** (ej. municipios, predios, cuencas hidrográficas, estaciones climáticas).

Las reglas fundamentales para la definición de entidades establecen:

- **Pertinencia contextual:** La entidad debe poseer **relevancia** explícita en el ámbito del problema espacial u organizacional que se modela.
- **Unicidad léxica:** Cada entidad debe identificarse con un **nombre unívoco** dentro del modelo.
- **Estructura descriptiva:** Toda entidad debe estar descrita por un conjunto de **atributos**.
- **Conjunto de instancias:** Una entidad define una **clase** o conjunto de ocurrencias individuales que comparten las **mismas propiedades**.
- **Identificación inequívoca:** Cada instancia de una entidad debe poder **individualizarse** mediante el valor de un atributo o combinación de atributos (identificador único).

Gráficamente, la entidad se representa mediante un contenedor de bordes redondeados con su nombre en minúsculas y en plural, seguido por el listado de sus atributos descriptivos:

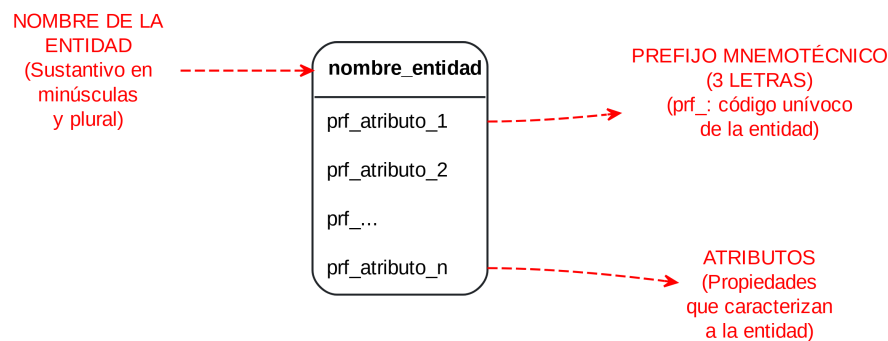


Figura 7.4: Estructura conceptual de una entidad con convenciones de nomenclatura y atributos.

7.2.2 Modelado de relaciones y reglas de lectura

Una relación es una asociación lógica y significativa que vincula instancias de una entidad con instancias de otra entidad. En la etapa preliminar de análisis conceptual, las relaciones deben concebirse considerando siempre su flujo bidireccional:

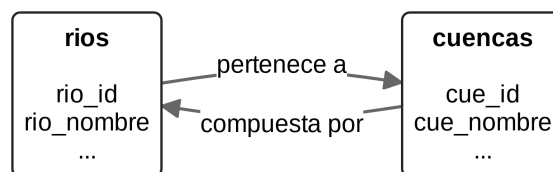


Figura 7.5: Flujo bidireccional de análisis en la asociación conceptual entre entidades espaciales.

Aunque analíticamente el flujo opera en dos direcciones, en el diagrama formal no se trazan dos flechas independientes. La metodología sintetiza la relación en una **única línea física compartida** entre ambas entidades, parametrizada a partir de tres componentes estructurales:

- **Nombre de la relación:** Verbo o descripción concisa en minúsculas que formaliza la regla operativa para cada sentido de lectura (ej. “compuesta por”, “pertenece a”).
- **Opcionalidad (cardinalidad mínima):** Expresa si la participación de una instancia en la asociación es mandatoria (1) o contingente (0).
- **Grado (cardinalidad máxima):** Establece el límite superior de instancias correlacionadas que pueden vincularse (1 o N).

Las convenciones gráficas estandarizadas por Richard Barker en el marco del *CASEMethod** traducen estos componentes en elementos visuales estandarizados:

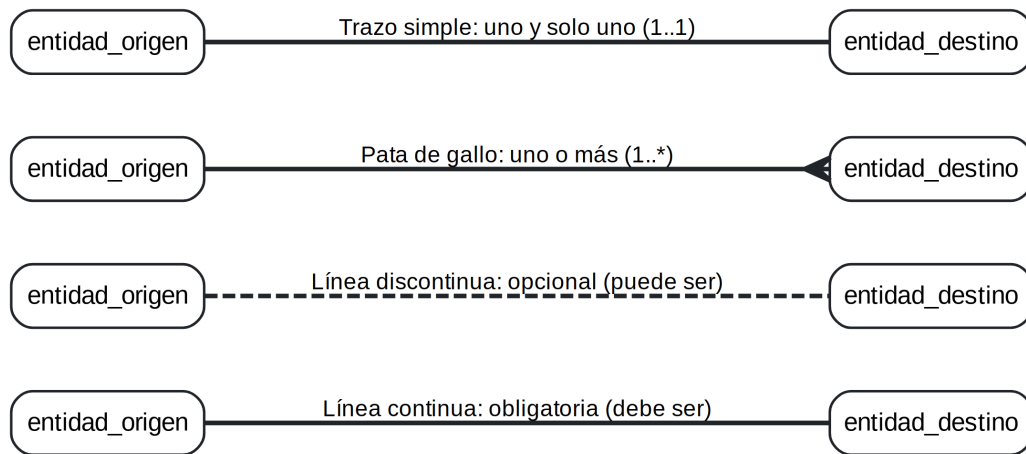


Figura 7.6: Simbología gráfica normalizada para relaciones en el modelo conceptual (Barker).

Tabla 7.1: Convenciones gráficas y semánticas para relaciones en el modelo conceptual

Componente formal	Modalidad / Grado	Notación	Significado formal y regla de lectura
Nombre	Regla operativa	<i>texto</i>	Verbo en minúsculas adyacente a la entidad de origen.
Opcionalidad	Obligatoria (<i>mandatoria</i>)	-----	Debe ser: La participación de la entidad es obligatoria (1).
Opcionalidad	Opcional (<i>contingente</i>)	- - - - -	Puede ser: La participación es contingente (0, tolera valores nulos).
Grado	Múltiple	----<	Uno o más (1..*): Pata de gallo (<i>crow's foot</i>).
Grado	Singular	-----	Uno y solo uno (1..1): Trazo simple sin bifurcación.

i Regla de composición de semirrectas

En el diagrama Entidad-Relación, la línea de relación se compone de dos semirrectas unidas en un punto medio. Cada semirrecta codifica de forma independiente la **opcionalidad** en el tramo de salida de la entidad de origen (trazo sólido o discontinuo) y el **grado** en el contacto con la entidad de llegada (trazo simple o pata de gallo).

i Regla canónica de lectura conceptual

Para verificar la validez semántica de una asociación, se formula una proposición estructurada en cada uno de los sentidos de navegación:

Cada [Entidad₁] (debe ser | puede ser) [Nombre de la relación] (una y solo una | una o más) [Entidad₂]

Ejemplo 1: asociación 1:M obligatoria en ambos extremos. En la Figura 7.7 se modela la jerarquía hidrológica continua entre cuencas y drenajes. La línea es 100% sólida debido a que la participación es mandatoria en ambos sentidos de navegación:

- Cada **cuenca** *debe estar compuesta por uno o más ríos* (tramo continuo de salida y pata de gallo en el extremo de ríos).
- Cada **río** *debe pertenecer a una y solo una cuenca* (tramo continuo de salida y trazo simple en el extremo de cuencas).

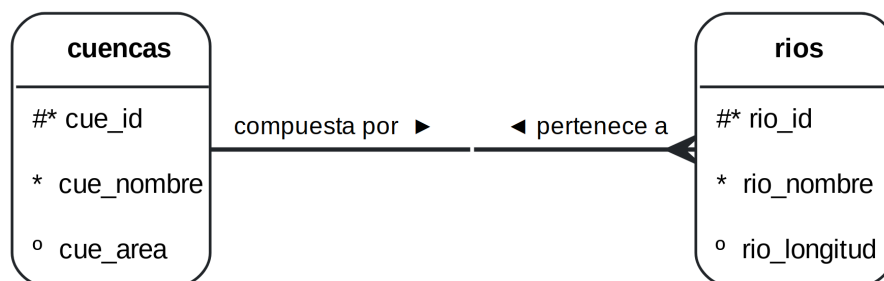


Figura 7.7: Asociación 1:M obligatoria en ambos extremos con línea 100% continua.

Ejemplo 2: asociación 1:M con opcionalidad. En la Figura 7.8 se ilustra la combinación de un tramo continuo con un tramo discontinuo, formalizando la delimitación técnica entre las entidades **microregiones** y **municipios**. Una microrregión se concibe como una agregación funcional de municipios:

La evaluación sintáctica de esta asociación ratifica las reglas de negocio del territorio:

- **Lectura directa (microregiones → municipios):**
 1. Sujeto: **microregiones**.
 2. Opcionalidad (trazo sólido continuo ---): Condición mandatoria → *debe*.
 3. Verbo adyacente: **agrupa**.
 4. Grado en el extremo de destino: Pata de gallo (**crow**) → *uno o más*.

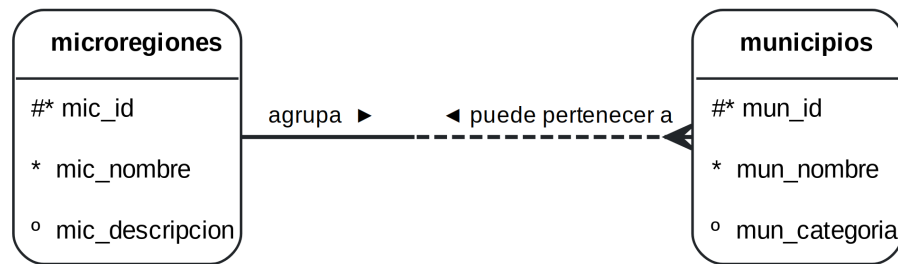


Figura 7.8: Asociación 1:M con opcionalidad mediante tramo continuo y tramo discontinuo.

5. Objeto de destino: **municipios**.

«Cada **microregión** debe agrupar a **uno o más municipios**»

• **Lectura inversa (municipios → microregiones):**

1. Sujeto: **municipios**.
2. Opcionalidad (trazo discontinuo - - -): Condición contingente → *puede*.
3. Verbo adyacente: **puede pertenecer a**.
4. Grado en el extremo de destino: Trazo simple → *una y solo una*.
5. Objeto de destino: **microregiones**.

«Cada **municipio** puede pertenecer a **una y solo una microregión**»

Esta formulación admite que existan municipios que no integran ninguna delimitación gremial o ambiental (anticipando la presencia de valores NULL en la clave foránea en la fase lógica), garantizando al mismo tiempo que aquellos adscritos participen a lo sumo en una microrregión.

Clasificación formal por grado y procedimiento de modelado. Atendiendo a su grado máximo de cardinalidad, las relaciones se tipifican en:

- **Relaciones uno a muchos (1 : M):** Presentan grado “uno y solo uno” en un sentido y “uno o más” en el opuesto. Constituyen el patrón jerárquico predominante en coberturas territoriales y estructuras padre-hijo.
- **Relaciones muchos a muchos (M : M):** Presentan grado “uno o más” en ambas direcciones de navegación.
- **Relaciones uno a uno (1 : 1):** Presentan grado “uno y solo uno” en ambos sentidos. En el modelado espacial frecuentemente sugieren una fragmentación innecesaria de atributos que pertenecen a una misma entidad física.

El modelado formal de relaciones sigue un procedimiento secuencial de cuatro pasos:

1. **Determinar la existencia:** Constatar la necesidad operativa de conectar ambas entidades a partir de los requerimientos.
2. **Nombrar la relación:** Asignar verbos precisos y activos en ambos sentidos de navegación.

3. **Establecer el grado:** Evaluar la cardinalidad máxima ($1 : 1$, $1 : M$, $M : M$).
4. **Determinar la opcionalidad:** Identificar si cada extremo es mandatorio u opcional mediante trazos continuos o discontinuos.

7.2.3 Cardinalidad mediante rangos numéricos y equivalencia con Barker

En herramientas de ingeniería inversa (como pgModeler) y en el estándar de diagramas de clases UML avalado por el Open Geospatial Consortium (OGC) y las normas ISO 19100, la cardinalidad se formaliza mediante pares ordenados de límites numéricos $[min..max]$ en lugar de trazos discontinuos o patas de gallo.

El límite inferior (min) formaliza la **opcionalidad** (0 para opcional, 1 para mandatorio), mientras que el límite superior (max) restringe el **grado** (1 para singular, $*$ o N para múltiple). La correspondencia matemática con la notación de Barker es directa:

Tabla 7.2: Mapeo formal entre la simbología de Barker y la notación de rangos numéricos

Notación gráfica de Barker	Rango numérico $[min..max]$	Modalidad conceptual	Interpretación en la base de datos
Línea continua + trazo simple	1..1	Obligatoria singular	Participación mandatoria única; restringe la clave foránea a NOT NULL.
Línea discontinua + trazo simple	0..1	Opcional singular	Participación contingente; admite valores NULL en la clave foránea.
Línea continua + pata de gallo	1..*	Obligatoria múltiple	Participación mandatoria múltiple; exige al menos una tupla asociada.
Línea discontinua + pata de gallo	0..*	Opcional múltiple	Participación contingente múltiple; admite cero o muchas asociaciones.

i Perspectiva de ubicación Look-Across

Al emplear rangos numéricos en esquemas conceptuales y relacionales, la etiqueta se ubica en el **extremo de destino** de la asociación (*filosofía Look-Across*, adoptada por UML y Barker): al evaluar la entidad **departamentos**, la multiplicidad $[1..*]$ y la etiqueta se posicionan sobre el contenedor de **municipios**, indicando que un departamento agrupa de una a múltiples subdivisiones municipales.

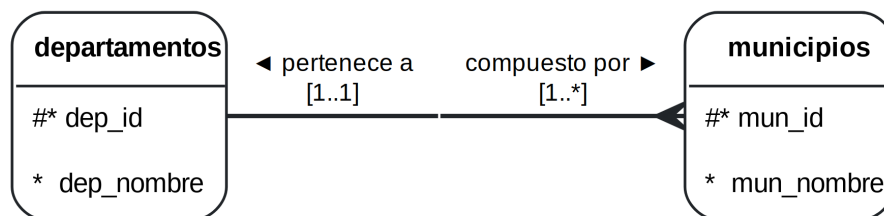


Figura 7.9: Concurrencia entre la notación gráfica de Barker y los rangos numéricos Look-Across.

7.2.4 Modelado de atributos y dominios

En el nivel de abstracción conceptual, las entidades representan conjuntos o clases de objetos del dominio, mientras que los atributos constituyen las **propiedades** elementales, **descriptivas**, **cuantitativas** o **identificadoras** que caracterizan a dichas entidades. Formalmente, una entidad define la estructura invariante o contrato de información, y cada **instancia u ocurrencia** particular de la entidad queda **materializada** por el **conjunto** específico de **valores** asignados a sus **atributos**.

Para preservar la pureza del diseño conceptual, la representación gráfica debe centrarse en la plantilla formal de la entidad (especificación) y en la noción de instancia como ocurrencia del mundo real, evitando la introducción prematura de tablas relacionales o estructuras físicas de almacenamiento.

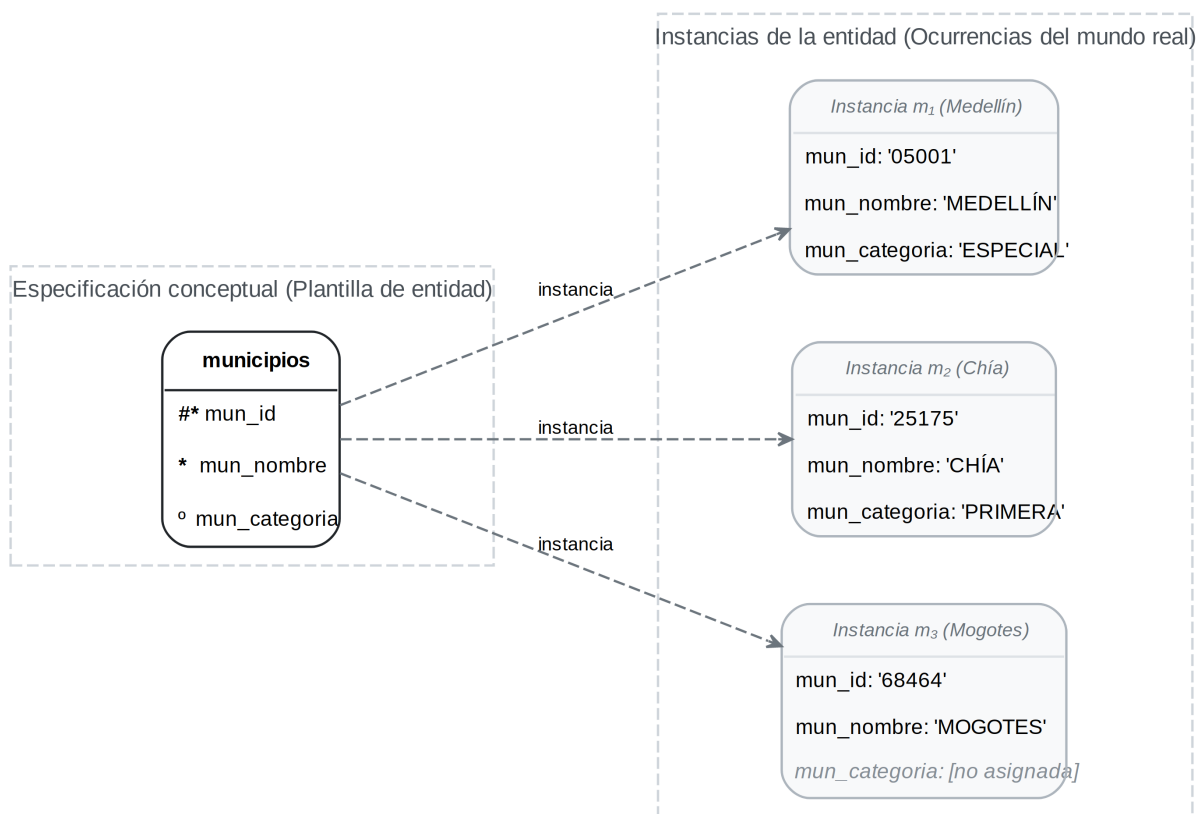


Figura 7.10: Correspondencia conceptual entre la especificación formal de la entidad municipios y la materialización de sus instancias territoriales.

i Semántica y opcionalidad de la categorización municipal

En Colombia, la categorización de los distritos y municipios se fundamenta en el artículo 6 de la Ley 136 de 1994, modificado por el artículo 2 de la Ley 617 de 2000, y es certificada anualmente por la Contaduría General de la Nación con base en la población censada por el DANE y los Ingresos Corrientes de Libre Destinación (ICLD) dictaminados por la Contraloría General de la República. La clasificación define

rangos que van desde la categoría Especial (ciudades principales como Medellín) hasta las categorías 1 a 6 (donde Chía califica en primera categoría y Mogotes en sexta categoría).

El atributo `mun_categoria` se modela formalmente con carácter opcional (°) debido a que representa una propiedad fiscal y administrativa dinámica sujeta a recertificación periódica. En consecuencia, durante las fases preliminares de ingesta cartográfica, levantamiento predial o integración de capas geográficas base, este atributo puede no estar asignado o registrarse temporalmente como un valor nulo sin invalidar la existencia de la unidad territorial en la base de datos espacial.

7.2.4.1 Criterios para la identificación de atributos

La identificación de atributos parte del levantamiento riguroso de requerimientos y del análisis de la documentación técnica o administrativa del dominio (formularios oficiales, actos administrativos, normatividad técnica o especificaciones cartográficas). Los patrones lingüísticos más frecuentes para detectar atributos corresponden a:

- **Sustantivos o adjetivos calificativos:** Términos que describen el estado o condición de una entidad (ej. *categoría, densidad, vigencia*).
- **Construcciones preposicionales:** Expresiones que vinculan una propiedad intrínseca con el sujeto (ej. *categoría por municipio, área por predio, población por departamento*).
- **Estructuras posesivas:** Denominaciones directas de pertenencia (ej. *nombre del municipio, código de la entidad, geometría del polígono*).

7.2.4.2 Convenciones formales de especificación

El estándar de diseño conceptual exige consistencia léxica y estructural para asegurar la transferibilidad del esquema hacia la fase lógica:

- **Formulación gramatical:** Emplear términos exclusivamente en singular y redactados en minúsculas.
- **Ubicación contextual:** Todo atributo debe confinarse dentro del recuadro estructural de la entidad a la que califica.
- **Semántica sin redundancia:** Los nombres deben ser descriptivos y concisos. Se debe evitar la duplicación del nombre completo de la entidad dentro de cada descriptor; en su lugar, se adoptan prefijos mnemotécnicos uniformes (ej. `mun_` para la entidad `municipios`), lo que garantiza trazabilidad semántica y previene colisiones léxicas en operaciones de cruce relacional.
- **Descomposición atómica:** Los atributos compuestos deben desarticularse en componentes elementales e indivisibles.

7.2.4.3 Principio de valor único y atomicidad

Tras determinar la lista inicial de atributos, es mandatorio comprobar que exista únicamente un valor posible para cada atributo en una instancia determinada.

Si una característica requiere registrar simultáneamente múltiples valores para un mismo objeto o entidad espacial (por ejemplo, múltiples normatividades de uso del suelo o varios códigos postales asociados a un municipio), el elemento no califica como atributo atómico de esa entidad. En tal caso, debe abstraerse como

una entidad independiente y modelarse a través de una relación formal (1:M), sentando las bases operativas de la primera forma normal desde la fase conceptual.

7.2.4.4 Modalidad y opcionalidad de los atributos

La opcionalidad describe si una propiedad es estrictamente indispensable para que una instancia exista válidamente dentro del sistema o si admite ausencia de información:

- **Atributos obligatorios (*):** Deben registrar un valor concreto en todas y cada una de las instancias de la entidad (ej. `mun_id` y `mun_nombre`). La ausencia de dato comprometería la validez del registro.
- **Atributos opcionales (°):** No requieren almacenar obligatoriamente un valor en todas las instancias (ej. `mun_categoria`). La entidad admite la no asignación, disponibilidad diferida o aplicabilidad condicional del atributo sin que se altere la integridad estructural del objeto.

7.2.4.5 Identificadores únicos en el nivel conceptual

Toda entidad conceptual requiere un mecanismo formal de identificación que permita discriminar inequívocamente cualquier instancia respecto a las demás dentro de la base de datos.

i Definición de identificador único (UID)

Atributo elemental o combinación mínima de atributos y/o relaciones cuyo valor distingue unívocamente a cada ocurrencia de la entidad en el universo del discurso.

En la notación gráfica de Barker, el identificador único se denota anteponiendo el símbolo numeral (#). Dado que una instancia no puede carecer de identidad ni presentar un identificador indeterminado, el UID simple es inherentemente obligatorio y se anota de manera compuesta como **#*** (ej. el código oficial DIVIPOLA `mun_id`). El identificador único conceptual constituye el fundamento semántico que en el modelo relacional lógico se formalizará como clave primaria (*Primary Key*).

7.2.4.6 Modelado de dominios conceptuales

En el nivel conceptual, la especificación de atributos se complementa con la definición rigurosa de sus **dominios**. Un dominio formaliza el conjunto exhaustivo de valores atómicos homogéneos y semánticamente válidos que un atributo puede asumir dentro del universo del discurso.

La delimitación de dominios en esta etapa asegura la coherencia ontológica del modelo, aislando las reglas de negocio de la implementación física del motor de base de datos. De acuerdo con el criterio de restricción y la estructura interna de los valores, los dominios se clasifican en:

- **Dominios predefinidos o primitivos:** Tipos base escalares determinados por la naturaleza del dato (cadenas alfabéticas, enteros, números reales, fechas o estados booleanos).
- **Dominios estructurados o regidos por patrón:** Cadenas de caracteres cuya validez depende del cumplimiento estricto de una máscara de formato, longitud invariante o expresión regular. Son característicos de sistemas de codificación oficial (códigos censales, identificadores prediales o nomenclaturas viales).
- **Dominios discretos o enumerados:** Conjuntos finitos y cerrados conformados por una lista taxativa de literales permitidos, propios de clasificaciones normativas o estados administrativos.

- **Dominios acotados por rango:** Intervalos continuos o discretos delimitados por extremos admisibles $[min, max]$, habituales en magnitudes físicas, densidades demográficas o coordenadas proyectadas.
- **Dominios espaciales:** Primitivas geométricas estandarizadas (puntos, líneas, polígonos o agregaciones multicomponente) asociadas a una dimensionalidad y a un sistema de referencia espacial (SRID).

7.2.4.7 Especificación de dominios para la entidad municipios

En la entidad `municipios`, la caracterización de los atributos `mun_id`, `mun_nombre` y `mun_categoria` exige definiciones semánticas contrastadas:

Atributo	Tipo conceptual de dominio	Definición y regla semántica del dominio	Expresión formal / Restricción
<code>mun_id</code>	Estructurado por patrón (cadena fija)	Código oficial DIVIPOLA municipal normalizado por el DANE. Cadena alfanumérica de exactamente 5 caracteres numéricos. Los primeros 2 caracteres corresponden al código del departamento (<code>dep_id</code>) y los 3 restantes a la subcodificación municipal.	Texto de longitud = 5, patrón $\wedge [0-9]\{5\}\$,$ donde los caracteres 1..2 $\equiv$ <code>dep_id</code>
<code>mun_nombre</code>	Primitivo textual general	Denominación toponímica oficial del municipio en el territorio nacional. Cadena alfabética estándar.	Longitud $\in [1, 100]$ caracteres
<code>mun_categoria</code>	Enumerado / Discreto	Clasificación fiscal y administrativa asignada bajo los criterios de la Ley 617 de 2000 y certificada anualmente por la Contaduría General de la Nación.	{ 'ESPECIAL', 'PRIMERA', 'SEGUNDA', 'TERCERA', 'CUARTA', 'QUINTA', 'SEXTA' }

i Naturaleza alfanumérica del código municipal DIVIPOLA

El modelado conceptual de identificadores espaciales o administrativos (como `mun_id` o `dep_id`) impone el uso estricto de dominios textuales o de caracteres, descartando categorizaciones numéricas enteras:

- **Preservación del cero a la izquierda:** En departamentos como Antioquia ('05'), Atlántico ('08') o Arauca ('81'), un tipo numérico escalar truncaría el cero inicial por optimización aritmética, convirtiendo el código de Medellín '05001' en 5001. Esto fractura la cardinalidad posicional fija (5 dígitos) y corrompe los cruces espaciales por coincidencia de subcadenas.
- **Ausencia de semántica cuantitativa:** Los códigos geográficos no representan magnitudes medibles ni admiten operaciones aritméticas (calcular sumas o promedios de códigos DIVIPOLA carece de sentido analítico). Corresponden a etiquetas nominales regidas por una composición jerárquica:

$$\text{mun_id} = \text{dep_id} \parallel \text{cod_municipio}$$

Donde $\text{dep_id} \in [0 - 9]2$ (ej. '05' para Antioquia, '25' para Cundinamarca, '68' para Santander) y $\text{cod_municipio} \in [0 - 9]3$ (ej. '001' para Medellín, '175' para Chía, '464' para Mogotes).

7.2.4.8 Clases espaciales y componentes geométricos en el modelo conceptual

En el diseño conceptual de bases de datos territoriales, las entidades se dividen en dos categorías según la naturaleza de sus atributos:

- **Clases espaciales (entidades geográficas):** Objetos del mundo real caracterizados por una **localización explícita** y una **extensión geométrica** en el espacio. Además de sus atributos descriptivos alfanuméricos, incorporan un atributo geométrico abstracto que, por **convención metodológica adoptada en este texto** (y no por imposición estricta de un estándar universal de la industria, donde también alternan denominaciones como *geom*, *geometry* o *shape*), se designa de manera uniforme bajo el identificador *the_geom*.
- **Entidades alfanuméricas (temáticas o de agregación):** Objetos o conceptos que **carecen de un atributo geométrico propio**. Su función radica en estructurar agrupaciones lógicas, clasificaciones normativas o atributos cualitativos.

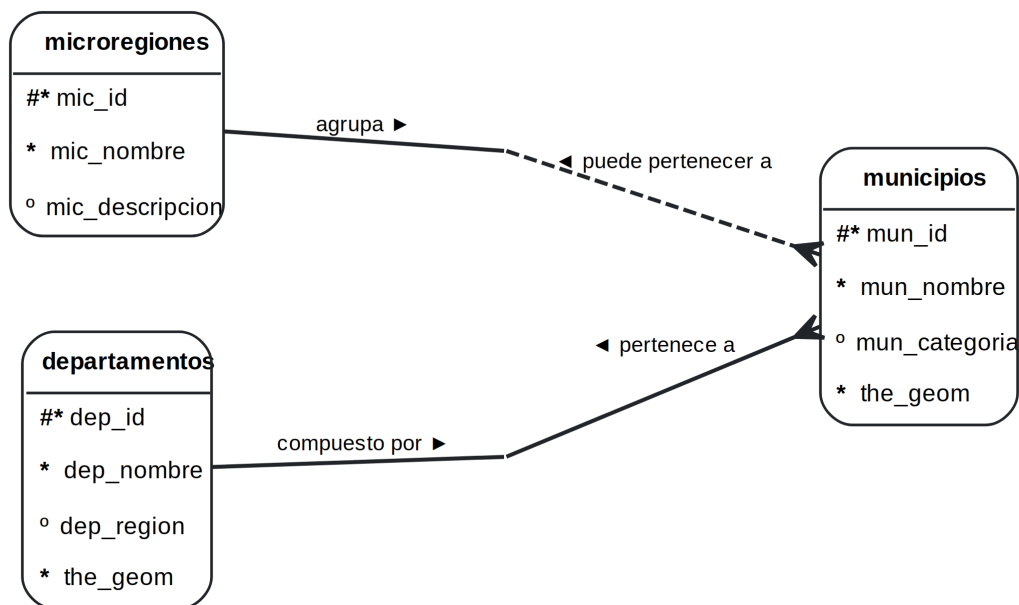


Figura 7.11: Integración conceptual de clases espaciales primarias y entidades alfanuméricas agregadas en notación Barker.

7.2.4.9 El dominio del atributo espacial *the_geom*

A diferencia de los atributos escalares convencionales (cadenas, enteros o fechas), el dominio del atributo *the_geom* no se limita a un tipo de dato primitivo. En el modelo conceptual, el dominio espacial queda unívocamente definido por tres restricciones ontológicas:

1. **Tipo de geometría (primitiva OGC):** Define la signatura topológica permitida según el estándar ISO 19125 / OGC Simple Features (`Point`, `LineString`, `Polygon` o sus variantes homogéneas `Multi*`). Para `municipios` y `departamentos`, el dominio formal corresponde a `MultiPolygon`, dado que las divisiones territoriales pueden comprender discontinuidades geográficas como islas, archipiélagos o enclaves administrativos.
2. **Dimensión de coordenadas:** Determina el número y la naturaleza de los ejes geométricos admisibles: dos dimensiones (XY), tres dimensiones (XYZ para coordenadas con cota métrica o XYM para sistemas con medición lineal), o cuatro dimensiones ($XYZM$). En la cartografía de límites político-administrativos, el dominio opera sobre el plano bidimensional (XY).
3. **Identificador de referencia espacial (SRID):** Especifica el marco geodésico, elipsoide y proyección cartográfica bajo los cuales se interpretan las coordenadas numéricas. Toda geometría carece de significado espacial si no se encuentra asociada a un SRID formalmente catalogado.

7.2.4.10 Derivación geométrica y agregación conceptual

Al articular las entidades `departamentos`, `municipios` y `microregiones`, se aplica el principio de no redundancia geométrica:

- Las entidades `municipios` y `departamentos` actúan como **clases espaciales primarias**, delimitadas formalmente mediante un atributo `the_geom` obligatorio (*).
- La entidad `microregiones` se modela estrictamente como una **entidad alfanumérica agregada**, sin columna `the_geom` en su estructura.

Una microrregión no posee una delimitación cartográfica independiente; su extensión territorial es una **propiedad espacial derivada** que surge conceptualmente a partir de la suma de las geometrías de los municipios que la integran.

Al vincular `microregiones` con `municipios` mediante una relación 1:M opcional, el modelo conceptual garantiza que la espacialidad de la microrregión se resuelva a través de la geometría de sus municipios miembros, preservando la coherencia topológica y evitando discrepancias de linderos antes de definir la implementación física en la base de datos.

7.2.5 Resumen de estándares y convenciones de diseño para entidades y atributos

En la ingeniería de bases de datos espaciales, antes de formalizar la transición hacia los modelos lógico y físico, es indispensable consolidar un conjunto homogéneo de convenciones léxicas y de nomenclatura a nivel conceptual. En esquemas territoriales complejos que integran múltiples coberturas geográficas, catálogos y dominios temáticos, la ausencia de acuerdos estructurales compartidos genera ambigüedad semántica, dificulta la lectura del diccionario de datos y entorpece la interoperabilidad de las especificaciones.

Para estructurar de manera uniforme las entidades, atributos y dominios dentro del modelo conceptual, se adoptan cuatro reglas metodológicas fundamentales:

- **Uso exclusivo de caracteres en minúscula (lowercase):** Todos los nombres de entidades y atributos deben definirse en minúsculas, prescindiendo de tildes, caracteres diacríticos, espacios intermedios o caracteres especiales. Esta práctica garantiza la neutralidad y portabilidad universal del esquema entre herramientas CASE, plataformas de diseño y motores de renderizado documental.

- **Separación de términos mediante guion bajo (snake_case):** Los términos compuestos deben articularse mediante guiones bajos (ej. `dep_nombre`, `mun_categoria`, `zonas_reserva`). Esta convención maximiza la legibilidad visual en diagramas conceptuales, diccionarios de datos y matrices de requerimientos, evitando la ambigüedad tipográfica de notaciones mixtas como CamelCase o PascalCase.
- **Entidades en plural y atributos en singular:** Una entidad define una clase, categoría o conjunto de objetos del mundo real (instancias) que comparten propiedades homogéneas; por tanto, se designa mediante sustantivos en plural o términos colectivos (ej. `departamentos`, `municipios`, `microregiones`). Por el contrario, un atributo describe una propiedad atómica o cualidad que caracteriza a una instancia individual particular, debiendo formularse rigurosamente en singular (ej. `id`, `nombre`, `categoria`, `the_geom`).
- **Prefijo mnemotécnico de tres letras en atributos:** Todo atributo conceptual debe comenzar con un prefijo mnemotécnico uniforme de tres letras (seguido de un guion bajo) que identifique de forma unívoca a la entidad a la que califica (ej. `dep_` para `departamentos`, `mun_` para `municipios`, `mic_` para `microregiones`). Esta convención aporta ventajas analíticas directas desde la etapa conceptual:
- **Trazabilidad léxica:** Facilita la identificación visual inmediata de la entidad de origen de cualquier descriptor al examinar especificaciones funcionales extensas, sin necesidad de consultar permanentemente la definición completa de la entidad.
- **Eliminación de colisiones semánticas y homonimias:** Previene la ambigüedad en el catálogo conceptual. Múltiples entidades territoriales independientes suelen compartir descriptores análogos (como nombre, código, área o fecha de creación); el prefijo mnemotécnico califica unívocamente a cada propiedad (diferenciando `dep_nombre` de `mun_nombre`) y elimina homonimias en la documentación del proyecto.
- **Facilitación del mapeo estructural:** En el diseño conceptual puro, las relaciones se representan mediante asociaciones semánticas y no mediante atributos derivados o claves foráneas. Estandarizar los identificadores con prefijo garantiza que, cuando el modelo transite hacia la fase lógica, las asociaciones propagadas conserven correspondencia léxica y uniformidad conceptual en todo el sistema.

La siguiente matriz ilustra la aplicación de estas directivas conceptuales sobre el subsistema territorial base:

Tabla 7.4: Convenciones de nomenclatura conceptual y prefijos mnemotécnicos en entidades territoriales.

Entidad conceptual	Prefijo	Identificador único (UID)	Atributos descriptivos	Atributo espacial (<code>the_geom</code>)
departamentos	<code>dep_</code>	<code>** dep_id</code>	<code>* dep_nombre^o</code> <code>dep_region</code>	<code>* the_geom</code> (<i>superficie compuesta 2D</i>)
municipios	<code>mun_</code>	<code>** mun_id</code>	<code>* mun_nombre^o</code> <code>mun_categoria</code>	<code>* the_geom</code> (<i>superficie compuesta 2D</i>)
microregiones	<code>mic_</code>	<code>** mic_id</code>	<code>* mic_nombre^o</code> <code>mic_descripcion</code>	(Ninguno: extensión derivada)

i Ausencia de claves foráneas en el esquema conceptual

En la Tabla 7.4 no existe una columna para «claves foráneas» (FK). En el nivel de abstracción conceptual, las vinculaciones territoriales (como la subordinación de un municipio a su departamento) no se resuelven agregando atributos artificiales en la entidad hija, sino a través de **asociaciones formales** con nombre, grado y opcionalidad. La inserción de identificadores externos como atributos escalares (**dep_id** dentro de **municipios**) constituye una decisión de implementación relacional que pertenece estrictamente a la fase subsiguiente de **diseño lógico**.

7.2.6 Paradigmas y alternativas de representación conceptual

El diseño conceptual dispone de diversas convenciones gráficas estandarizadas para abstraer las entidades del territorio y sus interacciones. La selección de la notación determina el nivel de compacidad visual, la precisión matemática de las restricciones y la facilidad de comunicación técnica entre especialistas temáticos y administradores de bases de datos. En la práctica profesional coexisten tres escuelas predominantes:

Notación clásica de Peter Chen (1976). Concibe el esquema mediante una separación topológica estricta entre tres constructos gráficos: rectángulos para las entidades, rombos intercalados para las relaciones y elipses radiales para cada atributo (con el identificador primario subrayado).

A pesar de su valor histórico y pedagógico en la introducción formal a las bases de datos, la notación de Chen resulta **poco práctica e ineficiente** en proyectos geoespaciales reales. La representación de cada atributo como un nodo independiente genera una severa dispersión gráfica: un esquema territorial corporativo compuesto por 20 entidades y un promedio de 15 atributos por cobertura requeriría cientos de elipses y aristas dispersas, saturando el espacio visual y oscureciendo la comprensión de la topología relacional del modelo.

Notación de ingeniería de la información y Barker (*Crow's Foot*). Para superar la dispersión visual del modelo de Chen, las corrientes de ingeniería de la información —particularmente la formalización de Richard Barker para el modelado conceptual— compactaron el diagrama. En este enfoque, los atributos se confinan verticalmente dentro del cuerpo de la propia entidad y los rombos intermedios se suprimen, expresando la asociación y la cardinalidad de manera directa sobre los extremos de la línea de enlace. Esta estructura ofrece una alta densidad de información en un diseño visualmente sobrio, constituyendo el estándar adoptado a lo largo de las secciones precedentes de este capítulo y en las herramientas CASE para el análisis y diseño conceptual de bases de datos.

Perfil conceptual geoespacial en diagramas de clases UML. Aunque la notación de Barker es óptima para transitar hacia motores relacionales transaccionales, carece de mecanismos formales para agrupar coberturas por paquetes de dominio, tipificar primitivas métricas o modelar la herencia de comportamientos espaciales. Para superar esta limitación, el consorcio OGC y el comité ISO/TC 211 estandarizaron el uso de perfiles de clases del Lenguaje Unificado de Modelado (UML), formalizando una aproximación orientada a objetos geográficos que se examina en detalle a continuación.

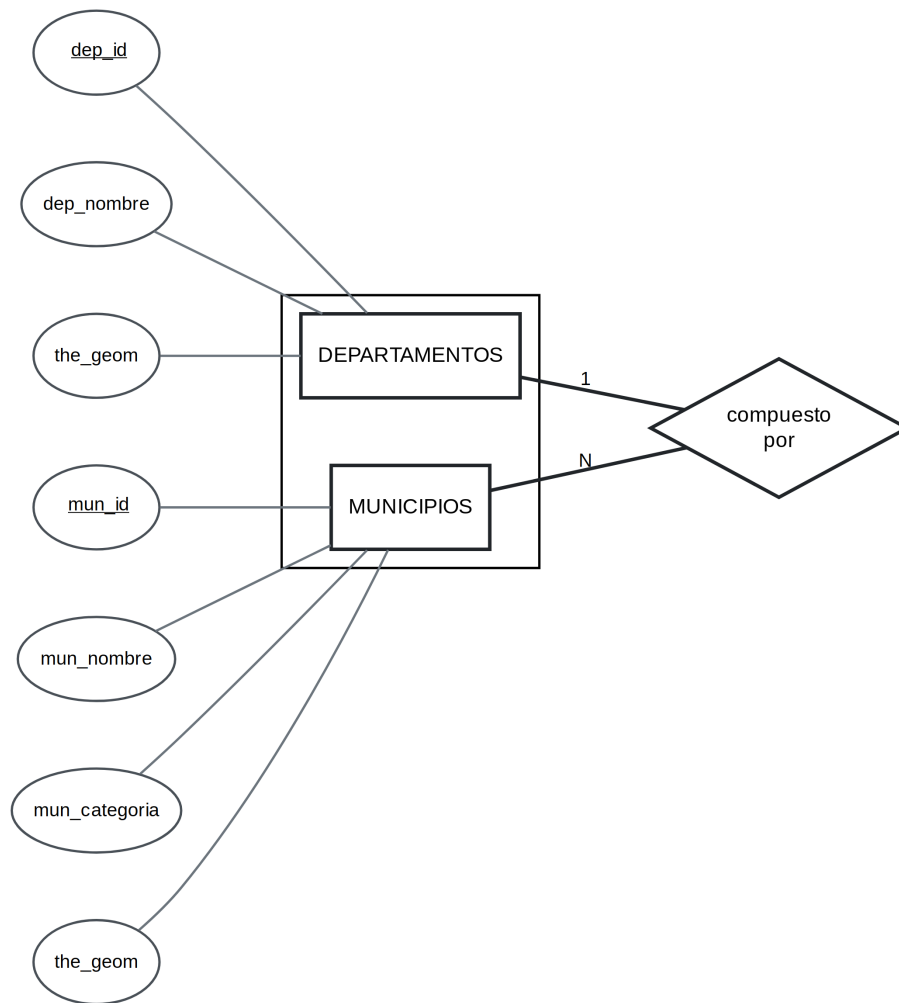


Figura 7.12: Representación de una asociación territorial binaria (1:N) bajo la notación clásica de Peter Chen.

7.3 Diseño conceptual geoespacial mediante diagramas de clases UML

El Lenguaje Unificado de Modelado (UML, *Unified Modeling Language*) representa el estándar formal adoptado por la industria geográfica internacional, respaldado formalmente por el Open Geospatial Consortium (OGC) y el comité técnico ISO/TC 211 mediante las normas **ISO 19103** (*Conceptual schema language*) e **ISO 19109** (*Rules for application schema*).

A diferencia de los esquemas relacionales tradicionales, UML concibe el territorio como un conjunto de clases de objetos interactuantes. Esta aproximación permite agrupar clases en **paquetes conceptuales** (equivalentes directos a **esquemas** de base de datos como **divpol** o **public**), explicitar el comportamiento métrico y tipológico de los datos espaciales mediante **estereotipos** y definir multiplicidades numéricas exactas bajo la perspectiva *Look-Across*.

i Definición de esquema en un SGBD

Contenedor lógico y espacio de nombres (*namespace*) dentro de una base de datos que define la estructura formal de la información y agrupa un conjunto cohesivo de objetos (tablas, vistas, funciones, dominios y secuencias), permitiendo la prevención de colisiones léxicas, la segregación temática y la administración granular de privilegios de acceso.

7.3.1 Fundamentos de orientación a objetos y correspondencia con el modelo Entidad-Relación

El modelado conceptual en UML transfiere los principios cardinales del paradigma orientado a objetos (POO) hacia la ingeniería de datos espaciales:

- **Clase frente a instancia:** Una **clase** define el contrato estructural, los atributos descriptivos y las restricciones espaciales invariantes para un tipo de fenómeno geográfico (**municipios**). Una **instancia** u objeto representa una ocurrencia particular y tangible sobre la superficie terrestre (como el municipio de Chía o de Mogotes).
- **Abstracción territorial:** Proceso de selección y aislamiento de los atributos físicos, jurídicos o administrativos relevantes para el universo del discurso, descartando el detalle irrelevante de la realidad continua.
- **Encapsulamiento conceptual:** Los descriptores alfanuméricos y la delimitación geométrica conforman una unidad semántica indivisible dentro del contenedor de la clase.
- **Generalización y especialización:** Capacidad de estructurar taxonomías mediante jerarquías de herencia, donde subclases concretas heredan atributos y relaciones de superclases abstractas (como la derivación de tipos de superficie a partir de la clase abstracta **Geometry**).

Para articular el diseño conceptual desarrollado previamente mediante la notación de Barker con el estándar formal UML, la Tabla 7.5 sistematiza la correspondencia entre ambos lenguajes de especificación.

Tabla 7.5: Correspondencia formal entre el modelo Entidad-Relación de Barker y el diagrama de clases UML

Concepto de modelado	Notación Entidad-Relación (Barker / CASE)	Perfil conceptual UML (ISO 19109 / OGC)	Significado en bases de datos espaciales
Unidad de modelado	Entidad: Contenedor redondeado; sustantivo en plural o colectivo (ej. municipios).	Clase: Contenedor rectangular; sustantivo en plural con estereotipo (<<FeatureType>>).	Define la estructura formal invariante para una cobertura o clase de entidad del territorio.
Atributo espacial	Atributo convencional obligatorio con identificador metodológico * the_geom .	Atributo tipado con primitiva abstracta: the_geom : MultiPolygon [1] .	Define la signature métrica, la dimensión topológica y la obligatoriedad geométrica.
Identificador único	Símbolo #* antepuesto al atributo (ej. #* mun_id).	Atributo tipado con multiplicidad unitaria obligatoria: mun_id : Character(5) [1] .	Propiedad que individualiza unívocamente a cada ocurrencia en el universo del discurso.
Opcionalidad	Trazo continuo (mandatorio) o discontinuo (opcional / contingente).	Multiplicidad mínima en el rango: 0 para opcional o 1 para mandatorio.	Determina si la participación de una instancia en la asociación admite ausencia de dato.
Grado de cardinalidad	Línea simple (singular) o pata de gallo (múltiple / crow's foot).	Multiplicidad máxima en el rango: 1 para singular o * para múltiple.	Establece el límite superior de instancias correlacionadas en la asociación.
Perspectiva de lectura	Semirrectas con verbos direccionales (,) evaluados de origen a destino.	Rangos numéricos ubicados en el extremo opuesto según la convención <i>Look-Across</i> .	Sintaxis estructurada para validar la integridad semántica de la asociación.
Espacio de nombres	No formalizado en el diagrama (delimitación implícita por contexto).	Paquete (<<Package>>): Contenedor estructural con etiqueta de dominio (ej. divpol).	Agrupar clases afines y prefigura la creación de esquemas relacionales independientes.

7.3.2 Componentes y convenciones del perfil geoespacial UML

En los diagramas de clases orientados al diseño conceptual de bases de datos territoriales, las especificaciones se rigen por las siguientes directivas estructurales:

- **Paquetes conceptuales (<<Package>>):** Agrupaciones lógicas de alto nivel que delimitan dominios temáticos de gobernanza de información. En el diseño de bases de datos espaciales, prefiguran la creación de esquemas relacionales dentro del clúster (por ejemplo, segregando la división político-administrativa oficial **divpol** de las entidades operativas en **public**).
- **Contenedor de clase:** Rectángulo dividido en dos compartimentos en la fase conceptual: la cabecera

(que incluye el estereotipo aplicable y el nombre de la clase) y el cuerpo de atributos tipados.

- **Estereotipos (<<...>>):** Modificadores semánticos que establecen la naturaleza ontológica del elemento dentro del modelo geográfico:
- **<<FeatureType>>** (Clase espacial): Modela fenómenos del territorio provistos de localización explícita y contorno cartográfico directo.
- **<<ObjectType>>** (Clase alfanumérica): Modela conceptos temáticos, agregaciones o catálogos administrativos que carecen de una columna geométrica propia en su estructura.
- **<<Enumeration>>:** Formaliza dominios discretos y cerrados conformados por una lista taxativa de literales válidos.
- **Sintaxis conceptual de atributos:** A diferencia del desarrollo de software orientado a objetos —donde se emplean modificadores de visibilidad como público (+), privado (-) o protegido (#) para gobernar el encapsulamiento de métodos—, en el diseño conceptual de bases de datos territoriales todos los atributos son consultables por definición. Para prevenir colisiones visuales con la simbología de identificadores (#) u obligatoriedad de Barker, se prescinde de modificadores de acceso de software y se formula la signatura conceptual:

nombre_atributo : Dominio/Tipo [multiplicidad]

Donde la multiplicidad especifica la opcionalidad mediante [1] para mandatorio (1..1) o [0..1] para opcional.

- **Atributo espacial the_geom:** Las clases tipificadas como <<FeatureType>> incorporan el atributo geométrico estandarizado bajo la denominación uniforme **the_geom** (convención metodológica adoptada en este texto para unificar la captura de primitivas, frente a denominaciones comerciales o variantes como **geom** o **shape**). Su dominio referencia la jerarquía formal de tipos geométricos ISO 19107 / OGC (ej. **MultiPolygon**, **Point**, **LineString**), con una dimensión métrica 2D explícita y sujeción a un sistema de referencia de coordenadas (SRS).
- **Multiplicidad de asociaciones (Look-Across):** Las asociaciones binarias vinculan las clases mediante aristas sólidas acompañadas de un verbo rector direccional. Las etiquetas numéricas se disponen en el extremo opuesto (*target*) respecto a la clase de referencia:
 - 1: Exactamente una instancia obligatoria (1..1).
 - 0..1: A lo sumo una instancia contingente.
 - 1..*: Una o muchas instancias obligatorias.
 - 0..*: Cero, una o múltiples instancias.

7.3.3 Modelado territorial integrado en UML

En el siguiente diagrama se formaliza el subsistema territorial compuesto por **departamentos**, **municipios** y la agrupación temática **microregiones**. Para reflejar la arquitectura de gobernanza y separación de responsabilidades:

- El paquete conceptual **divpol** confina a las clases espaciales primarias de la división territorial oficial (**departamentos** y **municipios**).
- El paquete conceptual **public** aloja a la entidad alfanumérica agregada **microregiones**.

Se ratifica que **microregiones** se modela estrictamente como una entidad temática: **carece de una columna**

espacial the_geom en su especificación. Su extensión territorial no surge de un levantamiento cartográfico aislado, sino que constituye una **propiedad espacial derivada** que emerge conceptualmente a partir de la agregación de los municipios que la componen.

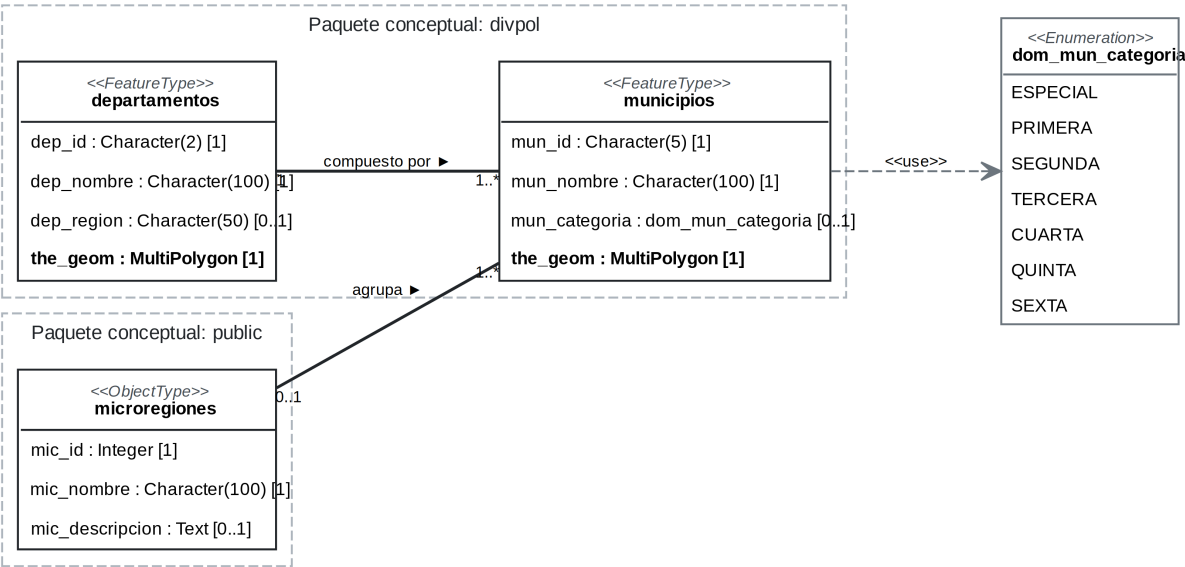


Figura 7.13: Esquema conceptual territorial en notación de clases UML (ISO 19109) estructurado por paquetes temáticos y dominios discretos.

Tabla 7.6: Convenciones semánticas y estructurales del perfil conceptual UML

Elemento UML	Mapeo conceptual y esquema	Interpretación semántica y regla territorial
<<Package>> public	Esquema relacional public	Espacio de nombres para tablas temáticas, agregaciones de gestión o entidades generales del sistema.
<<Package>> divpol	Esquema relacional divpol	Espacio de nombres reservado para la cartografía oficial de la división político-administrativa[cite: 9].
<<FeatureType>>	divpol.departamentos, divpol.municipios	Entidades territoriales primarias provistas de contorno cartográfico explícito (the_geom)[cite: 9].
<<ObjectType>>	public.microregiones	Entidad alfanumérica temática. Carece de columna the_geom ; su cobertura territorial se deriva espacialmente a partir de los municipios asociados.
<<Enumeration>>	dom_mun_categoria	Dominio discreto cerrado conforme a la categorización administrativa de la Ley 617 de 2000.
the_geom: MultiPolygon [1]	Atributo espacial obligatorio	Primitiva de superficie compuesta 2D que modela enclaves, áreas continuas e islas municipales o departamentales.

Elemento UML	Mapeo conceptual y esquema	Interpretación semántica y regla territorial
0..1 $\leftrightarrow$ 1..*	<code>public.microregiones</code> $\leftrightarrow$ <code>divpol.municipios</code>	Asociación interpaquete: Una microrregión agrupa de uno a muchos municipios. Un municipio puede pertenecer a lo sumo a una microrregión o a ninguna.
1 $\leftrightarrow$ 1..*	<code>divpol.departamentos</code> $\leftrightarrow$ <code>divpol.municipios</code>	Asociación intrapaquete: Cada departamento se compone de al menos un municipio. Todo municipio debe adscribirse estrictamente a un único departamento[cite: 9].

i Resolución conceptual de relaciones interpaquete

La articulación entre `public.microregiones` y `divpol.municipios` evidencia la capacidad de UML para modelar dependencias entre distintos dominios de gobernanza. En el diseño conceptual, la relación expresa que la delimitación territorial de la microrregión emerge de la agregación topológica de unidades pertenecientes a otro paquete temático. Las decisiones sobre si esta cobertura espacial se resolverá de forma dinámica bajo demanda o si se persistirá como una capa calculada de manera periódica corresponden estrictamente a las fases posteriores de diseño lógico e implementación física.

7.4 Transformación conceptual a lógica relacional

El diseño lógico constituye el proceso sistemático de traducir las entidades, atributos, dominios y asociaciones abstractas —definidas en el modelo conceptual mediante la notación de Barker o diagramas de clases UML— hacia las estructuras canónicas del **paradigma relacional**. En esta etapa se formalizan las **tablas**, **tuplas**, **columnas tipificadas** y **restricciones de integridad**, garantizando que el **esquema** quede preparado para su implementación física en el SGBD espacial sin incurrir en redundancias destructivas ni anomalías operacionales. En esta sección, el análisis se concentra en la implementación sobre PostgreSQL y su extensión PostGIS.

i Definiciones fundamentales del modelo relacional

Las siguientes definiciones condensan los componentes estructurales elementales del paradigma relacional:

- **Tabla (relación):** Estructura bidimensional de almacenamiento compuesta por un conjunto no ordenado de registros (tuplas) homogéneos y atributos (columnas), que materializa formalmente una clase de objetos o relación matemática en el esquema relacional.
- **Tupla:** Registro o fila individual dentro de una tabla que alberga un conjunto de valores atómicos correspondientes a cada atributo, representando una instancia u ocurrencia particular del mundo real.
- **Columna tipificada (atributo relacional):** Descriptor formal de la relación definido bajo un nombre unívoco y restringido a un tipo de dato o dominio específico (escalar o abstracto), garantizando la homogeneidad semántica de los valores almacenados en sus celdas.
- **Restricciones de integridad:** Reglas y predicados lógicos declarativos (claves primarias,

unicidad, restricciones de no nulidad NOT NULL, integridad referencial mediante claves foráneas y cláusulas CHECK) aplicados por el motor para preservar la consistencia interna y evitar anomalías transaccionales.

- **Esquema:** Contenedor lógico y espacio de nombres (*namespace*) dentro de la base de datos que agrupa un conjunto cohesivo de objetos (tablas, vistas, funciones, secuencias, tipos y dominios), facilitando la prevención de colisiones léxicas y la segregación temática de los datos territoriales.

7.4.1 El modelo relacional y reglas de derivación estructural

El modelo relacional, formulado originalmente por Edgar F. Codd (Codd, 1970), estructura los datos en **relaciones** (término formal para las tablas), concebidas matemáticamente como subconjuntos del producto cartesiano de una lista de dominios:

$$R \subseteq D_1 \times D_2 \times \dots \times D_n$$

Cada fila o registro de la tabla constituye una **tupla** que materializa una instancia u objeto del mundo real, mientras que cada columna representa un **atributo** tipificado bajo un dominio escalar o abstracto específico.

La transición desde el esquema conceptual hacia el esquema lógico relacional opera bajo un conjunto estricto de reglas formales de transformación:

- **Entidades conceptuales → Relaciones (tablas):** Cada entidad o clase conceptual da origen a una tabla independiente ubicada en el esquema de base de datos que le corresponde por gobernanza temática (ej. `divpol.departamentos`, `divpol.municipios`, `public.microregiones`).
- **Atributos conceptuales → Columnas atómicas:** Cada atributo conceptual se convierte en una columna con un tipo de dato explícito (VARCHAR, NUMERIC, INTEGER, GEOMETRY).
- **Opcionalidad → Restricciones de nulidad:** Los atributos obligatorios (*) se definen formalmente mediante la cláusula NOT NULL; los atributos conceptualmente opcionales (°) admiten valores nulos (NULL).
- **Identificador único (UID) → Clave primaria (Primary Key, PK):** El UID conceptual se materializa mediante una restricción PRIMARY KEY, asegurando unicidad física y obligatoriedad en todas las tuplas.
- **Asociaciones 1:M → Claves foráneas (Foreign Keys, FK):** En las relaciones uno a muchos, la clave primaria de la entidad del extremo “uno” (padre) se migra como una columna adicional en la tabla del extremo “muchos” (hijo).
- **Asociaciones M:M → Tablas de asociación intermedias:** Toda relación muchos a muchos se descompone obligatoriamente en dos relaciones 1:M asistidas por una tabla puente intermedia compuesta por las claves de las entidades participantes.

7.4.2 Notación estructural y código cromático de claves

Para documentar esquemas relacionales sin ambigüedades antes de escribir código DDL, se adopta un estándar formal de representación textual y cromática para la signatura de relaciones:

- **Nombre de la relación:** Se escribe en letras minúsculas en plural según el contexto del esquema, seguido de sus atributos entre paréntesis.
- **Clave primaria (PK):** Toda columna que integre la clave primaria debe aparecer **siempre subrayada** y se resalta convencionalmente en color **verde** (**green**).
- **Clave foránea (FK):** Toda columna que actúe como clave foránea referenciando a otra relación debe figurar **siempre en color rojo** (**red**).
- **Clave primaria compuesta con rol de clave foránea:** Cuando un atributo opera simultáneamente como componente de la clave primaria propia y como clave foránea hacia una tabla externa, se representa **subrayado y en color rojo**.

Aplicando esta convención cromática al subsistema territorial desarrollado en el modelo conceptual, el esquema lógico relacional se formula como:

- departamentos (dep_id, dep_nombre, dep_region, the_geom)
- microregiones (mic_id, mic_nombre, mic_descripcion)
- municipios (mun_id, mun_nombre, mun_categoria, dep_id, mic_id, the_geom)

En la relación `municipios`, la presencia de dep_id y mic_id materializa las dos asociaciones que en el diagrama conceptual se representaban como líneas conectadas: dep_id almacena un valor que debe coincidir con dep_id en `departamentos`, mientras que mic_id enlaza con mic_id en `microregiones`.

El siguiente diagrama contrasta de manera directa la especificación conceptual inicial (Barker) con el esquema lógico relacional resultante, explicitando el código cromático de claves y la navegación de punteros referenciales:

7.4.3 Reglas de integridad y gestión de identificadores

El modelo relacional garantiza la robustez y consistencia interna de los datos mediante dos principios fundamentales de integridad estructural:

1. **Integridad de entidad:** Toda tabla debe poseer una **clave primaria única** cuyos valores no admiten en ningún caso el valor nulo (NULL). La omisión o indeterminación de una clave primaria impediría distinguir una tupla de las demás, corrompiendo la noción matemática de relación.
2. **Integridad referencial:** Toda **clave foránea** que resida en una relación subordinada debe coincidir de forma exacta con un valor válido existente en la **clave primaria** de la relación padre referenciada, o bien albergar un valor NULL si la asociación es conceptualmente opcional. El motor de base de datos **intercepta y prohíbe** cualquier **operación de inserción o actualización** que produzca tuplas **huérfanas** en el sistema.

En la ingeniería de bases de datos territoriales, la formulación de claves primarias plantea una disyuntiva entre identificadores naturales y artificiales:

- **Claves naturales:** Atributos intrínsecos al dominio normativo o censal, tales como la codificación oficial DIVIPOLA del DANE (dep_id con 2 caracteres alfanuméricos, mun_id con 5 caracteres alfanuméricos).

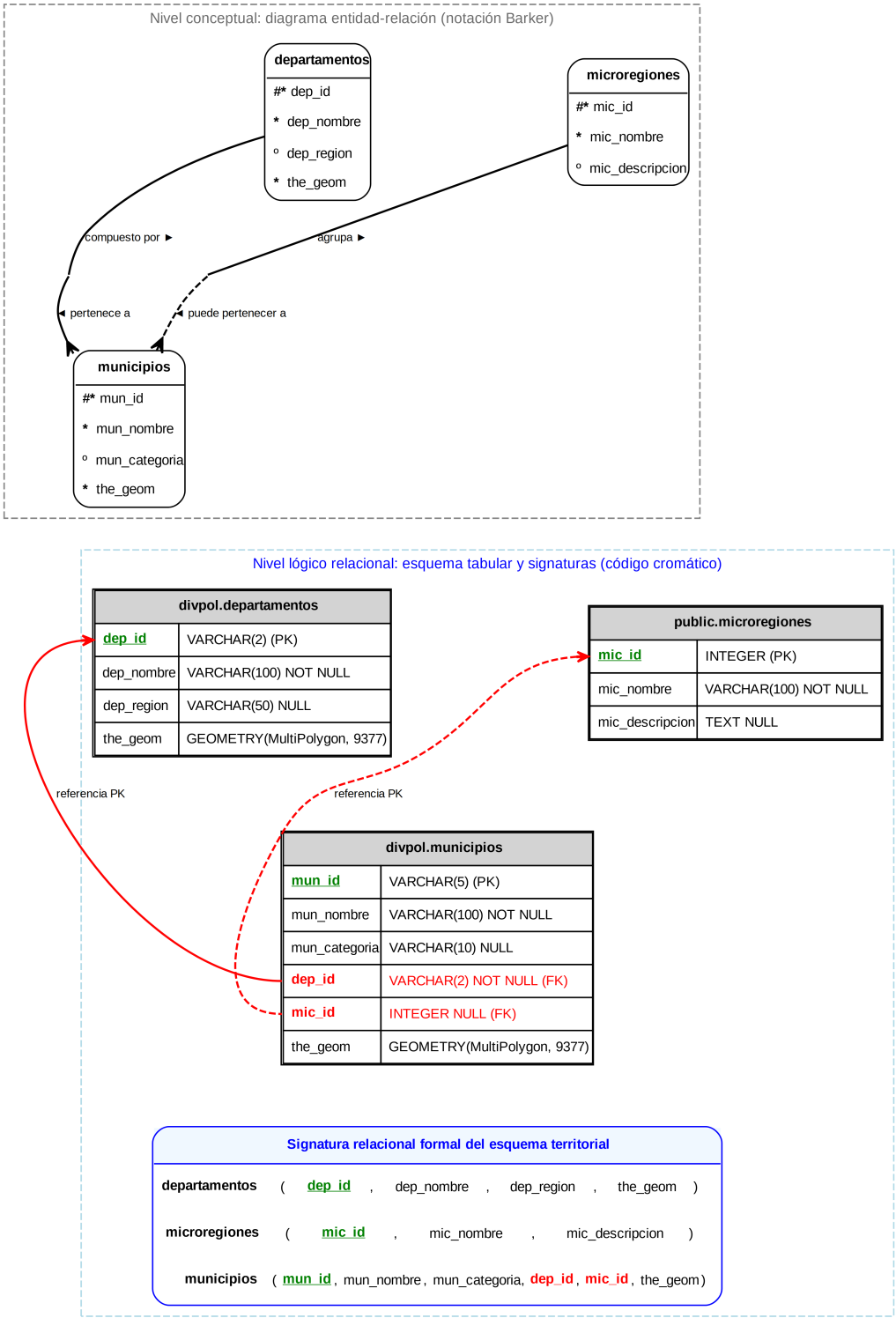


Figura 7.14: Transición del modelo conceptual (Barker) al diseño lógico relacional con código cromático de claves y signatura formal del esquema.

Poseen significado semántico directo y facilitan el intercambio de registros entre entidades del Estado sin recurrir a diccionarios intermedios.

- **Claves artificiales o subrogadas:** Valores numéricos enteros secuenciales generados de forma automática por el sistema gestor (tipos `SERIAL` o secuencias `IDENTITY`). Maximizan la eficiencia computacional en índices B-Tree y cruces relacionales, y satisfacen los requerimientos operativos de software SIG de escritorio (como QGIS o ArcGIS Pro), los cuales demandan históricamente una columna entera positiva monotónica (típicamente designada como `gid` u `objectid`) para renderizado acelerado y selección interactiva en pantalla.

En el diseño relacional del curso se adopta un **enfoque híbrido estandarizado**: para las capas cartográficas maestras se reserva la columna `gid SERIAL PRIMARY KEY` para garantizar interoperabilidad técnica con clientes SIG de escritorio, complementada formalmente por una restricción de unicidad y no nulidad (`UNIQUE NOT NULL`) sobre la clave natural DIVIPOLA (`mun_id, dep_id`), blindando de manera concurrente la integridad espacial y administrativa.

7.4.4 Operaciones de combinación relacional y espacial

La integración de la información distribuida en distintas tablas del modelo relacional se ejecuta mediante la operación de combinación (*JOIN*). Dependiendo de la naturaleza de los atributos involucrados, la combinación puede operar en el plano alfanumérico o en el plano geométrico:

- **Combinación relacional tradicional (*Attribute Join*):** Las tuplas de dos relaciones se articulan evaluando una condición de igualdad estricta entre la clave foránea y la clave primaria (`tabla_a.fk = tabla_b.pk`). En el caso territorial, un departamento se asocia con sus municipios evaluando `municipios.dep_id = departamentos.dep_id`.
- **Combinación espacial (*Spatial Join*):** Las tuplas se combinan evaluando un predicado topológico o geométrico estandarizado sobre sus respectivas columnas espaciales `the_geom` (`ST_Intersects`, `ST_Within`, `ST_Contains`), sin requerir la preexistencia de una clave foránea explícita entre ambas tablas.

La siguiente tabla sintetiza las propiedades de ambos mecanismos analíticos en el clúster relacional:

Tabla 7.7: Contraste estructural entre combinaciones relacionales y espaciales en motores objeto-relacionales.

Criterio de diseño	Combinación relacional (<i>Attribute Join</i>)	Combinación espacial (<i>Spatial Join</i>)
Criterio de enlace	Coincidencia de valores escalares: <code>FK = PK</code> .	Relación topológica euclidiana o geodésica.
Operador evaluador	Operador de igualdad estándar (<code>=</code>).	Funciones espaciales OGC (<code>ST_Within</code> , <code>ST_Intersects</code>).
Estructura de indexación	Índices univariados basados en árboles B+ (<i>B-Tree</i>).	Índices espaciales multidimensionales basados en árboles R (<i>GiST</i>).
Dependencia estructural	Exige la propagación formal de claves foráneas en el diseño lógico.	Opera sobre cualquier par de relaciones espaciales que compartan el mismo SRID.

7.4.5 Normalización relacional y formas normales avanzadas

La normalización es una disciplina formal del diseño lógico que estructura las relaciones para minimizar la redundancia y neutralizar las **anomalías de actualización, inserción y borrado**. Su aplicación se articula mediante niveles progresivos de rigor metodológico:

i Anomalías de actualización, inserción y borrado

Inconsistencias y fallos transaccionales originados por la redundancia estructural y dependencias funcionales inadecuadas en relaciones no normalizadas:

- **Anomalía de actualización:** Modificación parcial o asimétrica de datos duplicados a lo largo de múltiples registros, generando estados mutuamente contradictorios sobre un mismo atributo o hecho del mundo real.
- **Anomalía de inserción:** Imposibilidad de registrar los atributos válidos de una entidad independiente sin la concurrencia obligatoria de una entidad secundaria asociada, o la necesidad de introducir valores nulos artificiales para completar la tupla.
- **Anomalía de borrado:** Pérdida involuntaria e irreversible de información descriptiva o histórica de una entidad principal como consecuencia colateral de eliminar el registro de una entidad subordinada.

Para evaluar los requerimientos de descomposición relacional, es indispensable definir con precisión el instrumental teórico de claves y atributos:

i Definición de clave candidata

Columna o conjunto mínimo de columnas que identifica de manera unívoca cada tupla dentro de una relación, satisfaciendo simultáneamente los principios de unicidad (no existen dos tuplas con valores idénticos) e irreductibilidad o minimalidad (ningún subconjunto propio de sus atributos conserva la capacidad de identificación única).

A partir del conjunto de claves candidatas identificadas en una relación, el diseñador selecciona una como **clave primaria** (*Primary Key*), quedando las restantes categorizadas como **claves alternativas** (*Alternate Keys*).

i Atributos primos y no primos en el modelo relacional

En la teoría de la normalización relacional, la condición de un atributo se determina a partir de su pertenencia a las claves candidatas de la relación:

- **Atributo primo:** Atributo que forma parte de al menos una clave candidata (sea esta la clave primaria seleccionada o una clave alternativa) de la relación R . Si una tabla posee una clave candidata compuesta como (mun_id, via_codigo), tanto mun_id como via_codigo constituyen atributos primos de manera individual.
- **Atributo no primo:** Atributo que no forma parte de ninguna clave candidata de la relación R . Representa propiedades descriptivas, cuantitativas o temáticas (ej. mun_nombre, via_tipo o the_geom) y constituye el foco de evaluación de las dependencias parciales en la Segunda Forma Normal (2FN) y de las dependencias transitivas en la Tercera Forma Normal (3FN).

A partir de estas definiciones, las formas normales se estructuran jerárquicamente:

- **Primera forma normal (1FN):** Exige que todos los atributos contengan exclusivamente valores **atómicos** indivisibles dentro de su **dominio** (Codd, 1970). En la computación geoespacial, el atributo `the_geom` **no infringe la 1FN**: actúa como un Tipo Abstracto de Datos (ADT) encapsulado, procesado por el motor como una entidad matemática atómica unificada dentro del espacio geométrico.
- **Segunda forma normal (2FN):** Exige el cumplimiento de la 1FN y prohíbe que cualquier atributo no primo dependa parcialmente de una clave candidata compuesta (Codd, 1972). Si la clave primaria de una tabla es simple (como `mun_id`), la 2FN se satisface de manera directa.

i Ejemplo de infracción de la segunda forma normal (2FN)

Considérese una relación destinada a registrar la longitud de los tramos viales que cruzan por cada municipio:

```
tramos_municipio(mun_id, via_codigo, mun_nombre, via_tipo, longitud_km)
```

En esta estructura:

- **Clave primaria compuesta:** (`mun_id`, `via_codigo`)
- **Dependencia funcional completa:** (`mun_id`, `via_codigo`) $\rightarrow$ `longitud_km` (la longitud corresponde estrictamente al segmento de esa vía dentro de ese municipio específico).
- **Dependencias parciales (infracción de 2FN):**
 - `mun_id` $\rightarrow$ `mun_nombre` (`mun_nombre` depende únicamente del identificador municipal, ignorando la vía).
 - `via_codigo` $\rightarrow$ `via_tipo` (`via_tipo` depende exclusivamente del código de la vía, con independencia del municipio).

Dado que los atributos no primos `mun_nombre` y `via_tipo` no dependen de la totalidad de la clave primaria sino de subconjuntos de esta, la relación viola la 2FN y genera redundancia cada vez que una vía atraviesa múltiples municipios. La normalización exige descomponerla en tres relaciones:

1. `municipios(mun_id, mun_nombre)`
2. `vias(via_codigo, via_tipo)`
3. `tramos_municipio(mun_id, via_codigo, longitud_km)`

- **Tercera forma normal (3FN) y forma de Boyce-Codd (BCNF):** Elimina dependencias funcionales transitivas entre atributos no primos (Codd, 1972, 1974). En el ámbito territorial, almacenar el nombre del departamento dentro de la tabla municipal genera una cadena transitiva:

$$\text{mun_id} \rightarrow \text{dep_id} \quad \wedge \quad \text{dep_id} \rightarrow \text{dep_nombre}$$

Esto violaría la 3FN, obligando a registrar 'Antioquia' cientos de veces en disco. La 3FN exige desarticular el modelo en DEPARTAMENTOS y MUNICIPIOS, enlazados mediante **dep_id**.

i Dependencia funcional transitiva

Relación de determinación indirecta entre atributos dentro de un esquema relacional, en la cual un atributo no primo Z depende funcionalmente de una clave candidata X únicamente a través de un conjunto de atributos intermedio Y .

Formalmente, en una relación R , se manifiesta una dependencia funcional transitiva si se verifican simultáneamente las siguientes condiciones:

$$X \rightarrow Y \quad \wedge \quad Y \rightarrow Z$$

donde:

- $Y \nrightarrow X$ (el determinante intermedio Y no es una superclave ni determina de forma biunívoca a X).
- $Z \not\subseteq X \cup Y$ (la dependencia no es trivial).
- Z es un atributo no primo (no forma parte de ninguna clave candidata de la relación).

La existencia de dependencias transitivas transgrede los principios de la Tercera Forma Normal (3FN), induciendo redundancia en el almacenamiento y anomalías operativas de actualización y borrado al asociar descriptores que deberían estar segregados en su propia relación independiente (por ejemplo: $\text{mun_id} \rightarrow \text{dep_id} \rightarrow \text{dep_nombre}$).

- **Cuarta forma normal (4FN) y dependencias multivaluadas de Fagin:** Formulada por Ronald Fagin en 1977, aborda las **dependencias multivaluadas** ($X \twoheadrightarrow Y$), donde hechos independientes de cardinalidad múltiple se asocian a una misma clave (Fagin, 1977). La 4FN prohíbe almacenar en una sola relación dos **hechos independientes** (por ejemplo, resguardos indígenas y planes de ordenamiento de un municipio), exigiendo su descomposición en tablas independientes para prevenir redundancia combinatoria.
- **Quinta forma normal (5FN / PJNF):** Denominada formalmente *forma normal de proyección-uniión* (*Projection-Join Normal Form*, PJNF), garantiza que una relación que ha sido descompuesta en múltiples tablas pueda reconstruirse mediante operaciones de combinación (*join*) sin alterar la información original ni generar **tuplas espurias** (registros ficticios que afirman hechos falsos en la base de datos), vinculando las **dependencias de reunión** exclusivamente a las claves candidatas (Fagin, 1979).

i Conceptos clave y ejemplo práctico de la quinta forma normal (5FN / PJNF)

Una **dependencia de reunión** ocurre cuando una tabla ternaria no puede dividirse en dos relaciones sin corromper los datos, pero sí puede descomponerse en tres o más proyecciones sin pérdida.

Considérese la asignación de levantamientos catastrales donde rige la siguiente regla: un operador ejecuta un método en un municipio si y solo si el operador atiende el municipio, el operador domina el método, y el municipio autoriza dicho método.

`operaciones(operador, municipio, metodo)`

Tabla 7.8: Relación operaciones

operador	municipio	metodo
Geocad	Chía	GNSS
Geocad	Mogotes	GNSS
Geocad	Mogotes	Dron
Toposig	Mogotes	GNSS

Si esta relación se descompone únicamente en dos tablas:

- $R_1(\text{operador}, \text{municipio})$
- $R_2(\text{municipio}, \text{metodo})$

Al reconstruir la información mediante $R_1 \bowtie R_2$, en Mogotes el operador **Toposig** se asociará automáticamente con **Dron**, produciendo la **tupla espuria** (**Toposig**, **Mogotes**, **Dron**). Este dato es ficticio, pues **Toposig** no utiliza drones.

Para satisfacer la **5FN**, la relación debe descomponerse en **tres proyecciones simultáneas**:

1. $R_1(\text{operador}, \text{municipio})$
2. $R_2(\text{municipio}, \text{metodo})$
3. $R_3(\text{operador}, \text{metodo})$

Al evaluar la reunión natural de las tres tablas ($R_1 \bowtie R_2 \bowtie R_3$), la tabla R_3 descarta la combinación errónea (al no existir el par **Toposig** - **Dron**), recuperando exactamente las cuatro tuplas originales sin redundancias ni registros espurios.

- **Forma normal de dominio-clave (DKNF)**: Formulada por Ronald Fagin en 1981, representa la frontera teórica de la disciplina relacional:

i Teorema de Fagin sobre la forma normal de dominio-clave (DKNF)

Una relación relacional se encuentra en DKNF si y solo si toda restricción de integridad impuesta sobre los datos es una consecuencia lógica directa e inequívoca de la definición de los **dominios** de sus atributos y de las **claves** primarias o candidatas declaradas sobre el esquema (Fagin, 1981).

Una tabla que satisface la DKNF carece por definición de anomalías de modificación, inserción y borrado.

i Evaluación práctica de la DKNF en la jerarquía territorial DIVIPOLA

Considérese la relación de municipios articulada con su dependencia departamental:

municipios(mun_id, mun_nombre, dep_id)

1. Diagnóstico del estado de normalización actual:

Antes de contrastar la relación frente a la DKNF, se verifica su conformidad con la jerarquía clásica de formas normales:

- **1FN**: Cumple estrictamente, pues todos los atributos alojan valores atómicos dentro de sus dominios escalares.

- **2FN:** Satisface la condición al poseer una clave primaria simple (`mun_id`), descartando cualquier dependencia funcional parcial.
- **3FN y BCNF:** Se encuentra en 3FN y en BCNF debido a que la única dependencia funcional no trivial existente es $\text{mun_id} \rightarrow \{\text{mun_nombre}, \text{dep_id}\}$, donde el determinante es la clave candidata única (superclave). No existen dependencias transitivas entre atributos no primos.
- **4FN y 5FN:** Cumple trivialmente al no presentar hechos multivaluados independientes ($X \twoheadrightarrow Y$) ni dependencias de reunión cíclicas.

En consecuencia, **la relación se encuentra formalmente en BCNF y 5FN**. Sin embargo, este nivel de diseño resulta insuficiente para encapsular restricciones semánticas entre atributos interdependientes mediante mecanismos exclusivamente declarativos.

2. Especificación de claves y dominios:

- **Clave primaria:** `mun_id` (garantiza la unicidad de cada registro municipal).
- **Dominio de `mun_id`:** Cadena de exactamente 5 dígitos numéricos ($\text{Pattern}([0-9]^5)$).
- **Dominio de `dep_id`:** Cadena de exactamente 2 dígitos numéricos ($\text{Pattern}([0-9]^2)$).
- **Dominio de `mun_nombre`:** Cadena de texto de hasta 100 caracteres.

3. Restricción territorial general (C):

La estructura oficial de codificación DIVIPOLA del DANE impone la siguiente regla de jerarquía administrativa: «El código departamental `dep_id` asignado a un municipio debe coincidir de forma idéntica con los dos primeros caracteres de su identificador municipal `mun_id`»:

$$C : \text{dep_id} = \text{SUBSTRING}(\text{mun_id}, 1, 2)$$

4. Prueba de consecuencia lógica (infracción de DKNF):

Supóngase el intento de inserción de la siguiente tupla:

$$t_1 = ('05001', 'MEDELLÍN', '25')$$

Al contrastar la tupla frente a los mecanismos declarativos del esquema:

- ¿Satisface la restricción de clave primaria? **Sí**, '05001' no colisiona con ningún registro preexistente.
- ¿Satisface las restricciones de dominio? **Sí**, '05001' cumple con la longitud de 5 caracteres numéricos y '25' cumple con la de 2 caracteres numéricos.
- ¿Satisface la regla territorial C ? **No**, dado que $\text{SUBSTRING}('05001', 1, 2) = '05' \neq '25'$ (se registra a Medellín bajo la jurisdicción de Cundinamarca).

Dado que es factible formular tuplas que satisfacen rigurosamente todos los dominios y todas las claves pero violan la restricción territorial C , se demuestra formalmente que C **no es una consecuencia lógica de los dominios y claves declarados**. A pesar de hallarse en 5FN/BCNF, la relación municipios **no se encuentra en DKNF**, lo que obliga al gestor a implementar disparadores (*triggers*) o restricciones procedimentales complejas para impedir inconsistencias territoriales.

5. Rediseño hacia la forma normal de dominio-clave (DKNF):

Conforme al teorema de Fagin (Fagin, 1981), para que la restricción sea una consecuencia lógica directa, se descompone el identificador en sus componentes atómicos:

$$\text{municipios}(\text{dep_id}, \text{mun_consecutivo}, \text{mun_nombre})$$

- **Clave primaria compuesta:** (dep_id, mun_consecutivo).
- **Dominio de dep_id:** Cadena de 2 caracteres numéricos $[0 - 9]^2$ (clave foránea que referencia a departamentos).
- **Dominio de mun_consecutivo:** Cadena de 3 caracteres numéricos $[0 - 9]^3$ (subcódigo local municipal).

Bajo esta formulación:

- La tupla de Medellín se registra como ('05', '001', 'MEDELLÍN').
- El código DIVIPOLA integrado de 5 dígitos se deriva unívocamente mediante concatenación (dep_id || mun_consecutivo).
- Resulta imposible asociar a Medellín con el departamento '25' sin modificar explícitamente su clave primaria compuesta. La coherencia geográfica queda garantizada por la propia definición de las claves y dominios, **alcanzando plenamente la DKNF**.

7.4.6 Evolución paradigmática: almacenamiento relacional puro frente a objeto-relacional

Comprender las ventajas del diseño lógico relacional contemporáneo exige examinar la aproximación técnica adoptada en las décadas de 1980 y 1990. Ante la inexistencia de tipos abstractos de datos espaciales nativos en los motores de bases de datos, el cumplimiento estricto del principio de atomicidad de la Primera Forma Normal (1FN) forzaba la desarticulación tabular de cualquier objeto poligonal —como la delimitación geográfica de un municipio o departamento— en una jerarquía relacional de tablas alfanuméricas elementales.

i Notación de cardinalidad y opcionalidad pata de gallo (Crow's Foot)

A diferencia de la notación de Barker (donde la obligatoriedad se define mediante la alternancia de trazos continuos o discontinuos a lo largo del enlace), los diagramas entidad-relación generados en Mermaid implementan la simbología estándar formal **pata de gallo** (*Crow's Foot*), descomponiendo cada extremo de la asociación en dos glifos consecutivos:

- **Dos barras perpendiculares (||):** Formalizan una participación **estrictamente obligatoria y singular** (1..1). La barra interna especifica una cardinalidad mínima de uno (mandatorio), mientras que la barra externa fija la cardinalidad máxima en exactamente uno.
- **Barra perpendicular seguida de pata de gallo (|{):** Formaliza una participación **obligatoria de uno a muchos** (1..*). La barra codifica una cardinalidad mínima obligatoria de uno (no puede existir la entidad primaria sin al menos un registro dependiente), mientras que las tres líneas divergentes representan multiplicidad.
- **Círculo seguido de pata de gallo (o{):** Formaliza una relación **opcional de cero a muchos** (0..*). El círculo exterior codifica una cardinalidad mínima de cero (participación contingente), mientras que las tres líneas divergentes fijan la cardinalidad máxima múltiple.
- **Límites de la notación gráfica frente a reglas geométricas:** El estándar formal *Crow's Foot* únicamente admite cardinalidades mínimas de 0 o 1. Restricciones topológicas más estrictas —como la exigencia de un mínimo de tres vértices distintos para definir una superficie bidimensional, o cuatro coordenadas para clausurar un anillo poligonal según el estándar OGC ($N \geq 4$)— exceden la capacidad expresiva de los glifos de cardinalidad relacional. Estas condiciones geométricas no

se modelan en el conector visual, sino que se delegan a restricciones de integridad de dominio (*constraints* a nivel de tabla o disparadores).

Esta formulación tabular atomizada imponía limitaciones críticas en la gestión territorial:

- Renderizar la frontera cartográfica de un único municipio requería ejecutar operaciones de cruce (JOIN) consecutivas sobre cuatro tablas independientes y demandar un costo de ordenamiento secuencial (ORDER BY `pse_seq_num`) sobre miles de tuplas de nodos.
- Los predicados espaciales y las métricas topológicas básicas (cálculo de área, longitud perimetral o inclusión de predios) no podían resolverse en el servidor; el gestor debía transferir todas las coordenadas por la red hacia la estación de trabajo para reconstruir el polígono en la memoria del cliente SIG.
- Resultaba inviable estructurar métodos de indexación espacial multidimensional (como árboles R) sobre columnas escalares aisladas de tipo flotante (`pun_coord_x`, `pun_coord_y`).

La consolidación del modelo **objeto-relacional** y las especificaciones OGC/ISO transformaron el diseño de bases de datos territoriales al incorporar la geometría como un Tipo Abstracto de Datos (ADT) atómico nativo (`the_geom`). Esta arquitectura permite encapsular todas las primitivas y anillos que componen un municipio o departamento dentro de un único registro, habilitando el cómputo analítico en el servidor e integrando índices multidimensionales directos (árboles GiST) sobre la tabla relacional.

7.5 Implementación física y migración espacial desde el repositorio legado

El paso del modelo lógico al plano físico demanda materializar las relaciones en el gestor relacional y resolver la interoperabilidad con repositorios preexistentes. En los sistemas de información geográfica es habitual enfrentarse a bases de datos heredadas (*legacy*) que no se ajustan a las normas cartográficas vigentes ni a los principios de normalización relacional.

7.5.1 Arquitectura de la base de datos legado frente al nuevo repositorio local

El entorno de experimentación dispone de la base de datos `sig_db_unal`, la cual opera como repositorio de origen (*staging*). Al auditar su estructura técnica se constatan dos condiciones determinantes:

- **Sistema de referencia de coordenadas en desuso:** La totalidad de sus capas vectoriales está almacenada nativamente bajo el elipsoide Bogotá (origen Colombia Central, código **EPSG:21892**), sistema deprecado por la autoridad cartográfica nacional (IGAC) mediante la Resolución 471 de 2020 en favor del sistema unificado **EPSG:9377** (MAGNA-SIRGAS Origen Nacional).
- **Estructura tabular de origen:** A diferencia de esquemas cartográficos planos desarticulados, las tablas `divpol.departamentos` y `divpol.municipios` en `sig_db_unal` ya implementan identificadores DIVIOLA (`dep_id`, `mun_id`) y nombres normalizados (`dep_nombre`, `mun_nombre`). Sin embargo, carecen del atributo `mun_categoria` y requieren la migración formal hacia claves foráneas declarativas con integridad referencial en el nuevo motor.

Para subsanar estas limitaciones, se implementa en la instancia local la base de datos `bde_territorio_nacional`. Este nuevo repositorio adopta el estándar EPSG:9377, formaliza las claves foráneas del subsistema territorial

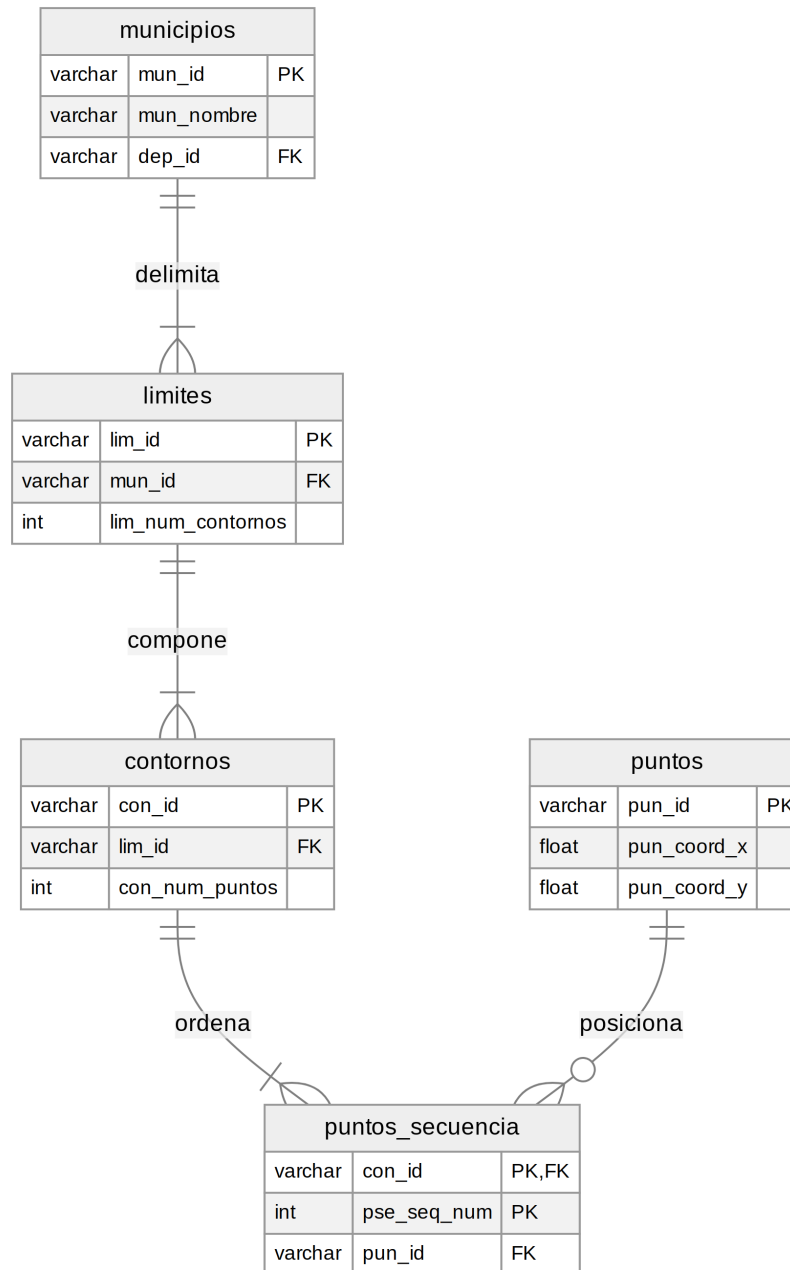


Figura 7.15: Desarticulación histórica de primitivas vectoriales bajo el modelo relacional puro sin tipos espaciales nativos.

(departamentos, municipios, microregiones) y resuelve la espacialidad de las entidades agregadas mediante álgebra computacional en el servidor.

7.5.2 Creación del catálogo físico y extensiones espaciales

La inicialización del repositorio operacional requiere la conexión administrativa a la base de datos por defecto del motor local (**postgres**), la terminación de sesiones concurrentes para garantizar idempotencia, la creación de la base de datos física y la posterior habilitación de la extensión PostGIS:

```
-- Importante: Usaremos la base de datos local!!

-- Terminación de conexiones activas y recreación limpia del repositorio
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'bde_territorio_nacional' AND pid <> pg_backend_pid();

DROP DATABASE IF EXISTS bde_territorio_nacional;

CREATE DATABASE bde_territorio_nacional
WITH ENCODING = 'UTF8'
TEMPLATE = template0;

\connect bde_territorio_nacional

-- Habilitación del motor espacial PostGIS y esquemas temáticos
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE SCHEMA IF NOT EXISTS divpol;
```

```
pg_terminate_backend
```

```
-----
(0 rows)
```

```
DROP DATABASE
CREATE DATABASE
CREATE EXTENSION
CREATE SCHEMA
```

7.5.3 Actualización del sistema de referencia espacial MAGNA-SIRGAS Origen Nacional (EPSG:9377)

Al crear un catálogo nuevo a partir de una plantilla básica, la tabla **spatial_ref_sys** no incluye por defecto la definición oficial del origen único nacional. Se asegura su registro en **bde_territorio_nacional** mediante la cláusula **ON CONFLICT**:

```
-- Importante: Estamos usando la base de datos local!!

-- Inserción o actualización del sistema MAGNA-SIRGAS Origen Nacional
INSERT INTO spatial_ref_sys (srid, auth_name, auth_srid, srtext, proj4text)
VALUES (
    9377,
    'EPSG',
    9377,
    'PROJCS["MAGNA-SIRGAS /
    ↪ Origen-Nacional",GEOGCS["MAGNA-SIRGAS",DATUM["Marco_Geocentrico_Nacional_de_Referencia",SPHEROID["GRS
    ↪ 1980",6378137,298.257222101,AUTHORITY["EPSG","7019"]],AUTHORITY["EPSG","6686"]],PRIMEM["Greenwich",0,AUTHORITY["EPSG",
    ↪
    ↪
    ↪ '+proj=tmerc +lat_0=4 +lon_0=-73 +k=0.9992 +x_0=5000000 +y_0=2000000 +ellps=GRS80
    ↪ +towgs84=0,0,0,0,0,0,0 +units=m +no_defs +type=crs'
    )
ON CONFLICT (srid) DO UPDATE
```

```
SET srtext = EXCLUDED.srtext, proj4text = EXCLUDED.proj4text;
```

```
INSERT 0 1
```

7.5.4 Estructura DDL con restricciones declarativas e índices

Siguiendo la signatura formal del modelo lógico territorial, se definen las tablas respetando el orden de dependencias referenciales: entidades padre (departamentos y microregiones) seguidas de la entidad dependiente (municipios). Se implementa la estrategia híbrida de identificadores mediante una columna artificial `gid SERIAL` para compatibilidad con SIG de escritorio, complementada por la clave primaria natural sobre el código oficial DIVIPOLA:

```
-- Importante: Estamos usando la base de datos local!!

-- 1. Entidad departamental (Clase espacial primaria)
CREATE TABLE divpol.departamentos (
  gid SERIAL,
  dep_id VARCHAR(2) NOT NULL,
  dep_nombre VARCHAR(100) NOT NULL,
  dep_region VARCHAR(50) NULL,
  the_geom GEOMETRY(MultiPolygon, 9377) NOT NULL,
  CONSTRAINT pk_divpol_departamentos PRIMARY KEY (dep_id),
  CONSTRAINT uq_departamentos_gid UNIQUE (gid)
);

-- 2. Entidad microrregiones (Entidad alfanumérica pura sin the_geom)
CREATE TABLE public.microregiones (
  mic_id INTEGER NOT NULL,
  mic_nombre VARCHAR(100) NOT NULL,
  mic_descripcion TEXT NULL,
  CONSTRAINT pk_public_microregiones PRIMARY KEY (mic_id),
  CONSTRAINT uq_microregiones_nombre UNIQUE (mic_nombre)
);

-- 3. Entidad municipal (Clase espacial subordinada)
CREATE TABLE divpol.municipios (
  gid SERIAL,
  mun_id VARCHAR(5) NOT NULL,
  mun_nombre VARCHAR(100) NOT NULL,
  mun_categoria VARCHAR(10) NULL,
  dep_id VARCHAR(2) NOT NULL,
  mic_id INTEGER NULL,
  the_geom GEOMETRY(MultiPolygon, 9377) NOT NULL,
  CONSTRAINT pk_divpol_municipios PRIMARY KEY (mun_id),
  CONSTRAINT uq_municipios_gid UNIQUE (gid),
  CONSTRAINT fk_municipios_departamentos FOREIGN KEY (dep_id)
    REFERENCES divpol.departamentos (dep_id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT,
  CONSTRAINT fk_municipios_microregiones FOREIGN KEY (mic_id)
    REFERENCES public.microregiones (mic_id)
    ON UPDATE CASCADE
    ON DELETE SET NULL,
  CONSTRAINT chk_mun_categoria CHECK (
    mun_categoria IN ('ESPECIAL', 'PRIMERA', 'SEGUNDA', 'TERCERA', 'CUARTA', 'QUINTA', 'SEXTA')
  )
);

-- Índices B-Tree para optimización de cruces relacionales (FK)
CREATE INDEX idx_municipios_dep_id ON divpol.municipios USING btree (dep_id);
CREATE INDEX idx_municipios_mic_id ON divpol.municipios USING btree (mic_id);

-- Índices espaciales GiST sobre cajas delimitadoras mínimas
CREATE INDEX sidx_departamentos_geom ON divpol.departamentos USING gist (the_geom);
CREATE INDEX sidx_municipios_geom ON divpol.municipios USING gist (the_geom);
```

```
NOTICE: table "municipios" does not exist, skipping
```

```
DROP TABLE
```

```
NOTICE: table "microregiones" does not exist, skipping
```

```

DROP TABLE
NOTICE:  table "departamentos" does not exist, skipping
DROP TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE INDEX

```

7.5.5 Migración de datos y reproyección de coordenadas

La transferencia de las coberturas vectoriales desde `sig_db_unal` hacia `bde_territorio_nacional` aplica transformaciones de coordenadas en vuelo. La función `ST_Transform(the_geom, 9377)` efectúa el cambio de datum y proyección cartográfica desde el elipsoide Bogotá (EPSG:21892) hacia el marco unificado nacional, mientras que `ST_Multi` garantiza la homogeneidad del tipo geométrico en caso de que existan polígonos simples en la tabla de origen:

```

-- Importante: Estamos usando la base de datos local!!

-- Ingesta y reproyección de departamentos desde sig_db_unal
INSERT INTO divpol.departamentos (dep_id, dep_nombre, the_geom)
SELECT
    dep_id,
    UPPER(TRIM(dep_nombre)) AS dep_nombre,
    ST_Multi(ST_Transform(the_geom, 9377)) AS the_geom
FROM dblink(
    'dbname=sig_db_unal user=postgres password=geomatica2025 host=127.0.0.1 port=5432',
    'SELECT dep_id, dep_nombre, the_geom FROM divpol.departamentos'
) AS d(dep_id VARCHAR(2), dep_nombre VARCHAR(100), the_geom GEOMETRY);

```

```

CREATE EXTENSION
DELETE 0
INSERT 0 32

```

i Resolución de nombres de red en dblink: cliente Docker frente a servidor anfitrión

La aparente contradicción al configurar los parámetros de red radica en el contexto de ejecución de cada componente del flujo de trabajo:

- **Invocación del cliente de línea de comandos (psql):** Cuando la sentencia se dispara desde la terminal integrada dentro del contenedor de desarrollo, el comando `psql` requiere el parámetro `-h "${LOCAL_PGHOST}"` (resuelto como `host.docker.internal`). Este nombre de dominio especial es provisto por el motor de Docker para permitir que un proceso cliente contenerizado enrute sus paquetes hacia el sistema operativo anfitrión.
- **Ejecución interna de la función dblink():** A diferencia del comando de terminal, la instrucción `SELECT * FROM dblink(...)` no la ejecuta el cliente en Docker, sino el proceso del servidor PostgreSQL que reside en el sistema anfitrión. Desde la perspectiva de ese motor, la base de datos de origen (`sig_db_unal`) está alojada localmente en su misma instancia física.

Si dentro de la cadena de conexión de `dblink` se declara `host=host.docker.internal`, el sistema operativo anfitrión intentará resolver ese dominio a través de su servidor DNS convencional, fallando con el error `could not translate host name "host.docker.internal"`. Por esta razón, la cadena de conexión interna de `dblink` debe declarar estrictamente `host=127.0.0.1` (o `localhost`), garantizando que la conexión interbase ocurra a través de la interfaz de bucle invertido (*loopback*) del propio equipo.

Para el poblamiento de `public.microrregiones`, se insertan las delimitaciones operativas y se vinculan a la tabla de municipios durante la migración. La categorización municipal (`mun_categoria`) se asigna formalmente para cumplir con la restricción `chk_mun_categoria`, clasificando las ciudades principales en `ESPECIAL`, Chía en `PRIMERA` y el resto en `SEXTA`:

```
-- Importante: Estamos usando la base de datos local!!

-- Inserción de catálogo temático de microrregiones
INSERT INTO public.microrregiones (mic_id, mic_nombre, mic_descripcion) VALUES
(1, 'Bajo Cauca', 'Región ganadera del norte de Antioquia y sur de Córdoba'),
(2, 'Valle del Sinú', 'Zona ganadera intensiva de la cuenca del río Sinú'),
(3, 'Sabanas de Sucre y Bolívar', 'Sistemas de doble propósito en sabanas del Caribe'),
(4, 'Altillanura Plana', 'Sabana nativa y pasturas mejoradas del oriente del Meta');

-- Ingesta de municipios con reproyección y asignación de claves foráneas
INSERT INTO divpol.municipios (mun_id, mun_nombre, mun_categoria, dep_id, mic_id, the_geom)
SELECT
    m.mun_id,
    UPPER(TRIM(m.mun_nombre)) AS mun_nombre,
    CASE
        WHEN m.mun_id IN ('05001', '11001', '76001', '08001') THEN 'ESPECIAL'
        WHEN m.mun_id = '25175' THEN 'PRIMERA'
        ELSE 'SEXTA'
    END AS mun_categoria,
    m.dep_id,
    CASE
        WHEN m.mun_id IN ('05154', '05250', '05495', '05790', '05837', '05893') THEN 1
        WHEN m.mun_id IN ('23001', '23079', '23168', '23182', '23417', '23555') THEN 2
        WHEN m.mun_id IN ('50006', '50226', '50568', '50577') THEN 4
        ELSE NULL
    END AS mic_id,
    ST_Multi(ST_Transform(m.the_geom, 9377)) AS the_geom
FROM dblink(
    'dbname=sig_db_unal user=postgres password=geomatica2025 host=127.0.0.1 port=5432',
    'SELECT mun_id, mun_nombre, dep_id, the_geom FROM divpol.municipios'
) AS m(mun_id VARCHAR(5), mun_nombre VARCHAR(50), dep_id VARCHAR(2), the_geom GEOMETRY);
```

```
DELETE 0
DELETE 0
INSERT 0 4
INSERT 0 1121
```

7.5.6 Geometría derivada: disolución topológica de microrregiones

Tal como se formalizó en el modelo conceptual UML y en el diseño lógico, la entidad `public.microrregiones` carece de almacenamiento geométrico redundante. La extensión geográfica de cada microrregión se computa dinámicamente agregando las geometrías de sus municipios subordinados mediante `ST_Union` y se publica en el catálogo como una vista espacial:

```
-- Importante: Estamos usando la base de datos local!!

-- Creación de la vista espacial con agregación topológica en tiempo de consulta
CREATE OR REPLACE VIEW public.v_microrregiones_espacial AS
SELECT
    mic.mic_id,
    mic.mic_nombre,
```

```
mic.mic_descripcion,
COUNT(mun.mun_id)::integer AS total_municipios,
ROUND((ST_Area(ST_Union(mun.the_geom)) / 10000.0)::numeric, 2) AS area_total_ha,
ST_Multi(ST_Union(mun.the_geom)::GEOMETRY(MultiPolygon, 9377) AS the_geom
FROM public.microregiones mic
INNER JOIN divpol.municipios mun ON mic.mic_id = mun.mic_id
GROUP BY mic.mic_id, mic.mic_nombre, mic.mic_descripcion;
```

NOTICE: view "v_microregiones_espacial" does not exist, skipping

DROP VIEW

CREATE VIEW

7.5.7 Operaciones de combinación relacional y espacial en el nuevo esquema

Una vez poblado el repositorio, es posible contrastar los dos mecanismos de articulación de información analizados en el diseño lógico: la combinación relacional gobernada por claves foráneas (*Attribute Join*) y la combinación espacial asistida por predicados topológicos (*Spatial Join*).

7.5.7.1 Combinación relacional: cuantificación municipal por departamento

La consulta siguiente evalúa la igualdad estricta `mun.dep_id = dep.dep_id` aprovechando el índice B-Tree creado sobre la clave foránea:

```
-- Importante: Estamos usando la base de datos local!!

SELECT
  dep.dep_id,
  dep.dep_nombre,
  COUNT(mun.mun_id) AS total_municipios,
  ROUND((ST_Area(dep.the_geom) / 1000000.0)::numeric, 2) AS area_dep_km2
FROM divpol.departamentos dep
INNER JOIN divpol.municipios mun ON dep.dep_id = mun.dep_id
GROUP BY dep.dep_id, dep.dep_nombre, dep.the_geom
ORDER BY total_municipios DESC
LIMIT 5;
```

dep_id	dep_nombre	total_municipios	area_dep_km2
05	ANTIOQUIA	125	63830.23
15	BOYACÁ	123	22915.50
25	CUNDINAMARCA	117	22510.08
68	SANTANDER	87	30387.59
52	NARIÑO	65	32826.21

(5 rows)

7.5.7.2 Combinación espacial: adscripción territorial independiente de identificadores

A diferencia de la combinación por atributos, un *Spatial Join* evalúa la inclusión topológica euclidiana mediante `ST_Intersects` sobre los índices GiST, permitiendo articular capas que no comparten claves foráneas explícitas:

```
-- Importante: Estamos usando la base de datos local!!

-- Verificación de contigüidad espacial entre la microrregión Valle del Sinú y municipios
SELECT
```

7.5. IMPLEMENTACIÓN FÍSICA Y MIGRACIÓN ESPACIAL DESDE EL REPOSITORIO LEGADO173

```
vm.mic_nombre,
mun.mun_nombre,
ROUND((ST_Area(ST_Intersection(mun.the_geom, vm.the_geom)) / 10000.0)::numeric, 2) AS area_traslape_ha
FROM public.v_microregiones_espacial vm
INNER JOIN divpol.municipios mun ON ST_Intersects(mun.the_geom, vm.the_geom)
WHERE vm.mic_id = 2
ORDER BY area_traslape_ha DESC;
```

mic_nombre	mun_nombre	area_traslape_ha
Valle del Sinú	MONTERÍA	302703.78
Valle del Sinú	PLANETA RICA	115764.43
Valle del Sinú	LORICA	93630.86
Valle del Sinú	BUENAVISTA	75494.56
Valle del Sinú	CHINÚ	53512.25
Valle del Sinú	CHIMÁ	34246.79
Valle del Sinú	VALENCIA	0.00
Valle del Sinú	PUERTO ESCONDIDO	0.00
Valle del Sinú	SAHAGÚN	0.00
Valle del Sinú	SAN ANDRÉS DE SOTAVENTO	0.00
Valle del Sinú	SAN ANTERO	0.00
Valle del Sinú	MOMIL	0.00
Valle del Sinú	MOÑITOS	0.00
Valle del Sinú	PUEBLO NUEVO	0.00
Valle del Sinú	AYAPEL	0.00
Valle del Sinú	CANALETE	0.00
Valle del Sinú	COTORRA	0.00
Valle del Sinú	ARBOLETES	0.00
Valle del Sinú	LA APARTADA	0.00
Valle del Sinú	SAN PEDRO DE URABÁ	0.00
Valle del Sinú	TIERRALTA	0.00
Valle del Sinú	SAN BENITO ABAD	0.00
Valle del Sinú	SAN BERNARDO DEL VIENTO	0.00
Valle del Sinú	CERETÉ	0.00
Valle del Sinú	SAN PELAYO	0.00
Valle del Sinú	CIÉNAGA DE ORO	0.00
Valle del Sinú	PURÍSIMA	0.00
Valle del Sinú	SAMPUÉS	0.00
Valle del Sinú	EL ROBLE	0.00
Valle del Sinú	CAIMITO	0.00
Valle del Sinú	LA UNIÓN	0.00
Valle del Sinú	MONTELÍBANO	0.00
Valle del Sinú	SAN CARLOS	0.00
Valle del Sinú	LOS CÓRDOBAS	0.00

(34 rows)

i Semántica topológica de ST_Intersects frente a colindancias de frontera

La presencia de 34 registros en el resultado obedece a la definición matemática del predicado `ST_Intersects(A, B)` según el modelo estándar DE-9IM del OGC:

- **Superposición superficial** ($D = 2$): Los 6 municipios constitutivos de la microrregión comparten área interior bidimensional, registrando superficies de traslape calculadas en miles de hectáreas.
- **Contacto perimetral** ($D \leq 1$): Los 28 municipios restantes son unidades vecinas que únicamente tocan la frontera exterior de la microrregión a lo largo de un límite compartido (`LineString`, satisfaciendo la condición topológica `ST_Touches`). Al evaluar `ST_Intersection` sobre entidades colindantes que solo comparten bordes, la geometría resultante es puramente lineal; en consecuencia, `ST_Area` evalúa su superficie como 0.00 ha.

Para aislar exclusivamente los municipios miembros y descartar colindancias de borde, la consulta puede restringirse añadiendo `AND ST_Area(ST_Intersection(mun.the_geom, vm.the_geom)) > 0` o evaluando la dimensión interior mediante `ST_Relate(mun.the_geom, vm.the_geom, '2*****')`.

7.5.8 Asociación muchos a muchos en el modelo conceptual

En la especificación de requerimientos de la Guía Ambiental para el Sector Ganadero Bovino de Colombia, el subsistema productivo e institucional plantea una interacción no jerárquica: múltiples dependencias técnicas gestionan de manera simultánea y concurrente múltiples sistemas de producción pecuaria.

En el nivel de abstracción conceptual, esta interacción se modela como una **asociación binaria muchos a muchos** ($M : M$) directa entre las clases `dependencias` y `sistemas_produccion` dentro del paquete `fgan`, con multiplicidades $[1..*] \leftrightarrow [1..*]$ evaluadas bajo la perspectiva *Look-Across*:

7.5.8.1 Resolución lógica mediante tabla asociativa intermedia

El modelo relacional estándar prohíbe implementar de forma directa una cardinalidad muchos a muchos entre dos relaciones simples. Almacenar listas de identificadores foráneos en una celda violaría el principio de atomicidad de la primera forma normal (1FN), mientras que duplicar filas de dependencias por cada sistema productivo induciría redundancias severas y anomalías de actualización.

Para resolver esta limitación en el diseño lógico, la asociación $M : M$ se descompone formalmente en dos relaciones uno a muchos ($1 : N$) mediante una **tabla asociativa intermedia** o tabla puente (`fgan.dependencias_sistemas_produccion`):

1. La tabla intermedia hereda los identificadores de ambas tablas maestras: **dpe_id** y **spr_id**.
2. Ambas columnas actúan simultáneamente como claves foráneas (referenciando a sus respectivos catálogos) y se combinan para conformar la **clave primaria compuesta** de la relación: (**dpe_id**, **spr_id**).
3. La tabla intermedia permite incorporar atributos propios de la relación que no pertenecen a ninguna de las entidades maestras por separado, tales como la fecha de formalización operativa (`dps_fecha_asignacion`).

El siguiente diagrama formaliza la descomposición lógica en el esquema `fgan`, detallando el código cromático de claves (verde para claves primarias simples, rojo subrayado para claves compuestas foráneas) y la signatura relacional resultante con sus punteros de integridad referencial:

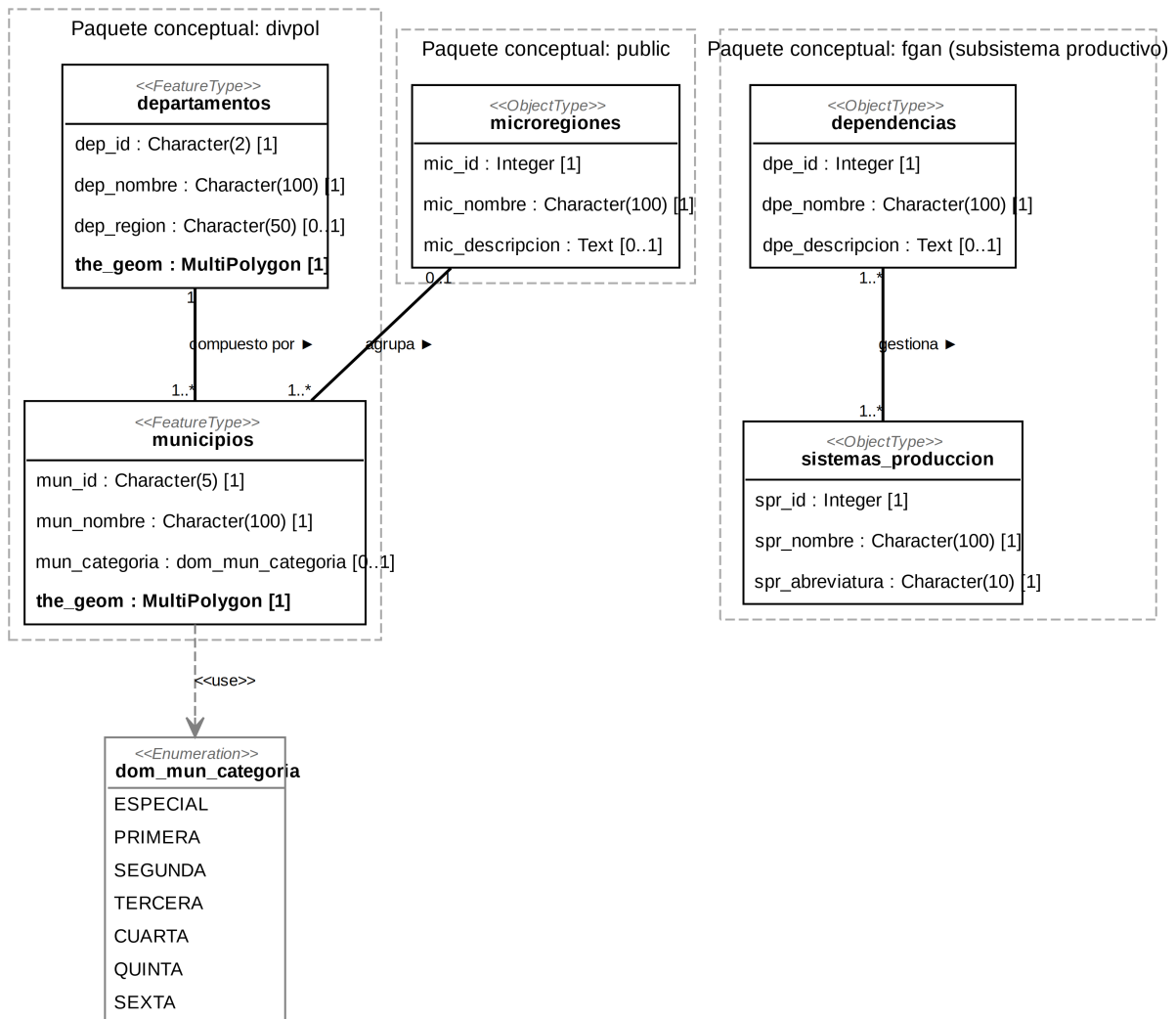


Figura 7.16: Modelo conceptual UML con paquetes temáticos y asociación muchos a muchos directa en el subsistema productivo.

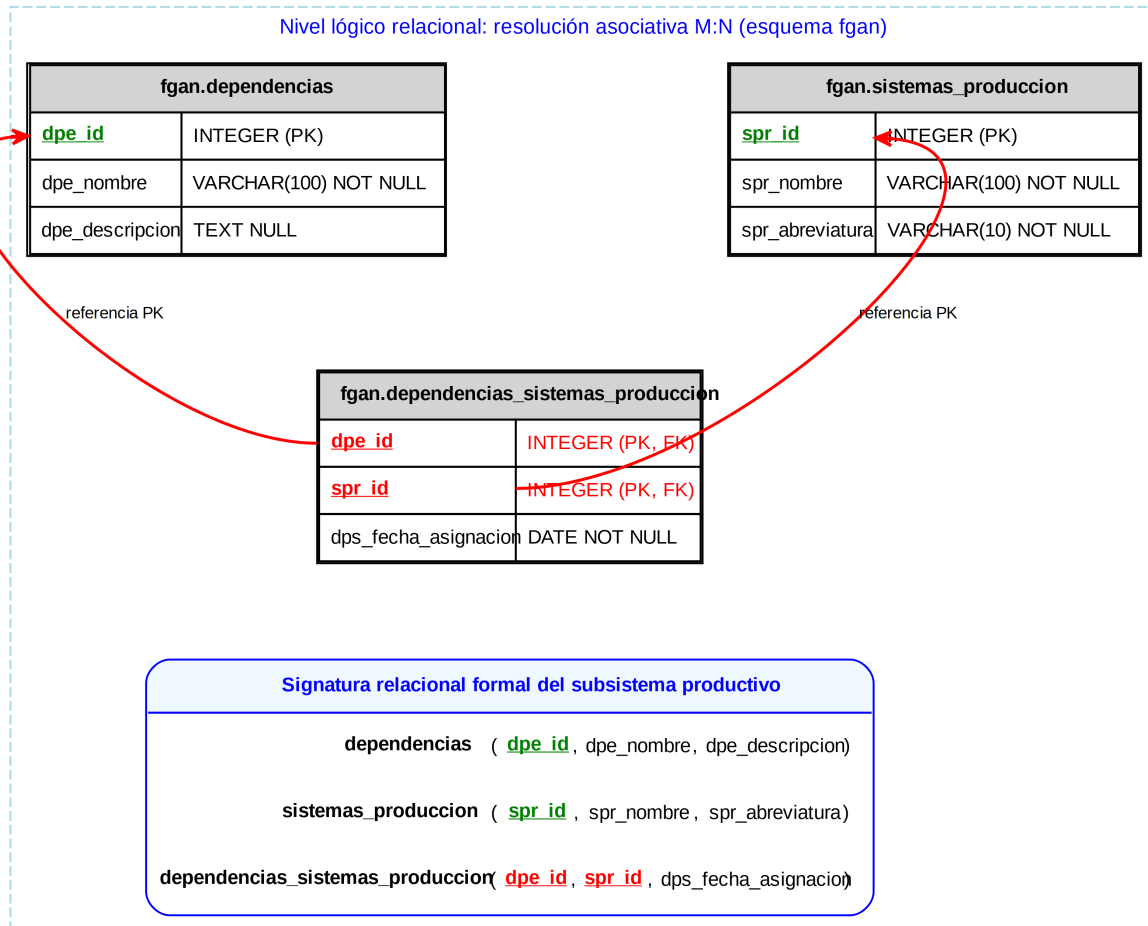


Figura 7.17: Resolución del esquema relacional M:N mediante tabla intermedia asociativa, código cromático y signatura formal.

7.5.8.2 Implementación física DDL del subsistema productivo

Para materializar este diseño en la base de datos operacional `bde_territorio_nacional`, se declara el esquema `fgan`, se implementan los catálogos y se codifica la tabla asociativa intermedia con sus restricciones declarativas de clave primaria compuesta e integridad referencial en cascada:

```
-- Importante: Estamos usando la base de datos local!!

CREATE SCHEMA IF NOT EXISTS fgan;

-- 1. Catálogo de dependencias maestras
CREATE TABLE fgan.dependencias (
    dpe_id INTEGER NOT NULL,
    dpe_nombre VARCHAR(100) NOT NULL,
    dpe_descripcion TEXT NULL,
    CONSTRAINT pk_fgan_dependencias PRIMARY KEY (dpe_id)
);

-- 2. Catálogo de sistemas de producción
CREATE TABLE fgan.sistemas_produccion (
    spr_id INTEGER NOT NULL,
    spr_nombre VARCHAR(100) NOT NULL,
    spr_abreviatura VARCHAR(10) NOT NULL,
    CONSTRAINT pk_fgan_sistemas_produccion PRIMARY KEY (spr_id)
);

-- 3. Tabla asociativa intermedia (Resolución física M:N)
CREATE TABLE fgan.dependencias_sistemas_produccion (
    dpe_id INTEGER NOT NULL,
    spr_id INTEGER NOT NULL,
    dps_fecha_asignacion DATE NOT NULL DEFAULT CURRENT_DATE,
    CONSTRAINT pk_dependencias_sistemas PRIMARY KEY (dpe_id, spr_id),
    CONSTRAINT fk_dps_dependencias FOREIGN KEY (dpe_id)
        REFERENCES fgan.dependencias (dpe_id)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_dps_sistemas FOREIGN KEY (spr_id)
        REFERENCES fgan.sistemas_produccion (spr_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);
```

```
CREATE SCHEMA
```

```
NOTICE: table "dependencias_sistemas_produccion" does not exist, skipping
```

```
DROP TABLE
```

```
NOTICE: table "sistemas_produccion" does not exist, skipping
```

```
DROP TABLE
```

```
NOTICE: table "dependencias" does not exist, skipping
```

```
DROP TABLE
```

```
CREATE TABLE
```

```
CREATE TABLE
```

```
CREATE TABLE
```

```
INSERT 0 2
```

```
INSERT 0 3
```

```
INSERT 0 4
```

7.6 Ejercicios de diseño conceptual, lógico e implementación física

1. **Normalización y dependencia funcional en infraestructura de riego:** Considérese la siguiente relación no normalizada para el registro de distritos de adecuación de tierras:

```
distritos_riego(dis_id, dis_nombre, cue_id, cue_nombre,
               mun_id, mun_nombre, area_beneficiada_ha, the_geom)
```

- Identifique formalmente las dependencias funcionales completas, parciales y transitivas presentes en la relación.
 - Aplique la descomposición normalizada hasta alcanzar la Tercera Forma Normal (3FN), especificando la signature relacional con código cromático (claves primarias en verde y foráneas en rojo) y prefijos mnemotécnicos de tres letras.
2. **Auditoría geodésica y validación topológica automatizada:** Escriba un bloque de consulta SQL que inspeccione el catálogo `geometry_columns` de PostgreSQL para detectar cualquier capa vectorial que resida en el esquema `divpol` bajo el sistema obsoleto `SRID = 21892`. Formule a continuación una sentencia DDL/DML que ejecute la reproyección hacia el sistema oficial `EPSG:9377` condicionada a que la geometría resultante satisfaga el predicado de validez topológica estricta (`ST_IsValid(the_geom) = TRUE`).
3. **Derivación analítica de la cobertura nacional mediante agregación:** En la arquitectura de la base de datos corporativa, la tabla física `divpol.nacion` no se adquiere como un vector cartográfico estático independiente, sino que debe derivarse a partir de la delimitación territorial de primer orden.
- Formule la expresión en álgebra relacional correspondiente a la unión espacial disuelta de la cobertura departamental.
 - Escriba la sentencia SQL que cree la tabla física `divpol.nacion` y la pueble con una única tupla de geometría continua (`MultiPolygon`, `EPSG:9377`) agregando los registros de `divpol.departamentos` mediante la función `ST_Union`.
4. **Modelado integral y resolución física de relaciones $M : N$ con atributos descriptivos:**

Una corporación autónoma regional requiere estructurar un repositorio espacial para el control ambiental de predios rurales que intersectan rondas hídricas de protección de fuentes superficiales. Las reglas operativas del territorio establecen:

- Un predio rural se individualiza mediante su código catastral oficial (`pre_codigo`), posee una matrícula inmobiliaria, área de terreno calculada y una delimitación poligonal continua (`the_geom`).
- Una ronda hídrica de protección se identifica mediante un código único (`ron_id`), el nombre del cauce hídrico asociado, el ancho legal de la franja de amortiguamiento en metros y una cobertura espacial poligonada (`the_geom`).
- La relación espacial entre ambas coberturas es no jerárquica: un mismo predio puede ser atravesado de manera concurrente por múltiples rondas hídricas, y una ronda de protección se extiende territorialmente a lo largo de decenas o cientos de predios independientes.
- La superposición genera una propiedad que no pertenece intrínsecamente al predio ni a la ronda de forma aislada, sino al fenómeno de intersección entre ambos: el porcentaje de afectación del predio (`afa_porcentaje_afectacion`), acompañado por la fecha del levantamiento técnico.

A partir de estos requerimientos, desarrolle los siguientes productos de ingeniería de datos:

- **Modelado conceptual Entidad-Relación (E-R):** Diseñe el diagrama E-R formal utilizando la notación pata de gallo (*Crow's Foot*), explicitando las entidades participantes, sus atributos descriptivos

e identificadores, y resolviendo la asociación muchos a muchos ($M : N$) mediante una entidad asociativa intermedia que aloje el atributo propio de la relación.

- **Modelado conceptual en diagramas de clases UML:** Construya el diagrama de clases conforme al perfil geoespacial ISO 19109/OGC estructurado en el paquete temático **ambiental**, tipificando las coberturas cartográficas con el estereotipo `<<FeatureType>>`, la entidad intermedia con `<<ObjectType>>`, e indicando las multiplicidades exactas bajo la perspectiva *Look-Across*.
- **Diseño lógico relacional y firmas con código cromático:** Formule las firmas relacionales formales del esquema resultante aplicando la convención de prefijos mnemotécnicos de tres letras, resaltando las claves primarias simples en color verde y subrayadas, y las claves foráneas en color rojo (indicando la clave primaria compuesta de la tabla intermedia mediante subrayado y color rojo simultáneo).
- **Implementación física DDL en PostgreSQL y PostGIS:** Escriba el bloque de sentencias DDL en PostgreSQL/PostGIS para materializar el esquema **ambiental**, creando las tablas maestras con geometrías `MultiPolygon` en el sistema unificado EPSG:9377 y la tabla puente intermedia, parametrizando:
 - Clave primaria compuesta sobre los identificadores foráneos.
 - Regla de integridad referencial `ON DELETE RESTRICT` y `ON UPDATE CASCADE` hacia ambas entidades padre.
 - Restricción declarativa `CHECK` que limite el porcentaje de afectación al intervalo numérico `[0.0, 100.0]`.
 - Índices relacionales B-Tree sobre las claves foráneas e índices espaciales GiST sobre las geometrías.

Parte III

Proyectos aplicados

Capítulo 8

Guía metodológica para el proyecto final

8.1 Estructura de la sesión de dos horas

El equipo expositor dispone de 2 horas (120 minutos) para presentar el proyecto de investigación y liderar la sesión práctica ante la cohorte. El tiempo se organiza en tres bloques secuenciales:

- **Bloque 1: Presentación del marco conceptual y arquitectura (40 minutos):** Exposición magistral apoyada en diapositivas interactivas sobre el estado del arte, la problemática de escalabilidad o cuello de botella del SGBD relacional clásico, el funcionamiento interno de la tecnología seleccionada (ej. Apache Sedona, DuckDB espacial, DGGS H3/S2, grafos con Neo4j/pgRouting, NoSQL, extensiones nativas en Rust o el stack GeoLibre) y el diseño experimental planteado.
- **Bloque 2: Taller guiado en tiempo real (65 minutos):** Desarrollo del laboratorio con la cohorte (*live coding*). La cohorte clona el repositorio o accede al entorno de contenedores y ejecuta el flujo de trabajo junto con los expositores desde la terminal integrada de Visual Studio Code o Positron. Se realizan pruebas de carga, particionamiento, construcción de índices espaciales y ejecución de consultas de *Spatial SQL* en vivo.
- **Bloque 3: Debate técnico y retroalimentación (15 minutos):** Espacio de discusión crítica, evaluación de métricas de rendimiento, análisis de planes de ejecución (DAG en Spark, **EXPLAIN ANALYZE** en PostgreSQL/DuckDB) y conclusiones metodológicas.

8.2 Estructura de carpetas y paquete de entregables

Cada equipo de investigación debe crear y estructurar la carpeta correspondiente a la temática asignada dentro del directorio `proyectos_finales/`, respetando estrictamente la siguiente jerarquía:

```
proyectos_finales/  
  tema_XX_[nombre_tema]/  
    XX_proyecto_[nombre_tema].qmd  
    XX_rubrica_[nombre_tema].qmd
```

```
XX_slides_[nombre_tema].qmd
XX_taller_[nombre_tema].qmd
references.bib
```

Donde `XX` representa el identificador numérico de dos dígitos asignado al tema (de 01 a 07) y `[nombre_tema]` corresponde al identificador en `snake_case` (ej. `01_duckdb_geoparquet`, `02_apache_sedona`, `07_geolibre_stack`). La orquestación global del libro mediante `_quarto.yml` y la configuración de contenedores residen exclusivamente en la raíz del repositorio central.

El paquete de entregables por equipo se compone de los siguientes artefactos:

- **`XX_proyecto_[nombre_tema].qmd`**: Documento de investigación estructurado bajo formato de reporte científico. Contiene la justificación teórica, objetivos, preguntas de investigación orientadoras, diseño experimental, análisis comparativo de rendimiento (CPU, memoria, E/S de disco) y discusión técnica. Este archivo se compilará como capítulo del libro del curso.
- **`XX_rubrica_[nombre_tema].qmd`**: Matriz de evaluación con criterios cuantitativos y cualitativos específicos para valorar el reporte analítico, la reproducibilidad del entorno y la ejecución del código.
- **`XX_slides_[nombre_tema].qmd`**: Presentación interactiva en Quarto RevealJS (`format: revealjs`) empleada como soporte visual durante el Bloque 1.
- **`XX_taller_[nombre_tema].qmd`**: Guía ejecutable paso a paso para el laboratorio guiado del Bloque 2, incluyendo preparación del entorno, bloques de código documentados y comandos de verificación.
- **`references.bib`**: Archivo BibTeX con la literatura académica y técnica citada en el reporte y en la presentación.

8.3 Estándares técnicos y directivas de código en Quarto

Todos los documentos `.qmd` generados por los equipos deben cumplir rigurosamente con los siguientes estándares de edición y reproducibilidad:

- **Sintaxis YAML en bloques de código**: Todo bloque ejecutable o demostrativo debe utilizar la sintaxis YAML de Quarto (`#| label: ...`, `#| eval: ...`, `#| echo: ...`). Está prohibido el uso de cabeceras heredadas de Knitr (`{r, ...}`).
- **Nomenclatura taxonómica de etiquetas**: Las etiquetas de los bloques (`label`) deben redactarse en `kebab-case` utilizando los siguientes prefijos según su función:
- **`setup-`**: Inicialización de entornos, librerías, conexiones a bases de datos o variables de entorno.
- **`docker-`**: Comandos de orquestación, verificación o ejecución dentro de contenedores.
- **`ddl-`**: Sentencias de definición de datos (`CREATE TABLE`, `CREATE INDEX`, particionamiento).
- **`dml-`**: Ingestión, inserción, actualización o transformación de datos espaciales.
- **`query-` o `sql-`**: Consultas analíticas espaciales, cruces topológicos y agregaciones.
- **`audit-`**: Verificación de integridad, conteo de registros o validación de geometrías.
- **`export-` o `backup-`**: Extracción modular de esquemas o generación de respaldos binarios.

- **Sintaxis estricta de línea de comandos (CLI):** En comandos de terminal (`psql`, `shp2pgsql`, `raster2pgsql`, `docker`, `docker compose`), es obligatorio incluir un espacio en blanco después de cada modificador corto (ej. `-p 5432`, `-h localhost`, `-U usuario`, `-d base_datos`, `-f script.sql`).
- **Neutralidad de plataforma:** Las instrucciones deben formularse de manera agnóstica al sistema operativo del estudiante, haciendo referencia a “la terminal integrada de Visual Studio Code / Positron”, “el equipo local” o “el sistema anfitrión (*host*)”.
- **Estilo editorial:** Redacción impersonal y técnica en *Sentence case* para todos los encabezados (`#`, `##`, `###`), manteniendo mayúsculas exclusivamente en siglas técnicas (SQL, OGC, SRID, EPSG, WKB, DAG). Se debe dejar una línea en blanco antes de iniciar cualquier lista o tabla para garantizar la correcta compilación en Quarto.

8.4 Pipeline de compilación y publicación continua

La integración y compilación de los proyectos se automatiza mediante GitHub Actions a partir del archivo `central_quarto.yml`:

1. **Indexación central:** El archivo `_quarto.yml` compila cada `XX_proyecto_[nombre_tema].qmd` como un capítulo secuencial del libro general de la asignatura.
2. **Publicación web:** Cada *Pull Request* validado genera el portal web del curso con los capítulos en HTML, las diapositivas interactivas (`XX_slides_[nombre_tema].html`) y las guías de laboratorio.
3. **Compilación PDF:** El flujo de integración compila el libro consolidado en formato PDF para impresión (vía Typst o LaTeX), unificando índices, referencias cruzadas y bibliografía centralizada.

Capítulo 9

Temas de proyecto final en bases de datos espaciales

Tabla 9.1: Matriz de asignación de temas para grupos de posgrado

Código	Tema de investigación	Paradigma tecnológico	Enfoque experimental del taller
Tema 01	DuckDB espacial y GeoParquet	OLAP embebido en memoria / Formatos Cloud-Native	Consultas analíticas vectoriales sobre almacenamiento en la nube
Tema 02	Apache Sedona y PySpark	Cómputo espacial distribuido (<i>Big Data</i>)	<i>Spatial Join</i> masivo con evaluación de particionamiento
Tema 03	Mallas globales H3 y S2 (DGGS)	Indexación espacial jerárquica discreta	Agregación de eventos y reclasificación multi-resolución
Tema 04	Neo4j Spatial vs. pgRouting	Grafos de propiedades vs. Relacional topológico	Ruteo dinámico y cálculo de isócronas en redes viales
Tema 05	Elasticsearch vs. MongoDB espacial	NoSQL orientado a documentos / Telemetría IoT	Ingestión continua y consultas de <i>geo-fencing</i> concurrentes
Tema 06	Lógica procedural y extensiones nativas	Procesamiento en servidor (PL/pgSQL, Rust)	Benchmark de serialización WKB y pipelines de análisis local
Tema 07	Ecosistemas analíticos desacoplados y flujos con GeoLibre	Arquitectura analítica abierta / <i>Modern Spatial Data Stack</i>	Pipelines reproducibles sin servidor sobre GeoParquet y PMTiles

9.1 Tema 01: Procesamiento analítico columnar y formatos nativos de nube (DuckDB espacial y GeoParquet)

- **Identificador de carpeta:** `proyectos_finales/tema_01_duckdb_geoparquet/`
- **Fundamento tecnológico:** DuckDB implementa un motor OLAP vectorial ejecutado en el espacio de memoria del cliente, eliminando la sobrecarga de la arquitectura cliente-servidor clásica de PostgreSQL/PostGIS. Permite la ejecución de consultas *Spatial SQL* directamente sobre archivos GeoParquet remotos o locales sin necesidad de ingestión previa a tablas relacionales.
- **Diseño experimental:** Comparación de tiempos de consulta, lectura de E/S y huella de memoria al procesar millones de geometrías (ej. polígonos catastrales o huellas de edificios) almacenadas en GeoParquet versus tablas indexadas en PostGIS.
- **Preguntas de investigación:**
 - ¿En qué magnitud reduce la compresión y el *predicate pushdown* de GeoParquet los tiempos de filtrado espacial frente a un escaneo indexado en PostGIS?
 - ¿Cuál es el límite de volumen de geometrías complejas que DuckDB puede procesar en memoria antes de experimentar degradación por paginación en disco?
- **Nota metodológica (ajuste a seis grupos):** En caso de consolidar la cohorte en seis equipos de trabajo, este proyecto integrará la plataforma **GeoLibre** como caso de estudio práctico y capa de aplicación en el taller guiado, utilizándola para demostrar la interoperabilidad de DuckDB y el consumo directo de capas GeoParquet y PMTiles desde una interfaz analítica abierta sin servidor.

9.2 Tema 02: Escalabilidad y procesamiento masivo distribuido (Apache Sedona y PySpark)

- **Identificador de carpeta:** `proyectos_finales/tema_02_apache_sedona/`
- **Fundamento tecnológico:** Transición del procesamiento espacial monolítico hacia arquitecturas distribuidas en clúster utilizando Apache Sedona. El foco radica en la distribución de estructuras geométricas, el particionamiento espacial (*Quad-Tree* / *KDB-Tree*) y la optimización de cruces espaciales (*Spatial Joins*) masivos en memoria distribuida.
- **Diseño experimental:** Ejecución de una prueba de carga cruzando capas de telemetría masiva (> 5 millones de registros) con polígonos de zonificación urbana/ambiental. Medición de tiempos de ejecución y análisis del grafo acíclico dirigido (*DAG*) en PySpark frente al límite de procesamiento de un único nodo en PostGIS.
- **Preguntas de investigación:**
 - ¿Cómo afecta la elección del árbol de particionamiento espacial a la minimización del *data shuffling* entre ejecutores Spark durante un *Spatial Join*?
 - ¿Cuál es el umbral de registros en el que la sobrecarga de coordinación de un clúster distribuido se justifica frente a un servidor relacional dedicado?

9.3 Tema 03: Sistemas de mallas espaciales discretas globales (DGGS: Uber H3 y Google S2)

- **Identificador de carpeta:** `proyectos_finales/tema_03_dggs_h3_s2/`
- **Fundamento tecnológico:** Reemplazo de los índices espaciales continuos basados en geometrías vectoriales clásicas (R-Tree / GiST) por sistemas de teselación jerárquica discreta (hexágonos H3 o celdas cuadrangulares S2). Este enfoque transforma costosas operaciones topológicas en operaciones enteras indexadas de alta velocidad.
- **Diseño experimental:** Agregación espacial de grandes volúmenes de eventos puntuales espacio-temporales y cálculo de densidad/vecindad mediante indexación H3 directa en base de datos frente a operaciones vectoriales tradicionales `ST_Intersects` y `ST_ClusterKMeans` en PostGIS.
- **Preguntas de investigación:**
 - ¿Qué balance existe entre la distorsión geométrica/área inherente a la resolución de H3/S2 y la ganancia en rendimiento computacional en consultas de agregación?
 - ¿Cómo impacta la conversión de geometrías continuas a identificadores discretos de 64 bits en el tamaño de almacenamiento e indexación en disco?

9.4 Tema 04: Bases de datos espaciales orientadas a grafos y análisis de redes (Neo4j Spatial vs. pgRouting)

- **Identificador de carpeta:** `proyectos_finales/tema_04_grafos_pg routing_neo4j/`
- **Fundamento tecnológico:** Modelado de topologías de transporte, redes de servicios públicos y conectividad ecológica. Se contrasta el modelo relacional espacial extendido para grafos (pgRouting sobre tablas de aristas y nodos en PostgreSQL) contra un motor de grafos nativo de propiedades (Neo4j con procedimientos espaciales Cypher).
- **Diseño experimental:** Construcción de una red vial de escala regional para la ejecución de algoritmos de caminos mínimos (Dijkstra, A*, Contraction Hierarchies) e isócronas bajo diferentes condiciones de concurrencia y pesos dinámicos.
- **Preguntas de investigación:**
 - ¿Cómo escala la traversabilidad en profundidad y el análisis de caminos mínimos en grafos nativos frente al uso de consultas relacionales recursivas e índices GiST?
 - ¿Qué limitaciones de integridad referencial y precisión métrica presentan los modelos de propiedades de grafos frente a los estándares OGC implementados en SGBD relacionales?

9.5 Tema 05: Indexación geoespacial en motores NoSQL para telemetría y flujos en tiempo real (Elasticsearch / OpenSearch vs. MongoDB)

- **Identificador de carpeta:** `proyectos_finales/tema_05_nosql_streaming_geo/`
- **Fundamento tecnológico:** Gestión de datos geoespaciales no estructurados y de alta velocidad de ingestión (IoT, sensores remotos, seguimiento de flotas). Estos motores emplean estructuras de

búsqueda especializadas como árboles BKD (*Block KD-Tree*) y *Geo-Hashes* optimizados para filtrado por proximidad (*k-NN*) y *geo-fencing*.

- **Diseño experimental:** Simulación de una carga de ingestión masiva en tiempo real con consultas simultáneas de contención (*geo_shape*) y proximidad (*geo_distance*). Comparación de latencia de escritura, uso de memoria e intervalos de refresco entre Elasticsearch y colecciones con índices *2dsphere* en MongoDB.
- **Preguntas de investigación:**
 - ¿Qué diferencias de rendimiento y consumo de recursos existen entre los índices BKD de Lucene y las mallas jerárquicas 2D de MongoDB ante operaciones concurrentes de lectura/escritura?
 - ¿Cómo se gestionan los cálculos geodésicos en geometrías que cruzan el antimeridiano o abarcan grandes extensiones territoriales en estos motores NoSQL?

9.6 Tema 06: Programación procedural espacial en servidor y extensiones nativas (PL/pgSQL avanzado, PL/Python y Rust/PGRX)

- **Identificador de carpeta:** `proyectos_finales/tema_06_procedimientos_plpgsql_rust/`
- **Fundamento tecnológico:** Ejecución de lógica analítica espacial avanzada directamente dentro del motor de base de datos para eliminar el cuello de botella de transferencia de datos (*serialization overhead* en formatos WKB/GeoJSON) entre el servidor y clientes analíticos externos como Python/GeoPandas.
- **Diseño experimental:** Implementación de un pipeline de limpieza topológica, simplificación y cálculo de métricas espaciales complejas mediante tres enfoques: 1) script cliente en GeoPandas, 2) función procedural interna en PL/pgSQL, y 3) extensión compilada en lenguaje de bajo nivel (Rust vía PGRX o C en PostgreSQL).
- **Preguntas de investigación:**
 - ¿Qué proporción del tiempo total de procesamiento en flujos espaciales complejos corresponde a la serialización/deserialización de geometrías entre cliente y servidor?
 - ¿Cuál es la ganancia en uso de CPU y vectorización al migrar algoritmos geométricos iterativos desde PL/pgSQL hacia extensiones compiladas nativas?

9.7 Tema 07: Ecosistemas analíticos desacoplados y flujos geoespaciales con GeoLibre

- **Identificador de carpeta:** `proyectos_finales/tema_07_geolibre_stack/`
- **Fundamento tecnológico:** Transición desde los sistemas de información geográfica monolíticos y los SGBD relacionales tradicionales hacia el *Modern Spatial Data Stack*. GeoLibre articula motores OLAP embebidos (DuckDB Spatial) con formatos *Cloud-Native Geospatial* (GeoParquet, PMTiles, FlatGeobuf) y entornos de ejecución en navegador/cliente, posibilitando pipelines de geoprocesamiento analítico y visualización interactiva sin desplegar infraestructura de servidor dedicada.
- **Diseño experimental:** Construcción y evaluación de un flujo analítico reproducible (extracción, transformación espacial, cálculo de métricas e indexación de teselas) comparando tres esquemas: 1) SGBD relacional clásico (PostgreSQL/PostGIS), 2) flujo cliente tradicional basado en scripts de

geoprocetamiento monolíticos, y 3) arquitectura desacoplada y ligera implementada sobre el ecosistema GeoLibre con DuckDB y almacenamiento en objetos.

- **Preguntas de investigación:**

- ¿En qué magnitud reduce la arquitectura desacoplada de GeoLibre los costos de infraestructura, la latencia de despliegue y el consumo de recursos frente a un SGBD relacional cliente-servidor?
- ¿Cuál es la eficiencia en la interoperabilidad entre formatos vectoriales columnares (GeoParquet) y formatos de teselas piramidales (PMTiles) al ejecutar consultas espaciales analíticas y renderizado dinámico en memoria?

Capítulo 10

Ficha de propuesta de tema de investigación

10.1 Información general

Tabla 10.1: Información general de la propuesta de investigación

Componente	Detalle
Tema principal	Arquitecturas de <i>Big Data</i> y procesadores distribuidos
Subtema	Escalabilidad y procesamiento masivo de datos espaciales
Título del proyecto	Evaluación del rendimiento y escalabilidad en el procesamiento vectorial masivo mediante Apache Sedona y PySpark
Nivel	Posgrado

10.2 Descripción del proyecto

El incremento en la frecuencia de adquisición y resolución de datos geoespaciales (telemetría IoT, registros de movilidad, fotogrametría y sensores remotos) ha expuesto las limitaciones de los sistemas de gestión de bases de datos relacionales (SGBD) tradicionales. Cuando los volúmenes de datos superan la memoria o la capacidad de cómputo de un solo nodo, las operaciones analíticas espaciales intensivas —como los *Spatial Joins*— experimentan cuellos de botella severos.

Este proyecto de investigación explora la transición desde el paradigma monolítico cliente-servidor hacia arquitecturas de cómputo distribuido en memoria mediante **Apache Sedona**. Se analizará cómo las especificaciones del *Open Geospatial Consortium* (OGC) y el estándar *Spatial SQL* se adaptan a entornos de procesamiento masivo, evaluando el impacto del particionamiento espacial, la indexación distribuida y los

formatos *Cloud-Native Geospatial*.

10.3 Objetivos de aprendizaje

Al finalizar el proyecto, el equipo de investigación estará en capacidad de:

1. **Evaluar** los límites de escalabilidad horizontal y vertical de los SGBD espaciales relacionales ante cargas de datos masivas.
2. **Analizar** la arquitectura interna de Apache Sedona, enfocándose en la distribución de estructuras de datos geométricos y ráster en clústeres computacionales.
3. **Formular** pipelines analíticos de *Spatial SQL* y PySpark optimizados para consultas espaciales a gran escala.
4. **Contrastar** la eficiencia de almacenamiento y E/S de los formatos tradicionales frente a formatos nativos de nube (*GeoParquet*, *GeoTIFF*, *Delta Lake*).

10.4 Preguntas de investigación orientadoras

Para estructurar el marco analítico, el proyecto de investigación debe dar respuesta a las siguientes interrogantes:

- ¿En qué umbral de volumen de datos o complejidad geométrica deja de ser eficiente un SGBD relacional tradicional frente a un procesador distribuido?
- ¿Cómo afecta la elección de la estrategia de particionamiento espacial (*Quad-Tree* frente a *KDB-Tree*) al fenómeno de *data shuffling* durante un *Spatial Join* en memoria distribuida?
- ¿Cuál es la ganancia en rendimiento de E/S y velocidad de consulta al migrar conjuntos de datos desde formatos tradicionales (*Shapefile* / *GeoJSON*) hacia *GeoParquet* en un entorno de lago de datos (*Data Lakehouse*)?

10.5 Desarrollo práctico y metodología del taller

El proyecto debe respaldarse con un diseño experimental replicable estructurado en tres fases:

1. **Fase 1: Configuración de la línea base y prueba de carga (SGBD monolítico):** Determinación del cuello de botella.
 - Seleccionar un conjunto de datos espaciales masivo (ej. puntos de telemetría GPS con más de cinco millones de registros y una capa poligonal de zonificación urbana o ambiental).
 - Configurar la base de datos relacional espacial en el contenedor del curso (PostgreSQL/PostGIS).
 - Ejecutar una consulta de cruce espacial (**ST_Intersects** o **ST_Contains**) documentando tiempos de ejecución, consumo de CPU, E/S de disco y uso de memoria RAM mediante **EXPLAIN ANALYZE**.
2. **Fase 2: Implementación distribuida en Apache Sedona:** Despliegue y ejecución en PySpark.
 - Desplegar el entorno de ejecución desde la terminal integrada de Visual Studio Code o Positron utilizando el contenedor Docker configurado con Apache Spark y Sedona.
 - Ingestar los datos en formato optimizado (*GeoParquet*).
 - Aplicar indexación espacial en memoria (*R-Tree*) y configurar el particionamiento de los DataFrames espaciales.

- Replicar el *Spatial Join* mediante PySpark y sentencias de *Spatial SQL*.

3. **Fase 3: Análisis comparativo y perfilamiento:** Evaluación de rendimiento.

- Comparar métricas de ejecución entre ambas arquitecturas (tiempo total, velocidad de lectura, consumo de recursos).
- Analizar el grafo acíclico dirigido (*DAG*) en la interfaz de Spark para evaluar la eficiencia del particionamiento espacial y la reducción de transferencias de datos entre ejecutores.

10.6 Entregables esperados

El equipo de investigación deberá estructurar y publicar en el repositorio central los siguientes artefactos:

- **02_proyecto_apache_sedona.qmd:** Reporte de investigación estructurado bajo formato científico con la justificación teórica, el diseño experimental, los gráficos comparativos de rendimiento y la discusión técnica.
- **02_rubrica_apache_sedona.qmd:** Matriz de evaluación del proyecto con criterios cuantitativos y cualitativos específicos.
- **02_slides_apache_sedona.qmd:** Presentación interactiva en Quarto RevealJS para la exposición magistral del Bloque 1.
- **02_taller_apache_sedona.qmd:** Guía ejecutable paso a paso del laboratorio guiado del Bloque 2, con celdas de código etiquetadas bajo la taxonomía estándar (**setup-**, **ddl-**, **dml-**, **query-**, **audit-**).
- **references.bib:** Archivo BibTeX con las fuentes bibliográficas y referencias citadas.

Capítulo 11

Rúbrica de evaluación para el reporte en QMD y el código

La evaluación del proyecto de investigación pondera cinco componentes fundamentales, integrando fundamentación teórica, rigor en el diseño experimental, reproducibilidad computacional y cumplimiento de directivas editoriales de Ciencia Abierta.

Tabla 11.1: Matriz de evaluación del proyecto de investigación y código fuente

Criterio	Peso	Sobresaliente (90-100%)	Competente (70-89%)	Insuficiente (<70%)
Rigor teórico y formulación metodológica	20%	Identifica y justifica con precisión el umbral de degradación del SGBD monolítico. Argumenta con solvencia técnica la elección de estructuras de datos, índices espaciales distribuidos (R-Tree / KDB-Tree) y el modelo de particionamiento.	Describe la problemática de escalabilidad y los índices espaciales, pero la justificación teórica o el análisis del particionamiento carecen de profundidad analítica.	No fundamenta teóricamente la transición hacia el procesamiento distribuido o presenta imprecisiones conceptuales severas en <i>Spatial SQL</i> y arquitecturas de <i>Big Data</i> .

Criterio	Peso	Sobresaliente (90-100%)	Competente (70-89%)	Insuficiente (<70%)
Implementación técnica en PySpark y Apache Sedona	25%	Código completamente funcional y modular en PySpark/Sedona. Aplica correctamente la conversión a GeoParquet, la indexación en memoria y la estrategia de particionamiento espacial sin errores de ejecución ni advertencias críticas.	Código funcional pero con optimizaciones limitadas en el particionamiento, uso ineficiente de estructuras distribuidas o justificación insuficiente en la selección del formato de almacenamiento.	Código incompleto, con errores de sintaxis/ejecución, o que no implementa la arquitectura distribuida requerida.
Perfilamiento de rendimiento y análisis comparativo	25%	Comparación empírica rigurosa entre PostgreSQL/PostGIS y Apache Sedona. Evalúa métricas cuantitativas de consumo de CPU, memoria RAM, E/S de disco y analiza el DAG de Spark para auditar el <i>data shuffling</i> con soporte gráfico claro.	Presenta mediciones de tiempo de ejecución pero omite el análisis detallado del plan de ejecución (DAG), la transferencia entre ejecutores o la auditoría de recursos de hardware.	Omite la batería de pruebas comparativas o la evaluación empírica carece de mediciones reproducibles y métricas observables.

Criterio	Peso	Sobresaliente (90-100%)	Competente (70-89%)	Insuficiente (<70%)
Reproducibilidad y entorno de ejecución	15%	Repositorio estructurado bajo control de versiones con Git/GitHub. Ejecución verificada en el contenedor Docker del curso desde la terminal integrada de Visual Studio Code o Positron, con dependencias fijadas y enlaces funcionales a datos abiertos.	Repositorio funcional pero con documentación incompleta en las instrucciones de despliegue o rutas de datos relativas no estandarizadas que dificultan la replicación inmediata.	Repositorio desorganizado, ausencia de control de versiones o incompatibilidad del flujo de trabajo con el entorno de ejecución provisto.
Estándares Quarto, sintaxis YAML y calidad editorial	15%	Documento .qmd estructurado impecablemente con sintaxis Quarto nativa. Bloques de código identificados con directivas YAML (<code># label: ...</code>) bajo taxonomía estándar (<code>setup-</code> , <code>ddl-</code> , <code>dml-</code> , <code>query-</code> , <code>audit-</code>), sin sintaxis heredada de Knitr. Uso correcto de referencias cruzadas (<code>@tbl-...</code> , <code>@fig-...</code>), ecuaciones en LaTeX, bibliografía BibTeX y redacción técnica impersonal en <i>Sentence case</i> .	Documento .qmd funcional pero con omisiones menores en la taxonomía de etiquetas, referencias bibliográficas incompletas o inconsistencias leves en la aplicación de <i>Sentence case</i> .	No aprovecha las capacidades analíticas de Quarto (texto plano sin formato estructurado), utiliza sintaxis obsoleta de Knitr, carece de citación formal en BibTeX o presenta deficiencias severas de redacción técnica.

Apéndice A

Quarto: orquestación y configuración

Quarto se basa en un archivo de configuración centralizado donde se definen los parámetros de renderizado, bibliografía y estilos.

A.1 Opciones del encabezado YAML

Ejemplo 1: Solo HTML

```
## label: setup-yaml-html
## eval: false
---
title: "Título de la Práctica"
author: "Nombre del Estudiante"
date: last-modified
format:
  html:
    toc: true
    toc-depth: 3
    toc-location: left
    number-sections: true
    theme: lux
---
```

Ejemplo 2: HTML y PDF

```
## label: setup-yaml-html-pdf
## eval: false
---
title: "Informe Técnico: Análisis Geoespacial"
author: "ID Correo UNAL"
date: today
format:
  html:
    toc: true
    number-sections: true
    theme: cosmo
  pdf:
    toc: true
    toc-title: "Contenido"
    number-sections: true
    documentclass: scrreprt
    geometry:
      - top=25mm
      - left=25mm
      - right=25mm
      - bottom=25mm
---
```

A.2 Resumen de comandos Quarto

Renderizar todos los formatos definidos en el encabezado `yaml`:

```
#| label: setup-render-all
#| eval: false
quarto render archivo.qmd --to all
```

Renderizar a HTML:

```
#| label: setup-render-html
#| eval: false
quarto render archivo.qmd --to html
```

Renderizar a PDF:

```
#| label: setup-render-pdf
#| eval: false
quarto render archivo.qmd --to pdf
```

Renderizar a PDF, mostrando todos los detalles en la consola:

```
#| label: setup-render-pdf-debug
#| eval: false
quarto render archivo.qmd --to pdf --trace --verbose
QUARTO_LOG_LEVEL=debug quarto render archivo.qmd --to pdf
```

Chequear el estado de la instalación de Quarto:

```
#| label: setup-quarto-check
#| eval: false
quarto check
```

A.3 Gestión eficiente de renderizado en Quarto

A continuación, se presenta la hoja de ruta para optimizar los tiempos de compilación del documento principal, especialmente cuando se integra procesamiento intensivo con motores de Julia y R.

A.3.1 Hoja de referencia: comandos de renderizado

La ejecución de estos comandos depende de la existencia del archivo `_quarto.yaml` en el directorio de trabajo. Si se requiere trabajar con archivos individuales, es posible alterar temporalmente el nombre de este archivo a `_quarto.yaml.back`.

Tabla A.1: Comandos esenciales para la gestión de flujos en Quarto

Acción	Comando
Trabajar en vivo (Previsualización dinámica)	<code>quarto preview</code>
Renderizar solo el capítulo actual	<code>quarto render archivo.qmd</code>
Generar el libro completo (Ensamblaje final)	<code>quarto render</code>
Forzar ejecución total (Ignorar el <code>freeze</code>)	<code>quarto render --execute</code>

A.3.2 Estrategias adicionales para el flujo de edición

Para evitar esperas innecesarias y optimizar la eficiencia, se recomienda aplicar las siguientes estrategias de previsualización y almacenamiento de resultados computacionales:

A.3.2.1 1. Flujo de edición en vivo (`quarto preview`)

En lugar de renderizar el documento manualmente tras cada modificación, se recomienda inicializar una sesión de previsualización desde la terminal integrada en la raíz del proyecto.

Al ejecutar este proceso, se levanta un servidor local. Cada vez que se registre un cambio en el archivo `.qmd` y se guarde el progreso, Quarto detectará la modificación y refrescará exclusivamente el capítulo activo en el navegador. Esta metodología mantiene las sesiones de procesamiento en memoria, permitiendo visualizar modificaciones en mapas y ecuaciones de forma inmediata.

A.3.2.2 2. Uso de caché para procesos analíticos pesados

Si un bloque de código ejecuta tareas de alto coste computacional (conexiones a bases de datos espaciales o análisis ráster masivo), es imperativo prevenir su re-ejecución automática. Para ello, se emplea la opción de caché a nivel de celda:

```
#| label: setup-analisis-costoso
#| eval: false
#| cache: true
#| results: asis

# Este código se ejecuta únicamente si se modifica el contenido interno del bloque.
# Quarto guarda los resultados en el directorio '_cache' para su reutilización.
j_eval(" ... proceso de Julia que tarda varios minutos ... ")
```

Nota técnica: Mientras que el parámetro global `freeze: auto` (definido en `_quarto.yml`) protege los capítulos finalizados para evitar su re-ejecución durante la compilación general, la directiva `#| cache: true` blindará el flujo de trabajo individual dentro del capítulo en edición activa.

A.4 Control de ejecución en Quarto: opciones de bloques

Para asegurar la formalidad, reproducibilidad y claridad de los reportes técnicos, es indispensable controlar con precisión la visibilidad del código fuente, la captura de resultados y la gestión de excepciones durante el renderizado. Quarto administra este comportamiento mediante directivas declaradas al inicio de cada bloque mediante sintaxis YAML (`#|` en Bash, R y Python, o `--|` en SQL, ver Tabla A.3).

Tabla A.2: Matriz de opciones de visibilidad, ejecución y captura de errores para bloques de código

Opción de bloque	¿Se ejecuta?	¿Muestra código?	¿Muestra salidas/errores?
<code># echo: false</code>	✓ SÍ	✗ NO	✓ SÍ
<code># include: false</code>	✓ SÍ	✗ NO	✗ NO
<code># code-fold: true</code>	✓ SÍ	Plegado	✓ SÍ
<code># eval: false</code>	✗ NO	✓ SÍ	✗ NO
<code># output: false</code>	✓ SÍ	✓ SÍ	✗ NO
<code># warning: false</code>	✓ SÍ	—	✗ NO
<code># message: false</code>	✓ SÍ	—	✗ NO
<code># error: true</code>	✓ SÍ	✓ SÍ	Captura error

A.4.1 Detalle funcional y aplicación técnica de las directivas

- **#| echo (Visibilidad del código fuente):** Controla si el código fuente del bloque se imprime en el documento final. Con `echo: false`, el bloque se evalúa y computa con normalidad, pero la sintaxis queda oculta al lector, mostrando exclusivamente las tablas, figuras o salidas generadas. Resulta ideal en secciones orientadas a resultados ejecutivos.
- **#| include (Inclusión integral del bloque):** Representa un modificador global que suprime tanto el código como sus salidas. Con `include: false`, el código se ejecuta en el proceso subyacente (registrando variables, cargando paquetes o modificando el entorno), pero no genera ningún rastro visible en el documento compilado. Se emplea típicamente en bloques de inicialización (`setup`).
- **#| code-fold (Plegado interactivo de código):** Habilita un mecanismo interactivo de colapso disponible en salidas HTML. El bloque oculta el código bajo una pestaña desplegable (*widget*), permitiendo al usuario inspeccionar la sintaxis bajo demanda sin saturar visualmente el texto principal.
- **#| eval (Evaluación de código):** Define si el motor de ejecución (Knitr, Jupyter o subshells de terminal) debe procesar el contenido del bloque. Con `eval: false`, el código se renderiza únicamente como texto formateado con resaltado de sintaxis, sin enviarse al intérprete. Es mandatorio para documentar comandos interactivos de terminal o metacomandos de cliente (como `\c` o `\q` en `psql`).
- **#| output (Control de salidas estándar):** Determina la inclusión del flujo de salida estándar (`stdout`). Al configurarse como `output: false`, el código se muestra y se ejecuta, pero se descartan las salidas textuales generadas por el intérprete (por ejemplo, salidas intermedias de sentencias SQL o confirmaciones de asignación de variables).
- **#| warning (Gestión de advertencias):** Regula la visualización de mensajes de advertencia (*warnings*) emitidos por librerías, controladores o funciones de cálculo espacial. Con `warning: false`, las alertas no detienen la ejecución ni se imprimen en el documento final, evitando ruidos visuales por funciones en proceso de depreciación.

- **#| message (Gestión de mensajes informativos):** Controla los flujos de mensajes complementarios emitidos durante la carga de módulos, paquetes o diagnósticos de conexión. Al fijarse en `message: false`, estos textos informativos se descartan del documento final.
- **#| error (Captura de excepciones y códigos de retorno):** Define el comportamiento del motor ante excepciones no controladas o códigos de salida distintos de cero (`status != 0`) en comandos de terminal. Por defecto (`error: false`), cualquier fallo aborta inmediatamente la compilación (`Execution halted`). Al declarar `error: true`, Quarto intercepta el error, lo formatea dentro del documento como una alerta técnica y permite continuar el renderizado de los capítulos posteriores sin interrumpir el flujo del libro.

A.4.2 Sintaxis de directivas según el motor de ejecución

La siguiente tabla resume los delimitadores requeridos para declarar metadatos y opciones de ejecución en los principales motores soportados por Quarto:

Tabla A.3: Sintaxis de directivas según el motor de ejecución

Motor / Lenguaje	Motor de bloque	Delimitador YAML	Ejemplo de declaración
Bash / Shell	<code>{bash}</code>	<code># </code>	<code># echo: false</code>
Python	<code>{python}</code>	<code># </code>	<code># echo: false</code>
R	<code>{r}</code>	<code># </code>	<code># echo: false</code>
Julia	<code>{julia}</code>	<code># </code>	<code># echo: false</code>
SQL	<code>{sql}</code>	<code>-- </code>	<code>-- connection: db</code>
Graphviz (DOT)	<code>{dot}</code>	<code>// </code>	<code>// fig-cap: "..."</code>
Mermaid	<code>{mermaid}</code>	<code>% </code>	<code>% fig-cap: "..."</code>
C / C++	<code>{cpp} / {c}</code>	<code>// </code>	<code>// eval: false</code>
Observable JS (D3)	<code>{ojs}</code>	<code>// </code>	<code>// echo: false</code>

A.4.3 Recomendaciones de uso según el contexto

Dependiendo de la sección del script activo, las siguientes configuraciones garantizan un documento estructurado:

Tabla A.4: Estrategias recomendadas de etiquetado por etapa analítica

Tipo de bloque	Opción recomendada	Propósito técnico
Configuración inicial	<code># include: false</code>	Ejecuta la carga de librerías y parámetros globales sin mostrar código ni resultados, evitando ruido visual al inicio del documento.
Limpieza de memoria	<code># include: false</code>	Ejecuta operaciones de liberación de recursos (p. ej. <code>rm()</code> , <code>gc()</code>) de forma silenciosa, ya que no aportan valor interpretativo al lector final.

Tipo de bloque	Opción recomendada	Propósito técnico
Procesamiento espacial	<code># code-fold: true</code>	Mantiene visible el código (relevante para la reproducibilidad metodológica) pero colapsado por defecto, permitiendo al lector expandirlo solo si necesita revisar la lógica de procesamiento.
Resultados analíticos	<code># echo: false</code>	Oculto el código fuente y muestra únicamente la salida (tablas, gráficos, estadísticas), priorizando la lectura de resultados sobre la implementación.

💡 Sintaxis correcta de opciones

Las opciones deben declararse obligatoriamente al inicio del bloque, precediendo cualquier otra instrucción del lenguaje:

```
#| label: setup-ejemplo-opciones
#| eval: false
#| echo: false
#| warning: false
library(terra)
# Código analítico...
```

A.5 Instalación de extensiones de Quarto

Las extensiones amplían las funcionalidades nativas de Quarto, integrando nuevos formatos de salida, filtros o componentes visuales. Para asegurar la descarga desde los repositorios remotos, es imperativo contar con conexión y privilegios de escritura en el directorio de trabajo.

A.5.1 Proceso de instalación

Quarto utiliza el subcomando `add` seguido de la ruta del repositorio. Esta directiva descarga los recursos en el directorio local `_extensions/`. Para instalar soporte de iconos vectoriales, se debe ejecutar la siguiente orden en la terminal integrada:

```
#| label: setup-instalar-extension
#| eval: false
quarto add quarto-ext/fontawesome
```

i Confirmación de seguridad

Durante la ejecución, la consola solicitará confirmación de confianza hacia los desarrolladores de la extensión. Es necesario introducir `Y` para autorizar la modificación del directorio.

A.5.2 Gestión de archivos del proyecto

Una vez confirmada la instalación, el árbol de directorios reflejará la siguiente estructura:

```
#| label: setup-arbol-directorios
#| eval: false
directorio-proyecto/
  _extensions/
    quarto-ext/
      fontawesome/
documento-principal.qmd
```

El contenido de la carpeta `_extensions` **no debe modificarse manualmente**, dado que el compilador de Quarto lo requiere estrictamente para inyectar las dependencias durante el renderizado.

A.5.3 Aplicación en el documento

Tras la integración exitosa, los componentes se invocan mediante sintaxis de código corto (*shortcodes*):

```
#| label: setup-ejemplo-iconos
#| eval: false
Para el análisis espacial se requerirán los siguientes insumos:

* {{< fa satellite >}} Datos de teledetección.
* {{< fa location-dot "red" >}} Puntos de control terrestre.
```

Actualización de dependencias

Para actualizar una extensión existente a su última compilación, se debe reejecutar el comando de instalación. El motor detectará el conflicto de versiones y solicitará autorización para sobrescribir los archivos locales.

A.6 Compilación de la guía del curso

El material general del curso se orquesta a través del archivo `_quarto.yml`, el cual establece las referencias explícitas hacia todos los documentos (`.qmd`) de capítulos, talleres y anexos.

A.6.1 Compilación del proyecto completo

Para compilar la totalidad del libro y generar todos los formatos de salida definidos, se debe ejecutar el comando base en el directorio que contiene el orquestador YAML:

```
#| label: setup-render-proyecto
#| eval: false
quarto render
```

A.6.2 Compilación de archivos individuales

Para procesar un único documento aislándolo de las políticas de numeración e indexación del proyecto global, se debe aplicar el siguiente protocolo:

1. Modificar temporalmente el nombre del orquestador de `_quarto.yml` a `_quarto.yml.back`.
2. Lanzar la instrucción de compilación apuntando al recurso específico:

- Generación concurrente (HTML y PDF):

```
#| label: setup-render-ind-all
#| eval: false
quarto render archivo.qmd --to all
```

- Generación de HTML:

```
#| label: setup-render-ind-html
#| eval: false
quarto render archivo.qmd --to html
```

- Generación de PDF:

```
#| label: setup-render-ind-pdf
#| eval: false
quarto render archivo.qmd --to pdf
```

A.6.3 Notas de estudio y personalización

Personalización del material

Quarto permite la inserción de bloques de notas personalizados mediante la sintaxis de llamadas (*callouts*). La estructura general para integrar bloques de observación es la siguiente:

```
#| label: setup-ejemplo-callout
#| eval: false
::: {.callout-tip icon="true" title="Bloque de notas"}
Espacio reservado para observaciones, recordatorios y apuntes técnicos.
:::
```

Apéndice B

Docker: gestión de contenedores e imágenes

```
# =====  
# 1. Directorio de ploteos  
# =====  
# Resolución condicional: valida si la sesión se ubica en la raíz o en chapters/  
if [ -d "plots" ]; then  
    export PLOTS_DIR="plots"  
else  
    export BACKUP_DIR="../plots"  
fi  
  
# =====  
# 2. Verificación de variables asignadas  
# =====  
echo "Directorio de ploteos: ${PLOTS_DIR}"
```

Directorio de ploteos: ../plots

El sistema Docker opera como un motor de virtualización (Inc., 2025) sobre el cual se desplegarán las máquinas virtuales basadas en Linux. Estas infraestructuras permiten la ejecución integrada de código analítico en Python, R y Julia, junto al soporte persistente de bases de datos geográficas.

B.1 Instalación del software

Descargue la versión oficial del orquestador desde el repositorio principal:

- **URL oficial:** <https://www.docker.com/products/docker-desktop/>
- Descargue el paquete instalador correspondiente a la arquitectura de hardware del equipo local.

B.1.1 Sistema anfitrión (Windows)

Para garantizar un rendimiento óptimo en Windows, el motor debe ejecutarse sobre la arquitectura nativa de virtualización de WSL 2 en lugar del hipervisor emulado Hyper-V.

1. Abra una terminal de PowerShell con privilegios de administrador y actualice los componentes del

subsistema:

```
#| label: setup-wsl-update
#| eval: false
wsl --update
```

2. Ejecute el instalador descargado. Durante el asistente de configuración:

- Asegúrese de marcar la casilla “Use WSL 2 instead of Hyper-V”.
- Si el asistente solicita reiniciar el sistema al finalizar la descompresión de binarios, reinicie el equipo para inicializar el controlador de virtualización.

3. **Instalación en una unidad secundaria (opcional):** Si la unidad principal del sistema no dispone de espacio suficiente para la instalación de los binarios base, es posible forzar una ruta de instalación personalizada desde PowerShell como administrador:

```
#| label: setup-install-docker-dir
#| eval: false
Start-Process "Docker Desktop Installer.exe" -ArgumentList "install",
↳ "--installation-dir=<unidad>:\Docker" -Wait
```

B.1.2 Procedimiento general (agnóstico)

En sistemas operativos tipo POSIX o arquitecturas alternativas:

1. Instale el paquete siguiendo los estándares del sistema anfitrión (gestores de paquetes **apt**, **brew** o asistentes nativos).
2. Asegúrese de que el demonio de Docker (**dockerd**) cuente con permisos de ejecución y esté habilitado para iniciar con el sistema.
3. Integre el usuario local dentro del grupo de seguridad **docker** para permitir la interacción sin necesidad de elevar privilegios (**sudo**).

B.2 Preparación del orquestador

Previo a la inicialización de las imágenes del curso, es indispensable dimensionar los recursos de almacenamiento y memoria del equipo local. Las imágenes políglotas y los motores espaciales del entorno consolidan un volumen de datos considerable:

- Se requiere un mínimo de **50 GB de espacio libre** en la unidad asignada a los discos virtuales.
- El despliegue integral de las dos imágenes base del curso genera un consumo acumulado de aproximadamente **45 GB**.

B.2.1 Reasignación del almacenamiento de imágenes virtuales

Si la unidad de almacenamiento principal carece del espacio libre requerido, se debe reubicar el catálogo donde el orquestador almacena los discos virtuales (archivos con extensión **.vhd**).

B.2.1.1 Sistema anfitrión (Windows)

Para mover la ubicación física del disco virtual de Docker Desktop a una unidad secundaria:

1. Abra la interfaz de **Docker Desktop**.
2. Haga clic en el ícono de engranaje en la barra superior para acceder a **Settings**.
3. En el panel lateral izquierdo, diríjase a **Resources** y seleccione la sección **Advanced**.
4. Localice el parámetro **Disk image location**.
5. Haga clic en el botón **Browse** y seleccione una carpeta en una unidad con suficiente capacidad de almacenamiento.
6. Haga clic en **Apply & restart** para que el servicio migre los datos y reinicie el motor.

B.2.1.2 Procedimiento general (agnóstico)

En configuraciones donde no se emplea Docker Desktop o se interactúa directamente con el demonio de Docker en el host:

1. Detenga el servicio **docker**.
2. Modifique el archivo `/etc/docker/daemon.json` agregando la directiva con la nueva ruta:

```
#| label: setup-docker-daemon-dir
#| eval: false
{
  "data-root": "<ruta_al_disco_secundario>/docker-data"
}
```

3. Mueva los archivos existentes e inicialice nuevamente el servicio.

B.2.2 Optimización de la memoria de intercambio (swap)

Para procesar conjuntos de datos ráster y vectoriales masivos sin interrupciones en R, Python o Julia, es imperativo configurar una partición de intercambio de alta capacidad. Esto evita la terminación forzada de contenedores por saturación de memoria RAM (*Out of Memory Killer*). Puede consultar más detalles técnicos en la Sección [B.13](#).

B.2.2.1 Sistema anfitrión (Windows)

El ajuste del espacio de intercambio en Windows requiere configurar la memoria virtual del sistema operativo y los parámetros del subsistema WSL 2.

Paso 1: Ajustar la memoria virtual en el sistema anfitrión

1. En la barra de búsqueda de Windows, escriba y seleccione “**Ver la configuración avanzada del sistema**”.
2. En la pestaña **Opciones avanzadas**, ubique la sección **Rendimiento** y haga clic en el botón **Configuración**.
3. Diríjase a la pestaña **Opciones avanzadas**, vaya al bloque **Memoria virtual** y presione el botón **Cambiar**.
4. Desmarque la casilla “Administrar automáticamente el tamaño del archivo de paginación para todas las unidades”.

5. Seleccione la unidad de almacenamiento principal o más veloz (SSD/NVMe), elija la opción **Tamaño personalizado** y defina los siguientes valores sugeridos (sujetos a la capacidad real del disco):

- **Tamaño inicial:** 16384 MB (16 GB).
- **Tamaño máximo:** 32768 MB (32 GB).

6. Presione el botón **Establecer**, luego **Aceptar** y reinicie el equipo.

Paso 2: Configurar los recursos globales de WSL 2 (.wslconfig)

Docker Desktop delega la gestión de RAM y swap en el archivo de configuración global del hipervisor:

1. Abra el explorador de archivos, ingrese %USERPROFILE% en la barra de direcciones y presione **Enter** para situarse en la raíz del usuario local.
2. Cree un archivo de texto con el nombre exacto `.wslconfig` (verifique que no conserve la extensión oculta `.txt`).
3. Incorpore la siguiente configuración para forzar el límite de RAM y dirigir el archivo de intercambio hacia un disco de alta velocidad:

```
#| label: setup-wslconfig
#| eval: false
[ws12]
memory=12GB # RAM física asignada al subsistema de virtualización
swap=32GB   # Memoria de intercambio para contingencia analítica
swapfile=<unidad>:\\<ruta_al_directorio_con_espacio>\\swap.vhdx
```

4. Reinicie el subsistema desde la terminal de comandos ejecutando `wsl --shutdown`.

B.2.2.2 Procedimiento general (agnóstico)

En entornos basados directamente en Linux, el espacio de intercambio se dimensiona en el host mediante la creación de un archivo de paginación estructurado:

```
#| label: setup-create-swapfile
#| eval: false
# 1. Crear un archivo reservado de 32 GB
sudo fallocate -l 32G /swapfile

# 2. Asignar permisos estrictos de lectura y escritura
sudo chmod 600 /swapfile

# 3. Formatear y activar el espacio de intercambio
sudo mkswap /swapfile
sudo swapon /swapfile
```

Auditoría de recursos asignados

Para comprobar si la memoria RAM y el espacio de intercambio se encuentran disponibles dentro del contenedor o en la terminal integrada de Visual Studio Code, ejecute:

```
#| label: audit-memory-resources
#| eval: false
free -h
```

B.3 Descarga y preparación de archivos

Ejecute los siguientes pasos para consolidar el entorno base:

1. **Obtención:** Descargue el archivo de recursos

- Archivo [j_eval_j_plot.r](#)
- Archivo [sig_unal_v1.12_completo_final.zip](#)
 - [sig_unal_v1.12_completo_final.tar](#)
 - [docker-compose.yml](#)
 - [init_unal_spatialdb.sql](#)
 - [Dockerfile.postgis](#)
- Archivo [backups_pg18.zip](#)

desde el siguiente link en Google Drive. 2. **Descompresión:** Extraiga el contenido del archivo comprimido [sig_unal_v1.12_completo_final.zip](#). 3. **Estructura del directorio:**

- [sig_unal_v1.12_completo_final.tar](#): Imagen consolidada del servidor espacial y del entorno analítico.
- [docker-compose.yml](#): Archivo de orquestación de puertos, redes y volúmenes persistentes.
- [init_unal_spatialdb.sql](#): Esquema base de datos espacial en texto plano.
- [j_eval_j_plot.r](#): Funciones de intercomunicación entre R y Julia.

Asegure que el directorio de trabajo local cumpla las siguientes especificaciones:

- Exclusión estricta de directorios sincronizados en la nube.
- Rutas absolutas sin espacios en blanco, tildes ni caracteres especiales.
- Omisión de carpetas temporales del sistema anfitrión.

B.4 Carga de contenedores orquestados

Este procedimiento ejecuta el aprovisionamiento local de dos imágenes estructuradas y configura la red interna para garantizar la comunicación bidireccional. Desde la terminal integrada ubicada en la raíz del proyecto, ejecute:

```
#| label: docker-carga-imagenes
#| eval: false
docker load -i sig_unal_v1.12_completo_final.tar
```

Nota: El intérprete de comandos pausará la respuesta hasta finalizar la descompresión binaria. Este proceso demanda tiempo y no debe interrumpirse bajo ninguna circunstancia.

B.5 Contenedores instalados

Finalizada la inyección de imágenes, ejecute el siguiente comando para auditar la disponibilidad de recursos:

```
#| label: docker-verificacion-ps
#| eval: false
docker ps -a
```

1. Contenedor `contenedor_sig_unal` basado en la imagen `image_sig_unal:v1.11_produccion_final`, equipado con:
 - **Python, R y Julia** y librerías geoespaciales.
 - **Quarto** (Ver Apéndice A).
2. Contenedor `contenedor_postgis_unal` basado en la imagen `postgis_unal:v1.12`, equipado con:
 - **PostgreSQL y PostGIS**, integrando el catálogo de estudio `sig_db_unal`.

B.6 Puesta en marcha y detención de contenedores

B.6.1 Inicio y reanudación de servicios

Para la provisión inicial del entorno, creación de redes virtuales y construcción de los contenedores a partir de la configuración declarativa:

```
#| label: docker-up-servicios
#| eval: false
docker compose up -d
```

Este comando valida la configuración, crea los contenedores definidos si no existen y levanta los servicios en segundo plano.

Si los contenedores ya fueron creados previamente y únicamente se encuentran en estado suspendido, se reactivan sus procesos sin recrear las redes ni alterar su capa mutable mediante:

```
#| label: docker-start-servicios
#| eval: false
docker compose start
```

Una vez reactivada la instancia, abra Visual Studio Code en el sistema anfitrión, presione **Ctrl + Shift + P**, seleccione **Dev Containers: Attach to Running Container...** y elija `/contenedor_sig_unal` para retomar el espacio de trabajo con todas sus extensiones previas intactas.

B.6.2 Detención y desmontaje de servicios

Al finalizar la sesión ordinaria, se debe suspender la ejecución de los servicios enviando una señal de apagado controlado (**SIGTERM**):

```
#| label: docker-stop-servicios
#| eval: false
docker compose stop
```

Esta operación detiene los procesos del motor relacional preservando intacta la capa de lectura y escritura del contenedor, lo cual asegura que las extensiones instaladas y la configuración del entorno **persistan** para la siguiente sesión.

Por el contrario, si se requiere reiniciar la infraestructura a su estado base o liberar los recursos de red del sistema anfitrión, se procede con el desmontaje:

```
#| label: docker-down-servicios
#| eval: false
docker compose down
```

Esta instrucción detiene los servicios y destruye los contenedores y las redes virtuales asociadas. Aunque los datos espaciales almacenados en los volúmenes físicos de PostgreSQL permanecen intactos, la capa de escritura del contenedor **se elimina**, lo que requerirá reinstalar extensiones adicionales de Visual Studio Code al momento de volver a ejecutar `docker compose up -d`.

B.7 Localización de archivos y persistencia

El directorio de trabajo local queda montado bidireccionalmente en la ruta `/home/rstudio/work` dentro de la imagen.

! Persistencia estructural

La topología garantiza que la caída del sistema anfitrión no comprometa las tablas almacenadas en el catálogo PostGIS ni los scripts depositados en `/home/rstudio/work`. El volumen configurado actúa como punto de montaje asíncrono.

B.8 Vinculación de contenedores en la terminal integrada

Para consolidar el entorno de desarrollo, acople el editor de texto (VSCode) directamente al núcleo del contenedor:

1. Levante los servicios mediante `docker compose up -d`.
2. Habilite la extensión *Dev Containers* en el equipo local.
3. Desde la paleta de comandos `Ctrl + Shift + P`, ejecute **Dev Containers: Attach to Running Container...** seleccionando el objetivo `contenedor_sig_unal`.
4. Verifique el directorio de montaje `/home/rstudio/work`.
5. Verifique las extensiones descritas en la Sección [D.6](#), o directamente en la Tabla [D.3](#).

B.9 Configuración de puertos y topología de conectividad

El diseño de red de la infraestructura basada en contenedores implementa un esquema de reenvío de puertos (*port forwarding*) estructurado para mitigar colisiones con servicios preexistentes en el equipo local y garantizar el acceso simultáneo a los entornos analíticos y al motor relacional espacial.

Tabla B.1: Mapeo de puertos para los servicios de la infraestructura dockerizada

Servicio	Puerto anfitrión	Puerto contenedor	Propósito técnico	Dirección de enlace
Jupyter Lab	8889	8888	Computación interactiva y notebooks	<code>http://127.0.0.1:8889</code>
Visor R (httpgd)	8788	8787	Renderizado gráfico SVG interactivo	<code>http://127.0.0.1:8788</code>
PostGIS (DB)	5434	5432	Motor de base de datos espacial relacional	<code>127.0.0.1:5434</code>

B.9.1 Coexistencia de clústeres y resolución de red según el contexto de ejecución

La resolución de nombres de red y los puertos de conexión varían sustancialmente según la ubicación del proceso emisor:

- **Peticiones originadas desde el sistema anfitrión (*host*):**
 - **Motor PostgreSQL físico local:** Se canaliza a través de la dirección de bucle local `localhost` (`127.0.0.1`) en el puerto TCP estándar **5432**.
 - **Motor espacial en el contenedor (*db-postgis*):** Se intercepta mediante la interfaz expuesta por Docker en `127.0.0.1` sobre el puerto **5434** (`5434:5432`), evitando interferencias con el servicio local predeterminado.
- **Peticiones originadas desde la terminal integrada en el contenedor:**
 - **Motor PostgreSQL físico local:** Al encontrarse aislado de la interfaz de red del anfitrión, la dirección `127.0.0.1` apunta al propio contenedor. La conexión hacia el motor físico del host requiere resolver el alias DNS especial `host.docker.internal` en el puerto nativo **5432**.
 - **Motor espacial en el contenedor (*db-postgis*):** La comunicación entre contenedores adyacentes vinculados a la misma red virtual (*bridge*) se efectúa mediante el nombre del servicio `db-postgis` en su puerto nativo **5432** (el puerto **5434** no existe en el espacio de nombres interno de Docker).

Tabla B.2: Matriz de enrutamiento y puertos según el vector de ejecución

Vector de conexión	Destino del servicio	Host / Interfaz	Puerto	Función técnica
Desde el sistema anfitrión	Instancia física local	<code>localhost</code>	5432	Puerto estándar del servicio local
Desde el sistema anfitrión	Clúster en contenedor	<code>127.0.0.1</code>	5434	Puerto externo expuesto por Docker

Vector de conexión	Destino del servicio	Host / Interfaz	Puerto	Función técnica
Desde la terminal del contenedor	Instancia física local	<code>host.docker.internal</code>	5432	Puente hacia el PostgreSQL del anfitrión
Desde la terminal del contenedor	Clúster en contenedor	<code>db-postgis</code>	5432	Resolución DNS interna del clúster

B.10 Credenciales de la base de datos

Parámetros de conexión estándar para utilidades cliente (QGIS, utilidades CLI, Python, R, Julia):

- **Catálogo:** `sig_db_unal`
- **Superusuario:** `profe_unal`
- **Autenticación:** `geomatica2025`
- **Enlace externo:** `127.0.0.1` (Puerto 5434)
- **Enlace interno:** `db-postgis` (Puerto 5432)

B.11 Lógica de arquitectura (Dockerfile y docker-compose)

La configuración se encuentra segmentada para garantizar aislamiento de memoria:

1. **Dockerfile:** Implementa directivas `EXPOSE 8888` y `EXPOSE 8787`, declarando canales de escucha independientes para evitar colisiones entre el servidor HTTP de R y el núcleo Jupyter.
2. **Orquestador YAML:** Ejecuta el mapeo `5434:5432` previniendo conflictos en el equipo local si existe un demonio PostgreSQL ejecutándose en el canal 5432.

B.12 Guía de acceso: topología de red

La comunicación difiere dependiendo del vector de entrada:

B.12.1 Conexión desde el sistema anfitrión (Exterior)

Las conexiones emitidas desde clientes locales deben apuntar a los puertos de exposición definidos en el enrutador virtual: Jupyter (8889), Base de Datos (5434).

B.12.2 Conexión intraclúster (Interior)

Los algoritmos ejecutados desde los cuadernos de trabajo operan bajo la red puente nativa. Por ende, deben apuntar directamente al host `db-postgis` en el puerto 5432.

B.13 Limpieza del entorno de contenedores

El ciclo de vida del desarrollo geoespacial genera capas de construcción, volúmenes de datos transitorios e imágenes intermedias que pueden comprometer el almacenamiento del equipo local o inducir conflictos durante la auto-inicialización declarativa de bases de datos.

A continuación se formaliza el protocolo sistemático para el diagnóstico, desmantelamiento ordenado, purga exhaustiva y auditoría de almacenamiento del subsistema de contenedores desde la terminal integrada de Visual Studio Code.

B.13.1 Diagnóstico de consumo de almacenamiento

Previo a cualquier operación de limpieza, se debe inspeccionar la volumetría física asignada a contenedores, imágenes, volúmenes locales y caché de compilación (*build cache*):

```
# Resumen global de espacio utilizado por subsistema
docker system df

# Desglose detallado por objeto, identificador y estado de referencia
docker system df -v
```

B.13.2 Desmantelamiento de servicios y volúmenes vinculados

Cuando un proyecto opera mediante Docker Compose, el desmantelamiento debe garantizar la remoción coordinada de contenedores, redes virtuales en puente y volúmenes de datos persistentes (-v):

```
# Detención y remoción de servicios, redes del proyecto y volúmenes nombrados
docker compose down -v --remove-orphans
```

Nota técnica: La bandera -v asegura la eliminación de volúmenes persistentes (tales como `postgis_data_unal`), permitiendo que despliegues posteriores ejecuten scripts de inicialización DDL/DML desde un estado completamente limpio.

B.13.3 Purga integral de artefactos y caché de compilación

Advertencia: Las siguientes instrucciones eliminan de forma irreversible contenedores detenidos, redes virtuales huérfanas, volúmenes sin asociar, imágenes sin etiquetar y cachés de construcción.

Para liberar el almacenamiento físico y purgar el subsistema BuildKit:

```
# 1. Purga exhaustiva de contenedores inactivos, redes, volúmenes huérfanos e imágenes no referenciadas
docker system prune -a --volumes -f

# 2. Eliminación de la memoria caché de construcción de BuildKit
docker builder prune -a -f
```

En escenarios administrativos donde se requiera forzar la remoción absoluta de la totalidad de objetos presentes en el subsistema (incluyendo imágenes base activas), se pueden encadenar las utilidades de listado y remoción forzada:

```
# Detención forzada y eliminación de todos los contenedores registrados
docker stop $(docker ps -a -q) 2>/dev/null || true
docker rm -f $(docker ps -a -q) 2>/dev/null || true

# Eliminación de la totalidad de imágenes locales y volúmenes huérfanos
docker rmi -f $(docker images -a -q) 2>/dev/null || true
docker volume rm $(docker volume ls -q) 2>/dev/null || true
```

B.13.4 Auditoría y confirmación de estado limpio

Finalizado el procedimiento de saneamiento, se valida que el subsistema de contenedores reporte una ocupación volumétrica nula (0 B) y que no existan entidades residuales en memoria o disco:

```
# Verificación de volumetría del motor
docker system df

# Comprobación de inventarios vacíos por subsistema
docker ps -a
docker images -a
docker volume ls
```

B.14 Verificación de registros de instalación

Para inspeccionar el flujo estándar de errores y salidas emitidas por el contenedor principal:

```
#!/ label: docker-logs-sig
#!/ eval: false
docker logs contenedor_sig_unal
```

B.15 Comandos de orquestación

Matriz referencial de utilidades CLI para la gestión de contenedores.

Tabla B.3: Matriz de comandos de orquestación de Docker

Fase	Invocación CLI	Función técnica
Construcción	<code>docker compose up --build -d</code>	Compilación integral y mapeo del árbol de imágenes.
Inicio	<code>docker compose up -d</code>	Reactivación silenciosa en segundo plano.
Monitoreo	<code>docker logs -f</code> <code>contenedor_sig_unal</code>	Captura continua del flujo estándar de salida.
Auditoría R	<code>docker exec -i -t</code> <code>contenedor_sig_unal R -e</code> <code>"library(sf)"</code>	Validación de motores PROJ/GEOS acoplados al R-Engine.

B.16 Resumen de la infraestructura instalada

Tabla B.4: Inventario de arquitectura y componentes del laboratorio

Componente	Especificación	Dependencia técnica
R Engine	4.3.3 (Angel Food Cake)	JuliaConnectoR acoplado al servidor httpgd.
Julia Stack	v1.10.x	ArchGDAL 3.12.1 resuelto vía enlaces simbólicos.
Python Stack	3.12.x	GeoPandas, Shapely y conectores psycopg2.
Persistencia	Volumen gestionado	Montaje bidireccional estable sobre /home/rstudio/work.

B.17 Validación del entorno Python

B.17.1 Ingreso al entorno interactivo

Desde la terminal integrada, asegure el arranque del servidor forzando los tokens de seguridad y los parámetros de red (evitando concatenaciones en las banderas cortas):

```
#| label: setup-jupyter-arranque
#| eval: false
jupyter lab --ip 0.0.0.0 --port 8889 --no-browser --allow-root --ServerApp.token geomatica2025
```

Acceso web: <http://127.0.0.1:8889> (Token: geomatica2025).

B.17.2 Guía visual del entorno

La carpeta /home/rstudio/work/ opera como un espejo asíncrono hacia el almacenamiento local, garantizando inmutabilidad frente a cortes abruptos del contenedor.

B.18 Validación del entorno R

B.18.1 Visualización interactiva con httpgd

Al tratarse de una infraestructura sin servidor X11 nativo (*headless*), se implementa httpgd como puente de renderizado de gráficos vectoriales para VSCode.

B.18.1.1 Inicio del servidor gráfico

Active el motor explícitamente al inicio de cada sesión interactiva:

```
#| label: setup-httpgd-arranque
#| eval: false
httpgd::hgd()
httpgd::hgd_details()
```

B.18.1.2 Resolución de conflictos

Si el sistema de red omite la inyección del puerto 8787, aplique una directiva forzada inhabilitando transitoriamente los tokens de seguridad de capa:

```

#| label: setup-httpgd-fuerza
#| eval: false
httpgd::hgd(host = "0.0.0.0", port = 8787, token = FALSE)

```

B.18.1.3 Función crítica del servidor gráfico

El dispositivo `httpgd` convierte el buffer en elementos interactivos de estándar SVG renderizables directamente por la API visual del editor, esquivando las restricciones naturales del despliegue en contenedores aislados.

B.19 Validación del entorno Julia

B.19.1 Ejecución de rutinas mediante evaluación cruzada

El sistema procesa bloques Quarto mediante `knitr`. La incompatibilidad histórica de `JuliaCall` con punteros de librerías geoespaciales requirió la creación de los wrappers `j_eval` y `j_plot` soportados por `JuliaConnector`.

Invoke las directivas de comunicación interlenguaje mediante el archivo base:

```

#| label: setup-julia-puente
#| code-fold: true
#| results: asis
source("../j_eval_j_plot.r")

```

B.20 Verificación de conectividad multilenguaje

B.20.1 Python

```

import psycopg2
import geopandas as gpd
from shapely.geometry import Point

try:
    conn = psycopg2.connect(
        host="db-postgis",
        dbname="sig_db_unal",
        user="profe_unal",
        password="geomatica2025"
    )
    print("Conexión a base de datos espacial exitosa")
    conn.close()
except Exception as e:
    print(f"Error relacional: {e}")

```

Conexión a base de datos espacial exitosa

B.20.2 R

```
library(sf)
```

Linking to GEOS 3.12.1, GDAL 3.8.4, PROJ 9.4.0; `sf_use_s2()` is TRUE

```
library(terra)
```

```
terra 1.8.93
```

```
Attaching package: 'terra'
```

```
The following object is masked from 'package:grid':
```

```
depth
```

```
conf <- sf_extSoftVersion()
cat(paste0("Motores de sistema detectados:\n",
  " - GDAL: ", conf["GDAL"], "\n",
  " - GEOS: ", conf["GEOS"], "\n",
  " - PROJ: ", conf["PROJ"], "\n"))
```

```
Motores de sistema detectados:
```

```
- GDAL: 3.8.4
- GEOS: 3.12.1
- PROJ: 9.4.0
```

B.20.3 Julia

```
j_eval('
using LibPQ
using DataFrames

try
  canal = LibPQ.Connection("host=db-postgis port=5432 dbname=sig_db_unal user=profe_unal
  ↪ password=geomatica2025")
  resultado_sql = execute(canal, "SELECT PostGIS_Version();")
  dataframe_version = DataFrame(resultado_sql)
  println("Enlace de datos espacial establecido con éxito en Julia. Versión: ", dataframe_version[1, 1])
  close(canal)
catch fallo_conexion
  println("Fallo de interconexión al orquestador: ", fallo_conexion)
end
')
```

```
Starting Julia ...
```

```
julia>
```

```
using LibPQ
```

```
julia>
```

```
using DataFrames
```

```
julia>
```

```
try
```

```
    canal = LibPQ.Connection("host=db-postgis port=5432 dbname=sig_db_unal user=profe_unal
    password=geomatica2025")
    resultado_sql = execute(canal, "SELECT PostGIS_Version();")
    dataframe_version = DataFrame(resultado_sql)
    println("Enlace de datos espacial establecido con éxito en Julia. Versión: ", datafram
```

```

    e_version[1, 1])
    close(canal)
catch fallo_conexion
    println("Fallo de interconexión al orquestador: ", fallo_conexion)
end
Enlace de datos espacial establecido con éxito en Julia. Versión: 3.6 USE_GEOS=1 USE_PROJ=
1 USE_STATS=1

```

```

plots_dir <- Sys.getenv("PLOTS_DIR", unset = ".")
if (!dir.exists(plots_dir)) dir.create(plots_dir, recursive = TRUE)
ruta_img <- file.path(plots_dir, "temp_grafica_julia.png")

j_plot('
ENV["GKSwtstype"] = "100"
using LibPQ
using DataFrames
using Plots
gr()

try
    canal = LibPQ.Connection("host=db-postgis port=5432 dbname=sig_db_unal user=profe_unal
    ↪ password=geomatica2025")

    query_autoridades = "SELECT auth_name, count(*)::int AS total FROM spatial_ref_sys WHERE auth_name IS
    ↪ NOT NULL GROUP BY auth_name ORDER BY total DESC LIMIT 5;"
    df_autoridades = DataFrame(execute(canal, query_autoridades))

    query_longitudes = "SELECT length(coalesce(srtext, '\\\\'\\\\\\\\\\\\\\\\'))::int AS longitud_wkt FROM
    ↪ spatial_ref_sys WHERE auth_name = '\\\\'EPSG\\\\\\\\' AND srtext IS NOT NULL LIMIT 1000;"
    df_longitudes = DataFrame(execute(canal, query_longitudes))

    close(canal)

    local p1 = bar(
        df_autoridades.auth_name,
        df_autoridades.total,
        color = :steelblue,
        xlabel = "Autoridad",
        ylabel = "Cantidad de SRID",
        title = "Sistemas de referencia por autoridad",
        xrotation = 15,
        legend = false
    )

    local p2 = histogram(
        df_longitudes.longitud_wkt,
        bins = 20,
        color = :coral,
        xlabel = "Longitud WKT EPSG (campo srtext)",
        ylabel = "Frecuencia",
        title = "Distribución de longitud WKT",
        legend = false
    )

    plot(
        p1, p2,
        layout = (1, 2),
        size = (900, 420),
        left_margin = 12Plots.mm,
        bottom_margin = 10Plots.mm,
        top_margin = 6Plots.mm,
        right_margin = 6Plots.mm
    )

catch fallo_proceso
    println("Error durante la extracción o graficación en Julia: ", fallo_proceso)

    # Gráfico de contingencia para evitar el colapso del compilador
    plot(
        title = "Fallo en ejecución de bloque espacial",
        framestyle = :none,
        annotations = (0.5, 0.5, text("Error: " * string(fallo_proceso), 10, :red, :center)),
        legend = false,
        size = (900, 420)
    )
end

```

```

', n = ruta_img, w = 900, h = 420)

julia>
ENV["GKSwstype"] = "100"
"100"

julia>
using LibPQ

julia>
using DataFrames

julia>
using Plots

julia>
gr()
Plots.GRBackend()

julia>
try
    canal = LibPQ.Connection("host=db-postgis port=5432 dbname=sig_db_unal user=profe_unal
        password=geomatica2025")

    query_autoridades = "SELECT auth_name, count(*)::int AS total FROM spatial_ref_sys WHE
RE auth_name IS NOT NULL GROUP BY auth_name ORDER BY total DESC LIMIT 5;"
    df_autoridades = DataFrame(execute(canal, query_autoridades))

    query_longitudes = "SELECT length(coalesce(srtext, '\'))::int AS longitud_wkt FROM sp
atial_ref_sys WHERE auth_name = '\EPSG\' AND srtext IS NOT NULL LIMIT 1000;"
    df_longitudes = DataFrame(execute(canal, query_longitudes))

    close(canal)

    local p1 = bar(
        df_autoridades.auth_name,
        df_autoridades.total,
        color = :steelblue,
        xlabel = "Autoridad",
        ylabel = "Cantidad de SRID",
        title = "Sistemas de referencia por autoridad",
        xrotation = 15,
        legend = false
    )

    local p2 = histogram(
        df_longitudes.longitud_wkt,

```

```

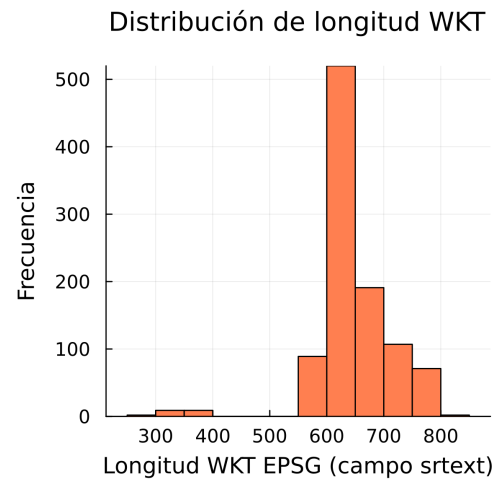
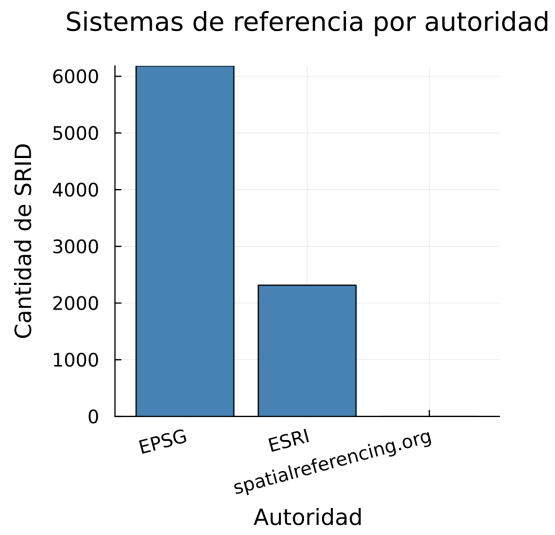
        bins = 20,
        color = :coral,
        xlabel = "Longitud WKT EPSG (campo srtext)",
        ylabel = "Frecuencia",
        title = "Distribución de longitud WKT",
        legend = false
    )

    plot(
        p1, p2,
        layout = (1, 2),
        size = (900, 420),
        left_margin = 12Plots.mm,
        bottom_margin = 10Plots.mm,
        top_margin = 6Plots.mm,
        right_margin = 6Plots.mm
    )

catch fallo_proceso
    println("Error durante la extracción o graficación en Julia: ", fallo_proceso)

    # Gráfico de contingencia para evitar el colapso del compilador
    plot(
        title = "Fallo en ejecución de bloque espacial",
        framestyle = :none,
        annotations = (0.5, 0.5, text("Error: " * string(fallo_proceso), 10, :red, :center
        )),
        legend = false,
        size = (900, 420)
    )
end

```



Apéndice C

Git y GitHub: Sincronización Local-Remota

C.1 Instalación de Git

C.2 Creación de cuenta en GitHub

C.3 Control de versiones - flujo de trabajo en PowerShell

Para gestionar el repositorio de forma robusta y tener control total sobre el historial de cambios, utilice la siguiente secuencia de comandos desde la terminal del contenedor:

Tabla C.1: Comandos esenciales para la gestión de Git en terminal

Acción	Comando	Propósito
Verificar estado	<code>git status</code>	Muestra la “verdad absoluta” local: archivos modificados o listos para commit.
Sincronizar índices	<code>git remote update</code>	Consulta el servidor para saber si hay cambios nuevos en la nube sin descargarlos aún.
Preparar cambios	<code>git add .</code>	Agrega todos los cambios detectados al área de preparación (<i>Staging</i>).
Confirmar versión	<code>git commit -m "mensaje"</code>	Crea un punto de control permanente en el historial local.
Enviar al remoto	<code>git push</code>	Sube los <i>commits</i> locales al servidor (GitHub).
Actualizar local	<code>git pull</code>	Descarga y fusiona los cambios del servidor en su entorno de trabajo.
Ver historial	<code>git log --oneline</code>	Muestra un resumen simplificado de la línea de tiempo del proyecto.

C.4 Diagnóstico de archivos “invisibles” o errores de rastreo

Si ha modificado un archivo pero este no aparece listado tras ejecutar `git status`, las causas técnicas suelen ser las siguientes:

1. **Interferencia de Repositorios Anidados:** Si clonó una carpeta que ya contenía un directorio `.git`, el repositorio principal la tratará como un objeto opaco (submódulo) y no rastreará sus archivos internos.
 - **Solución:** Verifique la presencia de carpetas ocultas con `ls -a` y elimine la carpeta `.git` interna si desea que el repositorio raíz tome el control: `rm -rf nombre_carpeta/.git`.
2. **Filtros en el archivo `.gitignore`:** El archivo puede tener reglas que excluyan carpetas enteras por nombre o extensión.
 - **Verificación:** Use el comando `git status --ignored` para listar todos los archivos que Git está omitiendo deliberadamente.
3. **Estado del sistema de archivos:** Git opera sobre los datos escritos en el disco duro. Asegúrese de que el editor de texto haya guardado efectivamente los cambios en el archivo físico antes de consultar el estado.

! Persistencia de Credenciales

Si la terminal le solicita usuario y contraseña en cada `push` o `pull`, puede configurar Git para que recuerde sus credenciales temporalmente en la memoria del contenedor con el siguiente comando: `git config --global credential.helper cache`

C.5 Subir el contenido (carpeta local hacia GitHub)

Si ya tiene sus archivos en Windows y quiere subirlos a un repositorio recién creado en GitHub:

1. **Instalar Git**
 - **Descarga:** Acceda a git-scm.com/download/win.
 - **Instalación:** Ejecute el instalador manteniendo las opciones por defecto; asegúrese de que la opción “Git from the command line and also from 3rd-party software” esté activa.
 - **Verificación:** Ejecute `git --version` en una terminal para confirmar la instalación.
2. **Crear una cuenta en GitHub**
3. **Abrir Terminal en la carpeta local:**
4. **Inicializar Git** en la carpeta local:

```
git init
```

5. Ver el **estado actual** del repositorio Git:

```
git status
```

6. **Archivo `.gitignore`:** Crear un archivo de texto `.gitignore` para indicarle a Git los archivos y

carpetas que nunca deben ser registrados con Git cuando se use el comando `git add` (siguiente item del procedimiento). Estos archivos que pueden estar en la raíz de la carpeta o dentro de otras sub-carpetas no se registrarán automáticamente cuando ejecute el comando `git add .` (ver **mas adelante**). Sin embargo puede forzar una carpeta o algunos tipos de archivos que estén marcados para ignorar dentro del `.gitignore` (ver **mas adelante**).

```
.quarto/  
_freeze/  
.ipynb_checkpoints/  
__pycache__/  
notebooks/  
scripts/  
data/
```

7. **Registrar (add)** el archivo `.gitignore` con Git:

```
git add .gitignore
```

8. Revise nuevamente el **estátus** y verifique cambios. Puede incluir el listado de los ignorados:

```
git status --ignored
```

9. **Salvar (commit)** el archivo en Git (local). Note que en el comando, el texto “*Archivo .gitignore*” es un comentario que describe lo que está salvando:

```
git commit -m "Archivo .gitignore"
```

10. Cambiar al nombre de **rama (branch)** a **principal (main)**. Pueden haber diferentes ramas con diferentes versiones del mismo archivo. Para cambiar a **main**:

```
git branch -M main
```

11. **Conectar** el repositorio local (Git) con nuestro repositorio en GitHub.

Para conectar Git con GitHub, puede hacerlo con SSH (debe realizar el proceso de instalación de llaves Sección C.6) o con HTTPS (debe generar un PAT desde github.com que usará como clave de autenticación). En este caso usaremos HTTPS.

Copie la url HTTP (Ej: <https://github.com/usuario/repositorio.git>) de un repositorio nuevo en GitHub.

```
git remote add origin https://github.com/usuario/repositorio.git
```

12. Procedimiento para **generar el PAT** (Personal Access Token) en GitHub.com {#ancla-github_pat}

- En lugar de escribir tu contraseña cuando el terminal te la pida, debes pegar un token generado en tu cuenta.

- Ve a tu cuenta de GitHub.com: **Settings > Developer settings > Personal access tokens > Tokens (classic)**.
- Genera un nuevo token **con el permiso repo activado**.
- Copia el token (no lo volverás a ver).

13. **Subir (push)** la información guardada localmente en Git a nuestro repositorio en GitHub:

```
git push -u origin main
# Cuanto solicite el password use el PAT/token generado previamente en github.com
```

Puedes ver el estatus `git status` y realizar un procedimiento similar para registrar (`add`), salvar (`commit`) y subir (`push`) mas archivos. Otras opciones de Git se describen a continuación:

- Registrar archivos específicos con Git (local):

```
git add file1 file2 ...
```

- **Forzar archivos** y/o carpetas que están marcados para ignorar (bandera `-f`): el comando a continuación sincronizará con Git toda la carpeta `_site/site_libs/` y su contenido, así contenga archivos o carpetas marcados para ignorar en `.gitignore`. `{#ancla-git_add_minus_f}`

```
git add -f _site/site_libs/
```

- Registrar **todos los archivos y carpetas** disponibles y sin registrar, sin embargo todos los archivos y carpetas dentro de `.gitignore` no serán tenidos en cuenta `{#ancla-git_add_punto}`:

```
git add .
```

- En **resumen** estos son los comandos que comúnmente usarás para mantener sincronizados tus archivos con Git y tu Git con GitHub, es decir el proceso de subir (`push`) archivos a la nube.

- `git add` (agregar archivos a Git)
- `git commit` (guardar archivos en Git)
- `git push` (subir archivos a GitHub)

- Los otros comandos vistos solo se ejecutan **una vez** o de acuerdo con **necesidades específicas**:

- `git init` (una vez)
- `git remote add origin` (una vez)
- `git branch -M main` (lo debes usar para cambiar a main en caso de tener múltiples ramas)

C.6 Autenticación segura SSH - Windows

La autenticación mediante SSH permite interactuar con GitHub de forma segura sin necesidad de ingresar credenciales manualmente en cada operación. Siga este procedimiento detallado para configurar su acceso.

C.6.1 Requisito previo: creación de cuenta en GitHub

El correo que use para crear esta cuenta **debe ser el mismo correo** usado para crear las llaves SSH para conexión automática.

C.6.2 Requisito previo: instalación de Git

Es indispensable que su sistema operativo cuente con el motor de Git antes de iniciar la configuración:

C.6.3 Preparación del entorno local

- **Navegación:** Diríjase mediante el Explorador de Archivos a la carpeta raíz de su proyecto (ejemplo: carpeta ID Unal).
- **Terminal:** Abra una ventana de PowerShell en esa ubicación (clic derecho -> “Abrir en Terminal”).
- **Inicialización:** Prepare el repositorio local mediante el comando:

```
git init
```

C.6.4 Gestión de llaves SSH en Windows

SSH (*Secure Shell*) es un protocolo de red diseñado para el acceso, administración y transferencia de datos entre un equipo local y un servidor remoto mediante un canal cifrado. En nuestro flujo de trabajo, implementamos el algoritmo **Ed25519**, un estándar de vanguardia basado en criptografía de curva elíptica (EdDSA). Este método no solo ofrece un nivel de seguridad superior frente a ataques de fuerza bruta, sino que permite una **autenticación transparente y sin contraseña**, actuando como un “pasaporte digital” que vincula de forma única su estación de trabajo con GitHub.

i ¿Por qué Ed25519?

A diferencia de los estándares antiguos (como RSA), **Ed25519** genera llaves más cortas, rápidas y significativamente más seguras, optimizando la latencia en cada comunicación con el repositorio.

- **Comprobación de llaves existentes:** Verifique si su usuario ya posee llaves configuradas:

```
ls ~/.ssh
```

- Si el directorio está vacío o solo contiene `known_hosts`, proceda a generar una nueva.
 - Si visualiza los archivos `id_ed25519` e `id_ed25519.pub`, puede saltar al paso de registro en GitHub.
- **Generación de llaves:** Cree un par de llaves seguras utilizando el algoritmo Ed25519.

El correo que use en el siguiente paso, **debe** ser el correo con el que creó la cuenta de GitHub: “su_correo@domimio.com”

```
ssh-keygen -t ed25519 -C "su_correo@domimio.com"
```

- **Nota:** Presione ENTER en todas las solicitudes para aceptar las rutas por defecto y no asignar una *passphrase*.

i Seguridad de la Identidad

La **llave pública** (`id_ed25519.pub`) es la que se entrega a GitHub, mientras que la **llave privada** (`id_ed25519`) funciona como su sello personal secreto y nunca debe salir de su equipo.

C.6.5 Registro de la identidad en GitHub

- **Copiar la firma pública:** Obtenga el contenido de su llave para registrarla:

```
cat $env:USERPROFILE\.ssh\id_ed25519.pub
```

Seleccione y copie toda la cadena de texto resultante.

- **Configuración en la plataforma:**
 - Acceda a su cuenta en [GitHub.com](https://github.com).
 - Entre a **Settings** -> **SSH and GPG keys**.
 - Seleccione **New SSH key**.
 - Asigne el título **Windows-UNAL-2025**, pegue el contenido en el campo **Key** y presione **Add SSH key**.

C.6.6 Activación del agente SSH del sistema

Para que Windows gestione su identidad de forma transparente, debe habilitar el servicio correspondiente:

- **Modo Administrador:** Cierre su terminal actual y abra una nueva sesión de **PowerShell como Administrador**.
- **Habilitar Servicio:** Ejecute los siguientes comandos para automatizar el arranque del agente:

```
Set-Service ssh-agent -StartupType Automatic  
Start-Service ssh-agent
```

- **Cargar la llave privada:** Registre su llave en el agente activo:

```
ssh-add $env:USERPROFILE\.ssh\id_ed25519
```

C.6.7 Verificación de la autenticación

- **Prueba de conexión:** Compruebe que el túnel de comunicación con los servidores de GitHub funciona correctamente:

```
ssh -T git@github.com
```

- **Resultado esperado:** Si la configuración es exitosa, recibirá un mensaje similar a: `Hi usuario!`
`You've successfully authenticated, but GitHub does not provide shell access.`

¡Túnel SSH Activo y Configurado!

Has establecido una identidad digital segura entre tu equipo y GitHub. A partir de este momento, la comunicación para sincronizar tus repositorios será transparente y automática.

- **Identidad Permanente:** Esta configuración se realiza **una sola vez** por computador; tu “firma digital” ya reside en el sistema.
- **Adiós a las Interrupciones:** GitHub ya no solicitará tu usuario, contraseña o tokens personales de acceso en cada operación.
- **Compatibilidad Total:** Esta llave es reconocida automáticamente por **PowerShell**, **VSCode**, **Positron**, **RStudio** y cualquier otra terminal científica que utilices.
- **Seguridad de Grado Profesional:** Tus envíos (push) viajan ahora por un canal cifrado bajo el estándar Ed25519.

C.7 Autenticación segura SSH - Linux (Docker)

La autenticación mediante **SSH** permite interactuar con GitHub de forma segura sin ingresar credenciales manualmente. En contenedores Docker, esto soluciona errores de validación SSL y facilita el uso de extensiones de Quarto.

C.7.1 Preparación del Entorno Global

Antes de generar las llaves, configura tu identidad de Git y verifica que el contenedor tenga salida a internet.

- **Configurar identidad:**

```
git config --global user.email "correo@dominio.com"
git config --global user.name "usuario_github"
```

- **Verificar conexión:**

```
curl -I https://github.com
```

⚠ Advertencia

Si recibes el error * “Could not resolve host”*, revisa la configuración de red de tu contenedor.

C.7.2 Gestión de llaves SSH (Algoritmo Ed25519)

Implementamos **Ed25519**, el estándar actual por su seguridad y velocidad.

- **Comprobar llaves existentes:**

```
ls -la ~/.ssh
```

- **Generar nueva llave:** Ejecuta el comando y presiona **ENTER** en todas las solicitudes (sin contraseña).

```
ssh-keygen -t ed25519 -C "alexysr@gmail.com"
```

i Nota

La llave pública (id_ed25519.pub) se registra en GitHub; la llave privada (id_ed25519) es tu firma secreta.

C.7.3 Registro de Identidad en GitHub

1. **Visualizar y copiar la llave pública:**

```
cat ~/.ssh/id_ed25519.pub
```

2. **Configurar en la web:**

- Ve a **Settings** -> **SSH and GPG keys** en tu cuenta de GitHub.
- Haz clic en **New SSH key**.
- Título: **Docker-RStudio-Geomatica**.
- Pega el contenido y guarda.

C.7.4 Activación del Agente SSH

Para que el sistema use tu llave automáticamente, inicia el agente y regístrala:

```
# Iniciar agente
eval "$(ssh-agent -s)"

# Cargar la llave privada
ssh-add ~/.ssh/id_ed25519
```

C.7.5 Verificación Final

Comprueba que el túnel de comunicación funciona correctamente:

```
ssh -T git@github.com
```

Resultado esperado:

```
Hi usuario_github! You've successfully authenticated, but GitHub does not provide
shell access.
```

C.7.5.1 ¡Configuración Exitosa!

Tu contenedor ahora es un entorno de confianza para GitHub. Comandos como `quarto add` o `git push` funcionarán de forma transparente.

C.8 Descargar contenido (GitHub hacia carpeta local)

Para explicar este procedimiento, usaremos el repositorio GitHub con el material de la clase. Los contenidos de la clase están en <https://github.com/alexysshr/ProgramacionSIG2026.git> (HTTPS) o `git@github.com:alexysshr/ProgramacionSIG2026.git` (SSH). La elección de uno de estos dos protocolos (HTTPS o SSH), depende de si ha configurado su identidad digital, ver Sección C.6.

⚠ ¡Advertencia sobre la Estructura de Carpetas!

Es fundamental para el orden del curso que el repositorio con los contenidos de la clase se conecte a una **carpeta independiente**.

No sincronice estos archivos dentro de su carpeta local con los contenedores ID UNAL. Mantener directorios separados garantiza que sus ejercicios personales y los materiales de lectura del profesor no se mezclen, evitando errores de sobreescritura y conflictos de Git al realizar actualizaciones del material.

C.8.1 Opción A: sincronizar mediante SSH (recomendado)

Utilice este método si ya configuró sus llaves siguiendo los pasos de la Sección C.6.

! Url SSH del Repositorio para la Clase

`git@github.com:alexysshr/ProgramacionSIG2026.git`

- **Vincular el repositorio remoto:** Si ya inicializó su carpeta local (no la de sus contenedores Docker) con `git init`, ejecute el siguiente comando para establecer el origen:

```
git remote add origin git@github.com:alexysshr/ProgramacionSIG2026.git
```

Nota: El siguiente comando funciona si el nombre de la rama en el servidor es `main`. * Para chequear el nombre de las ramas en el servidor use `git ls-remote --heads origin`.

- **Descargar el contenido (Pull):** Para traer los archivos del servidor a su computadora por primera vez de una rama llamada `main`:

```
git pull origin main
```

Para descargar los archivos a su computadora de una rama llamada `master`. Debido a que el repositorio local fue inicializado de forma independiente `git init`, es necesario permitir la fusión de historiales que no comparten un ancestro común mediante la bandera `--allow-unrelated-histories`

```
git pull origin master --allow-unrelated-histories
```

- **Verificación de estado:** Confirme que la carpeta local está sincronizada correctamente:

```
git status
```

Resultado esperado:

```
On branch main. Your branch is up to date. working tree clean.
```

C.8.2 Opción B: sincronizar mediante HTTPS

Utilice este método si aún no ha configurado llaves SSH o prefiere una descarga rápida. A diferencia del método anterior, este suele solicitar un **Personal Access Token (PAT)** (ver [líneas arriba](#)) para realizar operaciones de subida (*push*).

! Url HTTPS del Repositorio para la Clase

<https://github.com/alexysshr/ProgramacionSIG2026.git>

C.8.2.1 Opción B-1: Descarga rápida mediante `git clone`

Nota: Este procedimiento tiene la desventaja que el nombre de la carpeta es definido por el nombre del repositorio en GitHub, no por el usuario (usted).

- Este comando descarga el repositorio completo y crea automáticamente una subcarpeta con el nombre del proyecto:

```
git clone https://github.com/alexysshr/ProgramacionSIG2026.git
```

- Para chequear por actualizaciones en el repositorio remoto:

```
git remote -v
```

- Para descargar las actualizaciones a la carpeta local, es decir sincronizar el repositorio remoto con la carpeta local:

```
git pull
```

C.8.2.2 Opción B-2: Sincronizar repositorio GitHub con carpeta local existente

Utilice este procedimiento si usted ya creó una carpeta localmente (por ejemplo, `ProgramacionSIG2026`) y desea vincularla al repositorio oficial de GitHub para recibir actualizaciones sin perder sus archivos actuales.

Ejecute los siguientes comandos desde la terminal dentro de su carpeta de trabajo:

```
# 1. Iniciar el repositorio localmente
git init

# 2. Vincular la carpeta local con el servidor de GitHub
git remote add origin https://github.com/alexysshr/ProgramacionSIG2026.git

# 3. Asegurar que la rama principal se llame 'main'
git branch -M main

# 4. Sincronizar (traer los archivos del servidor)
# Usamos --allow-unrelated-histories para permitir la fusión de carpetas
# que no "nacieron" del mismo commit original.
git pull origin main --allow-unrelated-histories
```

! Manejo de conflictos en el primer pull

Al ejecutar el `git pull` en una carpeta existente, es posible que Git le pida resolver conflictos si usted tiene archivos con el mismo nombre que los del repositorio (ej. un `README.md` genérico).

Si desea forzar que sus archivos locales sean sobrescritos por los del repositorio oficial, puede usar: `git fetch --all git reset --hard origin/main`

💡 Verificación de la conexión

Para confirmar que su carpeta quedó bien vinculada, ejecute: `git remote -v`

Debería ver la URL de su repositorio tanto para `fetch` como para `push`.

C.9 Notas de flujo de trabajo

- **Persistencia:** Una vez clonado o vinculado el repositorio, Git recordará la ruta del servidor (ya sea SSH o HTTPS). No es necesario repetir los comandos de `remote add` en sesiones futuras.
- **Seguridad:** Si opta por HTTPS, recuerde que GitHub ya no acepta contraseñas básicas por terminal; deberá generar un PAT (Personal Access Token) en la configuración de su cuenta. Por esta razón, se recomienda priorizar el uso de **SSH**.

Apéndice D

Visual Studio Code (VScode): Extensiones e Interfaz

VScode es un editor de código para programadores gratuito, de código abierto y multiplataforma ([Microsoft Corporation, 2026](#)). Lo usaremos para editar archivos `.qmd` y para tener acceso a los contenedores Docker instalados para el curso.

D.1 Instalación

Descargue e instale la versión oficial de **VSCode IDE** correspondiente al sistema operativo de su máquina local desde el siguiente enlace:

- **URL:** <https://code.visualstudio.com/>
- **Instalación:** Seleccione el instalador `.exe` más reciente y que coincida con la arquitectura de su máquina.

D.2 Acceso y configuración en VSCode

Ver Sección [B.8](#).

D.3 Inicialización del visor gráfico

Ver Sección [B.18.1](#).

D.4 Atajos

D.4.1 La paleta de comandos VSCode

La mayoría de las funciones avanzadas de VSCode no tienen un botón visible. Se acceden mediante la **Paleta de Comandos** presionando `Ctrl + Shift + P`. A continuación, los comandos más utilizados en este curso:

Comando (Command Palette)	Propósito	¿Cuándo usarlo?
Developer: Reload Window	Reinicia la interfaz de VSCode sin cerrar el contenedor.	Cuando una extensión (como la de R o Julia) deja de responder.
Dev Containers: Rebuild Container	Reconstruye la imagen desde el Dockerfile.	Cuando se realizan cambios en la configuración del entorno.
Quarto: Preview	Abre una ventana lateral con la previsualización del documento.	Para ver los cambios en el <code>.qmd</code> en tiempo real.
R: Create R terminal	Fuerza la apertura de una nueva consola de R vinculada.	Si el prompt <code>></code> no aparece automáticamente.
Julia: Start REPL	Inicia la consola interactiva de Julia.	Al abrir el contenedor por primera vez para ejecutar código Julia.
Python: Select Interpreter	Permite elegir la versión de Python del contenedor.	Si VSCode no detecta automáticamente <code>/usr/bin/python3</code> .
PostgreSQL: New Query	Abre un editor SQL en blanco.	Para realizar consultas a la base de datos espacial.
Git: Pull / Git: Push	Sincroniza cambios con GitHub.	Para actualizar el repositorio de la clase o subir tareas.

! El comando de “Primer Auxilio”

Si notas que el autocompletado desaparece o que los gráficos no cargan en **htpugd**, el primer paso siempre es ejecutar **Developer: Reload Window**. Esto refresca todas las conexiones de las extensiones con el contenedor sin interrumpir los procesos que Docker está ejecutando de fondo.

D.4.2 Atajos de teclado rápidos VSCode

Además de la paleta, memorice estos tres para su flujo diario:

- **Ctrl + Shift + `** (acento grave): Abre una nueva Terminal.
- **Ctrl + J**: Mostrar la terminal/consola existente.
- **Ctrl + Enter**: Envía la línea de código actual desde el editor al REPL (R, Julia o Python).
- **Ctrl + Shift + V**: Previsualiza archivos Markdown (`.md`) de forma rápida.

D.4.3 Atajos de teclado JupyterLab (notebooks)

Acción	Atajo (Esc Mode)
Ejecutar Celda	Ctrl + Enter
Ejecutar y pasar a la siguiente	Shift + Enter
Convertir a Markdown	M
Convertir a Código	Y
Crear celda arriba / abajo	A / B

D.5 Instalación Tinytex

TinyTeX es un motor ligero de LaTeX necesario para compilar reportes profesionales (ej. en Quarto) en formato PDF (Xie, 2019). Si desea instalar TinyTeX para generar PDF por fuera del entorno Docker, desde la terminal de VSCode use el comando:

```
quarto install tinytex
```

La herramienta **Tinytex** viene instalada dentro del contenedor `contenedor_sig_unal`, así que no necesita instalarla.

D.6 Instalación de extensiones

Para garantizar la plena operatividad del entorno analítico, es indispensable verificar el despliegue del conjunto de extensiones en Visual Studio Code. La arquitectura de trabajo opera bajo un esquema desacoplado: una instancia del editor en el sistema anfitrión conectada a los servicios mediante el contenedor de desarrollo (`contenedor_sig_unal`).

El despliegue de las herramientas se estructura bajo una **estrategia de instalación modular**:

- **Herramientas base (en contenedor):** Conjunto preconfigurado e integrado en la capa interna del contenedor dentro de la imagen base. Al vincular la sesión mediante **Dev Containers: Attach to Running Container...** (Ctrl + Shift + P), este núcleo funcional se encuentra habilitado y disponible de forma inmediata dentro de la sesión remota del contenedor.
- **Herramientas del sistema anfitrión (host):** Extensiones que deben instalarse directamente en el Visual Studio Code local. Incluyen las herramientas de infraestructura requeridas para comunicarse con Docker y utilidades ligeras de diagramación y validación de formatos que se ejecutan localmente para optimizar la memoria del contenedor.
- **Herramientas diferidas bajo demanda:** Extensiones especializadas requeridas para los módulos avanzados de geodatabases corporativas, procesamiento columnar y procedimientos compilados (Ejes 3 y 4). Se instalarán puntualmente dentro del contenedor durante el desarrollo de los laboratorios respectivos.

D.6.1 Persistencia y ciclo de vida del contenedor

El aislamiento de procesos en contenedores introduce una distinción fundamental entre el estado de ejecución y la persistencia de datos. Cuando se instala una extensión adicional dentro del contenedor desde la interfaz de Visual Studio Code, los archivos binarios se escriben en la capa de lectura y escritura del contenedor activo (`/root/.vscode-server/extensions` o `/home/<usuario>/.vscode-server/extensions`).

Para conservar las nuevas extensiones instaladas en el contenedor y evitar la reconstrucción innecesaria del entorno, la gestión de sesiones debe regirse por las siguientes directivas:

1. **Uso exclusivo de `docker compose stop` y `docker compose start`:** Al finalizar la jornada de trabajo, el contenedor debe suspenderse mediante `docker compose stop`. Esta instrucción detiene los procesos del motor relacional preservando intacta la capa de escritura. Para reanudar la sesión en clases posteriores,

se ejecuta `docker compose start`, manteniendo las extensiones añadidas y el historial de comandos.

2. **Evitar comandos destructivos (`docker compose down`, `docker rm`):** Las instrucciones de eliminación de contenedores destruyen la capa mutable. Si el contenedor se remueve, cualquier extensión instalada manualmente se perderá y el entorno regresará al estado base empaquetado en la imagen original.

Una vez reiniciado el contenedor mediante `docker compose start`, inicie Visual Studio Code en el sistema anfitrión, presione `Ctrl + Shift + P`, seleccione **Dev Containers: Attach to Running Container...** y elija `/contenedor_sig_unal`.

D.6.2 Clasificación y catálogo de extensiones

A continuación se detalla el inventario técnico de extensiones, categorizado según el ámbito de ejecución y fase de disponibilidad en el curso:

Tabla D.3: Extensiones de Visual Studio Code para el entorno de bases de datos espaciales

Categoría	Herramienta	Propósito	ID de la extensión	Disponibilidad y ámbito
Infraestructura	Dev Containers	Conexión y gestión del contenedor activo	<code>ms-vscode-remote.remote-containers</code>	Sistema anfitrión
	Docker	Control de imágenes, redes, volúmenes y servicios	<code>ms-azuretools.vscode-docker</code>	Sistema anfitrión
Ecosistema R	R (REditorSupport)	Sintaxis, servidor de lenguaje (LSP) y terminal	<code>REditorSupport.r</code>	Base (en contenedor)
	R Debugger	Depuración interactiva de scripts en R	<code>RDebugger.r-debugger</code>	Base (en contenedor)
	R Extension Pack	Paquete de utilidades y snippets para análisis en R	<code>Ikuyadeu.r-pack</code>	Base (en contenedor)
Ecosistema Python	Python (Microsoft)	IntelliSense (Pylance), tipado y entornos virtuales	<code>ms-python.python</code>	Base (en contenedor)
	Python Debugger	Motor de depuración desacoplado para flujos Python	<code>ms-python.debugpy</code>	Base (en contenedor)

Categoría	Herramienta	Propósito	ID de la extensión	Disponibilidad y ámbito
Ecosistema Julia	Jupyter (Microsoft)	Cuadernos interactivos (.ipynb) y celdas de código	ms-toolsai.jupyter	Base (en contenedor)
	Julia	Soporte de lenguaje, REPL integrado y gráficos	julia-lang.language-julia	Base (en contenedor)
Bases de datos	PostgreSQL (Chris Kolkman)	Árbol visual para exploración de esquemas y tablas	ckolkman.vscode-postgres	Base (en contenedor)
Publicación	pgsql (psql runner)	Ejecución de scripts SQL en consola vía psql (Ctrl + F5)	doublefint.pgsql	Base (en contenedor)
	Quarto	Renderizado y compilación de documentos .qmd	quarto.quarto	Base (en contenedor)
	Live Server	Servidor HTTP local para previsualización de salidas HTML	ritwickdey.liveserver	Base (en contenedor)
Control de versiones	GitHub Repositories	Inspección y lectura de repositorios remotos en GitHub	github.remotehub	Base (en contenedor)
Calidad de código	Code Spell Checker	Corrector ortográfico en código y comentarios	streetsidesoftware.code-spell-checker	Base (en contenedor)
	Spanish - Code Spell Checker	Diccionario en español para el corrector ortográfico	streetsidesoftware.code-spell-checker-spanish	Base (en contenedor)
Formato y modelado	Rainbow CSV	Resaltado de columnas e inspección de geometrías WKT	mechatroner.rainbow-csv	Sistema anfitrión
	Mermaid Chart	Modelado y previsualización de diagramas Entidad-Relación	MermaidChart.vscodemermaid-chart	Sistema anfitrión

Categoría	Herramienta	Propósito	ID de la extensión	Disponibilidad y ámbito
Formatos avanzados	YAML Language Support	Validación de sintaxis en <code>_quarto.yml</code> y contenedores	<code>redhat.vscode-yaml</code>	Sistema anfitrión
	Geo Data Viewer	Previsualización rápida de capas GeoJSON y WKT	<code>RandomFractalsInc.geo-data-viewer</code>	Sistema anfitrión
	XML Language Support	Validación de esquemas XML Workspace y transformaciones	<code>redhat.vscode-xml</code>	Diferida (Semana 9)
	Parquet Viewer	Inspección tabular de archivos Parquet y GeoParquet	<code>dvirtz.parquet-viewer</code>	Diferida (Semana 11)
Sistemas compilados	rust-analyzer	Servidor de lenguaje para extensiones nativas con PGRX	<code>rust-lang.rust-analyzer</code>	Diferida (Semana 16)

! Ámbito de instalación: sistema anfitrión frente a contenedor

Las extensiones de infraestructura (**Dev Containers** y **Docker**) deben instalarse exclusivamente en el sistema anfitrión para gestionar la comunicación con el motor de virtualización.

En contraste, las herramientas analíticas (R, Python, Julia, PostgreSQL, Quarto) operan **dentro de la sesión remota del contenedor**. No es necesario instalarlas previamente en el sistema anfitrión: una vez ejecutada la directiva **Dev Containers: Attach to Running Container...**, cualquier extensión instalada desde la pestaña correspondiente se alojará directamente en el entorno de ejecución Linux del contenedor.

D.6.3 Notas técnicas sobre herramientas específicas

PostgreSQL (Chris Kolkman): Permite conectarse directamente al motor espacial PostGIS del contenedor sin abandonar Visual Studio Code, facilitando la exploración de la estructura de tablas y catálogos espaciales.

Cliente de terminal psql: La herramienta de línea de comandos `psql` se encuentra integrada de forma nativa en el sistema operativo del contenedor. No requiere configuración adicional para ejecutarse en la terminal integrada. La extensión `doublefint.psql` complementa esta utilidad canalizando bloques de código SQL desde el editor hacia la sesión activa mediante atajos de teclado.

Previsualización web con Live Server: Instancia un servidor HTTP local con recarga automática (*live reload*). Es requerida para examinar artefactos HTML interactivos fuera del entorno Quarto (mapas exportados con Leaflet, OpenLayers o MapLibre) y para ejecutar la función de búsqueda del sitio compilado del curso

(`_site/index.html`), evitando bloqueos originados por políticas CORS al abrir archivos locales directamente en el navegador.

Corrector ortográfico multilingüe: La extensión **Code Spell Checker** previene errores léxicos en comentarios de código y en la documentación técnica de archivos `.qmd`. Para habilitar la compatibilidad léxica con terminología en español, se integra el paquete **Spanish - Code Spell Checker** (`streetsidesoftware.code-spell-checker-spanish`).

D.6.4 Sinergia de las herramientas de PostgreSQL en Visual Studio Code

La coexistencia de `ckolkman.vscode-postgres` y `doublefint.pgsql` dentro del contenedor responde a una separación deliberada de funciones operativas: inspección estructural del catálogo frente a ejecución procedural interactiva.

Tabla D.4: Matriz funcional de herramientas para interacción con PostgreSQL

Extensión	Especialidad técnica	Alcance en el flujo analítico
PostgreSQL (Chris Kolkman)	Inspección del catálogo relacional	Navegación jerárquica de bases de datos, esquemas, llaves, tipos y columnas espaciales PostGIS.
pgsql (doublefint)	Ejecución procedural y scripts	Envío directo de bloques DDL/DML hacia el proceso <code>psql</code> en terminal, preservando variables de sesión.

D.6.4.1 Ventajas de la integración complementaria

- **Validación de esquemas sin bloqueo:** Durante la estructuración de consultas espaciales complejas, el árbol lateral de exploración permite corroborar nombres de atributos, proyecciones cartográficas (`SRID`) y tipos geométricos sin interrumpir el búfer de edición mediante consultas exploratorias de sondeo.
- **Preservación del contexto de sesión:** La extensión `doublefint.pgsql` opera contra la sesión nativa de `psql` en la terminal integrada, asegurando que directivas de entorno como `SET search_path`, esquemas de codificación (`client_encoding`) y transacciones manuales (`BEGIN ... COMMIT`) permanezcan consistentes.
- **Aislamiento de salidas analíticas:** Facilita mantener los resultados tabulares o planes de ejecución (`EXPLAIN ANALYZE`) desplegados en paneles independientes mientras se refactoriza el script principal en el editor.

D.6.4.2 Flujo de trabajo analítico recomendado

1. **Exploración:** Utilizar el explorador de objetos de `ckolkman.vscode-postgres` para verificar la conectividad, constatar la activación de extensiones espaciales (`postgis`, `postgis_topology`) y revisar la definición de vistas analíticas.
2. **Desarrollo y depuración:** Redactar los scripts de análisis espacial en archivos `.sql` y ejecutarlos mediante `doublefint.pgsql` presionando el atajo `Ctrl + F5` para canalizar el código a la terminal interactiva del contenedor.

Apéndice E

Temas técnicos sobre Windows

E.1 Herramientas de administración

Acceso Directo	Parámetro / Comando	Descripción
Windows + R	%UserProfile%	Abre tu carpeta personal de usuario (donde se crea el archivo .wslconfig).
Windows + R	cleanmgr	Abre el Liberador de espacio en disco para purgar basura del sistema.
Windows + R	taskmgr	[Ctrl + Shift + Esc] Abre el Administrador de Tareas para ver procesos.
Windows + R	resmon	Abre el Monitor de Recursos (ideal para ver el uso real de RAM y Swap).
Windows + R	sysdm.cpl	[Win + Pausa] Propiedades del sistema (configuración de Memoria Virtual).
Windows + R	%temp%	Abre la carpeta de Archivos temporales del usuario actual.
Windows + R	temp	Abre la carpeta de Archivos temporales del sistema (raíz).
Windows + R	prefetch	Carpeta de precarga de Windows (caché que puedes vaciar para liberar espacio).
Windows + R	diskmgmt.msc	Abre la Administración de discos (para vigilar tus unidades C: y D:).
Windows + R	appdata	Acceso a carpetas de configuración de aplicaciones (Roaming/Local).
Windows + R	cmd	[Win + X, luego C] Abre el Símbolo del Sistema (Terminal clásica).
Windows + R	powershell	[Win + X, luego A] Abre PowerShell con permisos de administrador.
Windows + R	control	Abre el Panel de Control clásico de Windows.

Acceso Directo	Parámetro / Comando	Descripción
Windows + R	<code>msconfig</code>	Configuración del sistema (gestión de servicios y arranque).
Windows + R	<code>ncpa.cpl</code>	Panel de Conexiones de red (configuración de adaptadores e IPs).
Nativo	Win + E	Abre instantáneamente el Explorador de Archivos .
Nativo	Win + I	Abre el panel de Configuración de Windows (Settings).
Nativo	Win + D	Minimiza/Restaura todo para ir al Escritorio rápidamente.
Nativo	Win + V	Abre el Historial del Portapapeles (muy útil para copiar varios códigos).

- Procesos de **WSL2** (Windows Subsystem for Linux 2): Docker y Vmmem

E.2 Comandos (PowerShell)

- **Ver uso de RAM:** `Get-Process | Sort-Object WorkingSet -Descending | Select-Object -First 10`
- **Ver estado de WSL:** `wsl --list --verbose`
- **Limpiar Cache de DNS:** `ipconfig /flushdns`

Apéndice F

Temas Técnicos sobre Linux

F.1 Aprovisionamiento y actualización del cliente PostgreSQL en el contenedor

```
# =====
# Aprovisionamiento y actualización del cliente PostgreSQL en el contenedor
# =====

# NOTA: En la base del sistema operativo (Ubuntu 24.04 LTS), el metapaquete
# estándar instala por defecto la versión 16:
# apt-get install -y --no-install-recommends postgresql-client

# Procedimiento para instalar PostgreSQL Client 18 desde el repositorio oficial PGDG
# (Resuelve el error 'server version mismatch' al ejecutar pg_dump contra PostgreSQL 18)

# 1. Sincronizar el catálogo de paquetes
apt-get update

# 2. Instalar utilidades para gestión de repositorios
apt-get install -y --no-install-recommends postgresql-common curl lsb-release ca-certificates

# 3. Registrar el repositorio oficial PGDG y sus llaves criptográficas
/usr/share/postgresql-common/pgdg/apt.postgresql.org.sh -y

# 4. Instalar los binarios cliente de PostgreSQL
apt-get install -y --no-install-recommends postgresql-client-18

# 5. Verificar versiones activas
psql --version
pg_dump --version
```

F.2 Instalación y verificación de Graphviz en el contenedor

La generación automatizada de diagramas entidad-relación y esquemas de dependencias relacionales requiere el motor de renderizado Graphviz y su utilidad de compilación de grafos `dot`. A continuación, se detalla el procedimiento para diagnosticar el estado del binario en el contenedor, aprovisionar el paquete mediante los repositorios oficiales de la distribución e inspeccionar la instalación resultante.

F.2.1 Diagnóstico previo del entorno

Antes de proceder con la instalación, es conveniente verificar si el ejecutable ya se encuentra en las rutas del sistema (PATH) o registrado en la base de datos de paquetes de Debian/Ubuntu:

```
#| label: setup-check-graphviz
#| eval: false
# 1. Comprobar la disponibilidad del ejecutable y su versión
dot -V

# 2. Localizar la ruta absoluta del binario en el sistema de archivos
which dot

# 3. Consultar el estado del paquete en el gestor dpkg
dpkg -l | grep graphviz
```

Si el comando `dot -V` retorna un error de comando no encontrado (`command not found`), se debe ejecutar el aprovisionamiento.

F.2.2 Aprovisionamiento del paquete Graphviz

Para asegurar la descarga de los metadatos más recientes y evitar fallos por catálogos locales vacíos, se debe sincronizar el índice de paquetes antes de invocar la instalación. Se incluye el modificador `--no-install-recommends` para evitar dependencias accesorias innecesarias en el contenedor:

```
#| label: setup-install-graphviz
#| eval: false
# 1. Sincronizar el catálogo de paquetes
apt-get update

# 2. Instalar el motor Graphviz
apt-get install -y --no-install-recommends graphviz
```

F.2.3 Verificación posterior de los binarios

Finalizada la instalación, se comprueba la correcta vinculación del ejecutable `dot` y la versión provista por la distribución:

```
#| label: setup-verify-graphviz
#| eval: false
dot -V
which dot
```

F.3 Comandos comunes de terminal

Al trabajar dentro de un contenedor Docker, la terminal es su línea directa con el sistema operativo (Ubuntu/Debian). Estos comandos le permitirán verificar que los motores geoespaciales y las librerías de sistema están correctamente instalados.

Comando	Propósito	¿Para qué sirve en el curso?
<code>dpkg -l \ grep -E "libpng\ libgdal\ libproj\ libgeos"</code>	Lista librerías instaladas	Verificar que los drivers de mapas (GDAL, PROJ, GEOS) están activos.

Comando	Propósito	¿Para qué sirve en el curso?
<code>gdalinfo --version</code>	Versión de GDAL	Confirmar que el motor de traducción de datos geográficos funciona.
<code>df -h</code>	Espacio en disco	Evitar errores de “Disk Full” al descargar imágenes satelitales pesadas.
<code>htop</code> (o <code>top</code>)	Monitor de recursos	Ver si un proceso de Julia o R está consumiendo toda la RAM o CPU.
<code>ls -la</code>	Listado detallado	Revisar permisos de archivos y carpetas (clave para errores de Git).
<code>psql --version</code>	Versión de Postgres	Confirmar que el cliente de base de datos está listo para usarse.
<code>which r / which julia</code>	Ruta del ejecutable	Localizar dónde están instalados los lenguajes para configurar el Path.
<code>fc-list \ grep -i "Symbola"</code>	Buscar fuentes del sistema	Localizar la ruta exacta de fuentes necesarias para emojis y símbolos en el PDF.

F.4 Verificación de librerías geoespaciales

El comando `dpkg -l | grep -E "libpng|libgdal|libproj|libgeos"` es su principal herramienta de diagnóstico. Si al ejecutarlo no obtiene resultados, las funciones de mapeo en R (`sf`), Python (`geopandas`) y Julia (`ArchGDAL`) fallarán, ya que todas dependen de estos binarios del sistema.

Tip de Productividad

En la terminal de VSCode, puede usar la tecla **Tab** para autocompletar nombres de archivos o comandos, y las **flechas arriba/abajo** para navegar por el historial de comandos ejecutados.

F.4.1 ¿Cómo leer la salida de `dpkg`?

Al ejecutar el comando de verificación, verá una lista con el prefijo `ii`. Esto significa:

- **i**: *Desired state* (Install)
- **i**: *Current state* (Installed)

Si ve algo diferente a `ii`, el paquete tiene problemas de instalación.

Apéndice G

PostgreSQL

G.1 Introducción al cliente interactivo y por lotes (psql)

La herramienta `psql` es la interfaz de línea de comandos (CLI) nativa y oficial de PostgreSQL. Construida directamente sobre la biblioteca de bajo nivel `libpq`, proporciona un canal de comunicación directo, ligero y eficiente con el motor de base de datos, libre del consumo de recursos de memoria y la sobrecarga gráfica asociados a entornos de escritorio.

En la administración avanzada y el análisis geoespacial, `psql` cumple dos funciones primordiales:

- **Modo interactivo:** Permite explorar catálogos, inspeccionar esquemas, auditar tablas y ejecutar funciones espaciales directamente desde la consola de comandos.
- **Modo por lotes (batch / scripting):** Permite ejecutar sentencias no interactivas (`-c`) o procesar archivos de transacciones DDL/DML masivas (`-f`), facilitando la automatización de flujos de trabajo reproducibles y la compilación dinámica de reportes en Quarto.

G.1.1 Anatomía de comandos: metacomandos de barra invertida frente a sentencias SQL

Una característica distintiva de `psql` es la separación estricta entre sentencias SQL estándar y metacomandos propios del cliente:

- **Sentencias SQL:** Instrucciones que se envían directamente al analizador sintáctico del motor relacional (`SELECT`, `INSERT`, `CREATE TABLE`). Deben terminar obligatoriamente con punto y coma (`;`).
- **Metacomandos de barra invertida (*backslash commands*):** Instrucciones interpretadas y ejecutadas internamente por `psql` en el lado del cliente para inspeccionar metadatos, gestionar parámetros de la sesión o interactuar con el sistema anfitrión sin requerir consultas directas a `information_schema` o `pg_catalog`. No requieren punto y coma al final.

Tabla G.1: Metacomandos esenciales de inspección y administración en psql

Metacomando	Función en la administración y análisis de bases de datos
\c [base_datos] [usuario]	Cambia la conexión de la sesión activa hacia otra base de datos o asume un rol de usuario diferente.
\l o \l+	Lista todas las bases de datos en el clúster con detalles de tamaño, propietario y codificación.
\dn o \dn+	Lista los esquemas disponibles en la base de datos conectada.
\dt [esquema].*	Muestra el catálogo de tablas relacionales existentes en un esquema específico.
\d+ [tabla]	Describe la estructura detallada de una tabla (columnas, tipos de datos, restricciones e índices).
\di o \di+	Lista los índices de la base de datos (fundamental para auditar la creación de índices espaciales GiST).
\dv o \dv+	Muestra el catálogo de vistas lógicas y vistas materializadas instanciadas en el esquema.
\df o \df+	Lista las funciones disponibles (útil para auditar el catálogo algorítmico ST_* de PostGIS).
\dx	Lista las extensiones instaladas en la base de datos activa (<code>postgis</code> , <code>postgis_raster</code> , etc.).
\du o \du+	Muestra los roles de usuario y los privilegios de acceso definidos en el clúster.
\x	Alterna la visualización de resultados de consola entre formato tabular y formato expandido (registro por fila).
\timing	Activa el cronómetro de sesión para medir el tiempo de ejecución analítica (esencial en perfilamiento espacial).
\i [archivo.sql]	Importa y ejecuta un archivo de procesamiento por lotes (<i>batch</i>) con sentencias SQL desde el sistema anfitrión.
\copy	Ejecuta transferencias eficientes en flujos de E/S entre tablas relacionales y archivos planos locales (CSV, TXT).
\e	Abre la última consulta ejecutada en el búfer utilizando el editor de texto predeterminado del sistema anfitrión.
\q	Cierra la sesión activa y finaliza el proceso del cliente interactivo en la terminal integrada.
\?	Despliega el manual de ayuda detallado del catálogo completo de metacomandos soportados por el cliente.

G.1.2 Parámetros de invocación y modificadores de línea de comandos

Al invocar utilidades cliente desde la consola de comandos o mediante scripts de automatización en la terminal integrada de Visual Studio Code, se utilizan modificadores estandarizados para controlar el destino, la autenticación y el comportamiento de la sesión.

Las herramientas cliente de PostgreSQL (`psql`, `pg_dump`, `pg_restore`) y los cargadores espaciales de PostGIS (`shp2pgsql`, `raster2pgsql`) comparten este conjunto de parámetros administrados por la biblioteca `libpq`:

A continuación se presenta la tabla integrada con los modificadores de control de formato y ajuste de salida para `psql` (`-x` y `-P`), manteniendo la estructura taxonómica, el espaciado estricto en banderas CLI y las convenciones de Quarto:

Tabla G.2: Parámetros y modificadores de línea de comandos para utilidades cliente de PostgreSQL y PostGIS

Bandera corta	Modificador largo	Utilidades compatibles	Variable de entorno	Descripción técnica y función operativa
<code>-h <host></code>	<code>--host=<host></code>	<code>psql</code> , <code>pg_dump</code> , <code>pg_restore</code> , <code>shp2psql</code>	<code>PGHOST</code>	Nombre de host, alias DNS de red o dirección IP de la interfaz donde el motor escucha conexiones.
<code>-p <puerto></code>	<code>--port=<puerto></code>	<code>psql</code> , <code>pg_dump</code> , <code>pg_restore</code> , <code>shp2psql</code>	<code>PGPORT</code>	Puerto TCP de enlace al servicio de base de datos (5432 por defecto).
<code>-U <usuario></code>	<code>--username=<usuario></code>	<code>psql</code> , <code>pg_dump</code> , <code>pg_restore</code> , <code>shp2psql</code>	<code>PGUSER</code>	Rol o cuenta de usuario con privilegios para autenticar la sesión.
<code>-d <base_datos></code>	<code>--dbname=<base_datos></code>	<code>psql</code> , <code>pg_dump</code> , <code>pg_restore</code> , <code>shp2psql</code>	<code>PGDATABASE</code>	Nombre del catálogo de base de datos relacional objetivo.
<code>-W</code>	<code>--password</code>	<code>psql</code> , <code>pg_dump</code> , <code>pg_restore</code>	<code>PGPASSWORD</code>	Fuerza la solicitud interactiva de la contraseña en la consola antes de abrir el socket de conexión.
<code>-c <consulta></code>	<code>--command=<consulta></code>	<code>psql</code>	N/A	Pasa una cadena con instrucciones SQL o metacomandos para ejecución no interactiva y finaliza la sesión.
<code>-f <archivo></code>	<code>--file=<archivo></code>	<code>psql</code> , <code>pg_dump</code> , <code>pg_restore</code>	N/A	En <code>psql</code> , lee e inyecta un script SQL; en <code>pg_dump/pg_restore</code> , especifica el archivo de respaldo de entrada o salida.

Bandera corta	Modificador largo	Variable		Descripción técnica y función operativa
		Utilidades compatibles	de entorno	
-x	--expanded	psql	N/A	Activa el modo de visualización expandida vertical (pares atributo-valor por registro) para prevenir desbordes horizontales en salidas tabulares extensas.
-P <opcion=valor>	--pset=<opcion=valor>	psql	N/A	Define parámetros de formato de impresión en la consola (por ejemplo, -P format=wrapped junto con -P columns=<ancho> para forzar el ajuste y envoltura de celdas).
-n <esquema>	--schema=<esquema>	pg_dump, pg_restore	N/A	Restringe la exportación o restauración exclusivamente a los objetos pertenecientes al esquema indicado.
-F <formato>	--format=<formato>	pg_dump, pg_restore	N/A	Define el formato del archivo de respaldo: p (SQL plano), c (<i>custom archive</i> comprimido), d (directorio) o t (<i>tar</i>).
-s	--schema-only	pg_dump, pg_restore	N/A	Extrae o restaura únicamente las definiciones estructurales DDL sin incluir registros de datos.
-a	--data-only	pg_dump, pg_restore, shp2pgsql	N/A	Extrae, restaura o procesa exclusivamente los registros de datos (DML) sin recrear esquemas ni tablas.
-v	--verbose	psql, pg_dump, pg_restore	N/A	Activa la emisión detallada de mensajes de diagnóstico y progreso en el canal de salida estándar.
-s <srid>	N/A	shp2pgsql, raster2pgsql	N/A	Asigna explícitamente el identificador espacial de referencia (SRID) a la columna geométrica o cobertura ráster.

Bandera corta	Modificador largo	Variable		Descripción técnica y función operativa
		Utilidades compatibles	de entorno	
-I	N/A	shp2pgsql, raster2pgsql	N/A	Agrega automáticamente la instrucción DDL para la creación del índice espacial GiST sobre la capa procesada.

G.1.3 Guía de inicio rápido para la administración con psql

La interacción con el motor relacional desde la terminal integrada de Visual Studio Code se estructura en dos modalidades operativas: la ejecución de utilidades del sistema anfitrión mediante el CLI y la emisión interactiva de sentencias DDL/DML o metacomandos dentro de una sesión activa de **psql**.

A continuación, se compilan las operaciones fundamentales de conexión, aprovisionamiento, ejecución de scripts y eliminación de bases de datos.

G.1.3.1 Registro del ejecutable en el sistema anfitrión

Para invocar las herramientas cliente desde cualquier ubicación en la terminal integrada de Visual Studio Code, es necesario incorporar el directorio que aloja los binarios (`<ruta_instalacion>/PostgreSQL/18/bin`) a la variable de entorno **PATH** del sistema anfitrión.

Una vez configurada la variable, se valida la disponibilidad del cliente interactivo:

```
#| label: setup-check-psql-version
#| eval: false
psql --version
```

G.1.3.2 Conexión a la base de datos de plantilla global

Para ejecutar tareas administrativas globales cuando aún no se ha creado una base de datos de usuario, se establece conexión con la base de datos **postgres**:

```
#| label: setup-connect-postgres
#| eval: false
psql -h localhost -p 5432 -U postgres -d postgres
```

G.1.3.3 Creación de bases de datos

El aprovisionamiento de un nuevo contenedor lógico se puede efectuar de forma interactiva (SQL) o mediante el ejecutable dedicado en la consola de comandos:

- Desde una sesión interactiva de **psql** (DDL):

```
#| label: ddl-create-database-quickstart
#| eval: false
CREATE DATABASE <nombre_bd>;
```

- Desde la terminal integrada de Visual Studio Code (CLI):

```
#| label: setup-createdb-cli
#| eval: false
createdb -h localhost -p 5432 -U postgres <nombre_bd>
```

G.1.3.4 Conexión y cambio de contexto de base de datos

Para autenticarse directamente en una base de datos específica o alternar la sesión activa entre diferentes bases de datos del clúster:

- Conexión directa desde la terminal integrada:

```
#| label: setup-connect-database-cli
#| eval: false
psql -h localhost -p 5432 -U <usuario> -d <nombre_bd>
```

- Cambio de contexto dentro de una sesión activa de psql:

```
#| label: setup-switch-database-metacommand
#| eval: false
\c <nombre_bd> <usuario>
```

G.1.3.5 Ejecución de archivos y scripts SQL por lotes

El procesamiento de archivos estructurados (.sql) que contienen secuencias DDL o cargas de datos se puede realizar de forma externa o interna:

- Ejecución externa desde la terminal integrada (modo batch):

```
#| label: setup-execute-script-cli
#| eval: false
psql -h localhost -p 5432 -U <usuario> -d <nombre_bd> -f <ruta_archivo>/<archivo.sql>
```

- Ejecución interna desde la sesión interactiva de psql:

```
#| label: setup-execute-script-metacommand
#| eval: false
\i <ruta_archivo>/<archivo.sql>
```

G.1.3.6 Eliminación de bases de datos

La eliminación física de una base de datos y sus esquemas asociados se realiza asegurando previamente que no existan conexiones activas concurrentes hacia el contenedor:

- Desde una sesión interactiva de psql (DDL):

```
#| label: ddl-drop-database-quickstart
#| eval: false
DROP DATABASE <nombre_bd>;
```

- Desde la terminal integrada de Visual Studio Code (CLI):

```
#| label: setup-dropdb-cli
#| eval: false
dropdb -h localhost -p 5432 -U <usuario> <nombre_bd>
```

G.1.3.7 Finalización de la sesión interactiva

Para concluir la sesión de `psql` y regresar al intérprete de la terminal integrada de Visual Studio Code:

- Ejecutar el metacomando de desconexión `\q`.
- Utilizar la combinación de interrupción de proceso `Ctrl + C` para cancelar una consulta bloqueante o `Ctrl + D` para enviar la señal de fin de transmisión (*EOF*).

Apéndice H

Lista de acrónimos y glosario de conceptos técnicos

H.1 Lista de acrónimos

En la Tabla H.1 se compilan las siglas y acrónimos técnicos empleados a lo largo del libro, indicando su equivalencia en inglés y su ámbito operativo.

Tabla H.1: Catálogo de acrónimos y siglas técnicas normalizadas.

Acrónimo	Denominación formal en español	Término original en inglés	Ámbito funcional
ACID	Atomicidad, consistencia, aislamiento y durabilidad	<i>Atomicity, Consistency, Isolation, Durability</i>	Transaccionalidad en motores de bases de datos.
ADT	Tipo abstracto de datos	<i>Abstract Data Type</i>	Tipado extensible en modelos objeto-relacionales.
AIO	Entrada/salida asíncrona	<i>Asynchronous Input/Output</i>	Subsistema de almacenamiento del motor gestor.
ANN	Vecinos más cercanos aproximados	<i>Approximate Nearest Neighbors</i>	Algoritmia de búsqueda en espacios vectoriales.
CBO	Optimizador basado en costos	<i>Cost-Based Optimizer</i>	Planificación y optimización de consultas SQL.
CRS	Sistema de referencia de coordenadas	<i>Coordinate Reference System</i>	Georreferenciación y geodesia computacional.

Acrónimo	Denominación formal en español	Término original en inglés	Ámbito funcional
DBA	Administrador de base de datos	<i>Database Administrator</i>	Gestión y gobierno de la infraestructura del motor.
DDL	Lenguaje de definición de datos	<i>Data Definition Language</i>	Subconjunto SQL para definición de esquemas.
DML	Lenguaje de manipulación de datos	<i>Data Manipulation Language</i>	Subconjunto SQL para consulta y edición de tuplas.
ECEF	Centrado en la Tierra, fijo a la Tierra	<i>Earth-Centered, Earth-Fixed</i>	Sistema cartesiano tridimensional geocéntrico.
EPSG	Grupo europeo de prospección petrolífera	<i>European Petroleum Survey Group</i>	Registro canónico de parámetros geodésicos.
EWKB	Binario extendido bien conocido	<i>Extended Well-Known Binary</i>	Serialización binaria de geometrías con SRID.
GiST	Árbol de búsqueda generalizado	<i>Generalized Search Tree</i>	Estructura de indexación para dominios no escalares.
GQL	Lenguaje de consulta de grafos	<i>Graph Query Language</i>	Estándar formal ISO/IEC 39075 para bases de datos de grafos.
HNSW	Grafos jerárquicos navegables de mundo pequeño	<i>Hierarchical Navigable Small World</i>	Estructura de indexación y búsqueda aproximada de vectores.
IGAC	Instituto Geográfico Agustín Codazzi	—	Autoridad cartográfica y geodésica de Colombia.
LSM	Árbol estructurado por fusión de registros	<i>Log-Structured Merge-tree</i>	Estructura de almacenamiento en motores de columnas anchas.
MBR	Rectángulo delimitador mínimo	<i>Minimum Bounding Rectangle</i>	Aproximación ortogonal planar de geometrías vectoriales.
MVCC	Control de concurrencia multiversión	<i>Multi-Version Concurrency Control</i>	Aislamiento transaccional sin bloqueos de lectura.
OGC	Consortio geoespacial abierto	<i>Open Geospatial Consortium</i>	Organización de estandarización geoespacial internacional.

Acrónimo	Denominación formal en español	Término original en inglés	Ámbito funcional
OLAP	Procesamiento analítico en línea	<i>Online Analytical Processing</i>	Cargas analíticas masivas y procesamiento columnar.
OLTP	Procesamiento de transacciones en línea	<i>Online Transaction Processing</i>	Transaccionalidad de alta concurrencia orientada a filas.
ORDBMS	Sistema gestor de bases de datos objeto-relacional	<i>Object-Relational Database Management System</i>	Modelo que combina álgebra relacional y tipos abstractos.
RBAC	Control de acceso basado en roles	<i>Role-Based Access Control</i>	Modelo de seguridad y asignación de privilegios en el motor.
SFS	Geometrías simples para SQL	<i>Simple Features for SQL</i>	Estándar OGC / ISO 19125 para tipos espaciales vectoriales.
SGBD	Sistema gestor de bases de datos	<i>Database Management System (DBMS)</i>	Infraestructura de software para administración de datos.
SGBD espacial	Sistema gestor de bases de datos espaciales	<i>Spatial Database Management System (SDBMS)</i>	Motor relacional extendido para geometrías y topología.
SIG	Sistema de información geográfica	<i>Geographic Information System (GIS)</i>	Software para captura, edición y análisis cartográfico.
SIMD	Instrucción única, datos múltiples	<i>Single Instruction, Multiple Data</i>	Procesamiento vectorial a nivel de registros de la CPU.
SQL	Lenguaje de consulta estructurado	<i>Structured Query Language</i>	Estándar declarativo relacional internacional.
SQL/PGQ	Consultas de grafos de propiedades en SQL	<i>SQL/Property Graph Queries</i>	Parte 16 del estándar ISO SQL:2023.
SRID	Identificador del sistema de referencia espacial	<i>Spatial Reference Identifier</i>	Clave numérica para identificación unívoca de CRS.
SSI	Aislamiento de instantáneas serializables	<i>Serializable Snapshot Isolation</i>	Implementación de serializabilidad transaccional sin bloqueos.

Acrónimo	Denominación formal en español	Término original en inglés	Ámbito funcional
TOAST	Técnica de almacenamiento de atributos sobredimensionados	<i>The Oversized-Attribute Storage Technique</i>	Mecanismo de compresión y desborde de páginas en PostgreSQL.
WAL	Bitácora de registro previo	<i>Write-Ahead Logging</i>	Mecanismo de persistencia y durabilidad transaccional.
WKB	Binario bien conocido	<i>Well-Known Binary</i>	Representación binaria estándar OGC para geometrías.
WKT	Texto bien conocido	<i>Well-Known Text</i>	Representación textual estándar OGC para geometrías y CRS.

H.2 Glosario de conceptos técnicos

- **Arquitectura canónica:** Modelo de referencia formal, teórico y universalmente aceptado en la literatura científica y los estándares internacionales para describir la estructura conceptual, la división de subsistemas y los principios operativos de una tecnología, con independencia de soluciones comerciales o implementaciones de código abierto particulares.
- **Atributo (o campo):** Unidad atómica nominada que define una característica, propiedad o dimensión de una entidad dentro de un dominio específico en el esquema relacional.
- **Atributo atómico frente a dato no atómico:** En el diseño relacional formal, clasifica los valores según su capacidad de descomposición estructural:
 - **Atributo atómico:** Valor escalar indivisible que carece de subestructuras con significado independiente dentro del modelo de datos (requisito indispensable de la primera forma normal o 1NF). Ejemplos: un código numérico `srId`, una marca temporal o un identificador entero.
 - **Dato no atómico:** Estructura compuesta, multivaluada o jerárquica cuyos componentes internos pueden descomponerse, direccionarse o evaluarse de forma individual sin alterar la validez del objeto. Ejemplos: arreglos (`ARRAY`), objetos semiestructurados (`JSONB`), tipos compuestos o entidades geométricas (`geometry`), conformadas internamente por secuencias ordenadas de pares o tripletas de coordenadas (X, Y, Z, M) y anillos topológicos.
- **Base de datos espacial (GeoDB):** Colección integrada y estructurada de datos geométricos, geodésicos y alfanuméricos persistidos bajo un esquema unificado, accesible de forma concurrente, con soporte de transaccionalidad ACID y gobernada por catálogos de referencia cartográfica.
- **Base de datos vectorial:** Sistema gestor de persistencia e indexación diseñado para almacenar coordenadas de alta dimensionalidad (*embeddings* en $\mathbb{R}^d$) y resolver consultas de afinidad mediante

métricas formales de distancia (euclidiana, coseno o producto escalar) estructuradas sobre algoritmos de búsqueda aproximada de vecinos más cercanos (ANN).

- **Búsqueda por similitud semántica:** Método de recuperación de información que evalúa la afinidad conceptual entre entidades no estructuradas proyectadas hacia un espacio métrico latente continuo mediante modelos de aprendizaje profundo, sustituyendo la coincidencia léxica exacta por cálculo de proximidad geométrica multidimensional.
- **Cartucho de datos (*cartridge*) o extensión:** Paquete de software desacoplado que se acopla a las interfaces de extensibilidad de un motor objeto-relacional para dotarlo de soporte nativo hacia un dominio específico (por ejemplo, tipos espaciales, clases de operadores GiST y funciones OGC provistas por la extensión `postgis`), sin requerir modificaciones ni recompilación en el código fuente nuclear del gestor.
- **Codificador (*encoder*) y representación densa (*embedding*):**
 - **Codificador (*encoder*):** Función de mapeo $f : \mathcal{X} \rightarrow \mathbb{R}^d$ implementada mediante una red neuronal profunda que transforma una entidad de entrada no estructurada desde un espacio disperso hacia un vector numérico denso en un espacio continuo de dimensión fija d .
 - **Representación densa (*embedding*):** Vector numérico resultante de la inferencia del codificador, donde la proximidad métrica relativa entre vectores condensa equivalencia o similitud semántica.
- **Código objeto:** Es la traducción en lenguaje de máquina binario (archivos `.o` o `.obj`) generada por el compilador a partir del código fuente. Contiene instrucciones comprensibles por la CPU, pero aún no tiene enlazadas las librerías del sistema ni los controladores externos.
- **Código objeto ejecutable:** Es el binario final compilado y enlazado (por ejemplo, un archivo binario ELF en Linux o un ejecutable `.exe` en Windows). El enlazador (`linker`) toma el código objeto de la aplicación y lo fusiona con las librerías de ejecución del cliente de base de datos (como `libpq` o las bibliotecas en tiempo de ejecución del SGBD).
- **Cómputo espacial distribuido en lagos de datos (*spatial lakehouse engine*):** Arquitectura analítica fundamentada en el desacoplamiento estricto entre el motor de ejecución y el sustrato de persistencia física, delegando el almacenamiento a formatos columnares abiertos (como GeoParquet o GeoArrow en repositorios de objetos) y el procesamiento algebraico a grafos acíclicos dirigidos (DAG) en la memoria volátil de un clúster. A diferencia de los SGBD relacionales —cuyo optimizador opera sobre páginas físicas locales y estructuras de indexación ligadas al disco (árboles GiST)—, este paradigma estructura la evaluación de consultas mediante **particionamiento espacial multidimensional distribuido** (*Quad-Tree*, *KDB-Tree* o *R-Tree* global), optimizando la distribución de teselas o entidades entre nodos de trabajo (*workers*) para mitigar el cuello de botella derivado de la transferencia masiva de datos por red (*data shuffling*).

La articulación de este modelo dentro de la disciplina responde a la necesidad de superar los límites de escalabilidad vertical (*scale-up*) propios de las instancias objeto-relacionales cuando se procesan volúmenes masivos de datos geográficos (telemetría continua, mallas de sensores o coberturas continentales). Al implementar formalmente la especificación OGC *Simple Feature Access* mediante sintaxis *Spatial SQL* sobre estructuras paralelas (DataFrames distribuidos), este enfoque faculta la comparación metodológica entre la resolución de predicados topológicos (`ST_Intersects`, `ST_Within`, `ST_Buffer`) gobernada por

un gestor transaccional centralizado frente a su cómputo particionado sobre memoria distribuida elástica.

- **Consistencia transaccional en sistemas distribuidos elásticos:** Garantía formal de cumplimiento determinista de las propiedades ACID y equivalencia secuencial estricta (*serializability* o *linearizability*) sobre datos particionados y distribuidos en múltiples nodos del clúster. Se fundamenta en algoritmos de consenso formal (Raft o Multi-Paxos) para la persistencia por quórum en bitácoras no volátiles (WAL), protocolos de compromiso en dos fases (2PC) para transacciones multinodo y sincronización temporal coordinada (relojes lógicos híbridos o tiempo físico acotado), asegurando un estado global válido e inalterable durante eventos de particionamiento autónomo, migración de rangos y escalado horizontal.
- **Control de acceso basado en roles** (Role-Based Access Control, RBAC): es un modelo de seguridad donde los privilegios sobre los recursos del sistema (conexión, lectura, escritura, ejecución de funciones o definición de esquemas) se asignan a entidades abstractas denominadas roles, y no de forma individual a cada cuenta de usuario. Los usuarios finales obtienen sus autorizaciones al ser vinculados como miembros de uno o varios roles predefinidos.
- **Ejecución vectorial** (*vectorized execution*): Paradigma de diseño del procesador de consultas en motores analíticos (OLAP) que sustituye el modelo iterador tupla a tupla por la evaluación de bloques homogéneos contiguos de valores de una misma columna en la memoria intermedia, permitiendo al compilador emitir instrucciones de hardware **SIMD** a nivel de registros de CPU.
- **Embeddings frente a análisis de componentes principales (PCA):** Enfoques de reducción de dimensionalidad orientados a proyectar entidades desde espacios de alta dimensionalidad ($\mathbb{R}^D$) hacia subespacios densos y continuos de menor dimensión ($\mathbb{R}^d$, con $d \ll D$). Mientras que el análisis de componentes principales (PCA) constituye una transformación lineal y ortogonal determinista que maximiza la varianza global proyectada mediante descomposición espectral o en valores singulares (SVD), los *embeddings* representan un mapeo no lineal aprendido mediante redes neuronales profundas que preserva vecindades locales y relaciones semánticas sobre variedades topológicas complejas (*manifolds*).

Tabla H.2: Comparación técnica y matemática entre PCA y modelos de embeddings.

	Análisis de componentes principales (PCA)	Representaciones densas (<i>embeddings</i>)
Dimensión analítica		
Naturaleza del mapeo	Transformación estrictamente lineal y ortogonal.	Mapeo continuo no lineal parametrizado por funciones de activación.
Método de resolución	Determinista y cerrado (descomposición espectral de la matriz de covarianza o SVD).	Iterativo y estocástico (optimización por descenso de gradiente sobre redes neuronales).
Criterio de optimización	Maximización de la varianza proyectada y minimización del error de reconstrucción cuadrático medio.	Minimización de funciones de pérdida contrastivas, tripletas o de entropía cruzada.

Dimensión analítica	Análisis de componentes principales (PCA)	Representaciones densas (<i>embeddings</i>)
Estructura de las dimensiones	Ejes ortogonales ordenados jerárquicamente de forma decreciente según la varianza explicada.	Dimensiones continuas no jerárquicas con semántica distribuida (la información reside en la dirección y distancia angular o métrica).
Captura de variedades (<i>manifolds</i>)	Proyecta sobre hiperplanos planos; no desenrolla curvaturas ni topologías no lineales en los datos.	Modela geometrías intrincadas y relaciones de vecindad no euclidianas en el espacio latente.
Supervisión y contexto	Técnica no supervisada orientada exclusivamente a la dispersión numérica de los datos de entrada.	Aprendizaje autosupervisado o supervisado guiado por tareas específicas (coocurrencia, similitud semántica o grafos).
Ámbito operativo en SGBD	Compresión de variables, decorrelación previa y reducción de dimensionalidad tabular.	Almacenamiento e indexación en bases de datos vectoriales mediante algoritmos ANN (HNSW, IVF-PQ) para búsqueda por proximidad semántica.

- **Encapsulamiento** (modelo objeto-relacional o Tipo 3): es el principio que agrupa la estructura interna y física de un dato complejo junto con la lógica algorítmica que lo manipula, ocultando la representación física en disco (arreglos binarios, compensaciones de memoria o índices internos) y exponiendo exclusivamente una interfaz declarativa de funciones estandarizadas.

En PostgreSQL y PostGIS, una columna de tipo **geometry** implementa este principio: el motor almacena internamente una estructura binaria de bajo nivel (**GSERIALIZED**) que integra coordenadas, dimensiones, caja envolvente (MBR) y SRID. El analista no accede a los *offsets* de bytes en disco, sino que opera el objeto a través de constructores y métodos conformes a los estándares OGC e ISO:

```
-- 1. Definición del tipo abstracto: la estructura física queda encapsulada
CREATE TABLE public.zonas_reserva (
  id serial PRIMARY KEY,
  nombre varchar(50) NOT NULL,
  geom geometry(Polygon, 9377) NOT NULL
);

-- 2. Manipulación mediante constructor formal OGC (oculta la codificación binaria)
INSERT INTO public.zonas_reserva (nombre, geom)
VALUES (
  'Parque Ecológico',
  ST_GeomFromText('POLYGON((5000000 2000000, 5001000 2000000, 5001000 2001000, 5000000 2001000, 5000000
↪ 2000000))', 9377)
);

-- 3. Invocación de métodos y predicados: evaluación sin deserialización manual
SELECT
  nombre,
  ST_Area(geom) AS area_m2,
  ST_AsText(geom) AS wkt_serializado
FROM public.zonas_reserva;
```

- **Encoder** (codificador frente a filtrado espacial clásico en teledetección): Modelo de extracción

y reducción dimensional que sustituye la convolución planar analítica por representaciones latentes profundas. Mientras que el procesamiento digital de imágenes (PDI) aplica núcleos invariantes estáticos (K , ej. Sobel 3×3) sobre una matriz monocanal ($H \times W$) para generar un ráster derivado que preserva la correspondencia biyectiva celda a celda ($H \times W \times 1$, persistido como GeoTIFF/COG para álgebra de mapas), un *encoder* procesa el tensor multispectral continuo completo ($H \times W \times B$). Mediante bancos convolucionales jerárquicos (W) optimizados por retropropagación y acoplados a activaciones no lineales, el codificador contrae progresivamente la resolución topológica de la escena mientras expande la profundidad semántica de los canales ($C_1 < C_2 < C_3$). Finalmente, suprime la grilla planar mediante agregación global (*Global Average Pooling*) para condensar la tesela en un único vector denso latente (*embedding* $1 \times d$, $\mathbf{z} \in \mathbb{R}^d$), estructurado para indexación métrica y recuperación por similitud (k -NN mediante índices HNSW) en bases de datos vectoriales como PostgreSQL con **pgvector**.

En la Figura H.1 se esquematiza el paralelo operativo entre la convolución analítica superficial de PDI y la transformación jerárquica profunda del codificador neuronal hacia el espacio latente.

- **Esquema de base de datos:** Definición estructural, lógica y semántica de las relaciones, tipos de datos, restricciones de dominio, claves y reglas de integridad que gobiernan un repositorio relacional, expresada formalmente mediante sentencias DDL.
- **Esquema en lectura (*schema-on-read*):** Paradigma de persistencia (característico de motores NoSQL documentales) donde los datos se almacenan sin someterse a una validación estructural previa contra un esquema DDL formal. El motor o la aplicación compila e interpreta la estructura, los tipos de datos y la jerarquía en el momento exacto de evaluar la consulta, contrastando con el esquema rígido en escritura (*schema-on-write*) propio de los sistemas relacionales.
- **Garantías ACID:** Conjunto de propiedades operativas que gobiernan el procesamiento transaccional en un SGBD:
 - **Atomicidad (*Atomicity*):** Garantía de indivisibilidad operativa; una transacción se consolida en su totalidad o revierte todas sus modificaciones mediante el uso coordinado del registro previo en bitácora (WAL) y registros de reversión (*rollback*).
 - **Consistencia (*Consistency*):** Preservación del estado válido del catálogo relacional; el motor invalida y aborta cualquier transacción que transgreda restricciones declarativas (PRIMARY KEY, FOREIGN KEY, CHECK) o disparadores.
 - **Aislamiento (*Isolation*):** Ejecución concurrente protegida contra anomalías; el gestor de transacciones aísla las modificaciones simultáneas valiéndose de control de concurrencia multiversión (MVCC) o protocolos de bloqueo.
 - **Durabilidad (*Durability*):** Garantía de persistencia no volátil; el motor asegura que, una vez confirmado el COMMIT, los cambios residan en disco persistente mediante sincronización del WAL (**fsync**), permitiendo su recuperación ante fallos del kernel o cortes eléctricos.
- **Grafos de mundo pequeño navegables y jerárquicos (HNSW):** Estructura de *indexación* basada en grafos multicapa superpuestos que optimiza la *búsqueda* aproximada de vecinos más cercanos (ANN) en espacios vectoriales continuos, logrando una complejidad temporal de escala logarítmica $O(\log N)$ mediante un recorrido voraz (*greedy routing*) descendente.
- **Herencia de tablas (en SGBD objeto-relacionales):** Mecanismo estructural del modelo relacional extendido que permite a una relación subordinada (tabla hija) adquirir automáticamente la totalidad

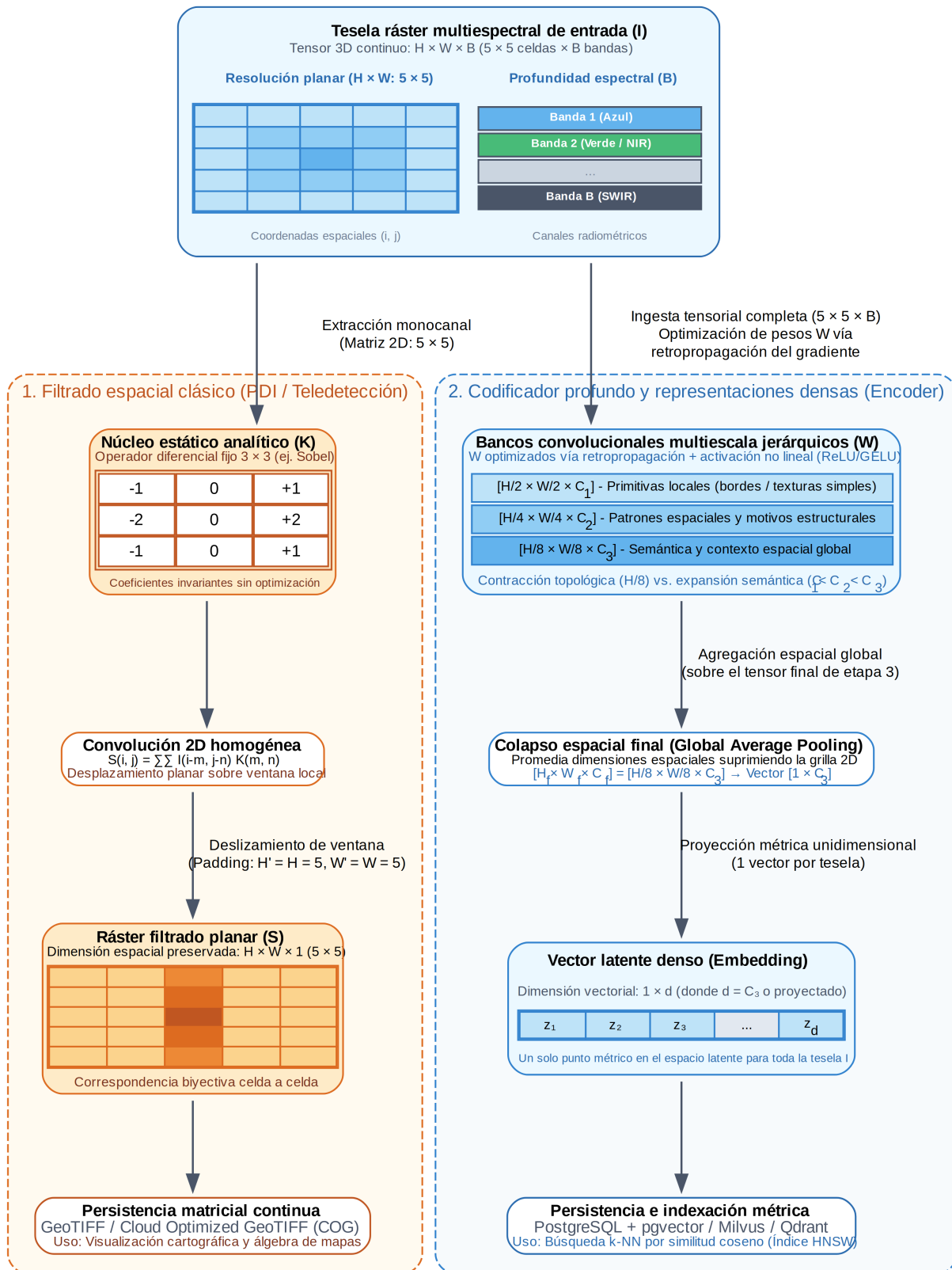


Figura H.1: Representación matricial y dimensional del filtrado espacial clásico frente a la extracción de embeddings mediante encoders.

de las columnas, tipos de datos y restricciones declaradas en una o más relaciones base (tablas padre) mediante la cláusula `INHERITS`. Las consultas de agregación o escaneo ejecutadas sobre la relación padre acceden de forma polimórfica y transparente a las tuplas de todas sus tablas descendientes, a menos que se limite el alcance a la relación superior mediante la cláusula declarativa `ONLY`.

En bases de datos espaciales, este mecanismo formaliza taxonomías geográficas jerárquicas (por ejemplo, derivar redes de acueducto o elementos de transporte a partir de una entidad abstracta con geometría común):

```
-- 1. Definición de la relación padre (esquema y geometría común)
CREATE TABLE public.elemento_red (
  id serial PRIMARY KEY,
  codigo varchar(20) NOT NULL UNIQUE,
  estado_operativo varchar(15) DEFAULT 'ACTIVO',
  geom geometry(Geometry, 9377) NOT NULL
);

-- 2. Creación de relaciones hijas mediante la cláusula INHERITS
CREATE TABLE public.valvula (
  diametro_pulgadas numeric(4, 2) NOT NULL,
  presion_nominal_psi integer NOT NULL
) INHERITS (public.elemento_red);

CREATE TABLE public.tuberia (
  material varchar(30) NOT NULL,
  rugosidad_coef numeric(5, 4) NOT NULL
) INHERITS (public.elemento_red);

-- 3. Inserción de tuplas en las relaciones especializadas
INSERT INTO public.valvula (codigo, estado_operativo, geom, diametro_pulgadas, presion_nominal_psi)
VALUES ('VAL-01', 'ACTIVO', ST_GeomFromText('POINT(5000000 2000000)', 9377), 4.0, 150);

INSERT INTO public.tuberia (codigo, estado_operativo, geom, material, rugosidad_coef)
VALUES ('TUB-01', 'ACTIVO', ST_GeomFromText('LINESTRING(5000000 2000000, 5000200 2000000)', 9377), 'PVC',
↵ 0.0090);

-- 4. Consulta polimórfica (recupera todas las hijas) frente a consulta restringida (ONLY)
SELECT codigo, ST_GeometryType(geom) AS tipo_geom FROM public.elemento_red;
SELECT codigo, ST_GeometryType(geom) AS tipo_geom FROM ONLY public.elemento_red;
```

- **Invariantes del modelo:** En teoría de bases de datos y diseño formal de sistemas, un invariante es una condición lógica o predicado que debe cumplirse y permanecer verdadero en todo momento, a través de cualquier transición de estado del sistema. Si una transacción intenta ejecutar una modificación que vulnere dicha condición, el motor aborta la operación y ejecuta una reversión (rollback) para impedir que la base de datos entre en un estado inconsistente. Ejemplos: Integridad de entidad (unicidad y no nulidad, ej: llave primaria); integridad referencial (llave foránea), integridad de dominio (check), validez topológica, homogeneidad de referencia espacial (SRID), consistencia dimensional). El diccionario de datos almacena estas especificaciones para que el gestor de integridad pueda contrastar cualquier operación DML contra ellas antes de permitir su persistencia física en las páginas de almacenamiento en disco.
- **Método de clase:** Función vinculada a la definición formal del tipo abstracto de datos (clase) y no al estado de un objeto particular en memoria o disco. Opera habitualmente como constructor o método de fábrica (*factory method*) para instanciar nuevos objetos espaciales a partir de argumentos escalares o serializaciones externas (WKT, WKB, coordenadas numéricas):

```
-- Invocación del constructor sin requerir una entidad preexistente en la base de datos
SELECT ST_GeomFromText('POINT(5000000 2000000)', 9377) AS nueva_geometria;
```

- **Método de instancia:** Función asociada a un objeto específico ya materializado en el motor, la cual opera accediendo a su estado interno encapsulado (secuencias de coordenadas, dimensiones, vértices o anillos topológicos) para calcular métricas, evaluar predicados espaciales o derivar propiedades:

```
-- Invocación sobre los atributos geométricos encapsulados de una tupla persistida
SELECT
  codigo,
  ST_Area(geom) AS area_m2,
  ST_GeometryType(geom) AS tipo_geometria
FROM public.predios;
```

- **Modelo de despliegue embebido (*in-process*):** Arquitectura donde el motor de base de datos se vincula como biblioteca de ejecución dentro del mismo proceso y espacio de memoria de la aplicación anfitriona (DuckDB o SQLite), eliminando la comunicación por red y la sobrecarga de sockets propia del esquema cliente-servidor.
- **OID (identificador de objeto y principio de identidad):** Valor numérico único, global e inmutable asignado automáticamente por el motor gestor en modelos orientados a objetos y objeto-relacionales que confiere distinción unívoca a cada entidad, con independencia del estado o valores contenidos en sus atributos (desacoplamiento entre identidad y estado). A diferencia del modelo relacional estricto—donde la unicidad depende de los valores escalares de una clave primaria—, dos objetos con atributos descriptivos y geometrías idénticas constituyen instancias independientes en memoria y disco al portar OID distintos:

```
-- Relación con entidades de idéntico estado espacial y temático
CREATE TABLE public.muestras_duplicadas (
  fid serial PRIMARY KEY, -- Identificador subrogado equivalente al OID
  sensor varchar(20) NOT NULL,
  geom geometry(Point, 9377) NOT NULL
);

-- Inserción de tuplas con valores y geometría idénticos
INSERT INTO public.muestras_duplicadas (sensor, geom) VALUES
('TERM-01', ST_GeomFromText('POINT(5000000 2000000)', 9377)),
('TERM-01', ST_GeomFromText('POINT(5000000 2000000)', 9377));

-- Estado idéntico preservando identidades independientes por identificador
SELECT fid AS oid_objeto, sensor, ST_AsText(geom) AS wkt
FROM public.muestras_duplicadas;
```

- **OLTP frente a OLAP como paradigmas arquitectónicos:** Patrones conceptuales de diseño, estructuración física del almacenamiento y optimización de ejecución que delimitan el perfil operativo de un motor de bases de datos. El procesamiento transaccional en línea (**OLTP**) prioriza la atomicidad ACID, alta concurrencia y baja latencia en operaciones puntuales de lectura/escritura sobre esquemas normalizados orientados a filas (*heap/row-oriented*); en contraste, el procesamiento analítico en línea (**OLAP**) maximiza el rendimiento en escaneos masivos, agregaciones complejas y proyecciones de atributos específicos sobre esquemas desnormalizados orientados a columnas (*columnar storage*).

Tabla H.3: Comparación técnica entre paradigmas arquitectónicos OLTP y OLAP.

Dimensión arquitectónica	Procesamiento transaccional (OLTP)	Procesamiento analítico (OLAP)
Objetivo operativo	Transacciones concurrentes atómicas en tiempo real.	Consultas complejas, analítica histórica y agregaciones masivas.
Disposición física	Orientada a filas (<i>heap storage</i> , páginas de 8 KB).	Orientada a columnas (<i>columnar blocks</i> contiguos por atributo).
Patrón de acceso	Lecturas y mutaciones atómicas (INSERT, UPDATE) de registros individuales.	Escaneos secuenciales masivos proyectando pocas columnas.
Esquema de datos	Normalizado (3NF/BCNF) para suprimir redundancias y anomalías de escritura.	Desnormalizado (esquemas en estrella, copo de nieve o tablas anchas).
Mecanismo de cómputo	Procesamiento transaccional de baja latencia con aislamiento ACID.	Paralelismo interprocesos, ejecución vectorial en CPU (SIMD) y alta compresión.

Aunque PostgreSQL fue concebido en su núcleo como un motor relacional **OLTP** basado en páginas físicas orientadas a filas (*heap storage* de 8 KB), su diseño extensible permite ejecutar cargas de trabajo analíticas de perfil **OLAP**. Esta convergencia se logra combinando la API *Table Access Method* (TAM) para almacenamiento y compresión columnar (**citus**, **hydra**), optimizaciones nativas del motor de ejecución (procesamiento paralelo, compilación JIT vía LLVM y poda dinámica de particiones), aceleración por hardware con instrucciones SIMD y la delegación de cómputo (*pushdown*) hacia formatos columnares o lagos de datos externos (como Apache Parquet) mediante *Foreign Data Wrappers* (FDW).

- **Modelo OSI (*Open Systems Interconnection*)**: Marco arquitectónico y conceptual de referencia estandarizado por la Organización Internacional de Normalización (ISO/IEC 7498-1) que descompone el proceso de comunicación e interoperabilidad en redes de datos en siete capas funcionales jerárquicas e independientes (Física, Enlace de datos, Red, Transporte, Sesión, Presentación y Aplicación). Cada estrato abstrae un conjunto específico de responsabilidades mediante interfaces bien definidas, prestando servicios a la capa superior e interactuando con la inferior mediante encapsulamiento sucesivo de cabeceras y unidades de datos de protocolo (PDU).

En la arquitectura de sistemas gestores de bases de datos objeto-relacionales (ORDBMS), este modelo formaliza el costo computacional de la topología cliente-servidor: cada consulta SQL y cada conjunto de geometrías transmitidas deben franquear la pila completa desde el protocolo de aplicación en el espacio de usuario (Capa 7), resolviendo la serialización binaria de objetos geométricos (Capa 6), la sesión autenticada (Capa 5) y la segmentación TCP en un puerto asignado como el 5432 (Capa 4), hasta el direccionamiento IP (Capa 3) y la transmisión de tramas sobre interfaces de red o adaptadores virtuales de bucle invertido (*loopback*).

- **Persistencia de clases (en bases de datos orientadas a objetos y objeto-relacionales)**: Mecanismo arquitectónico que permite a las instancias de una clase (u objetos en memoria con identidad propia mediante OID, atributos complejos y comportamiento asociado) almacenarse de forma directa y

no volátil en disco, sobreviviendo a la terminación del proceso que las creó sin requerir su descomposición forzada en múltiples tablas relacionales normalizadas (mitigación del desajuste de impedancia objeto-relacional). En motores objeto-relacionales, se materializa mediante tipos compuestos definidos por el usuario (CREATE TYPE) o tablas tipadas estructuradas a partir de tipos de datos abstractos:

```
-- 1. Definición del tipo compuesto estructurado (equivalente a la clase en memoria)
CREATE TYPE public.tipo_estacion_meteo AS (
  identificador varchar(15),
  altitud_msnm numeric(6, 2),
  sensores text[],
  ubicacion geometry(Point, 9377)
);

-- 2. Persistencia directa de instancias del tipo abstracto dentro de una relación
CREATE TABLE public.red_hidrometeorologica (
  id serial PRIMARY KEY,
  cuenca varchar(50) NOT NULL,
  estacion public.tipo_estacion_meteo NOT NULL
);

-- 3. Instanciación y almacenamiento del objeto preservando su estructura integral
INSERT INTO public.red_hidrometeorologica (cuenca, estacion)
VALUES (
  'Cuenca Alta Río Bogotá',
  ROW('EST-SABANA-01', 2550.00, ARRAY['Pluviómetro', 'Anemómetro'], ST_GeomFromText('POINT(5000000
    ↪ 2000000)', 9377))
);

-- 4. Acceso directo a los miembros y geometrías del objeto persistido
SELECT
  cuenca,
  (estacion).identificador AS codigo_estacion,
  (estacion).altitud_msnm AS elevacion,
  ST_AsText((estacion).ubicacion) AS wkt_posicion
FROM public.red_hidrometeorologica;
```

- **Polimorfismo (en SGBD objeto-relacionales):** Principio arquitectónico que faculta a una misma interfaz, operador o función para procesar argumentos de distintos tipos concretos pertenecientes a una misma jerarquía de herencia (*Geometry*), ejecutando en tiempo de ejecución (*runtime*) la rutina algorítmica específica según la signature morfológica del objeto (despacho dinámico). En esquemas relacionales, habilita la definición de columnas espaciales genéricas capaces de alojar simultáneamente entidades de distinta dimensión topológica (*Point*, *LineString*, *Polygon*) bajo una misma relación:

```
-- 1. Definición de columna espacial polimórfica (acepta cualquier subtipo OGC)
CREATE TABLE public.infraestructura_multitipo (
  id serial PRIMARY KEY,
  nombre varchar(50) NOT NULL,
  geom geometry(Geometry, 9377) NOT NULL
);

-- 2. Inserción de primitivas heterogéneas en la misma relación
INSERT INTO public.infraestructura_multitipo (nombre, geom) VALUES
('Punto de control', ST_GeomFromText('POINT(5000000 2000000)', 9377)),
('Línea de conducción', ST_GeomFromText('LINESTRING(5000000 2000000, 5000500 2000500)', 9377)),
('Predio operativo', ST_GeomFromText('POLYGON((5000000 2000000, 5000200 2000000, 5000200 2000200, 5000000
  ↪ 2000200, 5000000 2000000))', 9377));

-- 3. Despacho dinámico: ST_Area y ST_Length resuelven según la morfología del registro
SELECT
  nombre,
  ST_GeometryType(geom) AS subtipo,
  ST_Length(geom) AS longitud_m,
  ST_Area(geom) AS area_m2
FROM public.infraestructura_multitipo;
```

- **Propiedad tipada:** Atributo, campo o par clave-valor asociado a una entidad (en esquemas relacionales, objeto-relacionales o grafos de propiedades) cuyo dominio de valores admisibles, representación física en memoria/disco y operadores válidos están formalmente restringidos por un tipo de dato explícito

(primitivo, estructurado o abstracto). Su declaración impone validación de integridad semántica en tiempo de escritura (*schema-on-write*), previene anomalías por incoherencia de dominios y permite al motor de base de datos asignar métodos de acceso indexados y optimizadores algebraicos especializados (tales como predicados topológicos sobre columnas espaciales).

- **Relación (o tabla):** Estructura lógica bidimensional fundamental del modelo relacional, formalizada matemáticamente como un subconjunto no ordenado del producto cartesiano de los dominios de sus atributos, consistente con un esquema formal y compuesta por tuplas homogéneas.
- **Serialización:** Concepto con doble acepción en sistemas gestores de bases de datos:
 - **Serializabilidad transaccional:** Nivel supremo de aislamiento (**SERIALIZABLE**) que asegura que el resultado final de transacciones concurrentes sea equivalente a una ejecución estrictamente secuencial, previniendo anomalías complejas como el sesgo de escritura (*write skew*).
 - **Serialización de datos:** Proceso algorítmico de codificación que transforma estructuras dinámicas en memoria RAM (objetos C, topologías o grafos) en secuencias contiguas de bytes (como WKB/EWKB) o representaciones textuales (como WKT o GeoJSON) para su persistencia física o transporte en red.
- **Serialización SQL:** Transformación de objetos y estructuras de datos en sentencias o literales textuales interpretables por la gramática de SQL (como sentencias DML o llamadas a constructores tipo `ST_GeomFromText`) para su persistencia en scripts o transferencia en el motor.
- **Serializaciones XML/GML:** Codificación jerárquica basada en etiquetas de texto bajo el estándar OGC *Geography Markup Language* (ISO 19136), orientada al intercambio e interoperabilidad de entidades geográficas y sus atributos a través de servicios web (como OGC WFS).
- **Sistema distribuido elástico:** Arquitectura de base de datos compuesta por un clúster de nodos interconectados que opera ante el cliente como una única instancia lógica, con capacidad para aprovisionar (*scale-out*) o liberar (*scale-in*) recursos de procesamiento y almacenamiento de forma dinámica y automática según la demanda transaccional, sin degradación ni caída del servicio (*zero downtime*). En motores NewSQL y SQL distribuido, se fundamenta en el particionamiento horizontal autónomo (*auto-sharding*), la migración transparente de fragmentos de datos y la preservación de transaccionalidad ACID global mediante protocolos de consenso (Raft o Paxos).
- **Solución *ad-hoc*:** Implementación técnica construida a la medida para resolver una demanda analítica o de almacenamiento puntual, caracterizada por la ausencia de un marco arquitectónico reutilizable, carencia de esquemas formales y acoplamiento dependiente del formato local.
- **Tipo abstracto de datos (ADT):** Tipo de dato formal definido por el usuario o una extensión cuya estructura interna y métodos de serialización están encapsulados, operando en el motor como un tipo de dato nativo.
- **Tipo de dato geography:** Tipo abstracto de datos especializado en PostGIS para la persistencia y cómputo de relaciones topológicas sobre un elipsoide geodésico de referencia (comúnmente WGS 84, EPSG:4326). Opera bajo trigonometría esférica/elipsoidal, emitiendo resultados métricos en metros sin requerir transformaciones cartográficas planas intermedias, con una sobrecarga de cálculo superior al tipo plano *geometry*.

- **Tupla (registro o fila):** Unidad estructural elemental en una relación relacional que representa una instancia o entidad discreta del dominio, conformada por una colección finita de valores correspondientes a los atributos declarados en el esquema.

Apéndice I

Errata

Esta es la errata

Referencias Bibliográficas

- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377-387.
- Codd, E. F. (1972). Further normalization of the data base relational model. *Data base systems*, 6(1972), 33-64.
- Codd, E. F. (1974). *Recent Investigations in Relational Data Base Systems*. IBM Thomas J. Watson Research Division.
- Fagin, R. (1977). Multivalued dependencies and a new normal form for relational databases. *ACM Transactions on Database Systems (TODS)*, 2(3), 262-278.
- Fagin, R. (1979). Normal forms and relational database operators. *Proceedings of the 1979 ACM SIGMOD international conference on Management of data*, 153-160.
- Fagin, R. (1981). A normal form for relational databases that is based on domains and keys. *ACM Transactions on Database Systems (TODS)*, 6(3), 387-415.
- Inc., D. (2025). *Docker Desktop Documentation*. <https://docs.docker.com>
- Microsoft Corporation. (2026). *Visual Studio Code*. <https://code.visualstudio.com/>
- Open Geospatial Consortium. (2010). *OpenGIS Implementation Specification for Geographic Information - Simple Feature Access - Part 2: SQL Option* (OGC Implementation Standard OGC 06-104r4; Versión 1.2.1). Open Geospatial Consortium. <https://docs.ogc.org/is/06-104r4/06-104r4.pdf>
- Open Geospatial Consortium. (2011). *OpenGIS Implementation Specification for Geographic Information - Simple Feature Access - Part 1: Common Architecture* (OGC Implementation Standard OGC 06-103r4; Versión 1.2.1). Open Geospatial Consortium. <https://docs.ogc.org/is/06-103r4/06-103r4.pdf>
- Ramez, E. (2007). *Fundamentos de sistemas de bases de datos*. Pearson Educación.
- Silberschatz, A., Korth, H. F., Sudarshan, S., et al. (2002). *Database system concepts* (Vol. 5). McGraw-Hill New York.
- Xie, Y. (2019). TinyTeX: A lightweight, cross-platform, and easy-to-maintain LaTeX distribution based on TeX Live. *TUGboat*, 40(1), 30-32.

